

Plan Curricular 2026 de la Escuela Profesional de Computación Científica https://matematicas.unmsm.edu.pe/eap-computacion-cientifica.html

– Reporte Final–

Última modificación: Agosto de 2025

Este documento está basado en la propuesta internacional de *ACM/IEEE-CS Computing Curricula*, que a su vez sirvió como base de la versión en Español creada por la Sociedad Peruana de Computación (SPC)

ISBN:

Equipo de Trabajo

Dra. Maria Zegarra-Garay
Decana de la Facultad de Ciencias Matemáticas
email: *mzegarrag@unmsm.edu.pe*
Mg. Jorge Condado Jauregui
Profesor de la Facultad de Ciencias Matemáticas de la UNMSM
email: *jcondadoj@unmsm.edu.pe*
Dra. Frida López-Córdova
Profesora de la Facultad de Ciencias Matemáticas de la UNMSM
email: *flopezc@unmsm.edu.pe*
Mg. Victoria Ysabel Rojas Rojas
Profesora de la Facultad de Ciencias Matemáticas de la UNMSM
email: *vrojasr@unmsm.edu.pe*
Mg. Rubén Arbañil
Profesor de la Facultad de Ciencias Matemáticas de la UNMSM
email: *rarbanilr@unmsm.edu.pe*
Lic. Máximo Gerardo Gonzales Chávez
Profesor de la Facultad de Ciencias Matemáticas de la UNMSM
email: *mgonzalesc1@unmsm.edu.pe*
Mg. Oscar Benito Pacheco
Profesor de la Facultad de Ciencias Matemáticas de la UNMSM
email: *obenitop@unmsm.edu.pe*
Dra. Rosa Medina Aguilar
Profesora de la Facultad de Ciencias Matemáticas de la UNMSM
email: *rmedinaa1@unmsm.edu.pe*
Mg. Luz Teresa Quispe Vega
Profesora de la Facultad de Ciencias Matemáticas de la UNMSM
email: *lquispev@unmsm.edu.pe*
Mg. Miriam Gisell Pescorán Florencio
Profesora de la Facultad de Ciencias Matemáticas de la UNMSM
email: *mpescoranf@unmsm.edu.pe*
Lic. Joseph Wilmer Simpe Laura
Profesor de la Facultad de Ciencias Matemáticas de la UNMSM
email: *jsimpel@unmsm.edu.pe*
Mg. Francisco Felipe Quiroz Chueca
Profesor de la Facultad de Ciencias Matemáticas de la UNMSM
email: *fquirozc@unmsm.edu.pe*
Mg. Rómulo Fernando Lomparte Alvarado
Profesor de la Facultad de Ciencias Matemáticas de la UNMSM
email: *rlompartea@unmsm.edu.pe*
Dr. Omar Guzmán Rea
Profesor de la Facultad de Ciencias Matemáticas de la UNMSM
email: *oguzmanr@unmsm.edu.pe*
Ernesto Cuadros-Vargas
Orador distinguido para la *Association of Computing Machinery* (ACM) (2025-2027)
Miembro del Directorio de Gobernadores de la Sociedad de Computación del IEEE (2020-2023)
Miembro del *Steering Committee ACM/IEEE-CS Computing Curricula 2020 (CS2020)*
Miembro del *Steering Committee ACM/IEEE-CS Computing Curricula/Computer Science (CS2013)*
Secretario Ejecutivo del Centro Latinoamericano de Estudios en Informática (CLEI) 2008-2016
email: *ecuadros@spc.org.pe*

Representantes estudiantiles

Maria Fernanda Rupay Caballero
Estudiante de Computación Científica, UNMSM, 2024
email: *maria.rupayc@unmsm.edu.pe*
Sebastián Alfredo Borja Ramos
Estudiante de Computación Científica, UNMSM, 2024
email: *sebastian.borja@unmsm.edu.pe*
Angel Aarón Reyes Cáceres
Estudiante de Computación Científica, UNMSM, 2024
email: *angel.reyesc@unmsm.edu.pe*
Carlos Jesús López Bravo
Estudiante de Computación Científica, UNMSM, 2024
email: *carlos.lopez22@unmsm.edu.pe*
Julio André Carrillo Montero
Scientific Computing Student, UNMSM, 2023
email: *julio.carrillom@unmsm.edu.pe*
Adrian Enrique Borstelmann Segura
Estudiante de Computación Científica, UNMSM, 2025
email: *adrian.borstelmanns@unmsm.edu.pe*
Jan Lucas Mancinelli Vite
Estudiante de Computación Científica, UNMSM, 2023
email: *jan.mancinelli@unmsm.edu.pe*
Adriano Emer Valdivia
Estudiante de Computación Científica, UNMSM, 2025
email: *adriano.valdivia@unmsm.edu.pe*
Cristian Josue Bernal Nuñez
Estudiante de Computación Científica, UNMSM, 2023
email: *cristian.bernal@unmsm.edu.pe*
Jhanilde Pozo Hernandez
Estudiante de Computación Científica, UNMSM, 2023
email: *jhanilde.pozo@unmsm.edu.pe*
Franco David Zevallos Silva
Estudiante de Computación Científica, UNMSM, 2025
email: *franco.zevalloss@unmsm.edu.pe*
Angello Andre Manyari Medina
Estudiante de Computación Científica, UNMSM, 2023
email: *angello.manyari@unmsm.edu.pe*

Resumen ejecutivo

Este documento representa el informe de la nueva malla curricular 2026 del Escuela Profesional de Computación Científica (Matemática Computacional) de la Universidad Nacional Mayor de San Marcos (UNMSM) (*https://www.unmsm.edu.pe*) en la ciudad de Lima-Perú.

Este documento presenta el diseño curricular de la carrera de Matemática Computacional, una disciplina fundamental que se sitúa en la intersección entre la matemática pura y aplicada, y las ciencias de la computación. El diseño se alinea con los estándares internacionales de acreditación en matemáticas y se nutre de los marcos de referencia del *Computing Curricula* CC2020, desarrollado por la *Association for Computing Machinery* (ACM) y la *IEEE Computer Society* (IEEE-CS). Esta doble vertiente garantiza una formación sólida y moderna.

La propuesta adapta estos referentes globales a nuestro contexto regional, incorporando:

- Demandas específicas del sector productivo local en modelado, simulación y análisis de datos.
- Fortalezas institucionales en investigación matemática y su aplicación a problemas complejos.
- Tendencias emergentes que requieren un pensamiento abstracto y algorítmico robusto.

La Matemática Computacional se distingue de otros perfiles de la computación por su enfoque central en la teoría y los fundamentos matemáticos para resolver problemas complejos. Se diferencia y complementa a:

- **Ciencia de la Computación (CS)**: Profundizando en la base matemática de los algoritmos y la teoría de la computación.
- **Ingeniería de Software (SE)**: Aportando el rigor formal para la verificación y corrección de sistemas.
- **Ciencia de Datos (DS)**: Proporcionando el andamiaje teórico para el aprendizaje automático y el análisis estadístico.
- **Ingeniería de Computación (CE)**: Colaborando en el modelado y simulación de sistemas físicos y digitales.

El modelo pedagógico se sustenta en tres ejes fundamentales:

- **Rigor Matemático**: Dominio profundo del análisis, álgebra, probabilidad y estructuras discretas como base de todo el desarrollo computacional.
- **Implementación Efectiva**: Capacidad para traducir modelos y teorías matemáticas en algoritmos eficientes y software robusto.
- **Aplicación Interdisciplinaria**: Habilidad para modelar problemas del mundo real en áreas como finanzas, bioinformática, inteligencia artificial y física computacional.

Esta malla producirá profesionales con:

- Capacidad para desarrollar modelos matemáticos novedosos y diseñar algoritmos para problemas sin solución trivial.
- Competencias para trabajar en equipos multidisciplinarios, actuando como el puente entre la teoría abstracta y la solución práctica.
- Una formación ética que le permita comprender las implicaciones de los modelos algorítmicos en la sociedad, promoviendo la transparencia y la equidad.

Esta propuesta incorpora mecanismos de actualización continua, asegurando su relevancia frente a la rápida evolución de los campos de la matemática aplicada y la computación, y manteniendo un alineamiento con los más altos estándares de acreditación internacional.

Índice general

Agradecimientos xxi

Abreviaturas xxiii

1. Introducción 1
1.1. Definiciones básicas . . . 1
1.2. Campo y mercado ocupacional . . . 1
1.3. Importancia de la carrera en la sociedad . . . 2
1.4. Información institutional . . . 3
1.4.1. Misión . . . 3
1.4.2. Visión . . . 3
1.5. Información de acreditacion . . . 3
1.5.1. Resultados de la carrera *Outcomes* . . . 3
1.5.2. Objetivos educacionales . . . 4
1.5.3. Aprendizaje esperado (*Learning Outcomes*) . . . 4
1.6. Perfiles . . . 4
1.6.1. Perfil del estudiante . . . 4
1.6.2. Perfil del graduado . . . 5
1.6.3. Perfil Profesional . . . 5
1.6.4. Perfil del docente . . . 6
1.7. Grados y Títulos . . . 6
1.8. Recursos para dictado de clases . . . 6
1.9. Organización del documento . . . 7

2. Cuerpo del conocimiento de Matemática Computacional 9
2.1. Análisis Numérico y Científico (NSA) . . . 10
2.1.1. NSA/Aritmética Computacional y Análisis de Error . . . 10
2.1.2. NSA/Álgebra Lineal Numérica . . . 10
2.1.3. NSA/Búsqueda de Raíces y Optimización Numérica Básica . . . 11
2.1.4. NSA/Interpolación y Aproximación de Funciones . . . 11
2.1.5. NSA/Diferenciación e Integración Numérica . . . 11
2.1.6. NSA/Métodos Numéricos para Ecuaciones Diferenciales Ordinarias (EDOs) . . . 12
2.2. Modelado Matemático y Simulación (MMS) . . . 12
2.2.1. MMS/Formulación de Modelos Matemáticos . . . 12
2.2.2. MMS/Simulación de Sistemas Discretos . . . 13
2.2.3. MMS/Simulación de Sistemas Continuos . . . 13
2.2.4. MMS/Análisis de Sensibilidad y Robustez . . . 13
2.2.5. MMS/Validación y Verificación de Modelos . . . 13
2.3. Probabilidad y Computación Estadística (PSC) . . . 14
2.3.1. PSC/Probabilidad Avanzada y Procesos Estocásticos . . . 14
2.3.2. PSC/Inferencia Estadística Computacional . . . 14
2.3.3. PSC/Diseño de Experimentos Computacionales . . . 15
2.3.4. PSC/Métodos de Remuestreo y Validación . . . 15
2.4. Estructuras de Datos y Algoritmos (DSA) . . . 15

2.4.1. DSA/Complejidad Algorítmica y Análisis Asintótico . . . 16
2.4.2. DSA/Estructuras de Datos Fundamentales y Avanzadas . . . 16
2.4.3. DSA/Algoritmos de Búsqueda y Ordenamiento . . . 16
2.4.4. DSA/Algoritmos de Grafos y Optimización Combinatoria . . . 16
2.4.5. DSA/Algoritmos Paralelos y Distribuidos . . . 17
2.5. Computación de Alto Rendimiento (HPC) . . . 17
2.5.1. HPC/Arquitecturas de Computadoras Paralelas . . . 17
2.5.2. HPC/Programación Paralela de Memoria Compartida . . . 18
2.5.3. HPC/Programación Paralela de Memoria Distribuida . . . 18
2.5.4. HPC/Programación de Aceleradores . . . 18
2.5.5. HPC/Métodos Numéricos Paralelos . . . 18
2.6. Optimización y Aprendizaje Automático (OML) . . . 19
2.6.1. OML/Optimización Convexa y No Convexa . . . 19
2.6.2. OML/Optimización con Restricciones . . . 19
2.6.3. OML/Optimización para Aprendizaje Automático . . . 20
2.6.4. OML/Optimización a Gran Escala y Distribuida . . . 20
2.6.5. OML/Metaheurísticas y Optimización Bioinspirada . . . 20
2.7. Matemáticas para Ciencia de Datos y Aprendizaje Automático (DSM) . . . 20
2.7.1. DSM/Fundamentos del Aprendizaje Estadístico . . . 21
2.7.2. DSM/Métodos de Aprendizaje Supervisado . . . 21
2.7.3. DSM/Métodos de Aprendizaje No Supervisado . . . 21
2.7.4. DSM/Evaluación y Selección de Modelos . . . 22
2.8. Matemáticas para Inteligencia Artificial (AIM) . . . 22
2.8.1. AIM/Teoría del Aprendizaje Profundo . . . 22
2.8.2. AIM/Arquitecturas Neuronales Avanzadas . . . 22
2.8.3. AIM/Aprendizaje por Refuerzo . . . 23
2.8.4. AIM/Procesamiento de Lenguaje Natural con Transformers . . . 23
2.8.5. AIM/Modelos Generativos y GANs . . . 23
2.9. Computación Simbólica y Discreta (SDC) . . . 24
2.9.1. SDC/Teoría de Grupos y Algoritmos Algebraicos . . . 24
2.9.2. SDC/Teoría de Números Computacional y Criptografía . . . 24
2.9.3. SDC/Sistemas de Computación Simbólica . . . 24
2.9.4. SDC/Lógica Computacional y Verificación Formal . . . 25
2.10. Señales, Imágenes y Visualización (SIV) . . . 25
2.10.1. SIV/Análisis de Fourier y Wavelets . . . 25
2.10.2. SIV/Procesamiento Digital de Imágenes y Visión Computacional . . . 26
2.10.3. SIV/Visualización Científica y Computación Visual . . . 26
2.10.4. SIV/Gráficos por Computadora y Geometría Computacional . . . 26
2.11. Computación Cuántica y Algoritmos (QCA) . . . 26
2.11.1. QCA/Fundamentos de Mecánica Cuántica para Computación . . . 27
2.11.2. QCA/Qubits y Circuitos Cuánticos . . . 27
2.11.3. QCA/Algoritmos Cuánticos . . . 27
2.11.4. QCA/Criptografía Post-Cuántica y Simulación Cuántica . . . 28
2.12. Ciberseguridad y Privacidad (CSP) . . . 28
2.12.1. CSP/Criptografía Simétrica . . . 28
2.12.2. CSP/Criptografía Asimétrica . . . 28
2.12.3. CSP/Computación con Preservación de la Privacidad . . . 29
2.12.4. CSP/Teoría de la Información y Compresión de Datos . . . 29
2.13. Matemáticas Computacionales para Dominios Específicos (CDM) . . . 29
2.13.1. CDM/Bioinformática y Biología Computacional . . . 30
2.13.2. CDM/Finanzas Computacionales y Ciencia del Riesgo . . . 30
2.13.3. CDM/Física Computacional y Química Cuántica . . . 30
2.13.4. CDM/Ciencia de Materiales Computacional . . . 31
2.13.5. CDM/Modelado Climático y Sistemas Complejos . . . 31
2.14. Gobernanza y Gestión de Datos (DGM) . . . 31
2.14.1. DGM/Ética en IA y Algoritmos Responsables . . . 31

2.14.2. DGM/Gestión de Proyectos de Datos y MLOps 32
2.14.3. DGM/Gobernanza de Datos y Cumplimiento Normativo 32

3. Cuerpo del conocimiento de Ciencia de la Computación–*Computer Science* 33
3.1. Inteligencia Artificial (AI) 34
3.1.1. AI/Introducción: Problemas Fundamentales 35
3.1.2. AI/Búsqueda 36
3.1.3. AI/Representación del Conocimiento y Razonamiento Fundamental 37
3.1.4. AI/Aprendizaje Automático 38
3.1.5. AI/Aplicaciones e Impacto Social 39
3.1.6. AI/Representación Lógica y Razonamiento 40
3.1.7. AI/Representación Probabilística y Razonamiento 41
3.1.8. AI/Planificación 41
3.1.9. AI/Agentes y Sistemas Cognitivos 42
3.1.10. AI/Procesamiento del Lenguaje Natural 43
3.1.11. AI/Robótica 43
3.1.12. AI/Percepción y Visión por Computadora 44
3.2. Fundamentos Algorítmicos (AL) 45
3.2.1. AL/Estructuras de Datos y Algoritmos Fundamentales 45
3.2.2. AL/Estrategias Algorítmicas 47
3.2.3. AL/Análisis de Complejidad 47
3.2.4. AL/Modelos Computacionales y Lenguajes Formales 49
3.2.5. AL/Sociedad, Ética y la Profesión 50
3.3. Arquitectura y Organización (AR) 50
3.3.1. AR/Lógica Digital y Sistemas Digitales 51
3.3.2. AR/Representación de Datos a Nivel de Máquina 52
3.3.3. AR/Organización de Máquina a Nivel de Ensamblador 52
3.3.4. AR/Jerarquía de Memoria 53
3.3.5. AR/Interconexión y Comunicación 53
3.3.6. AR/Organización Funcional 54
3.3.7. AR/Rendimiento y Eficiencia Energética 54
3.3.8. AR/Arquitecturas Heterogéneas 54
3.3.9. AR/Arquitecturas de Procesador Seguras 55
3.3.10. AR/Arquitecturas Cuánticas 55
3.3.11. AR/Problemas de Sostenibilidad 56
3.4. Gestión de Datos (DM) 56
3.4.1. DM/El Rol de los Datos y el Ciclo de Vida de los Datos 58
3.4.2. DM/Conceptos Fundamentales de Sistemas de Bases de Datos 58
3.4.3. DM/Modelado de Datos 59
3.4.4. DM/Bases de Datos Relacionales 59
3.4.5. DM/Construcción de Consultas 60
3.4.6. DM/Procesamiento de Consultas 61
3.4.7. DM/Internos de DBMS 61
3.4.8. DM/Sistemas NoSQL 62
3.4.9. DM/Seguridad y Privacidad de Datos 62
3.4.10. DM/Análisis de Datos 63
3.4.11. DM/Bases de Datos Distribuidas/Computación en la Nube 63
3.4.12. DM/Bases de Datos Semiestructuradas y No Estructuradas 63
3.4.13. DM/Sociedad, Ética y Profesión 64
3.5. Fundamentos de los Lenguajes de Programación (FPL) 64
3.5.1. FPL/Programación Orientada a Objetos 65
3.5.2. FPL/Programación Funcional 66
3.5.3. FPL/Programación Lógica 67
3.5.4. FPL/Scripting de Shell 68
3.5.5. FPL/Programación Dirigida por Eventos y Reactiva 68
3.5.6. FPL/Computación Paralela y Distribuida 68

3.5.7. FPL/Programación Orientada a Aspectos . . . 69
3.5.8. FPL/Sistemas de Tipos . . . 70
3.5.9. FPL/Modelo de Ejecución y Memoria de Sistemas . . . 71
3.5.10. FPL/Traducción y Ejecución de Lenguajes . . . 72
3.5.11. FPL/Abstracción y Representación de Programas . . . 72
3.5.12. FPL/Análisis Sintáctico . . . 73
3.5.13. FPL/Análisis Semántico del Compilador . . . 74
3.5.14. FPL/Análisis de Programas y Analizadores . . . 74
3.5.15. FPL/Generación de Código . . . 75
3.5.16. FPL/Comportamiento en Tiempo de Ejecución y Sistemas . . . 75
3.5.17. FPL/Construcciones de Programación Avanzadas . . . 76
3.5.18. FPL/Pragmática del Lenguaje . . . 77
3.5.19. FPL/Semántica Formal . . . 77
3.5.20. FPL/Metodologías de Desarrollo Formal . . . 77
3.5.21. FPL/Principios de Diseño de Lenguajes de Programación . . . 78
3.5.22. FPL/Sociedad, Ética y Profesión . . . 78
3.6. Gráficos y Técnicas Interactivas (GIT) . . . 79
3.6.1. GIT/Conceptos Fundamentales de Gráficos y Técnicas Interactivas . . . 79
3.6.2. GIT/Visualización . . . 80
3.6.3. GIT/Renderizado Aplicado y Técnicas . . . 80
3.6.4. GIT/Modelado Geométrico . . . 80
3.6.5. GIT/Sombreado y Renderizado Avanzado . . . 81
3.6.6. GIT/Animación por Computadora . . . 81
3.6.7. GIT/Simulación . . . 81
3.6.8. GIT/Inmersión . . . 81
3.6.9. GIT/Interacción . . . 82
3.6.10. GIT/Procesamiento de Imágenes . . . 82
3.6.11. GIT/Computación Tangible/Física . . . 82
3.6.12. GIT/Sociedad, Ética y Profesión . . . 83
3.7. Interacción Humano-Computadora (HCI) . . . 83
3.7.1. HCI/Comprender al Usuario: Objetivos individuales e interacciones con otros . . . 84
3.7.2. HCI/Responsabilidad y Rendición de Cuentas en el Diseño . . . 84
3.7.3. HCI/Accesibilidad y Diseño Inclusivo . . . 84
3.7.4. HCI/Evaluación del Diseño . . . 85
3.7.5. HCI/Diseño de Sistemas . . . 85
3.7.6. HCI/Sociedad, Ética y Profesión . . . 86
3.8. Fundamentos Matemáticos y Estadísticos (MSF) . . . 86
3.8.1. MSF/Matemáticas Discretas . . . 86
3.8.2. MSF/Probabilidad . . . 87
3.8.3. MSF/Estadística . . . 87
3.8.4. MSF/Álgebra Lineal . . . 88
3.8.5. MSF/Cálculo . . . 89
3.9. Redes y Comunicaciones (NC) . . . 89
3.9.1. NC/Fundamentos de Redes y Comunicaciones . . . 90
3.9.2. NC/Aplicaciones en Red . . . 90
3.9.3. NC/Soporte de Fiabilidad . . . 91
3.9.4. NC/Enrutamiento y Reenvío . . . 91
3.9.5. NC/Comunicación de Un Solo Salto . . . 91
3.9.6. NC/Seguridad de Red . . . 92
3.9.7. NC/Soporte de Movilidad . . . 92
3.9.8. NC/Temas Emergentes . . . 92
3.10. Sistemas Operativos (OS) . . . 93
3.10.1. OS/Rol y Propósito de los Sistemas Operativos . . . 93
3.10.2. OS/Principios del Sistema Operativo . . . 94
3.10.3. OS/Concurrencia . . . 95
3.10.4. OS/Protección y Seguridad . . . 95

3.10.5. OS/Planificación . . . 96
3.10.6. OS/Modelo de Proceso . . . 96
3.10.7. OS/Gestión de Memoria . . . 97
3.10.8. OS/Gestión de Dispositivos . . . 97
3.10.9. OS/API de Sistemas de Archivos e Implementación . . . 98
3.10.10. OS/Sistemas de Archivos Avanzados . . . 98
3.10.11. OS/Virtualización . . . 99
3.10.12. OS/Sistemas en Tiempo Real y Embebidos . . . 100
3.10.13. OS/Tolerancia a fallos . . . 100
3.10.14. OS/Sociedad, Ética y Profesión . . . 101
3.11. Computación Paralela y Distribuida (PDC) . . . 101
3.11.1. PDC/Programas . . . 102
3.11.2. PDC/Comunicación . . . 103
3.11.3. PDC/Coordinación . . . 104
3.11.4. PDC/Evaluación . . . 105
3.11.5. PDC/Algoritmos . . . 106
3.12. Fundamentos de Desarrollo de Software (SDF) . . . 108
3.12.1. SDF/Conceptos y Prácticas Fundamentales de Programación . . . 108
3.12.2. SDF/Estructuras de Datos Fundamentales . . . 109
3.12.3. SDF/Algoritmos . . . 110
3.12.4. SDF/Prácticas de Desarrollo de Software . . . 110
3.12.5. SDF/Sociedad, Ética y la Profesión . . . 110
3.13. Ingeniería de Software (SE) . . . 111
3.13.1. SE/Trabajo en Equipo . . . 112
3.13.2. SE/Herramientas y Entornos . . . 113
3.13.3. SE/Requisitos del Producto . . . 113
3.13.4. SE/Diseño de Software . . . 114
3.13.5. SE/Construcción de Software . . . 115
3.13.6. SE/Verificación y Validación de Software . . . 117
3.13.7. SE/Refactorización y Evolución del Código . . . 118
3.13.8. SE/Confiabilidad del Software . . . 119
3.13.9. SE/Métodos Formales . . . 120
3.14. Seguridad (SEC) . . . 120
3.14.1. SEC/Fundamentos de Seguridad . . . 121
3.14.2. SEC/Sociedad, Ética y la Profesión . . . 122
3.14.3. SEC/Codificación Segura . . . 123
3.14.4. SEC/Criptografía . . . 124
3.14.5. SEC/Análisis, Diseño e Ingeniería de la Seguridad . . . 125
3.14.6. SEC/Informática Forense . . . 126
3.14.7. SEC/Gobierno de la Seguridad . . . 127
3.15. Sociedad, Ética y la Profesión (SEP) . . . 128
3.15.1. SEP/Contexto Social . . . 128
3.15.2. SEP/Métodos para el Análisis Ético . . . 129
3.15.3. SEP/Ética Profesional . . . 130
3.15.4. SEP/Propiedad Intelectual . . . 131
3.15.5. SEP/Privacidad y Libertades Civiles . . . 133
3.15.6. SEP/Comunicación . . . 133
3.15.7. SEP/Sostenibilidad . . . 134
3.15.8. SEP/Historia de la Informática . . . 135
3.15.9. SEP/Economías de la Informática . . . 136
3.15.10. SEP/Políticas de Seguridad, Leyes y Delitos Informáticos . . . 137
3.15.11. SEP/Diversidad, Equidad, Inclusión y Accesibilidad . . . 138
3.16. Fundamentos de Sistemas (SF) . . . 139
3.16.1. SF/Visión General de los Sistemas Informáticos . . . 140
3.16.2. SF/Conceptos Básicos . . . 140

3.16.3. SF/Gestión de Recursos . . . 141
3.16.4. SF/Rendimiento del Sistema . . . 142
3.16.5. SF/Evaluación de Rendimiento . . . 143
3.16.6. SF/Confiabilidad del Sistema . . . 143
3.16.7. SF/Seguridad del Sistema . . . 144
3.16.8. SF/Diseño del Sistema . . . 144
3.16.9. SF/Sociedad, Ética y la Profesión . . . 145
3.17. Desarrollo de Plataformas Especializadas (SPD) . . . 145
3.17.1. SPD/Aspectos Comunes, Preocupaciones Compartidas . . . 146
3.17.2. SPD/Plataformas Web . . . 146
3.17.3. SPD/Plataformas Móviles . . . 147
3.17.4. SPD/Plataformas Robóticas . . . 147
3.17.5. SPD/Plataformas Embebidas . . . 148
3.17.6. SPD/Plataformas de Juegos . . . 149
3.17.7. SPD/Plataformas de Computación Interactiva . . . 150
3.17.8. SPD/SPD-SEP/Móvil . . . 151
3.17.9. SPD/SPD-SEP Web . . . 152
3.17.10. SPD/SPD-SEP/Juegos . . . 152
3.17.11. SPD/SPD-SEP/Robótica . . . 153
3.17.12. SPD/SPD-SEP/Interactiva . . . 154

4. Cuerpo del conocimiento de Matemática 155
4.1. Cálculo y Análisis (CAN) . . . 156
4.1.1. CAN/Cálculo Diferencial e Integral . . . 156
4.1.2. CAN/Cálculo Multivariable y Vectorial . . . 156
4.1.3. CAN/Análisis Real e Integración de Lebesgue . . . 157
4.1.4. CAN/Análisis Complejo y Mapeo Conforme . . . 157
4.1.5. CAN/Análisis Funcional y Espacios de Banach/Hilbert . . . 157
4.1.6. CAN/Análisis Armónico y Series de Fourier . . . 157
4.1.7. CAN/Cálculo de Variaciones . . . 158
4.1.8. CAN/Teoría de la Medida . . . 158
4.2. Álgebra y Teoría de Números (ANT) . . . 158
4.2.1. ANT/Álgebra Lineal Elemental y Avanzada . . . 158
4.2.2. ANT/Teoría de Grupos y Simetría . . . 159
4.2.3. ANT/Anillos, Cuerpos y Teoría de Galois . . . 159
4.2.4. ANT/Teoría de Módulos y Álgebra Homológica . . . 159
4.2.5. ANT/Teoría de Números Elemental y Analítica . . . 159
4.2.6. ANT/Teoría de Números Algebraica . . . 160
4.2.7. ANT/Teoría de Representación . . . 160
4.2.8. ANT/Criptografía y Teoría de Codificación . . . 160
4.3. Geometría y Topología (GTO) . . . 160
4.3.1. GTO/Geometría Euclidiana y Proyectiva . . . 161
4.3.2. GTO/Topología General (de Conjuntos de Puntos) . . . 161
4.3.3. GTO/Geometría Diferencial de Curvas y Superficies . . . 161
4.3.4. GTO/Geometría Riemanniana . . . 161
4.3.5. GTO/Topología Algebraica . . . 161
4.3.6. GTO/Geometría Simpléctica . . . 162
4.3.7. GTO/Teoría Geométrica de Grupos . . . 162
4.3.8. GTO/Topología Diferencial y Variedades . . . 162
4.4. Matemáticas Discretas y Combinatoria (DMC) . . . 162
4.4.1. DMC/Combinatoria Enumerativa . . . 163
4.4.2. DMC/Teoría de Grafos y Análisis de Redes . . . 163
4.4.3. DMC/Combinatoria Extremal y Probabilística . . . 163
4.4.4. DMC/Optimización Discreta y Matroides . . . 164
4.4.5. DMC/Teoría de Diseños . . . 164
4.4.6. DMC/Lógica, Teoría de Conjuntos y Teoría de Modelos . . . 164

4.5. Probabilidad y Estadística (PST) . . . 164
4.5.1. PST/Axiomas de Probabilidad y Variables Aleatorias . . . 165
4.5.2. PST/Teoremas de Límite . . . 165
4.5.3. PST/Procesos Estocásticos y Cadenas de Markov . . . 165
4.5.4. PST/Estadística Matemática y Teoría de la Estimación . . . 166
4.5.5. PST/Pruebas de Hipótesis y Modelos Lineales . . . 166
4.5.6. PST/Inferencia Bayesiana . . . 166
4.5.7. PST/Estadística de Alta Dimensión . . . 166
4.5.8. PST/Análisis de Series Temporales . . . 166
4.5.9. PST/Estadística No Paramétrica . . . 167
4.6. Análisis Numérico y Científico (NSA) . . . 167
4.6.1. NSA/Análisis de Errores y Aritmética de Punto Flotante . . . 167
4.6.2. NSA/Álgebra Lineal Numérica . . . 167
4.6.3. NSA/Teoría de Interpolación y Aproximación . . . 168
4.6.4. NSA/Integración y Diferenciación Numérica . . . 168
4.6.5. NSA/Métodos Numéricos para EDOs y EDPs . . . 168
4.6.6. NSA/Métodos de Elementos Finitos y Volúmenes Finitos . . . 168
4.6.7. NSA/Algoritmos de Optimización . . . 169
4.6.8. NSA/Computación Paralela para Matemáticas . . . 169
4.7. Modelado Matemático y Simulación (MMS) . . . 169
4.7.1. MMS/Ecuaciones Diferenciales Ordinarias (EDO) . . . 169
4.7.2. MMS/Ecuaciones Diferenciales Parciales (EDP) . . . 170
4.7.3. MMS/Sistemas Dinámicos y Teoría de Bifurcaciones . . . 170
4.7.4. MMS/Métodos de Perturbación y Análisis Asintótico . . . 170
4.7.5. MMS/Dinámica de Fluidos y Mecánica de Medios Continuos . . . 171
4.7.6. MMS/Teoría de Control y Control Óptimo . . . 171
4.7.7. MMS/Ecuaciones Diferenciales Estocásticas (EDE) . . . 171
4.7.8. MMS/Modelado Basado en Agentes . . . 171
4.8. Fundamentos Matemáticos de la Ciencia de Datos (FDS) . . . 171
4.8.1. FDS/Teoría del Aprendizaje Estadístico . . . 172
4.8.2. FDS/Optimización para el Aprendizaje Automático . . . 172
4.8.3. FDS/Análisis de Datos Topológicos (TDA) . . . 172
4.8.4. FDS/Reducción de Dimensionalidad . . . 172
4.8.5. FDS/Métodos de Núcleo (Kernel) y SVM . . . 173
4.8.6. FDS/Teoría de la Información y Entropía . . . 173
4.8.7. FDS/Teoría Matemática del Aprendizaje Profundo . . . 173
4.9. Geometría y Topología Computacional (CGT) . . . 173
4.9.1. CGT/Diagramas de Voronoi y Triangulaciones de Delaunay . . . 174
4.9.2. CGT/Algoritmos de Envoltura Convexa . . . 174
4.9.3. CGT/Topología Digital . . . 174
4.9.4. CGT/Homología Aplicada y Números de Betti . . . 174
4.9.5. CGT/Generación y Procesamiento de Mallas . . . 174
4.10. Información y Computación Cuántica (QIC) . . . 175
4.10.1. QIC/Fundamentos de Mecánica Cuántica para Matemáticos . . . 175
4.10.2. QIC/Lógica y Circuitos Cuánticos . . . 175
4.10.3. QIC/Algoritmos Cuánticos . . . 175
4.10.4. QIC/Entrelazamiento y Desigualdades de Bell . . . 176
4.10.5. QIC/Corrección de Errores Cuánticos . . . 176
4.10.6. QIC/Redes de Tensores y Sistemas de Muchos Cuerpos . . . 176
4.11. Biología Matemática y Ciencias de la Vida (MBL) . . . 176
4.11.1. MBL/Dinámica de Poblaciones y Ecología . . . 177
4.11.2. MBL/Epidemiología Matemática . . . 177
4.11.3. MBL/Biología de Sistemas y Redes Metabólicas . . . 177
4.11.4. MBL/Bioinformática y Alineamiento de Secuencias . . . 177
4.11.5. MBL/Neurociencia Matemática . . . 178
4.12. Matemáticas Financieras y Actuariales (FAM) . . . 178

4.12.1. FAM/Teoría del Interés y Anualidades . . . 178
4.12.2. FAM/Arbitraje y Completitud del Mercado . . . 178
4.12.3. FAM/Teoría de Black-Scholes-Merton . . . 178
4.12.4. FAM/Medidas de Riesgo . . . 179
4.12.5. FAM/Modelos Actuariales y Contingencias de Vida . . . 179
4.12.6. FAM/Finanzas Computacionales y Monte Carlo . . . 179

5. Cuerpo del conocimiento de Física 181
5.1. Mecánica Clásica (CME) . . . 182
5.1.1. CME/Cinemática y Dinámica de Partículas . . . 182
5.1.2. CME/Trabajo, Energía y Fuerzas Conservativas . . . 183
5.1.3. CME/Sistemas de Partículas y Momento Lineal . . . 184
5.1.4. CME/Movimiento Rotacional y Momento Angular . . . 185
5.1.5. CME/Dinámica del Cuerpo Rígido en 3D . . . 185
5.1.6. CME/Marcos de Referencia no Inerciales y Fuerzas Ficticias . . . 186
5.1.7. CME/Mecánica Lagrangiana y Hamiltoniana . . . 186
5.1.8. CME/Movimiento de Fuerzas Centrales y Mecánica Orbital . . . 187
5.1.9. CME/Teoría de Pequeñas Oscilaciones y Osciladores Acoplados . . . 188
5.1.10. CME/Fundamentos de la Mecánica de Medios Continuos . . . 188
5.2. Electromagnetismo (ELM) . . . 189
5.2.1. ELM/Electrostática: Campo Eléctrico y Ley de Gauss . . . 189
5.2.2. ELM/Potencial Eléctrico y Ecuaciones de Poisson/Laplace . . . 190
5.2.3. ELM/Energía Electrostática y Dieléctricos . . . 190
5.2.4. ELM/Magnetostática: Leyes de Biot-Savart y Ampère . . . 191
5.2.5. ELM/Propiedades Magnéticas de la Materia . . . 191
5.2.6. ELM/Electrodinámica: Ley de Faraday e Inducción . . . 192
5.2.7. ELM/Ecuaciones de Maxwell y Transformaciones de Norma . . . 192
5.2.8. ELM/Ondas Electromagnéticas y Propagación en Medios . . . 193
5.2.9. ELM/Teoría de la Radiación y Radiación Dipolar . . . 193
5.2.10. ELM/Relatividad Especial en Electromagnetismo . . . 194
5.3. Termodinámica y Mecánica Estadística (TSM) . . . 194
5.3.1. TSM/Leyes de la Termodinámica y Funciones de Estado . . . 194
5.3.2. TSM/Teoría Cinética de los Gases . . . 195
5.3.3. TSM/Máquinas Térmicas, Entropía y la Segunda Ley . . . 195
5.3.4. TSM/Potenciales Termodinámicos y Relaciones de Maxwell . . . 196
5.3.5. TSM/Transiciones de Fase y Fenómenos Críticos . . . 196
5.3.6. TSM/Colectivos Microcanónico, Canónico y Gran Canónico . . . 196
5.3.7. TSM/Estadística Cuántica: Fermi-Dirac y Bose-Einstein . . . 197
5.3.8. TSM/Radiación de Cuerpo Negro y Fonones . . . 197
5.3.9. TSM/Mecánica Estadística de No-Equilibrio . . . 197
5.3.10. TSM/Aplicaciones a Sistemas Biológicos y Complejos . . . 198
5.4. Mecánica Cuántica (QME) . . . 198
5.4.1. QME/Orígenes de la Teoría Cuántica y Dualidad Onda-Partícula . . . 199
5.4.2. QME/La Ecuación de Schrödinger y Funciones de Onda . . . 199
5.4.3. QME/Formalismo: Espacio de Hilbert y Notación de Dirac . . . 199
5.4.4. QME/Potenciales Unidimensionales (Pozos, Barreras, Tunelamiento) . . . 200
5.4.5. QME/El Oscilador Armónico Cuántico . . . 200
5.4.6. QME/Momento Angular y Física del Espín . . . 200
5.4.7. QME/El Átomo de Hidrógeno y Potenciales Centrales . . . 201
5.4.8. QME/Teoría de Perturbaciones Independiente y Dependiente del Tiempo . . . 201
5.4.9. QME/Partículas Idénticas y Simetría de Intercambio . . . 202
5.4.10. QME/Teoría de la Dispersión y Secciones Eficaces . . . 202
5.5. Relatividad y Cosmología (RCO) . . . 202
5.5.1. RCO/Relatividad Especial: Transformaciones de Lorentz y Cuadrivectores . . . 203
5.5.2. RCO/Dinámica Relativista y Energía-Momento . . . 203
5.5.3. RCO/Fundamentos de la Relatividad General: Principio de Equivalencia . . . 203

5.5.4. RCO/Curvatura del Espacio-Tiempo y Ecuaciones de Campo de Einstein . . . 204
5.5.5. RCO/Geometría de Schwarzschild y Agujeros Negros . . . 204
5.5.6. RCO/Ondas Gravitacionales . . . 204
5.5.7. RCO/Teoría del Big Bang y Ecuaciones de Friedmann . . . 205
5.5.8. RCO/Fondo Cósmico de Microondas (CMB) . . . 205
5.5.9. RCO/Materia Oscura y Energía Oscura . . . 205
5.5.10. RCO/Inflación del Universo Temprano y Nucleosíntesis . . . 205
5.6. Física de la Materia Condensada (CMP) . . . 206
5.6.1. CMP/Estructuras Cristalinas y Redes Recíprocas . . . 206
5.6.2. CMP/Difracción de Rayos X y Dinámica de Red . . . 206
5.6.3. CMP/Modelo de Electrones Libres y Teoría de Bandas de Sólidos . . . 207
5.6.4. CMP/Semiconductores, Aislantes y Metales . . . 207
5.6.5. CMP/Propiedades Magnéticas: Para-, Dia- y Ferromagnetismo . . . 207
5.6.6. CMP/Superconductividad: Teoría BCS y Alta Tc . . . 208
5.6.7. CMP/Física de la Materia Blanda y Polímeros . . . 208
5.6.8. CMP/Nanoestructuras y Sistemas de Baja Dimensionalidad . . . 208
5.6.9. CMP/Espintrónica y Aislantes Topológicos . . . 209
5.6.10. CMP/Sistemas de Electrones Fuertemente Correlacionados . . . 209
5.7. Física Nuclear y de Partículas (NPP) . . . 209
5.7.1. NPP/Estructura Nuclear, Energía de Enlace y Modelo de Capas . . . 210
5.7.2. NPP/Radiactividad y Desintegraciones Nucleares (Alfa, Beta, Gamma) . . . 210
5.7.3. NPP/Reacciones Nucleares, Fisión y Fusión . . . 211
5.7.4. NPP/Aceleradores y Detectores de Partículas . . . 211
5.7.5. NPP/El Modelo Estándar de la Física de Partículas . . . 211
5.7.6. NPP/Quarks, Leptones y Bosones de Norma . . . 212
5.7.7. NPP/Simetrías y Leyes de Conservación (CPT) . . . 212
5.7.8. NPP/Interacciones Fundamentales y Diagramas de Feynman . . . 212
5.7.9. NPP/Mecanismo de Higgs y más allá del Modelo Estándar . . . 213
5.7.10. NPP/Física de Neutrinos y Oscilación . . . 213
5.8. Información y Computación Cuántica (QIC) . . . 213
5.8.1. QIC/Qubits, Superposición y Entrelazamiento . . . 214
5.8.2. QIC/Comuertas y Circuitos Lógicos Cuánticos . . . 214
5.8.3. QIC/Algoritmos Cuánticos (Shor, Grover) . . . 215
5.8.4. QIC/Teletransportación y Criptografía Cuántica . . . 215
5.8.5. QIC/Decoherencia y Corrección de Errores Cuánticos . . . 215
5.8.6. QIC/Realizaciones Físicas de Computadoras Cuánticas . . . 216
5.8.7. QIC/Sensores Cuánticos y Metrología . . . 216
5.8.8. QIC/Redes Cuánticas y Comunicación . . . 216
5.9. Óptica y Fotónica (OPH) . . . 217
5.9.1. OPH/Óptica Geométrica e Instrumentos Ópticos . . . 217
5.9.2. OPH/Óptica Ondulatoria: Interferencia y Difracción . . . 218
5.9.3. OPH/Polarización y Óptica de Cristales . . . 218
5.9.4. OPH/Física de Láseres: Principios y Aplicaciones . . . 218
5.9.5. OPH/Óptica No Lineal y Conversión de Frecuencia . . . 219
5.9.6. OPH/Fibra Óptica y Cristales Fotónicos . . . 219
5.9.7. OPH/Óptica Ultrarrápida y Espectroscopía . . . 219
5.9.8. OPH/Nano-óptica y Plasmónica . . . 220
5.10. Física Computacional y Basada en Datos (CDP) . . . 220
5.10.1. CDP/Métodos Numéricos para Ecuaciones Diferenciales en Física . . . 221
5.10.2. CDP/Métodos y Simulaciones de Monte Carlo . . . 221
5.10.3. CDP/Computación de Alto Rendimiento (HPC) para Simulaciones a Gran Escala 222
5.10.4. CDP/Aprendizaje Automático para el Descubrimiento Físico . . . 222
5.10.5. CDP/Análisis de Datos y Cuantificación de la Incertidumbre . . . 222
5.10.6. CDP/Modelado de Sistemas Adaptativos Complejos . . . 223
5.10.7. CDP/Redes Neuronales para Sistemas Cuánticos . . . 223

5.10.8. CDP/Redes Neuronales Informadas por la Física (PINNs) . . . 224
5.11. Biofísica y Física Médica (BMP) . . . 224
5.11.1. BMP/Física de las Macromoléculas Biológicas . . . 224
5.11.2. BMP/Biomecánica y Fuerzas Celulares . . . 225
5.11.3. BMP/Física Neuronal y Procesamiento de Información en el Cerebro . . . 225
5.11.4. BMP/Física de Imágenes (MRI, CT, PET, Ultrasonido) . . . 225
5.11.5. BMP/Radioterapia y Dosimetría . . . 226
5.11.6. BMP/Mecánica Estadística de Redes Biológicas . . . 226
5.11.7. BMP/Microfluídica y Lab-on-a-Chip . . . 227
5.12. Astrofísica y Ciencias del Espacio (ASS) . . . 227
5.12.1. ASS/Estructura y Evolución Estelar . . . 227
5.12.2. ASS/Ciencia Planetaria y Exoplanetas . . . 228
5.12.3. ASS/Dinámica y Estructura Galáctica . . . 228
5.12.4. ASS/Astrofísica de Altas Energías (Rayos X, Rayos Gamma) . . . 228
5.12.5. ASS/Técnicas Observacionales y Telescopios . . . 229
5.12.6. ASS/Física Solar y Clima Espacial . . . 229
5.12.7. ASS/Astronomía de Multimensajeros (Ondas Gravitacionales + Neutrinos) . . . 229

6. Plan de estudios **231**
6.1. Codificación de los cursos . . . 231
6.2. Estructura Curricular . . . 232
6.3. Distribución de cursos en la carrera . . . 237

7. Equivalencias con otros planes curriculares **239**
7.1. Equivalencia del Plan 2018 al Plan2026 . . . 239
7.2. Equivalencia del Plan Plan2026 al 2018 . . . 243

8. Laboratorios **247**

9. Sumillas **253**
9.1. Primer Semestre . . . 253
9.1.1. CS111. Introducción a la Programación . . . 253
9.1.2. MA100. Matemática Básica . . . 253
9.1.3. FG101. Lenguaje . . . 253
9.1.4. FG105. Género y Sociedad . . . 253
9.1.5. FG109. Ambiente y Desarrollo Sostenible . . . 253
9.1.6. FG112. Métodos de gestión personal universitaria . . . 254
9.1.7. FG140. Fundamentos de Investigación Científica . . . 254
9.2. Segundo Semestre . . . 254
9.2.1. CS100. Introducción a la Ciencia de la Computación . . . 254
9.2.2. CS112. Programación Orientada a Objetos I . . . 254
9.2.3. AI161. IA Aplicada . . . 254
9.2.4. MA111. Cálculo I . . . 255
9.2.5. FI101. Física General . . . 255
9.3. Tercer Semestre . . . 255
9.3.1. CS113. Programación Orientada a Objetos II . . . 255
9.3.2. CS1D1. Estructuras Discretas I . . . 255
9.3.3. CS2B1. Desarrollo Basado en Plataformas . . . 255
9.3.4. MA112. Cálculo II . . . 255
9.3.5. MA121. Algebra Lineal I . . . 255
9.3.6. ID101. Inglés I . . . 256
9.4. Cuarto Semestre . . . 256
9.4.1. CS1D2. Estructuras Discretas II . . . 256
9.4.2. CS210. Algoritmos y Estructuras de Datos . . . 256
9.4.3. CS271. Bases de Datos I . . . 256
9.4.4. MA122. Algebra Lineal II . . . 256
9.4.5. MA210. Cálculo III . . . 256

9.4.6. ID102. Inglés II . . . 256
9.5. Quinto Semestre . . . 257
9.5.1. CS251. Computación Gráfica . . . 257
9.5.2. MA211. Cálculo IV . . . 257
9.5.3. MA221. Análisis Numérico I . . . 257
9.5.4. MA222. Análisis Real I . . . 257
9.5.5. MA251. Introducción a la Estadística . . . 257
9.5.6. MA281. Teoría de números . . . 257
9.5.7. ID201. Inglés III . . . 257
9.6. Sexto Semestre . . . 258
9.6.1. MA212. Ecuaciones Diferenciales Ordinarias . . . 258
9.6.2. MA325. Análisis Numérico II . . . 258
9.6.3. MA371. Análisis funcional I . . . 258
9.6.4. MA351. Estadística y Probabilidades . . . 258
9.6.5. ID202. Inglés IV . . . 258
9.7. Séptimo Semestre . . . 258
9.7.1. CS261. Inteligencia Artificial . . . 258
9.7.2. CS3P1. Computación Paralela y Distribuída . . . 258
9.7.3. MA213. Ecuaciones Diferenciales Parciales . . . 259
9.7.4. MA326. Análisis Numérico III . . . 259
9.7.5. MA361. Modelado Matemático . . . 259
9.7.6. ID203. Inglés V . . . 259
9.8. Octavo Semestre . . . 259
9.8.1. AI263. Introducción al Aprendizaje de Máquina . . . 259
9.8.2. MA401. Proyecto de final de carrera I . . . 259
9.8.3. CS2S1. Sistemas Operativos . . . 260
9.8.4. CS3I1. Seguridad en Computación . . . 260
9.8.5. QC251. Mecánica Cuántica . . . 260
9.8.6. CS253. Análisis exploratorio de datos . . . 260
9.8.7. CS370. Big Data y Analítica de Datos . . . 260
9.8.8. STA252. Estadística Inferencial . . . 260
9.8.9. MA327. Análisis Numérico IV . . . 260
9.8.10. MA352. Procesos estocásticos . . . 261
9.8.11. MA372. Análisis funcional II . . . 261
9.9. Noveno Semestre . . . 261
9.9.1. AI264. Aprendizaje Profundo . . . 261
9.9.2. MA402. Proyecto de final de carrera II . . . 261
9.9.3. CS231. Redes y Comunicación . . . 261
9.9.4. MA381. Estructuras Algebraicas . . . 261
9.9.5. AI262. Procesamiento de Lenguaje Natural . . . 261
9.9.6. CS273. Análisis de series temporales . . . 262
9.9.7. MA341. Optimización I . . . 262
9.9.8. MA355. Teoría de Aprendizaje Estadístico . . . 262
9.10. Décimo Semestre . . . 262
9.10.1. CS400. Prácticas Pre-profesionales . . . 262
9.10.2. AI268. Visión Computacional . . . 262
9.10.3. AI367. Tópicos en Inteligencia Artificial . . . 262
9.10.4. MA403. Proyecto de final de carrera III . . . 263
9.10.5. QC381. Introduction a la Computación Cuántica . . . 263
9.10.6. CS254. Visualización de Datos Avanzada . . . 263

Índice de figuras

6.1. Esquema de codificación para los cursos. 231
6.2. Créditos por área por semestre 237
6.3. Distribución de cursos por áreas considerando creditaje. 238
6.4. Distribución de créditos por niveles de cursos. 238

Índice de cuadros

6.1. Distribución de cursos por áreas 237

Agradecimientos

Además de los autores directos de este documento, también deseamos dejar manifiesto de nuestro agradecimiento a otros colegas de diversas universidades del país y del mundo que gentilmente han aportado parte de su tiempo a darnos sus sugerencias. Entre ellos debemos mencionar a:

- Todo de equipo de ACM/IEEE-CS Computing Curricula (CC2020, CS2023 y CS2013).
- Los docentes que participaron en las Jornadas Curriculares (JC).
- A todo el personal administrativo por la organización las JC.
- A los estudiantes que participaron en las JC.
- A los colegas de otras universidades que nos han dado sus sugerencias.
- Sociedad Peruana de Computación por apoyarnos y facilitarnos su documento de referencia.

Todo este equipo de trabajo asumió como premisa que el centro de nuestro esfuerzo, es la formación académica y humana de los estudiantes para formar agentes de cambio positivo y disruptivo en la sociedad.

A todos ellos deseamos agradecerles por su aporte que ha permitido generar este documento, único en su género en nuestro país, que servirá para sentar las bases de una carrera más sólida en esta fantástica área que nos ha tocado estudiar y de la cual nos sentimos orgullosos de formar parte.

Abreviaturas

.

Capítulo 1

Introducción

La computación ha sufrido un desarrollo impresionante en las últimas décadas, convirtiéndose en el motor del desarrollo científico, tecnológico, industrial, social, económico y cultural, transformando de manera significativa nuestro diario accionar.

El surgimiento del computador ha marcado una nueva era en la historia de la humanidad que era imposible de imaginar varias décadas atrás. La gran cantidad de aplicaciones que se han desarrollado en los últimos años están transformando el desarrollo de todas las disciplinas del saber, la comercialización en el ámbito globalizado en que vivimos, la manera en que nos comunicamos, los procesos de enseñanza-aprendizaje y hasta en la manera como nos entretenemos.

Para darnos una idea de la relevancia e importancia, que en nuestro país ha alcanzado esta disciplina, basta mencionar que actualmente se ofrecen aproximadamente más de 100 carreras de Computación a nivel nacional. Esto sin considerar los programas de nivel Técnico Superior No Universitario que se ofertan.

Todas estas carreras existentes tienen como centro de su estudio a la computación pero lo hacen con más de 50 nombres distintos como: Ingeniería de Sistemas, Ingeniería de Computación, Ingeniería de Computación y Sistemas, entre otros. A pesar de que todas ellas apuntan al mismo mercado de trabajo, resulta por lo menos sorprendente que no sea posible encontrar por lo menos dos que compartan la misma curricula aunque sea en el primer año.

Comúnmente, durante la década de los setenta, la Computación se desarrolló dentro de las Facultades de Ciencias en la mayoría de las universidades estadounidenses, británicas y de otros países. Durante la década de los ochenta, los grupos de computación en las universidades se esforzaron por lograr una legitimidad académica en su ámbito local. Frecuentemente, se transformaron en departamentos de Matemáticas y Computación, hasta finalmente dividirse en dos departamentos de Matemáticas y de Computación, en la década de los noventa. Es en esta década en que un número creciente de instituciones reconocieron la influencia penetrante de la Computación, creando unidades independientes como departamentos, escuelas o institutos dedicados a tal área de estudio, un cambio que ha demostrado tanto perspicacia como previsión.

En Perú, un número cada vez mayor de instituciones de educación superior han tratado de seguir el desarrollo de las universidades extranjeras (aunque no siempre en forma muy seria o exitosa), reconociendo a la Computación como un área de estudio en sí misma, así como su importancia estratégica en la educación, y creando departamentos, escuelas o institutos dedicados a su estudio. La Facultad de Ciencias Matemáticas no puede ser la excepción a este cambio, en el que ya se tiene un retraso relativo con muchas de las instituciones educativas dentro y fuera de Perú.

1.1. Definiciones básicas

1.2. Campo y mercado ocupacional

Nuestro egresado podrá prestar sus servicios profesionales en empresas e instituciones públicas y privadas que requieran sus capacidades en función del desarrollo que oferta, entre ellas:

- Empresas dedicadas a la producción de software con calidad internacional.

- Empresas, instituciones y organizaciones que requieran software de calidad para mejorar sus actividades y/o servicios ofertados.

Nuestro egresado puede desempeñarse en el mercado laboral sin ningún problema ya que, en general, la exigencia del mercado y campo ocupacional está mucho más orientada al uso de herramientas. Sin embargo, es poco común que los propios profesionales de esta carrera se pregunten: ¿qué tipo de formación debería tener si yo quisiera crear esas herramientas además de saber usarlas?. Ambos perfiles (usuario y creador) son bastante diferentes pues no sería posible usar algo que todavía no fue creado. En otras palabras, los creadores de tecnología son los que dan origen a nuevos puestos de trabajo y abren la posibilidad de que otros puedan usar esa tecnología.

Debido a la formación basada en la investigación, nuestro profesional debe siempre ser un innovador donde trabaje. Esta misma formación permite que el egresado piense también en crear su propia empresa de desarrollo de software. Considerando que países en vias de desarrollo tienen un costo de vida mucho menor que Norte América ó Europa, una posibilidad que se muestra interesante es la exportación de software pero eso requiere que la calidad del producto sea al mismo nivel de lo ofrecido a nivel internacional.

Este perfil profesional también posibilita que los egresados se queden en el país; producir software en nuestro país y venderlo fuera es más rentable que salir al extranjero y comercializarlo allá.

El campo ocupacional de un egresado es amplio y está en continua expansión y cambio. Prácticamente toda empresa u organización hace uso de servicios de computación de algún tipo, y la buena formación básica de nuestros egresados hace que puedan responder a los requerimientos de las mismas exitosamente. Este egresado, no sólo podrá dar soluciones a los problemas existentes sino que deberá proponer innovaciones tecnológicas que impulsen la empresa hacia un progreso constante.

A medida que la informatización básica de las empresas del país avanza, la necesidad de personas capacitadas para resolver los problemas de mayor complejidad aumenta y el plan de estudios que hemos desarrollado tiene como objetivo satisfacer esta demanda considerandola a mediano y largo plazo. El campo para las tareas de investigación y desarrollo de problemas complejos en computación es también muy amplio y está creciendo día a día a nivel mundial.

Debido a la capacidad innovadora de nuestro egresado, existe una mayor la probabilidad de registrar patentes con un alto nivel inventivo lo cual es especialmente importante en nuestros países.

1.3. Importancia de la carrera en la sociedad

Uno de los caminos que se espera que siga un profesional del área de Computación Científica es que el se dedique a producir software o que se integre a las empresas productoras de software. En el ámbito de la computación, es común observar que los países cuentas con Asociaciones de Productores de Software cuyas políticas están orientadas a la exportación. Siendo así, no tendría sentido preparar a nuestros alumnos sólo para el mercado local o nacional. Nuestros egresados deben estar preparados para desenvolverse en el mundo globalizado que nos ha tocado vivir.

Nuestros futuros profesionales deben estar orientados a crear nuevas empresas de base tecnológica que puedan incrementar las exportaciones de software peruano. Este nuevo perfil está orientado a generar industria innovadora. Si nosotros somos capaces de exportar software competitivo también estaremos en condiciones de atraer nuevas inversiones. Las nuevas inversiones generarían más puestos de empleo bien remunerados y con un costo bajo en relación a otros tipos de industria. Bajo esta perspectiva, podemos afirmar que esta carrera será un motor que impulsará al desarrollo del país de forma decisiva con una inversión muy baja en relación a otros campos.

Es necesario recordar que la mayor innovación de productos comerciales de versiones recientes utiliza tecnología que se conocía en el mundo académico hace varios años, décadas o más. Un ejemplo claro son las bases de datos que soportan datos y consultas espaciales desde hace muy pocos años. Sin embargo, utilizan estructuras de datos que ya existían hace algunas décadas. Es lógico pensar que la gente del área académica no se dedique a estudiar en profundidad la última versión de un determinado software cuando esa tecnología ya la conocían hace mucho tiempo. Por esa misma razón es raro en el mundo observar que una universidad tenga convenios con una transnacional de software para dictar solamente esa tecnología pues, nuestra función es generar esa tecnología **y no sólo saber usarla**.

Tampoco debemos olvidar que los alumnos que ingresan hoy saldrán al mercado dentro de 5 años aproximadamente y, en un mundo que cambia tan rápido, no podemos ni debemos enseñarles tomando

en cuenta solamente el mercado actual. Nuestros profesionales deben estar preparados para resolver los problemas que habrá dentro de 10 o 15 años y eso sólo es posible a través de la investigación.

1.4. Información institutional

1.4.1. Misión

Por lo antes mencionado, nuestra misión es:

Ser el referente nacional integral de alto nivel en Matemática, Estadística, Computación Científica e Investigación Operativa, en los niveles de Posgrado y Pregrado, aportando a la preservacion de la diversidad cultural del Perú y a su progreso científico y tecnológico. Sus miembros se distinguen por la aplicación del metodo científico para el desarrollo del conocimiento y la solución de los problemas del país, respetando y protegiendo su medio ambiente.

1.4.2. Visión

- Queremos ser una carrera profesional acreditada con estándares internacionales que cuente con el reconocimiento en función de la calidad y competitividad de sus docentes y egresados.
- Queremos ser una carrera que trascienda por la relevancia y pertinencia de sus proyectos de investigación básica y aplicada.
- Buscamos ser una carrera que promueva el desarrollo de la industria del software relacionada a Computación Científica a nivel internacional, incorporando a sus egresados a la industria ya establecida o generando nuevas empresas en este mismo rubro.
- Queremos ser una carrera que comparta y difunda el conocimiento con todos los sectores de la población y contribuya a la solución de los problemas estratégicos de nuestra sociedad.

1.5. Información de acreditacion

1.5.1. Resultados de la carrera *Outcomes*

Al finalizar esta carrera, el(la) egresado(a), habrá logrado los siguientes resultados (*Outcomes*):

ABET-1) Analizar un problema computacional complejo y aplicar los principios computacionales y otras disciplinas relevantes para identificar soluciones.

ABET-2) Diseñar, implementar y evaluar una solución basada en la computación para satisfacer un conjunto dado de requisitos de computación en el contexto de la disciplina del programa.

ABET-3) Comunicarse efectivamente en diversos contextos profesionales.

ABET-4) Reconocer las responsabilidades profesionales y tomar decisiones informadas en la práctica de la computación basadas en principios legales y éticos.

ABET-5) Funcionar efectivamente como miembro o líder de un equipo involucrado en actividades apropiadas a la disciplina del programa.

ABET-6) Aplicar la teoría de la computación y los fundamentos del desarrollo de software para producir soluciones basadas en computación.

Las competencias y/o resultados propuestas en esta malla curricular corresponden a las competencias y/o resultados propuestas por ABET (http://www.abet.org).

Por otro lado, las habilidades adicionales son aportes de este grupo de trabajo.

1.5.2. Objetivos educacionales

Después de cinco años de egresado de la carrera profesional de Computación Científica, nuestros profesionales deben ser capaces de:

1. Cumplir y superar las expectativas de trabajo definidas por el entorno laboral.
2. Desempeñarse como miembro, o líder, de un equipo de trabajo tanto especializado como multidisciplinario.
3. Proponer soluciones al contexto laboral, donde se desenvuelve, basadas en la implementación o mejora del estado del arte en Computación Científica.
4. Comunicar de forma efectiva propuestas tecnológicas a personas de distintos niveles de conocimiento y de diferentes ámbitos sociales.
5. Actualizarse y adaptarse a nuevos conocimientos en Computación Científica y a diferentes ámbitos laborales, de forma autónoma o mediante estudios complementarios.
6. Demostrar un claro entendimiento de las consecuencias que surgen a partir de creaciones tecnológicas relacionadas a la Computación Científica en aspectos tales como lo social, ético, humano, moral, legal, ambiental, económico, entre otros.

1.5.3. Aprendizaje esperado (*Learning Outcomes*)

Cada KU dentro de un KA enumera tanto un conjunto de temas como los resultados de aprendizaje (*Learning Outcomes*) que los estudiantes deben alcanzar en lo que respecta a los temas especificados. Los resultados de aprendizaje no son de igual tamaño y no tienen una asignación uniforme de horas curriculares; temas con el mismo número de horas pueden tener muy diferentes números de resultados de aprendizaje asociados.

Cada resultado de aprendizaje tiene un nivel asociado de dominio. En la definición de los diferentes niveles tomamos como referencia otros enfoques curriculares, especialmente la taxonomía de Bloom, que ha sido bien explorada dentro de la Computación Científica. En este documento no se aplicaron directamente los niveles de Bloom en parte porque varios de ellos están determinados por el contexto pedagógico, lo que introduciría demasiada pluralidad en un documento de este tipo; y en parte porque los niveles de dominio pretenden ser indicativos y no imponer restricciones teóricas sobre los usuarios de este documento.

Nosotros usamos tres niveles de dominio esperados, que son:

Nivel 1 Familiarizarse(*Familiarity*) : El estudiante **comprende** un concepto básicamente. Responde: **¿Qué sabe sobre esto?**

Nivel 2 Usar(*Usage*) : El estudiante **aplica** conceptos en situaciones prácticas (ej: programación). Responde: **¿Cómo lo haría?**

Nivel 3 Evaluar(*Assessment*) : El estudiante **evalúa y justifica** enfoques. Responde: **¿Por qué este método?**

Por ejemplo, para evaluar los niveles de dominio, consideremos la noción de derivada en el cálculo. En el plano de la "familiaridad", se espera que un estudiante conozca la definición de derivada, entienda su significado como tasa de cambio y sepa por qué es una herramienta fundamental en ciencias básicas.

Con el fin de mostrar el dominio del nivel "Uso", el estudiante debe ser capaz de calcular derivadas de funciones elementales, aplicar reglas de derivación y utilizarlas para resolver problemas sencillos de optimización o de razones de cambio.

En el nivel de "Evaluación", se requeriría que el estudiante comprenda las múltiples interpretaciones de la derivada (geométrica, física, económica, etc.), analice sus limitaciones y sea capaz de seleccionar el enfoque más adecuado para modelar y resolver fenómenos complejos en distintos contextos científicos.

1.6. Perfiles

1.6.1. Perfil del estudiante

El aspirante a ingresar a la Escuela Profesional de Computación Científica de la Universidad Nacional Mayor de San Marcos (UNMSM) debe tener:

Conocimientos de:

1. Conceptos fundamentales de álgebra, geometría, trigonometría y análisis matemático elemental.
2. Fenómenos físicos y químicos básicos de su entorno cotidiano.
3. Nociones elementales de probabilidad y estadística descriptiva.

Habilidades para:

1. Identificar relaciones causa-efecto en sistemas naturales y abstractos, y predecir sus comportamientos.
2. Distinguir entre observaciones empíricas y modelos teóricos, reconociendo sus límites y aplicaciones.
3. Percibir y describir patrones cuantitativos y cualitativos en datos y fenómenos.
4. Plantear hipótesis, diseñar procedimientos de verificación e interpretar resultados de manera rigurosa.
5. Comunicar argumentos y conclusiones de forma clara, tanto oralmente como por escrito, utilizando lenguaje técnico apropiado.
6. Mantener la concentración en tareas analíticas que demanden esfuerzo sostenido.
7. Sintetizar información compleja y extraer conceptos esenciales.
8. Adquirir y aplicar metodologías de estudio sistemático y autorregulado.

Actitudes de:

1. Interés genuino y curiosidad por las ciencias exactas y naturales, y sus aplicaciones.
2. Disposición para el trabajo colaborativo en laboratorios, talleres y proyectos académicos.
3. Iniciativa para explorar problemas más allá del currículo y perseverancia ante desafíos intelectuales.

1.6.2. Perfil del graduado

El graduado del Programa Profesional de Computación Científica es capaz de:

1. Aplicar fundamentos teóricos de matemática pura (álgebra lineal, análisis matemático, teoría de números) para modelar problemas complejos en ciencia e ingeniería.
2. Diseñar e implementar algoritmos numéricos y simbólicos eficientes para resolver ecuaciones diferenciales, sistemas de optimización y problemas de álgebra computacional.
3. Desarrollar software matemático utilizando lenguajes de programación científica (Python, Julia, MATLAB, C++) y bibliotecas especializadas para computación de alto rendimiento.
4. Modelar y simular fenómenos utilizando métodos Monte Carlo, análisis de estabilidad, teoría de grafos y optimización combinatoria en aplicaciones reales.
5. Aplicar técnicas de machine learning, deep learning y análisis de datos masivos para extraer patrones, construir predictores y resolver problemas de clasificación y clustering.
6. Colaborar en proyectos interdisciplinarios de bioinformática, criptografía, finanzas cuantitativas, física computacional e inteligencia artificial, aportando rigor matemático.
7. Analizar la complejidad computacional de algoritmos, implementar técnicas de paralelización y optimizar el uso de recursos en clusters y arquitecturas GPU.
8. Comunicar resultados matemáticos y computacionales mediante publicaciones científicas, visualización de datos interactiva y documentación técnica rigurosa para diversas audiencias.
9. Adaptar herramientas matemáticas para resolver problemas industriales en sectores como telecomunicaciones, logística, ciberseguridad, minería de datos y desarrollo de videojuegos.
10. Continuar estudios de posgrado en matemática aplicada, ciencia de datos o computación científica, contribuyendo a la generación de conocimiento en áreas emergentes.

1.6.3. Perfil Profesional

El profesional en Computación Científica tiene las siguientes características:

- Aplicar teorías fundamentales de matemática pura y aplicada para modelar y resolver problemas complejos en ciencia e ingeniería.
- Diseñar e implementar algoritmos numéricos y simbólicos eficientes usando lenguajes científicos (Python, Julia, MATLAB, C++).
- Desarrollar modelos computacionales para simular fenómenos, resolver ecuaciones diferenciales y optimizar sistemas mediante métodos matemáticos avanzados.

- Aplicar técnicas de machine learning, deep learning y análisis de datos masivos para extraer patrones y construir predictores robustos.
- Trabajar con arquitecturas de computación de alto rendimiento (clusters, GPU) para resolver problemas de gran escala en tiempo razonable.
- Colaborar en proyectos interdisciplinarios en bioinformática, criptografía, finanzas cuantitativas, física computacional e inteligencia artificial.
- Comunicar resultados matemáticos y computacionales con rigor mediante publicaciones, visualizaciones interactivas y documentación técnica especializada.
- Analizar la complejidad computacional de algoritmos, implementar técnicas de paralelización y optimizar uso de recursos computacionales.
- Adaptar herramientas matemáticas para resolver desafíos en sectores como telecomunicaciones, minería, logística, ciberseguridad y desarrollo tecnológico.
- Continuar estudios de posgrado en matemática aplicada, ciencia de datos o computación científica, generando conocimiento original de impacto.

1.6.4. Perfil del docente

El docente de la carrera de Computación Científica es una persona que:

- Promueve la formación integral del estudiante mediante la enseñanza, investigación y extensión, transmitiendo conocimientos sólidos con claridad y simplicidad.
- Fomenta la investigación y el aprendizaje continuo para que los estudiantes se adapten a los avances del área y busquen el bien común de individuos, sociedad y organizaciones.
- Mantiene formación académica avanzada de calidad, con estudios a tiempo completo, buscando la excelencia académica a nivel internacional.
- Publica investigaciones en eventos internacionales y revistas indexadas de alto impacto, contribuyendo al avance del conocimiento y manteniéndose actualizado.
- Establece redes de cooperación con instituciones de nivel internacional, enriqueciendo la experiencia educativa.
- Participa activamente en la formación e integración del estudiante a la comunidad académica , promoviendo la exigencia continua y rigor académico.

1.7. Grados y Títulos

Cumplida la aprobación de los créditos académicos propuestos en la malla curricular y luego de haber cumplido con los requisitos estipulados en el reglamento de grados y títulos de la carrera, el egresado recibirá:

Grado Académico: Bachiller en Computación Científica y

Titulo Profesional: Licenciado en Computación Científica

1.8. Recursos para dictado de clases

Un profesional innovador debe estar al tanto de los últimos avances de su área siempre. Los últimos avances de esta área no son presentados en los libros necesariamente. Debemos utilizar publicaciones de revistas indexadas de circulación mundial.

Por esa razón, tomamos como base la suscripción institucional a la ACM y a la IEEE-CS. Es recomendado que el docente use este material para discutir en clase las tendencias en todas las áreas.

Los laboratorios de cómputo también deben ser renovados de acuerdo a la rapidez propia de esta área y esto implica renovaciones constantes para poder garantizar que los estudiantes estén actualizados.

El recurso físico en cuanto a laboratorios para cada curso está detallado en el Capítulo 8.

1.9. Organización del documento

El resto de este documento está organizado de la siguiente forma: el Capítulo 1, define y explica el campo de acción de la Computación Científica, además se hace una muy breve explicación de las distintas carreras relacionadas a esta disciplina. .

El Capítulo 5, muestra las áreas de Conocimiento de Computación Científica, indicando los tópicos y objetivos de aprendizaje de los temas, pertenecientes a estos grupos.

El Capítulo 6 contiene la distribución por semestres, por áreas, por niveles, visión gráfica de la malla curricular, comparación con las diversas propuestas internacionales, distribución de tópicos por curso así como la distribución de habilidades por materia.

El Capítulo line:42 **??** contiene información detallada para cada uno de los cursos incluyendo las habilidades con las cuales contribuye, bibliografía por cada unidad así como el número de horas mínimas por cada unidad.

En el Capítulo line:47 **??** podemos observar un listado de cursos por profesor y de profesor por curso.

En el Capítulo 7 se presentan las tablas de equivalencias con otros planes curriculares.

En el Capítulo 8 se presenta una sugerencia de los laboratorios requeridos para el dictado de clases las mismas que podrían variar de acuerdo al volumen de alumnos que se tenga.

Capítulo 2

Cuerpo del conocimiento de Matemática Computacional

Las 14 áreas de conocimiento en Matemática Computacional son:

2.1 Análisis Numérico y Científico (NSA) (Pág. 10)
- 2.1.1 Aritmética Computacional y Análisis de Error (Pág. 10)
- 2.1.2 Álgebra Lineal Numérica (Pág. 10)
- 2.1.3 Búsqueda de Raíces y Optimización Numérica Básica (Pág. 11)
- 2.1.4 Interpolación y Aproximación de Funciones (Pág. 11)
- 2.1.5 Diferenciación e Integración Numérica (Pág. 11)
- 2.1.6 Métodos Numéricos para Ecuaciones Diferenciales Ordinarias (EDOs) (Pág. 12)

2.2 Modelado Matemático y Simulación (MMS) (Pág. 12)
- 2.2.1 Formulación de Modelos Matemáticos (Pág. 12)
- 2.2.2 Simulación de Sistemas Discretos (Pág. 13)
- 2.2.3 Simulación de Sistemas Continuos (Pág. 13)
- 2.2.4 Análisis de Sensibilidad y Robustez (Pág. 13)
- 2.2.5 Validación y Verificación de Modelos (Pág. 13)

2.3 Probabilidad y Computación Estadística (PSC) (Pág. 14)
- 2.3.1 Probabilidad Avanzada y Procesos Estocásticos (Pág. 14)
- 2.3.2 Inferencia Estadística Computacional (Pág. 14)
- 2.3.3 Diseño de Experimentos Computacionales (Pág. 15)
- 2.3.4 Métodos de Remuestreo y Validación (Pág. 15)

2.4 Estructuras de Datos y Algoritmos (DSA) (Pág. 15)
- 2.4.1 Complejidad Algorítmica y Análisis Asintótico (Pág. 16)
- 2.4.2 Estructuras de Datos Fundamentales y Avanzadas (Pág. 16)
- 2.4.3 Algoritmos de Búsqueda y Ordenamiento (Pág. 16)
- 2.4.4 Algoritmos de Grafos y Optimización Combinatoria (Pág. 16)
- 2.4.5 Algoritmos Paralelos y Distribuidos (Pág. 17)

2.5 Computación de Alto Rendimiento (HPC) (Pág. 17)
- 2.5.1 Arquitecturas de Computadoras Paralelas (Pág. 17)
- 2.5.2 Programación Paralela de Memoria Compartida (Pág. 18)
- 2.5.3 Programación Paralela de Memoria Distribuida (Pág. 18)
- 2.5.4 Programación de Aceleradores (Pág. 18)
- 2.5.5 Métodos Numéricos Paralelos (Pág. 18)

2.6 Optimización y Aprendizaje Automático (OML) (Pág. 19)
- 2.6.1 Optimización Convexa y No Convexa (Pág. 19)
- 2.6.2 Optimización con Restricciones (Pág. 19)
- 2.6.3 Optimización para Aprendizaje Automático (Pág. 20)
- 2.6.4 Optimización a Gran Escala y Distribuida (Pág. 20)
- 2.6.5 Metaheurísticas y Optimización Bioinspirada (Pág. 20)

2.7 Matemáticas para Ciencia de Datos y Aprendizaje Automático (DSM) (Pág. 20)
- 2.7.1 Fundamentos del Aprendizaje Estadístico (Pág. 21)
- 2.7.2 Métodos de Aprendizaje Supervisado (Pág. 21)
- 2.7.3 Métodos de Aprendizaje No Supervisado (Pág. 21)
- 2.7.4 Evaluación y Selección de Modelos (Pág. 22)

2.8 Matemáticas para Inteligencia Artificial (AIM) (Pág. 22)
- 2.8.1 Teoría del Aprendizaje Profundo (Pág. 22)
- 2.8.2 Arquitecturas Neuronales Avanzadas (Pág. 22)
- 2.8.3 Aprendizaje por Refuerzo (Pág. 23)
- 2.8.4 Procesamiento de Lenguaje Natural con Transformers (Pág. 23)
- 2.8.5 Modelos Generativos y GANs (Pág. 23)

2.9 Computación Simbólica y Discreta (SDC) (Pág. 24)
- 2.9.1 Teoría de Grupos y Algoritmos Algebraicos (Pág. 24)
- 2.9.2 Teoría de Números Computacional y Criptografía (Pág. 24)
- 2.9.3 Sistemas de Computación Simbólica (Pág. 24)
- 2.9.4 Lógica Computacional y Verificación Formal (Pág. 25)

2.10 Señales, Imágenes y Visualización (SIV) (Pág. 25)
- 2.10.1 Análisis de Fourier y Wavelets (Pág. 25)
- 2.10.2 Procesamiento Digital de Imágenes y Visión Computacional (Pág. 26)
- 2.10.3 Visualización Científica y Computación Visual (Pág. 26)
- 2.10.4 Gráficos por Computadora y Geometría Computacional (Pág. 26)

2.11 Computación Cuántica y Algoritmos (QCA) (Pág. 26)
- 2.11.1 Fundamentos de Mecánica Cuántica para Computación (Pág. 27)
- 2.11.2 Qubits y Circuitos Cuánticos (Pág. 27)
- 2.11.3 Algoritmos Cuánticos (Pág. 27)
- 2.11.4 Criptografía Post-Cuántica y Simulación Cuántica (Pág. 28)

2.12 Ciberseguridad y Privacidad (CSP) (Pág. 28)
- 2.12.1 Criptografía Simétrica (Pág. 28)
- 2.12.2 Criptografía Asimétrica (Pág. 28)
- 2.12.3 Computación con Preservación de la Privacidad (Pág. 29)
- 2.12.4 Teoría de la Información y Compresión de Datos (Pág. 29)

2.13 Matemáticas Computacionales para Dominios Específicos (CDM) (Pág. 29)
- 2.13.1 Bioinformática y Biología Computacional (Pág. 30)
- 2.13.2 Finanzas Computacionales y Ciencia del Riesgo (Pág. 30)
- 2.13.3 Física Computacional y Química Cuántica (Pág. 30)
- 2.13.4 Ciencia de Materiales Computacional (Pág. 31)
- 2.13.5 Modelado Climático y Sistemas Complejos (Pág. 31)

2.14 Gobernanza y Gestión de Datos (DGM) (Pág. 31)

- 2.14.1 Ética en IA y Algoritmos Responsables (Pág. 31)
- 2.14.2 Gestión de Proyectos de Datos y MLOps (Pág. 32)
- 2.14.3 Gobernanza de Datos y Cumplimiento Normativo (Pág. 32)

2.1. Análisis Numérico y Científico (NSA)

Esta área cubre los métodos numéricos fundamentales y su análisis que forman la base de las matemáticas computacionales. Se enfoca en el desarrollo, implementación y análisis de algoritmos para resolver problemas matemáticos numéricamente, con un fuerte énfasis en comprender y controlar los errores que surgen de la aritmética de precisión finita y las aproximaciones discretas. Este conocimiento es esencial para todas las áreas posteriores en matemáticas computacionales, ya que proporciona las herramientas para garantizar que las soluciones calculadas sean precisas, estables y confiables.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
2.1.1 Aritmética Computacional y Análisis de Error (Pág. 10)			No
2.1.2 Álgebra Lineal Numérica (Pág. 10)			No
2.1.3 Búsqueda de Raíces y Optimización Numérica Básica (Pág. 11)			No
2.1.4 Interpolación y Aproximación de Funciones (Pág. 11)			No
2.1.5 Diferenciación e Integración Numérica (Pág. 11)			No
2.1.6 Métodos Numéricos para Ecuaciones Diferenciales Ordinarias (EDOs) (Pág. 12)			No

2.1.1. NSA/Aritmética Computacional y Análisis de Error

Temas:
Core Tier1

- Aritmética de punto flotante y el estándar IEEE
- Fuentes de error: truncamiento, redondeo y discretización
- Propagación de error y número de condición
- Estabilidad de algoritmos numéricos
- Error absoluto, relativo y directo/inverso

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Definir los diferentes tipos de errores en computación numérica (redondeo, truncamiento, discretización) [Familiarizarse]
2. Explicar los principios de representación y aritmética de punto flotante según el estándar IEEE [Familiarizarse]
3. Calcular el error absoluto y relativo de una aproximación [Usar]
4. Analizar la propagación de errores en una secuencia de operaciones aritméticas [Usar]
5. Distinguir entre la condición de un problema y la estabilidad de un algoritmo [Evaluar]
6. Calcular el número de condición de un problema simple (ej., multiplicación matriz-vector) [Usar]

2.1.2. NSA/Álgebra Lineal Numérica

Temas:
Core Tier1

- Descomposición LU y sus aplicaciones
- Descomposición QR y problemas de mínimos cuadrados
- Descomposición en Valores Singulares (SVD) y sus aplicaciones
- Número de condición y estabilidad numérica de sistemas lineales
- Métodos iterativos para sistemas lineales (Jacobi, Gauss-Seidel, Gradiente Conjugado)

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Implementar algoritmos de descomposición LU y QR para resolver sistemas lineales [Usar]

2. Explicar el significado geométrico y algebraico de la Descomposición en Valores Singulares [Familiarizarse]
3. Usar SVD para aplicaciones como aproximación matricial y análisis de componentes principales [Usar]
4. Analizar la estabilidad de un sistema lineal usando su número de condición [Evaluar]
5. Seleccionar e implementar un método iterativo apropiado para resolver sistemas lineales grandes y dispersos [Evaluar]

2.1.3. NSA/Búsqueda de Raíces y Optimización Numérica Básica

Temas:
Core Tier1
- Método de bisección e iteración de punto fijo
- Método de Newton y método de la secante para búsqueda de raíces
- Análisis de convergencia y orden de convergencia
- Descenso de gradiente para optimización univariada
- Tasa de convergencia y criterios de parada

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Implementar algoritmos de búsqueda de raíces (bisección, Newton, secante) y analizar su convergencia [Usar]
2. Clasificar el orden de convergencia de métodos iterativos [Evaluar]
3. Aplicar descenso de gradiente para resolver problemas simples de optimización univariada [Usar]
4. Diseñar criterios de parada apropiados para métodos iterativos [Usar]

2.1.4. NSA/Interpolación y Aproximación de Funciones

Temas:
Core Tier1
- Interpolación polinomial (formas de Lagrange, Newton)
- Interpolación por splines (splines cúbicos)
- Aproximación por mínimos cuadrados
- Fenómeno de Runge y nodos de Chebyshev
- Análisis de error para interpolación

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Construir polinomios interpoladores usando diferentes métodos [Usar]
2. Implementar interpolación por splines cúbicos [Usar]
3. Aplicar el método de mínimos cuadrados para ajuste de datos [Usar]
4. Explicar el fenómeno de Runge y estrategias para mitigarlo [Familiarizarse]
5. Analizar el error de un esquema de interpolación [Evaluar]

2.1.5. NSA/Diferenciación e Integración Numérica

Temas:
Core Tier1
- Aproximaciones por diferencias finitas (adelantadas, atrasadas, centradas)
- Extrapolación de Richardson
- Fórmulas de Newton-Cotes (regla del trapecio, regla de Simpson)
- Cuadratura gaussiana
- Análisis de error para integración numérica

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Calcular derivadas de funciones usando esquemas de diferencias finitas [Usar]
2. Aplicar extrapolación de Richardson para mejorar la precisión de aproximaciones numéricas [Usar]

3. Implementar reglas de integración numérica (Trapecio, Simpson, Gaussiana) [Usar]
4. Comparar la precisión y eficiencia de diferentes métodos de integración numérica [Evaluar]
5. Analizar el error para fórmulas de diferenciación e integración numérica [Evaluar]

2.1.6. NSA/Métodos Numéricos para Ecuaciones Diferenciales Ordinarias (EDOs)

Temas:
Core Tier1
- Método de Euler y sus variantes (explícito, implícito)
- Métodos de Runge-Kutta
- Métodos multipaso (Adams-Bashforth, Adams-Moulton)
- Análisis de estabilidad y convergencia para solucionadores de EDOs
- Sistemas de EDOs y EDOs de orden superior

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Implementar solucionadores básicos de EDOs (Euler, Runge-Kutta) [Usar]
2. Analizar las propiedades de estabilidad y convergencia de métodos numéricos para EDOs [Evaluar]
3. Seleccionar un método numérico apropiado para una EDO dada basado en requisitos de estabilidad y precisión [Evaluar]
4. Resolver sistemas de EDOs y convertir EDOs de orden superior a sistemas de primer orden [Usar]

2.2. Modelado Matemático y Simulación (MMS)

Esta área se enfoca en el proceso de crear, analizar y validar modelos matemáticos para representar sistemas del mundo real. Cubre tanto paradigmas de modelado continuo como discreto, así como enfoques deterministas y estocásticos. Los estudiantes aprenden a formular modelos, implementar simulaciones y evaluar críticamente los resultados mediante análisis de sensibilidad y cuantificación de incertidumbre. Este conocimiento es esencial para aplicar matemáticas computacionales a problemas complejos en ciencia, ingeniería y otros dominios.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
2.2.1 Formulación de Modelos Matemáticos (Pág. 12)			No
2.2.2 Simulación de Sistemas Discretos (Pág. 13)			No
2.2.3 Simulación de Sistemas Continuos (Pág. 13)			No
2.2.4 Análisis de Sensibilidad y Robustez (Pág. 13)			No
2.2.5 Validación y Verificación de Modelos (Pág. 13)			No

2.2.1. MMS/Formulación de Modelos Matemáticos

Temas:
Core Tier1
- Proceso y ciclo de modelado
- Modelos continuos vs. discretos
- Modelos deterministas vs. estocásticos
- Identificación de supuestos y técnicas de simplificación
- Análisis dimensional y escalado

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Describir el ciclo completo de modelado desde la definición del problema hasta la interpretación de la solución [Familiarizarse]
2. Clasificar modelos como continuos/discretos y deterministas/estocásticos [Evaluar]
3. Identificar y justificar supuestos apropiados para simplificación de modelos [Evaluar]
4. Aplicar análisis dimensional para reducir la complejidad de modelos físicos [Usar]

2.2.2. MMS/Simulación de Sistemas Discretos

Temas:
Core Tier1
- Teoría de colas y modelos
- Autómatas celulares y modelos basados en agentes
- Métodos de simulación Monte Carlo
- Cadenas y procesos de Markov
- Simulación de eventos discretos

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Formular y analizar modelos simples de colas [Usar]
2. Implementar autómatas celulares y simulaciones basadas en agentes [Usar]
3. Aplicar métodos Monte Carlo para simulación probabilística [Usar]
4. Analizar cadenas de Markov para transiciones de estado y comportamiento en estado estacionario [Evaluar]
5. Diseñar simulaciones de eventos discretos para análisis de sistemas [Usar]

2.2.3. MMS/Simulación de Sistemas Continuos

Temas:
Core Tier1
- Modelos de Ecuaciones Diferenciales Ordinarias (EDOs)
- Modelos de Ecuaciones Diferenciales Parciales (EDPs)
- Métodos de diferencias finitas para EDPs
- Selección e implementación de solucionadores
- Sistemas stiff y métodos adaptativos

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Formular modelos de EDOs y EDPs para sistemas físicos [Usar]
2. Implementar métodos de diferencias finitas para EDPs simples [Usar]
3. Seleccionar solucionadores numéricos apropiados para diferentes tipos de EDOs/EDPs [Evaluar]
4. Manejar sistemas stiff usando métodos numéricos apropiados [Usar]

2.2.4. MMS/Análisis de Sensibilidad y Robustez

Temas:
Core Tier1
- Análisis de sensibilidad local (derivadas parciales)
- Análisis de sensibilidad global (índices de Sobol)
- Método de Morris y técnicas de screening
- Análisis de robustez y escenarios del peor caso
- Propagación de incertidumbre

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Realizar análisis de sensibilidad local usando métodos basados en derivadas [Usar]
2. Aplicar técnicas de análisis de sensibilidad global (ej., índices de Sobol) [Usar]
3. Usar el método de Morris para screening de factores [Usar]
4. Evaluar la robustez del modelo bajo variaciones de parámetros [Evaluar]
5. Cuantificar la propagación de incertidumbre a través de modelos [Evaluar]

2.2.5. MMS/Validación y Verificación de Modelos

Temas:
Core Tier1
- Verificación: corrección del código numérico
- Validación: comparación con datos experimentales

- Cuantificación de Incertidumbre (UQ) en modelos
- Calibración de modelos y estimación de parámetros
- Pruebas y métricas de bondad de ajuste

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Distinguir entre verificación y validación en modelado [Familiarizarse]
2. Aplicar técnicas de verificación para garantizar la corrección del código [Usar]
3. Validar modelos usando datos experimentales y pruebas estadísticas [Usar]
4. Calibrar parámetros del modelo usando técnicas de optimización [Usar]
5. Aplicar métodos de cuantificación de incertidumbre para evaluar la confiabilidad del modelo [Usar]

2.3. Probabilidad y Computación Estadística (PSC)

Esta área conecta los fundamentos teóricos de probabilidad y estadística con su implementación computacional. Cubre modelado probabilístico avanzado, métodos de inferencia computacional y el diseño de experimentos in silico. Los estudiantes aprenden a aplicar procesos estocásticos, métodos bayesianos y técnicas de remuestreo para analizar sistemas complejos y tomar decisiones basadas en datos bajo incertidumbre. Este conocimiento es crucial para el análisis de datos moderno, el aprendizaje automático y la computación científica.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
2.3.1 Probabilidad Avanzada y Procesos Estocásticos (Pág. 14)			No
2.3.2 Inferencia Estadística Computacional (Pág. 14)			No
2.3.3 Diseño de Experimentos Computacionales (Pág. 15)			No
2.3.4 Métodos de Remuestreo y Validación (Pág. 15)			No

2.3.1. PSC/Probabilidad Avanzada y Procesos Estocásticos

Temas:
Core Tier1

- Fundamentos de probabilidad medida-teórica
- Variables aleatorias y distribuciones
- Caracterización de procesos estocásticos
- Cadenas y procesos de Markov
- Movimiento browniano y procesos de difusión

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Aplicar conceptos medida-teóricos a espacios de probabilidad [Familiarizarse]
2. Caracterizar diferentes tipos de procesos estocásticos [Evaluar]
3. Analizar cadenas de Markov para comportamiento transitorio y en estado estacionario [Usar]
4. Modelar sistemas usando procesos estocásticos en tiempo continuo [Usar]

2.3.2. PSC/Inferencia Estadística Computacional

Temas:
Core Tier1

- Estimación de Máxima Verosimilitud (MLE)
- Métodos de inferencia bayesiana
- Métodos de Monte Carlo mediante Cadenas de Markov (MCMC)
- Algoritmo Expectación-Maximización (EM)
- Remuestreo bootstrap y jackknife

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Implementar Estimación de Máxima Verosimilitud para varias distribuciones [Usar]

2. Aplicar métodos bayesianos para estimación de parámetros [Usar]
3. Implementar algoritmos MCMC para muestreo posterior [Usar]
4. Usar el algoritmo EM para modelos con variables latentes [Usar]
5. Aplicar métodos bootstrap para intervalos de confianza [Usar]

2.3.3. PSC/Diseño de Experimentos Computacionales

Temas:
Core Tier1

- Diseños factoriales y factoriales fraccionados
- Metodología de superficie de respuesta
- Diseños de llenado de espacio (hipercubo latino)
- Diseño óptimo de experimentos
- Experimentos computacionales y modelado sustituto

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Diseñar experimentos factoriales para screening de factores [Usar]
2. Aplicar metodología de superficie de respuesta para optimización [Usar]
3. Implementar diseños de llenado de espacio para experimentos computacionales [Usar]
4. Seleccionar diseños óptimos para objetivos específicos [Evaluar]
5. Construir modelos sustitutos para simulaciones costosas [Usar]

2.3.4. PSC/Métodos de Remuestreo y Validación

Temas:
Core Tier1

- Estimación bootstrap e intervalos de confianza
- Técnicas de validación cruzada
- Pruebas de permutación y aleatorización
- Selección de modelos y criterios de información
- Métodos de ensamblado y bagging

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Implementar bootstrap para estimación de error estándar [Usar]
2. Aplicar validación cruzada para evaluación de modelos [Usar]
3. Usar pruebas de permutación para contraste de hipótesis [Usar]
4. Comparar modelos usando criterios de información (AIC, BIC) [Evaluar]
5. Implementar bagging para reducción de varianza [Usar]

2.4. Estructuras de Datos y Algoritmos (DSA)

Esta área proporciona el conjunto de herramientas fundamental para diseñar y analizar soluciones computacionales eficientes. Cubre los fundamentos teóricos de la complejidad algorítmica, la implementación práctica de estructuras de datos fundamentales y avanzadas, y los principios de diseño detrás de algoritmos eficientes para búsqueda, ordenamiento, procesamiento de grafos y optimización combinatoria. Este conocimiento forma el núcleo computacional que permite a los estudiantes desarrollar soluciones eficientes en todas las demás áreas de las matemáticas computacionales.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
2.4.1 Complejidad Algorítmica y Análisis Asintótico (Pág. 16)			No
2.4.2 Estructuras de Datos Fundamentales y Avanzadas (Pág. 16)			No
2.4.3 Algoritmos de Búsqueda y Ordenamiento (Pág. 16)			No
2.4.4 Algoritmos de Grafos y Optimización Combinatoria (Pág. 16)			No
2.4.5 Algoritmos Paralelos y Distribuidos (Pág. 17)			No

2.4.1. DSA/Complejidad Algorítmica y Análisis Asintótico

Temas:
Core Tier1
- Medidas de complejidad y notación asintótica
- Análisis del mejor caso, caso promedio y peor caso
- Relaciones de recurrencia y técnicas de resolución
- Análisis empírico de algoritmos
- Complejidad espacial y compensaciones

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Analizar algoritmos usando notación asintótica (Big O, Omega, Theta) [Usar]
2. Diferenciar entre complejidades del mejor caso, caso promedio y peor caso [Evaluar]
3. Resolver relaciones de recurrencia para análisis de algoritmos [Usar]
4. Comparar algoritmos mediante mediciones empíricas [Evaluar]

2.4.2. DSA/Estructuras de Datos Fundamentales y Avanzadas

Temas:
Core Tier1
- Arreglos, listas enlazadas y arreglos dinámicos
- Pilas, colas y deques
- Tablas hash y resolución de colisiones
- Árboles y árboles de búsqueda binaria
- Montículos y colas de prioridad

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Implementar estructuras de datos básicas desde cero [Usar]
2. Analizar complejidad temporal/espacial de operaciones de estructuras de datos [Evaluar]
3. Seleccionar estructuras de datos apropiadas para problemas dados [Evaluar]
4. Manejar colisiones hash usando diferentes estrategias [Usar]

2.4.3. DSA/Algoritmos de Búsqueda y Ordenamiento

Temas:
Core Tier1
- Búsqueda lineal y binaria
- Ordenamiento basado en comparación (QuickSort, MergeSort, HeapSort)
- Ordenamiento sin comparación (RadixSort, CountingSort)
- Algoritmos de selección y estadísticas de orden
- Ordenamiento externo y procesamiento a gran escala

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Implementar y comparar algoritmos de búsqueda [Usar]
2. Analizar complejidad temporal/espacial de algoritmos de ordenamiento [Evaluar]
3. Seleccionar algoritmo de ordenamiento apropiado para características de datos [Evaluar]
4. Implementar algoritmos de estadísticas de orden [Usar]

2.4.4. DSA/Algoritmos de Grafos y Optimización Combinatoria

Temas:
Core Tier1
- Representación de grafos y recorrido (BFS, DFS)
- Algoritmos de camino más corto (Dijkstra, Bellman-Ford)
- Árboles de expansión mínima (Prim, Kruskal)
- Flujo en redes y emparejamiento
- NP-completitud y algoritmos de aproximación

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Implementar algoritmos de recorrido de grafos [Usar]
2. Resolver problemas de camino más corto usando algoritmos apropiados [Usar]
3. Construir árboles de expansión mínima [Usar]
4. Analizar complejidad de algoritmos de grafos [Evaluar]
5. Diseñar algoritmos de aproximación para problemas NP-difíciles [Usar]

2.4.5. DSA/Algoritmos Paralelos y Distribuidos

Temas:
Core Tier1

- Modelos de computación paralela (PRAM)
- Algoritmos paralelos básicos (reducción, scan)
- Algoritmos distribuidos y consenso
- Sincronización y balanceo de carga
- MapReduce y procesamiento distribuido de datos

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Analizar algoritmos paralelos usando el modelo trabajo-span [Evaluar]
2. Implementar operaciones de reducción y scan paralelas [Usar]
3. Diseñar algoritmos distribuidos para consenso [Usar]
4. Aplicar el paradigma MapReduce al procesamiento de datos [Usar]

2.5. Computación de Alto Rendimiento (HPC)

Esta área se enfoca en los principios y prácticas de desarrollar soluciones computacionales que aprovechan los recursos de computación paralela y distribuida para resolver problemas científicos y de ingeniería a gran escala. Cubre arquitecturas de computadoras, modelos de programación paralela, técnicas de optimización para hardware moderno y la implementación de métodos numéricos en sistemas de alto rendimiento. Este conocimiento permite a los estudiantes abordar problemas computacionalmente intensivos que son inviables en sistemas de un solo procesador.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
2.5.1 Arquitecturas de Computadoras Paralelas (Pág. 17)			No
2.5.2 Programación Paralela de Memoria Compartida (Pág. 18)			No
2.5.3 Programación Paralela de Memoria Distribuida (Pág. 18)			No
2.5.4 Programación de Aceleradores (Pág. 18)			No
2.5.5 Métodos Numéricos Paralelos (Pág. 18)			No

2.5.1. HPC/Arquitecturas de Computadoras Paralelas

Temas:
Core Tier1

- Taxonomía de Flynn y arquitecturas paralelas
- Jerarquías de memoria y coherencia de caché
- Procesadores multi-núcleo y memoria compartida
- Sistemas de memoria distribuida y clusters
- Aceleradores (GPUs, FPGAs) y computación heterogénea

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Clasificar arquitecturas paralelas usando la taxonomía de Flynn [Familiarizarse]
2. Analizar efectos de la jerarquía de memoria en el rendimiento [Evaluar]
3. Comparar sistemas multi-núcleo y de memoria distribuida [Evaluar]
4. Seleccionar aceleradores apropiados para tareas computacionales específicas [Evaluar]

2.5.2. HPC/Programación Paralela de Memoria Compartida

Temas:
Core Tier1
- Modelo de programación OpenMP
- Creación y gestión de hilos
- Primitivas de sincronización (cerrojos, barreras)
- Compartición de datos y alcance
- Optimización de rendimiento y balanceo de carga

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Implementar algoritmos paralelos usando OpenMP [Usar]
2. Gestionar hilos y paralelismo de tareas [Usar]
3. Aplicar mecanismos de sincronización correctamente [Usar]
4. Optimizar el rendimiento de memoria compartida [Usar]

2.5.3. HPC/Programación Paralela de Memoria Distribuida

Temas:
Core Tier1
- Modelo de programación MPI
- Comunicación punto a punto
- Operaciones de comunicación colectiva
- Topologías de procesos y grupos
- Análisis de escalabilidad y optimización

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Implementar algoritmos distribuidos usando MPI [Usar]
2. Manejar comunicaciones punto a punto y colectivas [Usar]
3. Diseñar topologías de procesos eficientes [Usar]
4. Analizar escalabilidad de aplicaciones distribuidas [Evaluar]

2.5.4. HPC/Programación de Aceleradores

Temas:
Core Tier1
- Arquitectura GPU y modelos de programación
- Modelo de programación CUDA
- Directivas OpenACC
- Diseño y optimización de kernels
- Gestión de memoria en aceleradores

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Programar GPUs usando CUDA [Usar]
2. Aplicar directivas OpenACC para programación de aceleradores [Usar]
3. Diseñar kernels GPU eficientes [Usar]
4. Gestionar la jerarquía de memoria GPU efectivamente [Usar]

2.5.5. HPC/Métodos Numéricos Paralelos

Temas:
Core Tier1
- Algoritmos de álgebra lineal paralela
- Solucionadores paralelos de EDPs
- Métodos de descomposición de dominio
- Balanceo de carga para computaciones numéricas
- Afinamiento de rendimiento y profiling

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Implementar algoritmos numéricos paralelos [Usar]
2. Aplicar descomposición de dominio para EDPs [Usar]
3. Balancear carga computacional entre procesadores [Usar]
4. Afinar el rendimiento de códigos numéricos paralelos [Usar]

2.6. Optimización y Aprendizaje Automático (OML)

Esta área cubre los fundamentos matemáticos y métodos computacionales para optimización, con especial énfasis en técnicas relevantes para el aprendizaje automático. Incluye optimización convexa y no convexa, optimización con restricciones, métodos de optimización a gran escala y metaheurísticas. El área conecta la teoría de optimización tradicional con aplicaciones modernas de aprendizaje automático, proporcionando las herramientas necesarias para entrenar modelos complejos y resolver problemas de optimización a gran escala eficientemente.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
2.6.1 Optimización Convexa y No Convexa (Pág. 19)			No
2.6.2 Optimización con Restricciones (Pág. 19)			No
2.6.3 Optimización para Aprendizaje Automático (Pág. 20)			No
2.6.4 Optimización a Gran Escala y Distribuida (Pág. 20)			No
2.6.5 Metaheurísticas y Optimización Bioinspirada (Pág. 20)			No

2.6.1. OML/Optimización Convexa y No Convexa

Temas:
Core Tier1

- Conjuntos y funciones convexas
- Condiciones de optimalidad
- Métodos de gradiente para optimización convexa
- Desafíos de optimización no convexa
- Análisis de convergencia

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Caracterizar problemas de optimización convexa [Evaluar]
2. Aplicar condiciones de optimalidad para verificar soluciones [Usar]
3. Implementar métodos basados en gradiente para problemas convexos [Usar]
4. Analizar desafíos en optimización no convexa [Evaluar]

2.6.2. OML/Optimización con Restricciones

Temas:
Core Tier1

- Programación lineal (método simplex)
- Programación entera
- Multiplicadores de Lagrange y condiciones KKT
- Métodos de penalización y barrera
- Teoría de dualidad

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Resolver problemas de programación lineal [Usar]
2. Formular y resolver problemas de programación entera [Usar]
3. Aplicar condiciones KKT a problemas con restricciones [Usar]
4. Usar métodos de penalización para optimización con restricciones [Usar]

2.6.3. OML/Optimización para Aprendizaje Automático

Temas:
Core Tier1

- Descenso de gradiente estocástico
- Momentum y métodos adaptativos (Adam, RMSProp)
- Regularización y dispersidad
- Optimización distribuida para AA
- Métodos de segundo orden para AA

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Implementar variantes de descenso de gradiente estocástico [Usar]
2. Aplicar métodos de optimización adaptativos [Usar]
3. Usar regularización para generalización de modelos [Usar]
4. Diseñar estrategias de optimización distribuidas [Usar]

2.6.4. OML/Optimización a Gran Escala y Distribuida

Temas:
Core Tier1

- Métodos de primer orden para problemas a gran escala
- Descenso coordenado y métodos por bloques
- Consenso y optimización distribuida
- Optimización de aprendizaje federado
- Optimización distribuida eficiente en comunicación

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Aplicar métodos de primer orden a problemas a gran escala [Usar]
2. Implementar algoritmos de descenso coordenado [Usar]
3. Diseñar algoritmos distribuidos basados en consenso [Usar]
4. Optimizar sistemas de aprendizaje federado [Usar]

2.6.5. OML/Metaheurísticas y Optimización Bioinspirada

Temas:
Core Tier1

- Algoritmos genéticos y estrategias evolutivas
- Inteligencia de enjambre (PSO, colonia de hormigas)
- Temple simulado
- Búsqueda tabú
- Metaheurísticas híbridas

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Implementar algoritmos genéticos [Usar]
2. Aplicar métodos de inteligencia de enjambre [Usar]
3. Usar temple simulado para optimización global [Usar]
4. Diseñar enfoques metaheurísticos híbridos [Usar]

2.7. Matemáticas para Ciencia de Datos y Aprendizaje Automático (DSM)

Esta área proporciona los fundamentos matemáticos subyacentes a la ciencia de datos moderna y el aprendizaje automático. Cubre la teoría de aprendizaje estadístico, métodos de aprendizaje supervisado y no supervisado, técnicas de evaluación de modelos y los principios matemáticos que gobiernan el comportamiento de los algoritmos de aprendizaje. El enfoque está en comprender por qué funcionan

los métodos de aprendizaje automático, sus garantías teóricas y sus limitaciones, en lugar de solo su aplicación.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
2.7.1 Fundamentos del Aprendizaje Estadístico (Pág. 21)			No
2.7.2 Métodos de Aprendizaje Supervisado (Pág. 21)			No
2.7.3 Métodos de Aprendizaje No Supervisado (Pág. 21)			No
2.7.4 Evaluación y Selección de Modelos (Pág. 22)			No

2.7.1. DSM/Fundamentos del Aprendizaje Estadístico

Temas:
Core Tier1
- Compensación sesgo-varianza
- Teoría de Vapnik-Chervonenkis (VC)
- Minimización de riesgo empírico
- Cotass de generalización
- Fundamentos de teoría de aprendizaje

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Analizar la compensación sesgo-varianza en selección de modelos [Evaluar]
2. Aplicar teoría VC para analizar complejidad de modelos [Usar]
3. Implementar minimización de riesgo empírico [Usar]
4. Interpretar cotas de generalización [Familiarizarse]

2.7.2. DSM/Métodos de Aprendizaje Supervisado

Temas:
Core Tier1
- Modelos lineales y extensiones
- Máquinas de Vectores Soporte (SVM)
- Árboles de decisión y métodos de ensamblado
- Métodos de kernel
- Aprendizaje supervisado probabilístico

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Implementar y analizar modelos lineales [Usar]
2. Aplicar SVM para clasificación y regresión [Usar]
3. Construir métodos de ensamblado (bagging, boosting) [Usar]
4. Usar métodos de kernel para problemas no lineales [Usar]

2.7.3. DSM/Métodos de Aprendizaje No Supervisado

Temas:
Core Tier1
- Algoritmos de clustering (k-means, jerárquico)
- Reducción de dimensionalidad (PCA, t-SNE)
- Estimación de densidad
- Aprendizaje de variedades
- Autoencoders y aprendizaje profundo no supervisado

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Aplicar algoritmos de clustering para exploración de datos [Usar]
2. Implementar PCA y otros métodos de reducción de dimensionalidad [Usar]
3. Usar estimación de densidad para aprendizaje no supervisado [Usar]
4. Analizar técnicas de aprendizaje de variedades [Evaluar]

2.7.4. DSM/Evaluación y Selección de Modelos

Temas:
Core Tier1

- Estrategias de validación cruzada
- Métricas de rendimiento para clasificación y regresión
- Comparación de modelos y pruebas estadísticas
- Optimización de hiperparámetros
- Curvas de aprendizaje y herramientas de diagnóstico

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Implementar validación cruzada para evaluación de modelos [Usar]
2. Seleccionar métricas de rendimiento apropiadas [Evaluar]
3. Comparar modelos usando pruebas estadísticas [Usar]
4. Optimizar hiperparámetros usando métodos sistemáticos [Usar]

2.8. Matemáticas para Inteligencia Artificial (AIM)

Esta área cubre los fundamentos matemáticos avanzados de la inteligencia artificial moderna, con énfasis en la teoría del aprendizaje profundo, arquitecturas neuronales avanzadas, aprendizaje por refuerzo y modelos generativos. Proporciona la comprensión teórica necesaria para desarrollar arquitecturas de IA novedosas, analizar sus propiedades y entender sus limitaciones. El enfoque está en los principios matemáticos que permiten a las máquinas aprender representaciones complejas y tomar decisiones inteligentes.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
2.8.1 Teoría del Aprendizaje Profundo (Pág. 22)			No
2.8.2 Arquitecturas Neuronales Avanzadas (Pág. 22)			No
2.8.3 Aprendizaje por Refuerzo (Pág. 23)			No
2.8.4 Procesamiento de Lenguaje Natural con Transformers (Pág. 23)			No
2.8.5 Modelos Generativos y GANs (Pág. 23)			No

2.8.1. AIM/Teoría del Aprendizaje Profundo

Temas:
Core Tier1

- Teoremas de aproximación universal
- Paisaje de optimización de redes profundas
- Generalización en aprendizaje profundo
- Teoría de aprendizaje de representaciones
- Dinámicas de entrenamiento y flujo de gradiente

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Explicar los teoremas de aproximación universal y sus implicaciones [Familiarizarse]
2. Analizar el paisaje de optimización de redes profundas [Evaluar]
3. Estudiar fenómenos de generalización en aprendizaje profundo [Evaluar]
4. Analizar dinámicas de entrenamiento de redes neuronales [Evaluar]

2.8.2. AIM/Arquitecturas Neuronales Avanzadas

Temas:
Core Tier1

- Redes Neuronales Convolucionales (CNNs)
- Redes Neuronales Recurrentes (RNNs) y LSTMs

- Arquitecturas Transformer
- Mecanismos de atención
- Redes Neuronales de Grafos (GNNs)

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Diseñar arquitecturas CNN para tareas visuales [Usar]
2. Implementar RNNs para modelado de secuencias [Usar]
3. Aplicar arquitecturas Transformer a varios dominios [Usar]
4. Usar mecanismos de atención en redes neuronales [Usar]

2.8.3. AIM/Aprendizaje por Refuerzo

Temas:
Core Tier1

- Procesos de Decisión de Markov (MDPs)
- Iteración de valor e iteración de política
- Q-learning y aprendizaje por diferencias temporales
- Métodos de gradiente de política
- Aprendizaje por refuerzo profundo

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Formular problemas como Procesos de Decisión de Markov [Usar]
2. Implementar iteración de valor y de política [Usar]
3. Aplicar Q-learning para RL libre de modelo [Usar]
4. Usar métodos de gradiente de política [Usar]

2.8.4. AIM/Procesamiento de Lenguaje Natural con Transformers

Temas:
Core Tier1

- Arquitectura Transformer y auto-atención
- BERT y modelos de lenguaje pre-entrenados
- Modelos GPT y generación autoregresiva
- Transformers multimodales
- Variantes eficientes de transformers

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Implementar modelos transformer desde cero [Usar]
2. Afinar modelos de lenguaje pre-entrenados [Usar]
3. Aplicar modelos GPT para generación de texto [Usar]
4. Usar transformers multimodales para tareas cross-modales [Usar]

2.8.5. AIM/Modelos Generativos y GANs

Temas:
Core Tier1

- Redes Generativas Antagónicas (GANs)
- Autoencoders Variacionales (VAEs)
- Flujos normalizantes
- Modelos de difusión
- Evaluación de modelos generativos

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Implementar GANs para generación de imágenes [Usar]
2. Aplicar VAEs para aprendizaje de representaciones latentes [Usar]
3. Usar modelos de difusión para generación [Usar]

4. Evaluar la calidad de modelos generativos [Evaluar]

2.9. Computación Simbólica y Discreta (SDC)

Esta área conecta disciplinas matemáticas clásicas con métodos computacionales, enfocándose en computación simbólica, algoritmos algebraicos, aplicaciones de teoría de números y lógica computacional. Cubre la manipulación de expresiones matemáticas en forma exacta, los aspectos computacionales de las matemáticas discretas y la aplicación de estas técnicas a criptografía y verificación formal. Este conocimiento es esencial para desarrollar sistemas de álgebra computacional, protocolos criptográficos y software formalmente verificado.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
2.9.1 Teoría de Grupos y Algoritmos Algebraicos (Pág. 24)			No
2.9.2 Teoría de Números Computacional y Criptografía (Pág. 24)			No
2.9.3 Sistemas de Computación Simbólica (Pág. 24)			No
2.9.4 Lógica Computacional y Verificación Formal (Pág. 25)			No

2.9.1. SDC/Teoría de Grupos y Algoritmos Algebraicos

Temas:
Core Tier1

- Fundamentos de teoría de grupos y representación computacional
- Algoritmos algebraicos para operaciones de grupos
- Grupos de permutación y simetría computacional
- Aplicaciones en criptografía y teoría de códigos
- Complejidad de algoritmos algebraicos

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Implementar operaciones fundamentales de grupos computacionalmente [Usar]
2. Analizar algoritmos algebraicos para teoría de grupos [Evaluar]
3. Aplicar grupos de permutación a problemas de simetría [Usar]
4. Usar teoría de grupos en aplicaciones criptográficas [Usar]

2.9.2. SDC/Teoría de Números Computacional y Criptografía

Temas:
Core Tier1

- Aritmética modular y algoritmos
- Pruebas de primalidad y factorización
- Criptografía de curva elíptica
- Problema del logaritmo discreto
- Fundamentos de criptografía post-cuántica

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Implementar algoritmos de aritmética modular [Usar]
2. Aplicar métodos de prueba de primalidad [Usar]
3. Usar curvas elípticas en protocolos criptográficos [Usar]
4. Analizar sistemas basados en logaritmo discreto [Evaluar]

2.9.3. SDC/Sistemas de Computación Simbólica

Temas:
Core Tier1

- Sistemas de álgebra computacional (Mathematica, SymPy)
- Integración y diferenciación simbólica

- Resolución de ecuaciones y simplificación
- Demostración de teoremas y razonamiento automatizado
- Aplicaciones en computación científica

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Usar sistemas de álgebra computacional para computación matemática [Usar]
2. Implementar algoritmos de computación simbólica [Usar]
3. Aplicar técnicas de demostración de teoremas [Usar]
4. Conectar computación simbólica y numérica [Evaluar]

2.9.4. SDC/Lógica Computacional y Verificación Formal

Temas:
Core Tier1

- Lógica proposicional y de predicados
- Solucionadores SAT y satisfacibilidad
- Verificación de modelos y lógica temporal
- Verificación de programas y lógica de Hoare
- Demostradores de teoremas interactivos

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Aplicar lógica computacional a resolución de problemas [Usar]
2. Usar solucionadores SAT para satisfacción de restricciones [Usar]
3. Implementar algoritmos de verificación de modelos [Usar]
4. Verificar programas usando métodos formales [Usar]

2.10. Señales, Imágenes y Visualización (SIV)

Esta área cubre los fundamentos matemáticos y computacionales para procesar, analizar y visualizar señales, imágenes y datos. Incluye análisis de Fourier y wavelets, procesamiento digital de imágenes, algoritmos de visión computacional, técnicas de visualización científica y gráficos por computadora. El enfoque está en desarrollar la comprensión matemática y las habilidades computacionales necesarias para extraer información de datos sensoriales y crear representaciones visuales efectivas de información compleja.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
2.10.1 Análisis de Fourier y Wavelets (Pág. 25)			No
2.10.2 Procesamiento Digital de Imágenes y Visión Computacional (Pág. 26)			No
2.10.3 Visualización Científica y Computación Visual (Pág. 26)			No
2.10.4 Gráficos por Computadora y Geometría Computacional (Pág. 26)			No

2.10.1. SIV/Análisis de Fourier y Wavelets

Temas:
Core Tier1

- Series y transformadas de Fourier
- Transformada de Fourier Discreta (DFT) y FFT
- Transformadas wavelet y análisis multirresolución
- Diseño de filtros y procesamiento de señales
- Aplicaciones en análisis de señales e imágenes

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Implementar algoritmos FFT para procesamiento de señales [Usar]

2. Aplicar transformadas wavelet para análisis multiescala [Usar]
3. Diseñar filtros digitales para procesamiento de señales [Usar]
4. Analizar señales en el dominio de la frecuencia [Evaluar]

2.10.2. SIV/Procesamiento Digital de Imágenes y Visión Computacional

Temas:
Core Tier1
- Representación de imágenes y espacios de color
- Filtrado y mejora de imágenes
- Segmentación de imágenes y detección de características
- Procesamiento morfológico de imágenes
- Algoritmos de visión computacional

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Implementar algoritmos de filtrado de imágenes [Usar]
2. Aplicar técnicas de segmentación de imágenes [Usar]
3. Usar operaciones morfológicas para análisis de imágenes [Usar]
4. Desarrollar aplicaciones de visión computacional [Usar]

2.10.3. SIV/Visualización Científica y Computación Visual

Temas:
Core Tier1
- Principios de visualización y percepción
- Visualización de flujo y campos vectoriales
- Renderizado volumétrico e isosuperficies
- Visualización de información
- Analítica visual y exploración interactiva

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Diseñar visualizaciones científicas efectivas [Usar]
2. Implementar técnicas de visualización de flujo [Usar]
3. Aplicar renderizado volumétrico a datos científicos [Usar]
4. Crear herramientas de analítica visual interactiva [Usar]

2.10.4. SIV/Gráficos por Computadora y Geometría Computacional

Temas:
Core Tier1
- Pipeline de renderizado y transformaciones
- Algoritmos geométricos y estructuras de datos
- Representación de curvas y superficies
- Detección de colisiones y geometría computacional
- Gráficos en tiempo real y programación GPU

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Implementar algoritmos básicos de renderizado [Usar]
2. Aplicar algoritmos de geometría computacional [Usar]
3. Diseñar curvas y superficies usando representaciones matemáticas [Usar]
4. Desarrollar aplicaciones gráficas aceleradas por GPU [Usar]

2.11. Computación Cuántica y Algoritmos (QCA)

Esta área introduce los principios fundamentales de la computación cuántica y los algoritmos cuánticos, tendiendo un puente entre la mecánica cuántica y las matemáticas computacionales. Cubre

bits cuánticos, compuertas cuánticas, circuitos cuánticos y el desarrollo de algoritmos cuánticos que ofrecen posibles aceleraciones sobre enfoques clásicos. El enfoque está en comprender los fundamentos matemáticos de la computación cuántica e implementar algoritmos cuánticos usando herramientas de simulación clásica.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
2.11.1 Fundamentos de Mecánica Cuántica para Computación (Pág. 27)			No
2.11.2 Qubits y Circuitos Cuánticos (Pág. 27)			No
2.11.3 Algoritmos Cuánticos (Pág. 27)			No
2.11.4 Criptografía Post-Cuántica y Simulación Cuántica (Pág. 28)			No

2.11.1. QCA/Fundamentos de Mecánica Cuántica para Computación

Temas:
Core Tier1
- Estados cuánticos y vectores de estado
- Superposición y entrelazamiento
- Medición cuántica y observables
- Espacios de Hilbert y notación de Dirac
- Evolución unitaria y dinámica cuántica

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Describir estados cuánticos usando notación de Dirac [Familiarizarse]
2. Explicar fenómenos de superposición y entrelazamiento [Familiarizarse]
3. Aplicar principios de medición cuántica [Usar]
4. Analizar transformaciones unitarias en sistemas cuánticos [Evaluar]

2.11.2. QCA/Qubits y Circuitos Cuánticos

Temas:
Core Tier1
- Representación de qubits y esfera de Bloch
- Compuertas y circuitos cuánticos
- Conjuntos universales de compuertas cuánticas
- Ruido cuántico y modelos de error
- Simulación de circuitos cuánticos

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Representar qubits usando formalismo matemático [Usar]
2. Diseñar circuitos cuánticos usando compuertas estándar [Usar]
3. Implementar conjuntos universales de compuertas cuánticas [Usar]
4. Simular circuitos cuánticos en computadoras clásicas [Usar]

2.11.3. QCA/Algoritmos Cuánticos

Temas:
Core Tier1
- Algoritmo de Deutsch-Jozsa
- Algoritmo de búsqueda de Grover
- Algoritmo de factorización de Shor
- Transformada Cuántica de Fourier
- Algoritmos de caminata cuántica

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Implementar el algoritmo de Deutsch-Jozsa [Usar]

2. Aplicar el algoritmo de Grover para problemas de búsqueda [Usar]
3. Analizar el algoritmo de factorización de Shor [Evaluar]
4. Usar la Transformada Cuántica de Fourier en algoritmos [Usar]

2.11.4. QCA/Criptografía Post-Cuántica y Simulación Cuántica

Temas:
Core Tier1
- Amenaza cuántica a la criptografía clásica
- Criptografía basada en retículos
- Criptografía basada en códigos
- Simulación cuántica de sistemas físicos
- Corrección de errores cuánticos

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Analizar amenazas cuánticas a sistemas criptográficos [Evaluar]
2. Implementar esquemas criptográficos basados en retículos [Usar]
3. Simular sistemas cuánticos usando computadoras clásicas [Usar]
4. Aplicar códigos de corrección de errores cuánticos [Usar]

2.12. Ciberseguridad y Privacidad (CSP)

Esta área cubre los fundamentos matemáticos de las tecnologías modernas de ciberseguridad y preservación de la privacidad. Incluye protocolos criptográficos, análisis de seguridad, tecnologías de mejora de la privacidad y aplicaciones de teoría de la información. El enfoque está en desarrollar la comprensión matemática necesaria para diseñar, analizar e implementar sistemas seguros y algoritmos de preservación de la privacidad en el contexto de las matemáticas computacionales.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
2.12.1 Criptografía Simétrica (Pág. 28)			No
2.12.2 Criptografía Asimétrica (Pág. 28)			No
2.12.3 Computación con Preservación de la Privacidad (Pág. 29)			No
2.12.4 Teoría de la Información y Compresión de Datos (Pág. 29)			No

2.12.1. CSP/Criptografía Simétrica

Temas:
Core Tier1
- Cifrados de bloque y modos de operación
- Cifrados de flujo
- Funciones hash criptográficas
- Códigos de autenticación de mensajes
- Técnicas de criptoanálisis

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Implementar algoritmos de cifrado de bloque [Usar]
2. Aplicar modos de operación para cifrado [Usar]
3. Usar funciones hash criptográficas [Usar]
4. Analizar ataques de criptoanálisis [Evaluar]

2.12.2. CSP/Criptografía Asimétrica

Temas:
Core Tier1
- Principios de criptografía de clave pública

- Criptosistema RSA
- Criptografía de curva elíptica
- Intercambio de clave Diffie-Hellman
- Firmas digitales

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Implementar cifrado y descifrado RSA [Usar]
2. Aplicar criptografía de curva elíptica [Usar]
3. Usar Diffie-Hellman para intercambio seguro de claves [Usar]
4. Implementar esquemas de firma digital [Usar]

2.12.3. CSP/Computación con Preservación de la Privacidad

Temas:
Core Tier1

- Privacidad diferencial
- Cifrado homomórfico
- Computación multipartita segura
- Privacidad en aprendizaje federado
- Métricas y análisis de privacidad

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Aplicar mecanismos de privacidad diferencial [Usar]
2. Usar cifrado homomórfico para computación privada [Usar]
3. Implementar protocolos de computación multipartita segura [Usar]
4. Analizar garantías de privacidad de algoritmos [Evaluar]

2.12.4. CSP/Teoría de la Información y Compresión de Datos

Temas:
Core Tier1

- Entropía y medidas de información
- Teorema de codificación de fuente
- Capacidad de canal y canales ruidosos
- Algoritmos de compresión
- Seguridad criptográfica y teoría de la información

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Calcular entropía e información mutua [Usar]
2. Aplicar codificación de fuente a compresión de datos [Usar]
3. Analizar capacidad de canal para comunicación [Evaluar]
4. Implementar algoritmos de compresión [Usar]

2.13. Matemáticas Computacionales para Dominios Específicos (CDM)

Esta área aplica métodos de matemáticas computacionales a dominios de aplicación específicos, demostrando cómo el modelado matemático, la simulación y las técnicas de análisis de datos se utilizan para resolver problemas del mundo real en varios campos científicos y de ingeniería. Cubre biología computacional, finanzas cuantitativas, física computacional, ciencia de materiales y modelado climático, proporcionando a los estudiantes conocimiento específico del dominio y habilidades interdisciplinarias de resolución de problemas.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
2.13.1 Bioinformática y Biología Computacional (Pág. 30)			No
2.13.2 Finanzas Computacionales y Ciencia del Riesgo (Pág. 30)			No
2.13.3 Física Computacional y Química Cuántica (Pág. 30)			No
2.13.4 Ciencia de Materiales Computacional (Pág. 31)			No
2.13.5 Modelado Climático y Sistemas Complejos (Pág. 31)			No

2.13.1. CDM/Bioinformática y Biología Computacional

Temas:
Core Tier1
- Alineamiento y análisis de secuencias
- Análisis de datos genómicos
- Predicción de estructura de proteínas
- Biología de sistemas y análisis de redes
- Construcción de árboles filogenéticos

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Implementar algoritmos de alineamiento de secuencias [Usar]
2. Analizar conjuntos de datos genómicos usando métodos estadísticos [Usar]
3. Modelar estructuras de proteínas usando métodos computacionales [Usar]
4. Aplicar enfoques de biología de sistemas [Usar]

2.13.2. CDM/Finanzas Computacionales y Ciencia del Riesgo

Temas:
Core Tier1
- Modelos de valoración de opciones
- Optimización de portafolios
- Gestión de riesgos y valor en riesgo
- Cálculo estocástico en finanzas
- Estrategias de trading algorítmico

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Valorar opciones usando métodos numéricos [Usar]
2. Optimizar portafolios de inversión [Usar]
3. Calcular medidas de riesgo y valor en riesgo [Usar]
4. Aplicar cálculo estocástico al modelado financiero [Usar]

2.13.3. CDM/Física Computacional y Química Cuántica

Temas:
Core Tier1
- Simulación de dinámica molecular
- Cálculos de química cuántica
- Métodos Monte Carlo en física
- Simulación de mecánica de medios continuos
- Cálculo de campos electromagnéticos

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Simular dinámica molecular [Usar]
2. Implementar algoritmos de química cuántica [Usar]
3. Aplicar métodos Monte Carlo a sistemas físicos [Usar]
4. Modelar problemas de mecánica de medios continuos [Usar]

2.13.4. CDM/Ciencia de Materiales Computacional

Temas:
Core Tier1

- Predicción de estructura cristalina
- Modelado de campo de fase
- Cálculos de Teoría del Funcional de la Densidad (DFT)
- Modelado multiescala de materiales
- Informática de materiales y aprendizaje automático

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Predecir estructuras cristalinas usando métodos computacionales [Usar]
2. Aplicar modelos de campo de fase para evolución de materiales [Usar]
3. Implementar cálculos DFT para estructura electrónica [Usar]
4. Usar enfoques de modelado multiescala [Usar]

2.13.5. CDM/Modelado Climático y Sistemas Complejos

Temas:
Core Tier1

- Modelado del sistema climático
- Dinámica de fluidos y modelos atmosféricos
- Sistemas complejos y teoría de redes
- Modelado de ecosistemas
- Cuantificación de incertidumbre en modelos climáticos

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Modelar sistemas climáticos usando ecuaciones diferenciales [Usar]
2. Simular dinámica de fluidos para modelado atmosférico [Usar]
3. Analizar sistemas complejos usando teoría de redes [Evaluar]
4. Cuantificar incertidumbre en modelos ambientales [Usar]

2.14. Gobernanza y Gestión de Datos (DGM)

Esta área cubre los aspectos éticos, gerenciales y organizacionales de la ciencia de datos y las matemáticas computacionales. Aborda el desarrollo y despliegue responsable de sistemas de IA, metodologías de gestión de proyectos para proyectos intensivos en datos, marcos de gobernanza de datos y cumplimiento regulatorio. El enfoque está en desarrollar las habilidades profesionales necesarias para liderar proyectos de matemáticas computacionales de manera responsable y efectiva en contextos del mundo real.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
2.14.1 Ética en IA y Algoritmos Responsables (Pág. 31)			No
2.14.2 Gestión de Proyectos de Datos y MLOps (Pág. 32)			No
2.14.3 Gobernanza de Datos y Cumplimiento Normativo (Pág. 32)			No

2.14.1. DGM/Ética en IA y Algoritmos Responsables

Temas:
Core Tier1

- Equidad algorítmica y mitigación de sesgos
- Transparencia e IA explicable (XAI)
- Responsabilidad y rendición de cuentas en sistemas de IA
- Marcos éticos para el desarrollo de IA
- Principios FAT/ML (Equidad, Responsabilidad, Transparencia)

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Detectar y mitigar sesgos algorítmicos [Usar]
2. Implementar técnicas de IA explicable [Usar]
3. Aplicar marcos éticos a proyectos de IA [Evaluar]
4. Evaluar sistemas usando principios FAT/ML [Evaluar]

2.14.2. DGM/Gestión de Proyectos de Datos y MLOps

Temas:
Core Tier1
- MLOps y gestión del ciclo de vida de modelos
- Metodologías de gestión de proyectos de datos
- Control de versiones para modelos y datos
- Despliegue y monitoreo de modelos
- Infraestructura para ciencia de datos

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Implementar prácticas MLOps [Usar]
2. Gestionar proyectos de ciencia de datos efectivamente [Usar]
3. Desplegar y monitorear modelos de aprendizaje automático [Usar]
4. Configurar infraestructura para ciencia de datos [Usar]

2.14.3. DGM/Gobernanza de Datos y Cumplimiento Normativo

Temas:
Core Tier1
- Marcos de gobernanza de datos
- GDPR y regulaciones de protección de datos
- Gestión de calidad de datos
- Ética de datos y administración
- Monitoreo y auditoría de cumplimiento

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Implementar marcos de gobernanza de datos [Usar]
2. Aplicar GDPR y regulaciones de privacidad [Usar]
3. Gestionar calidad de datos a lo largo del ciclo de vida [Usar]
4. Garantizar cumplimiento normativo en proyectos de datos [Usar]

Capítulo 3

Cuerpo del conocimiento de Ciencia de la Computación–*Computer Science*

Las 17 áreas de conocimiento en Ciencia de la Computación–*Computer Science* son:

3.1 Inteligencia Artificial (AI) (Pág. 34)
- 3.1.1 Introducción: Problemas Fundamentales (Pág. 35)
- 3.1.2 Búsqueda (Pág. 36)
- 3.1.3 Representación del Conocimiento y Razonamiento Fundamental (Pág. 37)
- 3.1.4 Aprendizaje Automático (Pág. 38)
- 3.1.5 Aplicaciones e Impacto Social (Pág. 39)
- 3.1.6 Representación Lógica y Razonamiento (Pág. 40)
- 3.1.7 Representación Probabilística y Razonamiento (Pág. 41)
- 3.1.8 Planificación (Pág. 41)
- 3.1.9 Agentes y Sistemas Cognitivos (Pág. 42)
- 3.1.10 Procesamiento del Lenguaje Natural (Pág. 43)
- 3.1.11 Robótica (Pág. 43)
- 3.1.12 Percepción y Visión por Computadora (Pág. 44)

3.2 Fundamentos Algorítmicos (AL) (Pág. 45)
- 3.2.1 Estructuras de Datos y Algoritmos Fundamentales (Pág. 45)
- 3.2.2 Estrategias Algorítmicas (Pág. 47)
- 3.2.3 Análisis de Complejidad (Pág. 47)
- 3.2.4 Modelos Computacionales y Lenguajes Formales (Pág. 49)
- 3.2.5 Sociedad, Ética y la Profesión (Pág. 50)

3.3 Arquitectura y Organización (AR) (Pág. 50)
- 3.3.1 Lógica Digital y Sistemas Digitales (Pág. 51)
- 3.3.2 Representación de Datos a Nivel de Máquina (Pág. 52)
- 3.3.3 Organización de Máquina a Nivel de Ensamblador (Pág. 52)
- 3.3.4 Jerarquía de Memoria (Pág. 53)
- 3.3.5 Interconexión y Comunicación (Pág. 53)
- 3.3.6 Organización Funcional (Pág. 54)
- 3.3.7 Rendimiento y Eficiencia Energética (Pág. 54)
- 3.3.8 Arquitecturas Heterogéneas (Pág. 54)
- 3.3.9 Arquitecturas de Procesador Seguras (Pág. 55)
- 3.3.10 Arquitecturas Cuánticas (Pág. 55)
- 3.3.11 Problemas de Sostenibilidad (Pág. 56)

3.4 Gestión de Datos (DM) (Pág. 56)
- 3.4.1 El Rol de los Datos y el Ciclo de Vida de los Datos (Pág. 58)
- 3.4.2 Conceptos Fundamentales de Sistemas de Bases de Datos (Pág. 58)
- 3.4.3 Modelado de Datos (Pág. 59)
- 3.4.4 Bases de Datos Relacionales (Pág. 59)
- 3.4.5 Construcción de Consultas (Pág. 60)
- 3.4.6 Procesamiento de Consultas (Pág. 61)
- 3.4.7 Internos de DBMS (Pág. 61)
- 3.4.8 Sistemas NoSQL (Pág. 62)
- 3.4.9 Seguridad y Privacidad de Datos (Pág. 62)
- 3.4.10 Análisis de Datos (Pág. 63)
- 3.4.11 Bases de Datos Distribuidas/Computación en la Nube (Pág. 63)
- 3.4.12 Bases de Datos Semiestructuradas y No Estructuradas (Pág. 63)
- 3.4.13 Sociedad, Ética y Profesión (Pág. 64)

3.5 Fundamentos de los Lenguajes de Programación (FPL) (Pág. 64)
- 3.5.1 Programación Orientada a Objetos (Pág. 65)
- 3.5.2 Programación Funcional (Pág. 66)
- 3.5.3 Programación Lógica (Pág. 67)
- 3.5.4 Scripting de Shell (Pág. 68)
- 3.5.5 Programación Dirigida por Eventos y Reactiva (Pág. 68)
- 3.5.6 Computación Paralela y Distribuida (Pág. 68)
- 3.5.7 Programación Orientada a Aspectos (Pág. 69)
- 3.5.8 Sistemas de Tipos (Pág. 70)
- 3.5.9 Modelo de Ejecución y Memoria de Sistemas (Pág. 71)
- 3.5.10 Traducción y Ejecución de Lenguajes (Pág. 72)
- 3.5.11 Abstracción y Representación de Programas (Pág. 72)
- 3.5.12 Análisis Sintáctico (Pág. 73)
- 3.5.13 Análisis Semántico del Compilador (Pág. 74)
- 3.5.14 Análisis de Programas y Analizadores (Pág. 74)
- 3.5.15 Generación de Código (Pág. 75)
- 3.5.16 Comportamiento en Tiempo de Ejecución y Sistemas (Pág. 75)
- 3.5.17 Construcciones de Programación Avanzadas (Pág. 76)
- 3.5.18 Pragmática del Lenguaje (Pág. 77)
- 3.5.19 Semántica Formal (Pág. 77)
- 3.5.20 Metodologías de Desarrollo Formal (Pág. 77)
- 3.5.21 Principios de Diseño de Lenguajes de Programación (Pág. 78)
- 3.5.22 Sociedad, Ética y Profesión (Pág. 78)

3.6 Gráficos y Técnicas Interactivas (GIT) (Pág. 79)
- 3.6.1 Conceptos Fundamentales de Gráficos y Técnicas Interactivas (Pág. 79)
- 3.6.2 Visualización (Pág. 80)
- 3.6.3 Renderizado Aplicado y Técnicas (Pág. 80)
- 3.6.4 Modelado Geométrico (Pág. 80)
- 3.6.5 Sombreado y Renderizado Avanzado (Pág. 81)
- 3.6.6 Animación por Computadora (Pág. 81)
- 3.6.7 Simulación (Pág. 81)
- 3.6.8 Inmersión (Pág. 81)
- 3.6.9 Interacción (Pág. 82)
- 3.6.10 Procesamiento de Imágenes (Pág. 82)
- 3.6.11 Computación Tangible/Física (Pág. 82)
- 3.6.12 Sociedad, Ética y Profesión (Pág. 83)

3.7 Interacción Humano-Computadora (HCI) (Pág. 83)
- 3.7.1 Comprender al Usuario: Objetivos individuales e interacciones con otros (Pág. 84)
- 3.7.2 Responsabilidad y Rendición de Cuentas en el Diseño (Pág. 84)
- 3.7.3 Accesibilidad y Diseño Inclusivo (Pág. 84)
- 3.7.4 Evaluación del Diseño (Pág. 85)
- 3.7.5 Diseño de Sistemas (Pág. 85)
- 3.7.6 Sociedad, Ética y Profesión (Pág. 86)

3.8 Fundamentos Matemáticos y Estadísticos (MSF) (Pág. 86)
- 3.8.1 Matemáticas Discretas (Pág. 86)
- 3.8.2 Probabilidad (Pág. 87)
- 3.8.3 Estadística (Pág. 87)
- 3.8.4 Álgebra Lineal (Pág. 88)
- 3.8.5 Cálculo (Pág. 89)

3.9 Redes y Comunicaciones (NC) (Pág. 89)
- 3.9.1 Fundamentos de Redes y Comunicaciones (Pág. 90)
- 3.9.2 Aplicaciones en Red (Pág. 90)
- 3.9.3 Soporte de Fiabilidad (Pág. 91)
- 3.9.4 Enrutamiento y Reenvío (Pág. 91)
- 3.9.5 Comunicación de Un Solo Salto (Pág. 91)
- 3.9.6 Seguridad de Red (Pág. 92)
- 3.9.7 Soporte de Movilidad (Pág. 92)
- 3.9.8 Temas Emergentes (Pág. 92)

3.10 Sistemas Operativos (OS) (Pág. 93)
- 3.10.1 Rol y Propósito de los Sistemas Operativos (Pág. 93)
- 3.10.2 Principios del Sistema Operativo (Pág. 94)
- 3.10.3 Concurrencia (Pág. 95)
- 3.10.4 Protección y Seguridad (Pág. 95)
- 3.10.5 Planificación (Pág. 96)
- 3.10.6 Modelo de Proceso (Pág. 96)
- 3.10.7 Gestión de Memoria (Pág. 97)
- 3.10.8 Gestión de Dispositivos (Pág. 97)
- 3.10.9 API de Sistemas de Archivos e Implementación (Pág. 98)
- 3.10.10 Sistemas de Archivos Avanzados (Pág. 98)
- 3.10.11 Virtualización (Pág. 99)
- 3.10.12 Sistemas en Tiempo Real y Embebidos (Pág. 100)
- 3.10.13 Tolerancia a fallos (Pág. 100)
- 3.10.14 Sociedad, Ética y Profesión (Pág. 101)

3.11 Computación Paralela y Distribuida (PDC) (Pág. 101)
- 3.11.1 Programas (Pág. 102)
- 3.11.2 Comunicación (Pág. 103)
- 3.11.3 Coordinación (Pág. 104)
- 3.11.4 Evaluación (Pág. 105)
- 3.11.5 Algoritmos (Pág. 106)

3.12 Fundamentos de Desarrollo de Software (SDF) (Pág. 108)
- 3.12.1 Conceptos y Prácticas Fundamentales de Programación (Pág. 108)
- 3.12.2 Estructuras de Datos Fundamentales (Pág. 109)
- 3.12.3 Algoritmos (Pág. 110)
- 3.12.4 Prácticas de Desarrollo de Software (Pág. 110)
- 3.12.5 Sociedad, Ética y la Profesión (Pág. 110)

3.13 Ingeniería de Software (SE) (Pág. 111)
- 3.13.1 Trabajo en Equipo (Pág. 112)
- 3.13.2 Herramientas y Entornos (Pág. 113)
- 3.13.3 Requisitos del Producto (Pág. 113)
- 3.13.4 Diseño de Software (Pág. 114)
- 3.13.5 Construcción de Software (Pág. 115)
- 3.13.6 Verificación y Validación de Software (Pág. 117)
- 3.13.7 Refactorización y Evolución del Código (Pág. 118)
- 3.13.8 Confiabilidad del Software (Pág. 119)
- 3.13.9 Métodos Formales (Pág. 120)

3.14 Seguridad (SEC) (Pág. 120)
- 3.14.1 Fundamentos de Seguridad (Pág. 121)
- 3.14.2 Sociedad, Ética y la Profesión (Pág. 122)
- 3.14.3 Codificación Segura (Pág. 123)
- 3.14.4 Criptografía (Pág. 124)
- 3.14.5 Análisis, Diseño e Ingeniería de la Seguridad (Pág. 125)
- 3.14.6 Informática Forense (Pág. 126)
- 3.14.7 Gobierno de la Seguridad (Pág. 127)

3.15 Sociedad, Ética y la Profesión (SEP) (Pág. 128)
- 3.15.1 Contexto Social (Pág. 128)
- 3.15.2 Métodos para el Análisis Ético (Pág. 129)
- 3.15.3 Ética Profesional (Pág. 130)
- 3.15.4 Propiedad Intelectual (Pág. 131)
- 3.15.5 Privacidad y Libertades Civiles (Pág. 133)
- 3.15.6 Comunicación (Pág. 133)
- 3.15.7 Sostenibilidad (Pág. 134)
- 3.15.8 Historia de la Informática (Pág. 135)
- 3.15.9 Economías de la Informática (Pág. 136)
- 3.15.10 Políticas de Seguridad, Leyes y Delitos Informáticos (Pág. 137)
- 3.15.11 Diversidad, Equidad, Inclusión y Accesibilidad (Pág. 138)

3.16 Fundamentos de Sistemas (SF) (Pág. 139)
- 3.16.1 Visión General de los Sistemas Informáticos (Pág. 140)
- 3.16.2 Conceptos Básicos (Pág. 140)
- 3.16.3 Gestión de Recursos (Pág. 141)
- 3.16.4 Rendimiento del Sistema (Pág. 142)
- 3.16.5 Evaluación de Rendimiento (Pág. 143)
- 3.16.6 Confiabilidad del Sistema (Pág. 143)
- 3.16.7 Seguridad del Sistema (Pág. 144)
- 3.16.8 Diseño del Sistema (Pág. 144)
- 3.16.9 Sociedad, Ética y la Profesión (Pág. 145)

3.17 Desarrollo de Plataformas Especializadas (SPD) (Pág. 145)
- 3.17.1 Aspectos Comunes, Preocupaciones Compartidas (Pág. 146)
- 3.17.2 Plataformas Web (Pág. 146)
- 3.17.3 Plataformas Móviles (Pág. 147)
- 3.17.4 Plataformas Robóticas (Pág. 147)
- 3.17.5 Plataformas Embebidas (Pág. 148)
- 3.17.6 Plataformas de Juegos (Pág. 149)
- 3.17.7 Plataformas de Computación Interactiva (Pág. 150)
- 3.17.8 SPD-SEP/Móvil (Pág. 151)
- 3.17.9 SPD-SEP Web (Pág. 152)
- 3.17.10 SPD-SEP/Juegos (Pág. 152)
- 3.17.11 SPD-SEP/Robótica (Pág. 153)
- 3.17.12 SPD-SEP/Interactiva (Pág. 154)

3.1. Inteligencia Artificial (AI)

La inteligencia artificial (IA) estudia problemas que son difíciles o impracticables de resolver con enfoques algorítmicos tradicionales. Estos problemas a menudo recuerdan a aquellos que se consideran que requieren inteligencia humana, y las estrategias de solución de IA resultantes típicamente se generalizan sobre clases de problemas. Las técnicas de IA ahora son omnipresentes en la informática, apoyando aplicaciones cotidianas como el correo electrónico, las redes sociales, la fotografía, los mercados financieros y los asistentes virtuales inteligentes (por ejemplo, Siri, Alexa). Estas técnicas también se utilizan en el diseño y análisis de agentes autónomos que perciben su entorno e interactúan racionalmente con él, como vehículos autónomos y otros robots.

Tradicionalmente, la IA ha incluido una mezcla de enfoques simbólicos y subsimbólicos. Las soluciones que proporciona se basan en un amplio conjunto de esquemas de representación del conocimiento generales y especializados, mecanismos de resolución de problemas y técnicas de optimización. Estos enfoques abordan la percepción (por ejemplo, reconocimiento de voz, comprensión del lenguaje natural, visión por computadora), la resolución de problemas (por ejemplo, búsqueda, planificación, optimización), la generación (por ejemplo, narrativa, conversación, imágenes, modelos, recomendaciones), la actuación (por ejemplo, robótica, automatización de tareas, control) y las arquitecturas necesarias para soportarlas (por ejemplo, agentes únicos, sistemas multiagente). El aprendizaje automático puede

utilizarse dentro de cada uno de estos aspectos e incluso puede emplearse de extremo a extremo en todos ellos. El estudio de la Inteligencia Artificial prepara a los estudiantes para determinar cuándo un enfoque de IA es apropiado para un problema dado, identificar representaciones y mecanismos de razonamiento apropiados, implementarlos y evaluarlos tanto con respecto al rendimiento como a su impacto social más amplio.

Durante la última década, el término ”inteligencia artificial” se ha vuelto común en empresas, artículos de noticias y conversaciones cotidianas, impulsado en gran medida por una serie de aplicaciones de aprendizaje automático de alto impacto. Estos avances fueron posibles gracias a la amplia disponibilidad de grandes conjuntos de datos, el aumento de la potencia computacional y las mejoras algorítmicas. En particular, ha habido un cambio desde representaciones diseñadas manualmente hacia representaciones aprendidas automáticamente a través de la optimización sobre grandes conjuntos de datos. Los avances resultantes han puesto términos como ”redes neuronales” y ”aprendizaje profundo” en el vocabulario cotidiano. Las empresas ahora anuncian soluciones basadas en IA como valor añadido a sus servicios, de modo que ”inteligencia artificial” es ahora tanto un término técnico como una palabra de moda de marketing. Otras disciplinas, como la biología, el arte, la arquitectura y las finanzas, utilizan cada vez más técnicas de IA para resolver problemas dentro de sus disciplinas.

Por primera vez en nuestra historia, la población en general tiene acceso a herramientas sofisticadas impulsadas por IA, incluyendo herramientas para generar ensayos o poemas a partir de un indicador, obras de arte a partir de una descripción, y fotografías o videos falsos que representan personas reales. La tecnología de IA ahora se utiliza ampliamente en el comercio de acciones, la curación de nuestras noticias y feeds de redes sociales, la evaluación automatizada de solicitantes de empleo, la detección de condiciones médicas y la influencia en las sentencias penales a través de la predicción de reincidencia. En consecuencia, la tecnología de IA puede tener impactos sociales significativos y consideraciones éticas que deben entenderse y considerarse al desarrollarla y aplicarla.

Unidad de Conocimiento (KU)	**Core Tier1**	**Core Tier2**	**Alternativo**
3.1.1 Introducción: Problemas Fundamentales (Pág. 35)			No
3.1.2 Búsqueda (Pág. 36)			No
3.1.3 Representación del Conocimiento y Razonamiento Fundamental (Pág. 37)			No
3.1.4 Aprendizaje Automático (Pág. 38)			No
3.1.5 Aplicaciones e Impacto Social (Pág. 39)			No
3.1.6 Representación Lógica y Razonamiento (Pág. 40)			No
3.1.7 Representación Probabilística y Razonamiento (Pág. 41)			No
3.1.8 Planificación (Pág. 41)			No
3.1.9 Agentes y Sistemas Cognitivos (Pág. 42)			No
3.1.10 Procesamiento del Lenguaje Natural (Pág. 43)			No
3.1.11 Robótica (Pág. 43)			No
3.1.12 Percepción y Visión por Computadora (Pág. 44)			No

3.1.1. AI/Introducción: Problemas Fundamentales

Temas:
CS Core

- Visión general de los problemas de IA, Ejemplos de aplicaciones recientes exitosas de IA
- Definiciones de agentes con ejemplos (por ejemplo, reactivos, deliberativos)
- ¿Qué es el comportamiento inteligente?: 1. La prueba de Turing y sus defectos 2. Entrada y salida multimodal 3. Simulación del comportamiento inteligente 4. Razonamiento racional versus no racional
- Características del problema: 1. Totalmente observable versus parcialmente observable 2. Agente único versus multiagente 3. Determinista versus estocástico 4. Estático versus dinámico 5. Discreto versus continuo
- Naturaleza de los agentes: 1. Autonomía autónoma, semiautónoma, de iniciativa mixta 2. Reflexivos, basados en metas y basados en utilidad 3. Toma de decisiones bajo incertidumbre y

con información incompleta 4. La importancia de la percepción y las interacciones ambientales 5. Agentes basados en aprendizaje 6. Agentes corporeizados: *a*) sensores, dinámicas, efectores
- Visión general de las Aplicaciones de IA, crecimiento e impacto (económico, social, ético)

KA Core
- Practicar la identificación de características del problema en entornos de ejemplo
- Profundidad adicional sobre la naturaleza de los agentes con ejemplos
- Profundidad adicional sobre Aplicaciones de IA, Crecimiento e Impacto (económico, social, ético, seguridad)

Non Core
- Problemas filosóficos
- Historia de la IA

Aprendizaje esperado (*Learning Outcomes*):
CSCore:
1. Describir la prueba de Turing y el experimento mental de la "Habitación China" [Explicar]
2. Diferenciar entre razonamiento/comportamiento óptimo y razonamiento/comportamiento similar al humano [Evaluar]
3. Diferenciar los términos: IA, aprendizaje automático y aprendizaje profundo [Explicar]
4. Enumerar las características de un problema específico [Aplicar]

3.1.2. AI/Búsqueda

Temas:
CS Core
- Representación del espacio de estados de un problema: 1. Especificación de estados, metas y operadores 2. Factorización de estados en representaciones (espacios de hipótesis) 3. Resolución de problemas mediante búsqueda en grafos: *a*) por ejemplo, Grafos como un espacio y recorridos de árboles como exploración de ese espacio *b*) Construcción dinámica del grafo (no dada de antemano)
- Búsqueda en grafo no informada para resolución de problemas: 1. Búsqueda en amplitud 2. Búsqueda en profundidad: *a*) Con profundización iterativa 3. Búsqueda de costo uniforme
- Búsqueda en grafo heurística para resolución de problemas: 1. Construcción heurística y admisibilidad 2. Ascenso de colinas 3. Mínimos locales y el paisaje de búsqueda: *a*) Soluciones locales versus globales 4. Búsqueda primero el mejor voraz 5. Búsqueda A*
- Complejidades de espacio y tiempo de los algoritmos de búsqueda en grafos

KA Core
- Búsqueda bidireccional
- Búsqueda en haz
- Juegos adversariales de dos jugadores: 1. Búsqueda minimax 2. Poda alfa-beta: *a*) Corte por profundidad
- Implementación de la búsqueda A*
- Satisfacción de restricciones

Non Core
- Comprensión del espacio de búsqueda: 1. Construcción de árboles de búsqueda 2. Espacios de búsqueda dinámicos 3. Explosión combinatoria del espacio de búsqueda 4. Topología del espacio de búsqueda (por ejemplo, crestas, puntos de silla, mínimos locales)
- Búsqueda local
- Búsqueda tabú
- Variaciones de A* (IDA*, SMA*, RBFS)
- Juegos adversariales de dos jugadores: 1. El efecto horizonte 2. Libros de apertura/soluciones de finales 3. Qué significa "resolver" un juego (por ejemplo, damas)
- Implementación de búsqueda minimax, búsqueda en haz
- Búsqueda expectimax (resolución MDP) y nodos de azar
- Búsqueda estocástica: 1. Recocido simulado 2. Algoritmos genéticos 3. Búsqueda en árbol Monte Carlo

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Diseñar la representación del espacio de estados para un rompecabezas (por ejemplo, N-reinas o problema de los 3 jarrones) [Aplicar]
2. Seleccionar e implementar un algoritmo de búsqueda no informada apropiado para un problema (por ejemplo, tres en raya), y caracterizar sus complejidades de tiempo y espacio [Aplicar]
3. Seleccionar e implementar un algoritmo de búsqueda informada apropiado para un problema después de diseñar una función heurística útil (por ejemplo, un robot navegando en un mundo de cuadrícula 2D) [Aplicar]
4. Evaluar si una heurística para un problema dado es admisible/puede garantizar una solución óptima [Evaluar]

KACore:

5. Aplicar búsqueda minimax en un juego adversarial de dos jugadores (por ejemplo, conecta 4), usando evaluación heurística en una profundidad particular para calcular las puntuaciones a propagar [Aplicar]
6. Diseñar e implementar búsqueda A*/en haz para resolver un problema, y compararla con otros algoritmos de búsqueda en términos del costo de la solución, número de nodos expandidos, etc [Aplicar]
7. Aplicar búsqueda minimax con poda alfa-beta para podar el espacio de búsqueda en un juego adversarial de dos jugadores (por ejemplo, conecta 4) [Aplicar]
8. Comparar y contrastar algoritmos genéticos con técnicas de búsqueda clásicas, explicando cuándo es más apropiado usar un algoritmo genético para aprender un modelo versus otras formas de optimización (por ejemplo, descenso de gradiente) [Evaluar]
9. Comparar y contrastar varias búsquedas heurísticas con respecto a la aplicabilidad a un problema dado [Evaluar]
10. Modelar un rompecabezas lógico o Sudoku como un problema de satisfacción de restricciones, resolverlo con búsqueda con retroceso y determinar cuánto la consistencia de arcos puede reducir el espacio de búsqueda [Aplicar]

NonCore:

11. Diseñar e implementar una solución de algoritmo genético para un problema [Aplicar]
12. Diseñar e implementar un programa de recocido simulado para evitar mínimos locales en un problema [Aplicar]

3.1.3. AI/Representación del Conocimiento y Razonamiento Fundamental

Temas:

CS Core

- Tipos de representaciones: 1. Simbólicas, lógicas: *a*) Crear una representación a partir de un enunciado de problema en lenguaje natural 2. Representaciones subsimbólicas aprendidas 3. Modelos gráficos (por ejemplo, Bayes ingenuo, red bayesiana)
- Repaso del razonamiento probabilístico, teorema de Bayes
- Razonamiento bayesiano: 1. Inferencia bayesiana

KA Core

- Variables aleatorias y distribuciones de probabilidad: 1. Axiomas de probabilidad 2. Inferencia probabilística 3. Regla de Bayes (derivación) 4. Inferencia bayesiana (ejemplos más complejos)
- Independencia
- Independencia condicional
- Cadenas de Markov y modelos de Markov
- Utilidad y toma de decisiones

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Dado un enunciado de problema en lenguaje natural, codificarlo como una representación simbólica o lógica [Aplicar]
2. Explicar cómo podemos tomar decisiones bajo incertidumbre, usando conceptos como el teorema de Bayes y la utilidad [Explicar]
3. Calcular una inferencia probabilística en un problema del mundo real usando el teorema de Bayes para determinar la probabilidad de una hipótesis dada la evidencia [Aplicar]

KACore:

4. Aplicar la regla de Bayes para determinar la probabilidad de una hipótesis dada la evidencia [Aplicar]
5. Calcular la probabilidad de resultados y probar si los resultados son independientes [Aplicar]

3.1.4. AI/Aprendizaje Automático

Temas:
CS Core

- Definición y ejemplos de una amplia variedad de tareas de aprendizaje automático: 1. Aprendizaje supervisado: *a*) Clasificación *b*) Regresión 2. Aprendizaje por refuerzo 3. Aprendizaje no supervisado: *a*) Agrupamiento
- Ideas fundamentales: 1. Teorema de no hay almuerzo gratis: ningún aprendiz puede resolver todos los problemas; las decisiones de diseño de representación tienen consecuencias. 2. Fuentes de error e indecidibilidad en el aprendizaje automático
- Un aprendizaje supervisado simple basado en estadísticas como regresión lineal o árboles de decisión: 1. Enfocarse en cómo funcionan sin entrar en detalles matemáticos o de optimización; suficiente para entender y usar implementaciones existentes correctamente
- El problema del sobreajuste/control de la complejidad de la solución (regularización, poda - solo intuición): 1. El compromiso entre sesgo (subajuste) - varianza (sobreajuste)
- Trabajar con Datos: 1. Preprocesamiento de datos: *a*) Importancia y peligros de las elecciones de preprocesamiento 2. Manejo de valores faltantes (imputación, marcar como faltante): *a*) Implicaciones de imputar versus marcar como faltante 3. Codificación de variables categóricas, codificación de datos de valor real 4. Normalización/estandarización 5. Énfasis en datos reales, no ejemplos de libros de texto
- Representaciones: 1. Espacios de hipótesis y complejidad 2. Expansión de características de base simple, como elevar al cuadrado características univariadas 3. Representaciones de características aprendidas
- Evaluación del aprendizaje automático: 1. Separación de conjuntos de entrenamiento, validación y prueba 2. Métricas de rendimiento para clasificadores 3. Estimación del rendimiento en prueba en datos retenidos 4. Ajuste de los parámetros de un modelo de aprendizaje automático con un conjunto de validación 5. Importancia de entender qué está haciendo un modelo, dónde están sus peligros/limitaciones y las implicaciones de sus decisiones
- Redes neuronales básicas: 1. Fundamentos de comprensión de cómo funcionan las redes neuronales y su proceso de entrenamiento, sin detalles de los cálculos 2. Introducción básica a redes neuronales generativas (por ejemplo, modelos de lenguaje grandes)
- Ética para el Aprendizaje Automático: 1. Enfocarse en datos reales, escenarios reales y estudios de caso 2. Sesgo de conjunto de datos/algorítmico/de evaluación y consecuencias no deseadas

KA Core

- Formulación del aprendizaje automático simple como un problema de optimización, como regresión lineal por mínimos cuadrados o regresión logística: 1. Función objetivo 2. Descenso de gradiente 3. Regularización para evitar sobreajuste (formulación matemática)
- Conjuntos de modelos: 1. Combinación de mayoría ponderada simple
- Aprendizaje profundo: 1. Redes feed-forward profundas (solo intuición, sin matemáticas) 2. Redes neuronales convolucionales (solo intuición, sin matemáticas) 3. Visualización de representaciones de características aprendidas de redes profundas 4. Otras arquitecturas (NN generativas, NN recurrentes, transformers, etc.)
- Evaluación de rendimiento: 1. Otras métricas para clasificación (por ejemplo, error, precisión, exhaustividad) 2. Métricas de rendimiento para regresores 3. Matriz de confusión 4. Validación cruzada: *a*) Ajuste de parámetros (búsqueda en cuadrícula/aleatoria, mediante validación cruzada)
- Visión general de los métodos de aprendizaje por refuerzo
- Dos o más aplicaciones de algoritmos de aprendizaje automático: 1. Por ejemplo, medicina y salud, economía, visión, lenguaje natural, robótica, juego
- Ética para el Aprendizaje Automático: 1. Continuar enfocándose en datos reales, escenarios reales y estudios de caso 2. Privacidad 3. Equidad 4. Propiedad intelectual 5. Explicabilidad

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Describir las diferencias entre los tres estilos principales de aprendizaje (supervisado, por refuerzo y no supervisado) y determinar cuál es apropiado para un dominio de problema particular [Explicar]
2. Diferenciar los términos: IA, aprendizaje automático y aprendizaje profundo [Evaluar]
3. Formular una aplicación como un problema de clasificación, incluyendo las características de entrada disponibles y la salida a predecir (por ejemplo, identificar caracteres alfabéticos a partir de entrada de cuadrícula de píxeles) [Aplicar]
4. Aplicar dos o más algoritmos de aprendizaje estadístico simple a una tarea de clasificación y medir la precisión de los clasificadores [Aplicar]
5. Identificar el sobreajuste en el contexto de un problema y curvas de aprendizaje y describir soluciones al sobreajuste [Evaluar]

KACore:

6. Explicar cómo funciona el aprendizaje automático como un proceso de optimización/búsqueda [Explicar]
7. Implementar un algoritmo de aprendizaje estadístico y el proceso de optimización correspondiente para entrenar el clasificador y obtener una predicción en nuevos datos [Aplicar]
8. Describir el proceso de entrenamiento de redes neuronales y las representaciones aprendidas resultantes [Explicar]
9. Explicar los procedimientos adecuados de evaluación de AA, incluyendo las diferencias entre rendimiento en entrenamiento y prueba, y qué puede salir mal con el proceso de evaluación llevando a informes inexactos del rendimiento de AA [Explicar]
10. Comparar dos algoritmos de aprendizaje automático en un conjunto de datos, implementando la metodología de preprocesamiento de datos y evaluación (por ejemplo, métricas y manejo de divisiones entrenamiento/prueba) desde cero [Aplicar]
11. Visualizar el progreso del entrenamiento de una red neuronal a través de curvas de aprendizaje en un kit de herramientas establecido (por ejemplo, TensorBoard) y visualizar las características aprendidas de la red [Aplicar]
12. Comparar y contrastar varias técnicas de aprendizaje (por ejemplo, árboles de decisión, regresión logística, Bayes ingenuo, redes neuronales y redes de creencia), proporcionando ejemplos de cuándo cada estrategia es superior [Comparar]
13. Evaluar el rendimiento de un sistema de aprendizaje simple en un conjunto de datos del mundo real [Evaluar]

NonCore:

14. Caracterizar el estado del arte en teoría del aprendizaje, incluyendo sus logros y limitaciones [Explicar]
15. Explicar el problema del sobreajuste, junto con técnicas para detectar y manejar el problema [Explicar]
16. Explicar el triple compromiso entre el tamaño de un espacio de hipótesis, el tamaño del conjunto de entrenamiento y la precisión del rendimiento [Explicar]
17. Dada una aplicación real de aprendizaje automático, describir problemas éticos respecto a las elecciones de datos, pasos de preprocesamiento, selección de algoritmo y visualización/presentación de resultados [Aplicar]

3.1.5. AI/Aplicaciones e Impacto Social

Nota: Hay un beneficio sustancial en estudiar aplicaciones y ética/equidad/confianza/explicabilidad en un plan de estudios junto con los métodos y teoría que aplican, en lugar de cubrir la ética en una sesión de clase separada y dedicada. Siempre que sea posible, el estudio de estos temas debe integrarse junto con otros módulos, como explorar cómo los árboles de decisión podrían aplicarse a un problema específico en sostenibilidad ambiental como la asignación de uso de la tierra, y luego evaluar las implicaciones sociales/ambientales/éticas de hacerlo.

Temas:

CS Core

- Al menos una aplicación de IA a un problema y campo específico, como medicina, salud, sostenibilidad, redes sociales, economía, educación, robótica, etc. (elegir al menos uno para el Núcleo

CS): 1. Formular y evaluar una aplicación específica como un problema de IA: *a*) Cómo lidiar con problemas mal especificados o mal planteados 2. Disponibilidad/escasez y limpieza de datos: *a*) Limpieza y preprocesamiento básico de datos *b*) Sesgo del conjunto de datos 3. Sesgo algorítmico 4. Sesgo de evaluación 5. Evaluación de las implicaciones sociales de la aplicación

- Modelos generativos profundos desplegados: 1. Visión general de alto nivel de modelos generativos de imágenes profundas (por ejemplo, a 2023, DALL-E, Midjourney, Stable Diffusion, etc.), sus usos y sus limitaciones/peligros. 2. Visión general de alto nivel de modelos de lenguaje grandes (por ejemplo, a 2023, ChatGPT, Bard, etc.), sus usos y sus limitaciones/peligros.
- Visión general del impacto social de la IA: 1. Ética 2. Equidad 3. Confianza/explicabilidad 4. Privacidad y uso de datos de entrenamiento 5. Autonomía humana y supervisión/regulaciones/requisitos legales 6. Sostenibilidad

KA Core

- Una o más aplicaciones adicionales de IA a un amplio conjunto de problemas y diversos campos, como medicina, salud, sostenibilidad, redes sociales, economía, educación, robótica, etc. (elegir un área diferente de la elegida para el Núcleo CS): 1. Formular y evaluar una aplicación específica como un problema de IA: *a*) Cómo lidiar con problemas mal especificados o mal planteados 2. Disponibilidad/escasez y limpieza de datos: *a*) Limpieza y preprocesamiento básico de datos *b*) Sesgo del conjunto de datos 3. Sesgo algorítmico 4. Sesgo de evaluación 5. Evaluación de las implicaciones sociales de la aplicación
- Profundidad adicional sobre modelos generativos profundos desplegados: 1. Introducción a cómo funcionan los modelos generativos de imágenes profundas (por ejemplo, a 2023, DALL-E, Midjourney, Stable Diffusion) incluyendo discusión de atención 2. Introducción a cómo funcionan los modelos de lenguaje grandes (por ejemplo, a 2023, ChatGPT, Bard) incluyendo discusión de atención 3. Idea de modelos fundamentales, cómo usarlos, y los beneficios/problemas con entrenarlos a partir de grandes datos
- Análisis y discusión del impacto social de la IA: 1. Ética 2. Equidad 3. Confianza/explicabilidad 4. Privacidad y uso de datos de entrenamiento 5. Autonomía humana y supervisión/regulaciones/requisitos legales 6. Sostenibilidad

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Dado un dominio de aplicación y problema del mundo real, formular una solución de IA para ello, identificando datos/entrada apropiados, preprocesamiento, representaciones, técnicas de IA y métricas/metodología de evaluación [Aplicar]
2. Analizar el impacto social de una o más aplicaciones específicas de IA del mundo real, identificando problemas respecto a ética, equidad, sesgo, confianza y explicabilidad [Evaluar]
3. Describir algunos de los modos de falla de los modelos generativos profundos actuales para lenguaje o imágenes, y cómo esto podría afectar su uso en una aplicación [Describir]

3.1.6. AI/Representación Lógica y Razonamiento

Temas:
Non Core

- Repaso de lógica proposicional y de predicados
- Resolución y demostración de teoremas (solo lógica proposicional): 1. Encadenamiento hacia adelante, encadenamiento hacia atrás
- Problemas de representación del conocimiento: 1. Lógicas de descripción 2. Ingeniería de ontologías
- Web semántica
- Razonamiento no monótono (por ejemplo, lógicas no clásicas, razonamiento por defecto)
- Argumentación
- Razonamiento sobre acción y cambio (por ejemplo, cálculo de situaciones y eventos)
- Razonamiento temporal y espacial
- Programación lógica: 1. Prolog, Programación de conjuntos de respuestas
- Sistemas expertos basados en reglas
- Redes semánticas
- Razonamiento basado en modelos y basado en casos

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Traducir una oración en lenguaje natural (por ejemplo, inglés) a una declaración de lógica de predicados [Aplicar]
2. Convertir una declaración lógica a forma causal [Aplicar]
3. Aplicar resolución a un conjunto de declaraciones lógicas para responder a una consulta [Aplicar]
4. Comparar y contrastar los modelos más comunes utilizados para la representación estructurada del conocimiento, destacando sus fortalezas y debilidades [Evaluar]
5. Identificar los componentes del razonamiento no monótono y su utilidad como mecanismo representacional para sistemas de creencias [Evaluar]
6. Comparar y contrastar las técnicas básicas para representar la incertidumbre [Evaluar]
7. Comparar y contrastar las técnicas básicas para la representación cualitativa [Evaluar]
8. Aplicar cálculo de situaciones y eventos a problemas de acción y cambio [Aplicar]
9. Explicar la distinción entre razonamiento temporal y espacial, y cómo se interrelacionan [Explicar]
10. Explicar la diferencia entre técnicas de razonamiento basadas en reglas, basadas en casos y basadas en modelos [Explicar]
11. Definir el concepto de un sistema de planificación y cómo difiere de las técnicas de búsqueda clásicas [Definir]
12. Describir las diferencias entre planificación como búsqueda, planificación basada en operadores y planificación proposicional, proporcionando ejemplos de dominios donde cada una es más aplicable [Describir]
13. Explicar la distinción entre inferencia monótona y no monótona [Explicar]

3.1.7. AI/Representación Probabilística y Razonamiento

Temas:
Non Core

- Repaso de independencia condicional
- Representaciones del conocimiento: 1. Redes bayesianas: *a*) Inferencia exacta y su complejidad *b*) Mantas de Markov y d-separación *c*) Métodos de muestreo aleatorizado (Monte Carlo) (por ejemplo, muestreo de Gibbs) 2. Redes de Markov 3. Modelos de probabilidad relacional 4. Modelos ocultos de Markov
- Teoría de la decisión: 1. Preferencias y funciones de utilidad 2. Maximización de la utilidad esperada 3. Teoría de juegos

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Calcular la probabilidad de una hipótesis dada la evidencia en una red bayesiana [Aplicar]
2. Explicar cómo las afirmaciones de independencia condicional permiten una mayor eficiencia de los sistemas probabilísticos [Explicar]
3. Identificar ejemplos de representaciones del conocimiento para razonar bajo incertidumbre [Listar/Enumerar]
4. Establecer la complejidad de la inferencia exacta. Identificar métodos para inferencia aproximada [Enunciar/Establecer]
5. Diseñar e implementar al menos una representación del conocimiento para razonar bajo incertidumbre [Aplicar]
6. Describir las complejidades del razonamiento probabilístico temporal [Describir]
7. Diseñar e implementar un HMM como un ejemplo de sistema probabilístico temporal [Aplicar]
8. Describir la relación entre preferencias y funciones de utilidad [Describir]
9. Explicar cómo las funciones de utilidad y el razonamiento probabilístico pueden combinarse para tomar decisiones racionales [Explicar]

3.1.8. AI/Planificación

Temas:
Non Core

- Repaso de lógica proposicional y de primer orden
- Operadores de planificación y representaciones de estado
- Planificación de orden total
- Planificación de orden parcial
- Grafos de plan y GraphPlan
- Planificación jerárquica
- Lenguajes y representaciones de planificación: 1. PDDL
- Planificación multiagente
- Planificación basada en MDP
- Interconexión de planificación, ejecución y replanificación dinámica: 1. Planificación condicional 2. Planificación continua 3. Planificación probabilística

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Construir la representación de estado, meta y operadores para un problema de planificación dado [Aplicar]
2. Codificar un problema de planificación en PDDL y usar un planificador para resolverlo [Aplicar]
3. Dado un conjunto de operadores, estado inicial y estado meta, dibujar el grafo de planificación de orden parcial e incluir restricciones de orden para resolver todos los conflictos [Aplicar]
4. Construir el grafo de planificación completo para que GraphPlan resuelva un problema dado [Aplicar]

3.1.9. AI/Agentes y Sistemas Cognitivos

Temas:
Non Core

- Arquitecturas de agentes (por ejemplo, reactivas, en capas, cognitivas)
- Teoría de agentes (incluyendo formalismos matemáticos)
- Racionalidad, Teoría de juegos: 1. Agentes decisorios-teóricos 2. Procesos de decisión de Markov (MDP) 3. Algoritmos de bandidos
- Agentes de software, asistentes personales y acceso a información: 1. Agentes colaborativos 2. Agentes de recolección de información 3. Agentes creíbles (personajes sintéticos, modelado de emociones en agentes)
- Agentes de aprendizaje
- Sistemas cognitivos: 1. Arquitecturas cognitivas (por ejemplo, ACT-R, SOAR, ICARUS, FORR) 2. Capacidades (por ejemplo, percepción, toma de decisiones, predicción, mantenimiento del conocimiento) 3. Representación, organización, utilización, adquisición y refinamiento del conocimiento 4. Aplicaciones y evaluación de sistemas cognitivos
- Sistemas multiagente: 1. Agentes colaboradores 2. Equipos de agentes 3. Agentes competitivos (por ejemplo, subastas, votación) 4. Sistemas de enjambre y modelos biológicamente inspirados 5. Aprendizaje multiagente
- Interacción humano-agente: 1. Metodologías de comunicación (verbal y no verbal) 2. Problemas prácticos 3. Aplicaciones: *a*) Agentes comerciales, gestión de cadena de suministro *b*) Problemas éticos de las interacciones de IA con humanos *c*) Regulación y requisitos legales de los sistemas de IA para interactuar con humanos

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Caracterizar y contrastar las arquitecturas estándar de agentes [Evaluar]
2. Describir las aplicaciones de la teoría de agentes a dominios como agentes de software, asistentes personales y agentes creíbles, y discutir las implicaciones éticas asociadas [Describir]
3. Describir los paradigmas principales utilizados por los agentes de aprendizaje [Describir]
4. Demostrar usando ejemplos apropiados cómo los sistemas multiagente soportan la interacción de agentes [Demostrar]
5. Construir un agente inteligente usando una arquitectura cognitiva establecida (ACT-R, SOAR) para resolver un problema específico [Aplicar]

3.1.10. AI/Procesamiento del Lenguaje Natural

Temas:
Non Core

- Gramáticas deterministas y estocásticas
- Algoritmos de análisis sintáctico: 1. CFGs y analizadores de tabla (por ejemplo, CYK) 2. CFGs probabilísticos y CYK ponderado
- Representación del significado/Semántica: 1. Representaciones del conocimiento basadas en lógica 2. Roles semánticos 3. Representaciones temporales 4. Creencias, deseos e intenciones
- Métodos basados en corpus
- N-gramas y HMMs
- Suavizado y retroceso
- Ejemplos de uso: Etiquetado POS y morfología
- Recuperación de información: 1. Modelo de espacio vectorial: *a*) TF & IDF 2. Precisión y exhaustividad
- Extracción de información
- Traducción de lenguaje
- Clasificación de texto, categorización: 1. Modelo de bolsa de palabras
- Aprendizaje profundo para PLN: 1. RNNs 2. Transformers 3. Incrustaciones multimodales (por ejemplo, imágenes + texto) 4. Modelos de lenguaje generativos

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Definir y contrastar gramáticas deterministas y estocásticas, proporcionando ejemplos para mostrar la adecuación de cada una [Definir]
2. Simular, aplicar o implementar algoritmos clásicos y estocásticos para analizar lenguaje natural [Aplicar]
3. Identificar los desafíos de representar el significado [Analizar]
4. Listar las ventajas de usar corpus estándar. Identificar ejemplos de corpus actuales para una variedad de tareas de PLN [Listar/Enumerar]
5. Identificar técnicas para recuperación de información, traducción de lenguaje y clasificación de texto [Analizar]
6. Implementar una transformación TF/IDF, usarla para extraer características de un corpus, y entrenar un algoritmo de aprendizaje automático estándar usando esas características para hacer clasificación de texto [Aplicar]

3.1.11. AI/Robótica

Temas:
Non Core

- Visión general: problemas y progreso: 1. Sistemas robóticos de última generación, incluyendo sus sensores y una visión general de su procesamiento sensorial 2. Arquitecturas de control de robots, por ejemplo, control deliberativo versus reactivo y vehículos de Braitenberg 3. Modelado del mundo y modelos del mundo 4. Incertidumbre inherente en la percepción y en el control
- Sensores y efectores: 1. Sensores: por ejemplo, LIDAR, sonar, visión, profundidad, estereoscópico, cámaras de eventos, micrófonos, háptica 2. Efectores: por ejemplo, ruedas, brazos, pinzas
- Marcos de coordenadas, traslación y rotación (2D y 3D)
- Espacio de configuración y mapas ambientales
- Interpretación de datos sensoriales inciertos
- Localización y mapeo
- Navegación y control
- Cinemática directa e inversa
- Planificación de trayectorias de movimiento y optimización de trayectorias
- Manipulación y agarre
- Control y dinámica de articulaciones
- Control basado en visión
- Coordinación y colaboración múltiple de robots

- Interacción humano-robot: 1. Espacios de trabajo compartidos 2. Equipos humano-robot e HRI físico 3. Robots de asistencia social 4. Predicción de movimiento/tarea/meta 5. Colaboración y comunicación (explícita vs implícita, verbal o simbólica vs no verbal o visual) 6. Confianza
- Aplicaciones y Problemas Sociales, Económicos y Éticos: 1. Implicaciones sociales, económicas, de derecho al trabajo 2. Implicaciones éticas y de privacidad de las aplicaciones robóticas 3. Responsabilidad en robótica autónoma 4. Armas autónomas y ética 5. Supervisión y control humano

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Listar capacidades y limitaciones de los sistemas robóticos de última generación actuales, incluyendo sus sensores y el procesamiento sensorial crucial que informa a esos sistemas [Listar/Enumerar]
2. Integrar sensores, actuadores y software en un robot diseñado para emprender una tarea específica [Aplicar]
3. Programar un robot para realizar tareas simples usando arquitecturas de control deliberativas, reactivas y/o híbridas [Aplicar]
4. Implementar algoritmos fundamentales de planificación de movimiento dentro de un espacio de configuración de robot [Aplicar]
5. Caracterizar las incertidumbres asociadas con sensores y actuadores robóticos comunes; articular estrategias para mitigar estas incertidumbres [Caracterizar]
6. Listar las diferencias entre las representaciones del entorno externo de los robots, incluyendo sus fortalezas y limitaciones [Listar/Enumerar]
7. Comparar y contrastar al menos tres estrategias para la navegación de robots dentro de entornos conocidos y/o desconocidos, incluyendo sus fortalezas y limitaciones [Comparar]
8. Describir al menos un enfoque para coordinar las acciones y percepción de varios robots para realizar una única tarea [Describir]
9. Comparar y contrastar un enfoque de coordinación multirobot y un enfoque de colaboración humano-robot y atribuir sus diferencias a diferencias entre los entornos del problema [Comparar]
10. Analizar los problemas sociales, económicos y éticos de una aplicación robótica del mundo real [Analizar]

3.1.12. AI/Percepción y Visión por Computadora

Temas:
Non Core

- Visión por computadora: 1. Adquisición, representación, procesamiento y propiedades de imágenes 2. Representación de formas, reconocimiento de objetos y segmentación 3. Análisis de movimiento 4. Modelos generativos
- Audio y reconocimiento de voz
- Tacto y propiocepción
- Otras modalidades (por ejemplo, olfato)
- Modularidad en el reconocimiento
- Enfoques para el reconocimiento de patrones: 1. Algoritmos de clasificación y medidas de calidad de clasificación 2. Técnicas estadísticas 3. Técnicas de aprendizaje profundo

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Resumir la importancia del reconocimiento de imágenes y objetos en IA e indicar varias aplicaciones significativas de esta tecnología [Resumir]
2. Listar al menos tres enfoques de segmentación de imágenes, como umbralización, algoritmos basados en bordes y basados en regiones, junto con sus características definitorias, fortalezas y debilidades [Listar/Enumerar]
3. Implementar reconocimiento de objetos 2D basado en representaciones de forma basadas en contorno y/o región [Aplicar]
4. Distinguir los objetivos del reconocimiento de sonido, reconocimiento de voz y reconocimiento de hablante, e identificar cómo se manejará diferente la señal de audio cruda en cada uno de estos casos [Distinguir]

5. Proporcionar al menos dos ejemplos de transformación de una fuente de datos de un dominio sensorial a otro, por ejemplo, datos táctiles interpretados como imágenes 2D de banda única [Generar]
6. Implementar un algoritmo de extracción de características en datos reales, por ejemplo, un detector de bordes o esquinas para imágenes o vectores de coeficientes de Fourier que describen un segmento corto de señal de audio [Aplicar]
7. Implementar un algoritmo que combine características en percepciones de nivel superior, por ejemplo, un contorno o polígono a partir de primitivas visuales o hipótesis de fonemas a partir de una señal de audio [Aplicar]
8. Implementar un algoritmo de clasificación que segmenta percepciones de entrada en categorías de salida y evalúa cuantitativamente la clasificación resultante [Aplicar]
9. Evaluar el rendimiento de la extracción de características subyacente, relativo a al menos un enfoque alternativo posible (ya sea implementado o no) en su contribución a la tarea de clasificación (8), anterior [Evaluar]
10. Describir al menos tres enfoques de clasificación, sus prerrequisitos para aplicabilidad, sus fortalezas y sus limitaciones [Describir]
11. Implementar y evaluar una solución de aprendizaje profundo para problemas en visión por computadora, como reconocimiento de objetos o escenas [Aplicar]

3.2. Fundamentos Algorítmicos (AL)

Los algoritmos y las estructuras de datos son fundamentales para la informática, ya que todo cálculo teórico y programa aplicado consiste en algoritmos que operan sobre elementos de datos que poseen alguna estructura subyacente. Seleccionar soluciones computacionales apropiadas para problemas del mundo real se beneficia de comprender las capacidades y limitaciones teóricas y prácticas de los algoritmos y paradigmas disponibles, incluyendo su impacto en el medio ambiente y la sociedad. Además, esta comprensión proporciona una visión de la naturaleza intrínseca de la computación, los problemas computacionales y la resolución de problemas computacionales, así como de las posibles técnicas de solución independientes del lenguaje de programación, el paradigma de programación, el hardware de la computadora u otros aspectos de implementación.

Esta área de conocimiento se centra en la naturaleza de la computación, incluyendo los conceptos y habilidades necesarios para diseñar y analizar algoritmos para resolver problemas computacionales del mundo real. Complementa la implementación de algoritmos y estructuras de datos que se encuentran en el área de conocimiento de Fundamentos de Desarrollo de Software (SDF). Como los algoritmos y las estructuras de datos son esenciales en todas las áreas avanzadas de la informática, esta área proporciona los fundamentos algorítmicos que se espera que todo graduado en informática conozca. La exposición a la amplitud de estos temas fundamentales de AL está diseñada para proporcionar a los estudiantes la base para estudiar estos temas en mayor profundidad, para estudiar temas adicionales de computación y algoritmos, y para aprender algoritmos avanzados en una variedad de áreas de conocimiento de CS y disciplinas CS+X.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
3.2.1 Estructuras de Datos y Algoritmos Fundamentales (Pág. 45)			No
3.2.2 Estrategias Algorítmicas (Pág. 47)			No
3.2.3 Análisis de Complejidad (Pág. 47)			No
3.2.4 Modelos Computacionales y Lenguajes Formales (Pág. 49)			No
3.2.5 Sociedad, Ética y la Profesión (Pág. 50)			No

3.2.1. AL/Estructuras de Datos y Algoritmos Fundamentales

Temas:
CS Core

- Tipo de Dato Abstracto (ADT) y operaciones sobre un ADT: 1. Operaciones de diccionario (insertar, eliminar, encontrar)

- Arreglos: 1. Numéricos vs no numéricos, cadenas de caracteres 2. Unidimensionales (vector) vs multidimensionales (matriz)
- Registros/Estructuras/Tuplas y Objetos
- Listas enlazadas (por razones históricas): 1. Simples vs Dobles y Lineales vs Circulares
- Pilas
- Colas y deques: 1. Cola de prioridad basada en montículo
- Tablas/mapas hash: 1. Resolución de colisiones y complejidad (por ejemplo, sondeo, encadenamiento, rehash)
- Grafos (por ejemplo, [no]dirigidos, [a]cíclicos, [no]conexos y [no]ponderados): 1. Representación de grafos: lista de adyacencia vs matriz
- Árboles: 1. Binarios, n-arios y árboles de búsqueda 2. Balanceados (por ejemplo, AVL, Rojo-Negro, Montículo)
- Conjuntos
- Algoritmos de búsqueda: 1. Complejidad $O(n)$ (por ejemplo, búsqueda lineal/secuencial en arreglo/lista) 2. Complejidad $O(\log_2 n)$ (por ejemplo, búsqueda binaria) 3. Complejidad $O(\log_b n)$ (por ejemplo, búsqueda en árbol no informada en profundidad/amplitud)
- Algoritmos de ordenamiento (por ejemplo, estables, inestables): 1. Complejidad $O(n^2)$ (por ejemplo, inserción, selección) 2. Complejidad $O(n \log n)$ (por ejemplo, quicksort, merge, timesort)
- Algoritmos de grafos: 1. Camino más corto (por ejemplo, Dijkstra, Floyd) 2. Árbol de expansión mínima (por ejemplo, Prim, Kruskal)

KA Core

- Algoritmos de ordenamiento: 1. Complejidad $O(n \log n)$ heapsort 2. Pseudo $O(n)$ complejidad (por ejemplo, bucket, counting, radix)
- Algoritmos de grafos: 1. Clausura transitiva (por ejemplo, Warshall) 2. Ordenamiento topológico

- Emparejamiento: 1. Emparejamiento eficiente de cadenas (por ejemplo, Boyer-Moore, Knuth-Morris-Pratt) 2. Emparejamiento de subsecuencia común más larga 3. Emparejamiento de expresiones regulares

Non Core

- Algoritmos de criptografía (por ejemplo, SHA-256)
- Algoritmos paralelos
- Algoritmos de consenso (por ejemplo, Blockchain): 1. Prueba de trabajo vs prueba de participación
- Algoritmos de computación cuántica: 1. Basados en oráculo (por ejemplo, Deutsch-Jozsa, Bernstein-Vazirani, Simon) 2. Aceleración superpolinomial mediante QFT (por ejemplo, Shor) 3. Aceleración polinomial mediante amplificación de amplitud (por ejemplo, Grover)
- Algoritmo de Transformada Rápida de Fourier (FFT)
- Algoritmo de evolución diferencial

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Para cada ADT/Estructura de Datos en esta unidad: *a*) Explicar su definición, propiedades, representación(es) y operaciones de ADT asociadas. *b*) Explicar paso a paso cómo las operaciones de ADT asociadas con la estructura de datos la transforman. [Explicar]
2. Para cada algoritmo en esta unidad explicar paso a paso cómo opera el algoritmo [Explicar]
3. Para cada enfoque algorítmico (por ejemplo, ordenamiento) en esta unidad aplicar un ejemplo prototípico del enfoque (por ejemplo, ordenamiento por mezcla) [Aplicar]
4. Dados los requisitos para un problema, desarrollar múltiples soluciones usando varias estructuras de datos y algoritmos. Posteriormente, evaluar la idoneidad, fortalezas y debilidades seleccionando un enfoque que satisfaga mejor los requisitos [Crear]
5. Explicar cómo se maneja la evitación de colisiones y la resolución de colisiones en tablas hash [Explicar]
6. Explicar factores más allá de la eficiencia computacional que influyen en la elección de algoritmos, como el tiempo de programación, la mantenibilidad y el uso de patrones específicos de la aplicación en los datos de entrada [Explicar]
7. Explicar la propiedad de montículo y el uso de montículos como una implementación de una cola de prioridad [Explicar]

KACore:

8. Para cada uno de los algoritmos y enfoques algorítmicos en los temas del Núcleo KA: *a*) Explicar un ejemplo prototípico del algoritmo, y *b*) Explicar paso a paso cómo opera el algoritmo. [Explicar]

NonCore:

9. Una apreciación de la computación cuántica y su aplicación a ciertos problemas [Explicar]

3.2.2. AL/Estrategias Algorítmicas

Temas:

CS Core

- Paradigmas: 1. Fuerza Bruta (por ejemplo, búsqueda lineal, ordenamiento por selección, viajante de comercio, mochila) 2. Disminuir y Vencer: *a*) Por una Constante (por ejemplo, ordenamiento por inserción, ordenamiento topológico) *b*) Por un Factor Constante (por ejemplo, búsqueda binaria) *c*) Por un Tamaño Variable (por ejemplo, Euclides) 3. Dividir y Vencer (por ejemplo, búsqueda binaria, quicksort, mergesort, Strassen) 4. Voraz (por ejemplo, Dijkstra, Kruskal, Mochila) 5. Transformar y Vencer: *a*) Simplificación de instancia (por ejemplo, encontrar duplicados mediante preordenamiento de lista) *b*) Cambio de representación (por ejemplo, heapsort) *c*) Reducción de problema (por ejemplo, mínimo común múltiplo, programación lineal) *d*) Programación dinámica (por ejemplo, Floyd, Marshall, Bellman-Ford) 6. Compromisos espacio vs tiempo (por ejemplo, hash)
- Manejo del crecimiento exponencial (por ejemplo, heurística A*, ramificación y poda, retroceso)
- Iteración vs recursión (por ejemplo, factorial, búsqueda en árbol)

KA Core

- Paradigmas: 1. Algoritmos de aproximación 2. Mejora iterativa (por ejemplo, Ford-Fulkerson, simplex) 3. Algoritmos aleatorizados/estocásticos (por ejemplo, corte máximo, bolas y cubos)

Non Core

- Computación cuántica

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Para cada uno de los paradigmas en esta unidad: *a*) Explicar sus características definitorias *b*) Explicar un ejemplo que demuestre el paradigma incluyendo cómo este ejemplo satisface las características del paradigma. [Explicar]
2. Para cada uno de los algoritmos en la unidad Estructuras de Datos y Algoritmos Fundamentales, explicar el paradigma utilizado por el algoritmo y cómo ejemplifica este paradigma [Explicar]
3. Dado un algoritmo, explicar el paradigma utilizado por el algoritmo y cómo ejemplifica este paradigma [Explicar]
4. Dar un problema del mundo real, evaluar paradigmas algorítmicos apropiados y algoritmos de estos paradigmas que aborden el problema incluyendo evaluar los compromisos entre los paradigmas y algoritmos seleccionados [Evaluar]
5. Dar ejemplos de algoritmos iterativos y recursivos que resuelvan el mismo problema, explicar los beneficios y desventajas de cada enfoque [Explicar]
6. Evaluar si un enfoque voraz conduce a una solución óptima [Evaluar]
7. Explicar varios enfoques para abordar problemas computacionales cuyas soluciones algorítmicas son exponenciales [Explicar]

3.2.3. AL/Análisis de Complejidad

Temas:

CS Core

- Marco de Análisis de Complejidad: 1. Rendimiento de un algoritmo en el mejor caso, caso promedio y peor caso 2. Mediciones empíricas y relativas (Orden de Crecimiento) 3. Tamaño de entrada y operaciones primitivas 4. Eficiencia de tiempo y espacio
- Análisis de complejidad asintótica (cotas promedio y peor caso): 1. Notaciones formales Big-O, Big-Omega y Big-Theta 2. Clases de Complejidad Fundamentales y Ejemplos/Problemas

Representativos: *a*) $O(1)$ Constante (por ejemplo, acceso a arreglo) *b*) $O(\log_2 n)$ Logarítmica (por ejemplo, búsqueda binaria) *c*) $O(n)$ Lineal (por ejemplo, búsqueda lineal) *d*) $O(n \log_2 n)$ Log Lineal (por ejemplo, mergesort) *e*) $O(n^2)$ Cuadrática (por ejemplo, ordenamiento por selección) *f*) $O(n^c)$ Polinomial (por ejemplo, $O(n^3)$ eliminación gaussiana) *g*) $O(2^n)$ Exponencial (por ejemplo, Mochila, Satisfactibilidad (SAT), Viajante de Comercio (TSP), todos los subconjuntos) *h*) $O(n!)$ Factorial (por ejemplo, circuito hamiltoniano, todas las permutaciones)

- Mediciones empíricas de rendimiento
- Tractabilidad e intractabilidad: 1. Clases de Complejidad P, NP y NP-Completo 2. Problemas NP-Completos (por ejemplo, SAT, Mochila, TSP) 3. Reducciones
- Compromisos tiempo-espacio en algoritmos

KA Core

- Notaciones Little-o, Little-Omega y Little Theta
- Análisis recursivo formal
- Análisis amortizado
- Modelos de complejidad basados en Máquinas de Turing: 1. Complejidad de tiempo: *a*) Clases P, NP, NP-C y EXP *b*) Teorema de Cook-Levin 2. Complejidad de espacio: *a*) NSpace y PSpace *b*) Teorema de Savitch

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Preparar una presentación que explique a estudiantes de primer año los conceptos básicos de complejidad algorítmica incluyendo comportamiento de algoritmo en mejor caso, caso promedio y peor caso, notaciones Big-O, Omega y Theta, clases de complejidad, compromisos tiempo-espacio, medición empírica e impacto en problemas prácticos [Explicar]
2. Usando ejemplos, explicar cada una de las clases de complejidad fundamentales en esta unidad [Explicar]
3. Para cada clase de complejidad fundamental en esta unidad, explicar un algoritmo que demuestre la complejidad de tiempo de ejecución asociada [Explicar]
4. Para cada algoritmo en la unidad Estructuras de Datos y Algoritmos Fundamentales, explicar su clase de complejidad de tiempo de ejecución y por qué pertenece a esta clase [Explicar]
5. Evaluar informalmente la clase de complejidad fundamental de algoritmos simples [Evaluar]
6. Dado un problema para programar para el cual puede haber varios enfoques algorítmicos, evaluarlos y determinar cuáles son factibles, y seleccionar uno que sea óptimo en implementación y comportamiento de tiempo de ejecución [Evaluar]
7. Desarrollar estudios empíricos para determinar y validar hipótesis sobre la complejidad de tiempo de ejecución de varios algoritmos ejecutando algoritmos con entradas de varios tamaños y comparando el rendimiento real con el análisis teórico [Crear]
8. Explicar ejemplos que ilustren los compromisos tiempo-espacio de algoritmos [Explicar]
9. Explicar cómo el balance del árbol afecta la eficiencia de las operaciones de árbol de búsqueda binaria [Explicar]
10. Explicar a una audiencia no técnica la importancia de los algoritmos tratables versus intratables usando una explicación intuitiva de la complejidad Big-O [Explicar]
11. Explicar la importancia de la NP-Completitud [Explicar]
12. Explicar cómo NP-Duro es una cota inferior y NP es una cota superior para la NP-Completitud [Explicar]
13. Explicar ejemplos de problemas NP-completos [Explicar]

KACore:

14. Usar relaciones de recurrencia para evaluar la complejidad de tiempo de algoritmos definidos recursivamente [Aplicar]
15. Aplicar relaciones de recurrencia elementales usando una forma del Teorema Maestro [Aplicar]
16. Aplicar la notación Big-O para dar cotas superiores en la complejidad tiempo/espacio de algoritmos [Aplicar]
17. Explicar el Teorema de Cook-Levin y la NP-Completitud de SAT [Explicar]
18. Explicar las clases P y NP [Explicar]
19. Demostrar que un problema es NP-Completo reduciendo un problema NP-C clásico conocido a él (por ejemplo, 3SAT y Clique) [Crear]
20. Explicar la clase P-espacio y su relación con la clase EXP [Explicar]

3.2.4. AL/Modelos Computacionales y Lenguajes Formales

Temas:
CS Core

- Autómatas formales: 1. Estado Finito 2. Pila 3. Linealmente Acotado 4. Máquina de Turing
- Lenguajes formales, gramáticas y Jerarquía de Chomsky: 1. Regulares (Tipo-3): *a*) Expresiones Regulares 2. Libres de Contexto (Tipo-2) 3. Sensibles al Contexto (Tipo-1) 4. Recursivamente Enumerables (Tipo-0)
- Relaciones entre autómatas formales, lenguajes y gramáticas
- Decidabilidad, (in)computabilidad y parada
- La tesis de Church-Turing
- Corrección algorítmica: 1. Invariantes (por ejemplo, en iteración, recursión, búsqueda en árbol)

KA Core

- Autómatas deterministas y no deterministas
- Demostraciones del Lema de Bombeo: 1. Demostración de la limitación de Estado Finito/Lenguaje Regular 2. Limitación de Autómatas de Pila/Lenguaje Libre de Contexto
- Decidabilidad: 1. Aritmetización y diagonalización
- Reducibilidad y reducciones
- Complejidad de tiempo basada en Máquina de Turing
- Complejidad de espacio (por ejemplo, Pspace, Teorema de Savitch)
- Modelos equivalentes de computación algorítmica: 1. Máquinas de Turing y Variaciones (por ejemplo, multicinta, no deterministas) 2. Cálculo Lambda 3. Funciones Mu-Recursivas

Non Core

- Computación cuántica: 1. Postulados de la mecánica cuántica: *a*) Espacio de estados *b*) Evolución del estado *c*) Composición de estados *d*) Medición del estado 2. Representaciones de vector columna de qubits 3. Representaciones matriciales de operaciones cuánticas 4. Compuertas cuánticas simples (por ejemplo, XNOT, CNOT)

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Para cada autómata formal en esta unidad: *a*) Explicar su definición comparando sus características con los otros autómatas de esta unidad *b*) Usando un ejemplo, explicar paso a paso cómo el autómata opera sobre la entrada incluyendo si acepta la entrada asociada *c*) Explicar un ejemplo de entradas que pueden y no pueden ser aceptadas por el autómata. [Explicar]
2. Dado un problema, desarrollar un autómata apropiado que aborde el problema [Crear]
3. Desarrollar una expresión regular para un lenguaje regular dado expresado en lenguaje natural [Crear]
4. Explicar la diferencia entre expresiones regulares (aceptadores Tipo-3) y las expresiones regulares (aceptadores Tipo-2) usadas en lenguajes de programación [Explicar]
5. Para cada modelo formal en esta unidad: *a*) Explicar su definición comparando sus características con las otras en esta unidad *b*) Explicar ejemplos de entradas que son y no pueden ser aceptadas por el lenguaje/gramática. [Explicar]
6. Explicar una Máquina de Turing universal y su operación [Explicar]
7. Presentar a una audiencia de compañeros de trabajo y gerentes la imposibilidad de proporcionarles un programa que verifique todos los otros programas, incluyendo algunos aparentemente simples, en busca de bucles infinitos incluyendo una explicación del problema de la parada, por qué no tiene solución algorítmica y su importancia para la computación algorítmica del mundo real [Explicar]
8. Explicar ejemplos de problemas clásicos incomputables [Explicar]
9. Explicar la Tesis de Church-Turing y su importancia para la computación algorítmica [Explicar]
10. Explicar cómo los invariantes (de bucle) pueden usarse para demostrar la corrección de un algoritmo [Explicar]

KACore:

11. Para cada autómata formal en esta unidad explicar (comparar/contrastar) sus capacidades deterministas y no deterministas [Explicar]
12. Aplicar lemas de bombeo, o medios alternativos, para demostrar las limitaciones de los autómatas

de Estado Finito y Pila [Aplicar]
13. Aplicar aritmetización y diagonalización para demostrar que el Problema de la Parada para Máquinas de Turing es Indecidible [Aplicar]
14. Dado un lenguaje indecidible conocido, aplicar una reducción por mapeo o historia computacional para demostrar que otro lenguaje es indecidible [Aplicar]
15. Convertir entre notaciones equivalentemente poderosas para un lenguaje, incluyendo entre DFAs, NFAs y expresiones regulares, y entre PDAs y CFGs [Aplicar]
16. Explicar el teorema de Rice y su importancia [Explicar]
17. Explicar un ejemplo de demostración de un problema que es incomputable reduciendo un problema incomputable clásico conocido a él [Explicar]
18. Explicar las funciones Recursivas Primitivas y Generales (cero, sucesor, selección, recursión primitiva, composición y Mu), su importancia e implementaciones en Máquina de Turing [Explicar]
19. Explicar cómo se realiza la computación en el Cálculo Lambda (por ejemplo, conversión alfa y reducción beta) [Explicar]

NonCore:

20. Para un sistema cuántico dar ejemplos que expliquen los siguientes postulados: *a*) Espacio de Estados - estado del sistema representado como un vector unitario en espacio de Hilbert *b*) Evolución del Estado - el uso de operadores unitarios para evolucionar el estado del sistema *c*) Composición de Estados - el uso de producto tensorial para componer estados del sistema *d*) Medición del Estado - la salida probabilística de medir un estado del sistema. [Explicar]
21. Explicar la operación de una compuerta cuántica XNOT o CNOT sobre un bit cuántico representado como una matriz y vector columna, respectivamente [Explicar]

3.2.5. AL/Sociedad, Ética y la Profesión

Temas:

CS Core

- Algoritmos sociales, éticos y seguros
- Equidad algorítmica
- Anonimato (por ejemplo, Privacidad Diferencial)
- Responsabilidad/Transparencia
- Algoritmos responsables
- Impactos económicos y otros de algoritmos ineficientes
- Sostenibilidad

KA Core

- Computación consciente del contexto

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Desarrollar soluciones algorítmicas a problemas sociales del mundo real, como enrutar una ambulancia a un hospital [Crear]
2. Explicar el impacto que un algoritmo puede tener en el medio ambiente y la sociedad cuando se usa para resolver un problema del mundo real considerando su sostenibilidad y que puede afectar a diferentes grupos sociales de diferentes maneras [Explicar]
3. Preparar una presentación que justifique la selección de estructuras de datos y/o algoritmos apropiados para resolver un problema del mundo real dado [Explicar]
4. Explicar un ejemplo que articule cómo la privacidad diferencial protege el conocimiento de los datos de un individuo [Explicar]
5. Explicar los impactos ambientales de las decisiones de diseño que se relacionan con el diseño de algoritmos [Explicar]
6. Explicar los compromisos involucrados en los algoritmos de prueba de trabajo y prueba de participación [Explicar]

3.3. Arquitectura y Organización (AR)

Los profesionales de la informática pasan considerable tiempo escribiendo código eficiente para resolver un problema particular en un dominio de aplicación. A medida que ocurre el cambio del

procesamiento secuencial al paralelo, se requiere una comprensión más profunda de las arquitecturas de computadoras subyacentes. La arquitectura ya no puede verse como una caja negra donde los principios de una arquitectura pueden aplicarse a otra. En cambio, los programadores deben mirar dentro de la caja negra y usar componentes específicos para mejorar el rendimiento del sistema y la eficiencia energética.

El área de conocimiento de Arquitectura y Organización (AR) tiene como objetivo desarrollar una comprensión más profunda de los entornos de hardware sobre los cuales se basa casi toda la computación, y las interfaces relevantes proporcionadas a las capas de software superiores. El hardware objetivo comprende desde procesadores de sistemas embebidos de baja gama hasta multiprocesadores empresariales de alta gama.

Los temas en esta área de conocimiento beneficiarán a los estudiantes permitiéndoles apreciar los principios arquitectónicos fundamentales de los sistemas informáticos modernos, incluyendo el desafío de aprovechar el paralelismo para sostener las mejoras de rendimiento y energía en el futuro. Esta KA ayudará a los estudiantes de informática a alejarse del enfoque de caja negra y a ser más conscientes del sistema informático subyacente y las eficiencias que arquitecturas específicas pueden lograr.

Unidad de Conocimiento (KU)	**Core Tier1**	**Core Tier2**	**Alternativo**
3.3.1 Lógica Digital y Sistemas Digitales (Pág. 51)			No
3.3.2 Representación de Datos a Nivel de Máquina (Pág. 52)			No
3.3.3 Organización de Máquina a Nivel de Ensamblador (Pág. 52)			No
3.3.4 Jerarquía de Memoria (Pág. 53)			No
3.3.5 Interconexión y Comunicación (Pág. 53)			No
3.3.6 Organización Funcional (Pág. 54)			No
3.3.7 Rendimiento y Eficiencia Energética (Pág. 54)			No
3.3.8 Arquitecturas Heterogéneas (Pág. 54)			No
3.3.9 Arquitecturas de Procesador Seguras (Pág. 55)			No
3.3.10 Arquitecturas Cuánticas (Pág. 55)			No
3.3.11 Problemas de Sostenibilidad (Pág. 56)			No

3.3.1. AR/Lógica Digital y Sistemas Digitales

Temas:

KA Core

- Lógica combinacional vs secuencial/arreglos de puertas programables en campo (FPGAs): 1. Lógica combinacional fundamental 2. Bloque de construcción de lógica secuencial
- Herramientas de diseño asistido por computadora que procesan representaciones de hardware y arquitectura
- Síntesis de alto nivel: 1. Notación de transferencia de registros 2. Lenguaje de descripción de hardware (por ejemplo, Verilog/VHDL/Chisel)
- Flujo de diseño de sistema en chip (SoC)
- Restricciones físicas: 1. Retardos de puertas 2. Fan-in y fan-out 3. Energía/potencia 4. Velocidad de la luz

Aprendizaje esperado (*Learning Outcomes*):

KACore:

1. Discutir la progresión de los componentes de tecnología informática desde tubos de vacío hasta VLSI, desde arquitecturas de computadoras mainframe hasta la organización de computadoras a escala de almacén [Debatir]
2. Describir el paralelismo y las dependencias de datos entre y dentro de componentes en una arquitectura de computadora heterogénea moderna [Describir]
3. Explicar la relación entre paralelismo y consumo de energía [Explicar]
4. Construir el diseño de bloques básicos para una computadora: unidad aritmético-lógica (nivel de puertas), registros (nivel de puertas), unidad central de procesamiento (nivel de transferencia de registros) y memoria (nivel de transferencia de registros) [Crear]
5. Evaluar bloques básicos simples (por ejemplo, unidad aritmético-lógica, registros, movimiento entre registros) de un diseño de computadora simple [Evaluar]

6. Analizar el comportamiento de temporización de un procesador segmentado, identificando problemas de dependencia de datos [Analizar]

3.3.2. AR/Representación de Datos a Nivel de Máquina

Temas:
CS Core

- Visión general e historia de la arquitectura de computadoras
- Bits, bytes y palabras
- Representaciones sin signo, con signo y complemento a dos
- Representación de datos numéricos y bases numéricas: 1. Punto fijo 2. Punto flotante
- Representación de datos no numéricos
- Representación de registros, arreglos y tipos de datos UTF

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Discutir por qué todo en las computadoras son datos, incluyendo instrucciones [Debatir]
2. Explicar cómo las representaciones numéricas de longitud fija pueden afectar la exactitud y precisión [Explicar]
3. Describir cómo se almacenan los enteros negativos en representaciones de magnitud con signo y complemento a dos [Describir]
4. Discutir cómo diferentes formatos pueden representar datos numéricos [Debatir]
5. Explicar la representación a nivel de bits de datos no numéricos, como caracteres, cadenas, registros y arreglos [Explicar]
6. Traducir datos numéricos de un formato a otro [Traducir]
7. Describir cómo un sumador único (sin detección de desbordamiento) puede manejar tanto entrada con signo (complemento a dos) como sin signo (binario) sin "saber" qué formato está usando una entrada dada [Describir]

3.3.3. AR/Organización de Máquina a Nivel de Ensamblador

Temas:
CS Core

- Arquitectura de máquina von Neumann
- Unidad de control: captación, decodificación y ejecución de instrucciones
- Introducción a SIMD vs MIMD y la taxonomía de Flynn
- Organización de multiprocesadores/multinúcleo de memoria compartida

KA Core

- Arquitectura del conjunto de instrucciones (ISA) (por ejemplo, x86, ARM y RISC-V): 1. Conjuntos de instrucciones de ancho fijo vs variable 2. Formatos de instrucción 3. Manipulación de datos, control, E/S 4. Modos de direccionamiento 5. Programación en lenguaje de máquina 6. Programación en lenguaje ensamblador
- Mecanismos de llamada y retorno de subrutinas
- E/S e interrupciones
- Segmentos de montículo, estáticos, pila y código

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Discutir cómo las unidades funcionales clásicas de von Neumann se implementan en sistemas embebidos, particularmente en memoria dentro y fuera del chip [Debatir]
2. Describir cómo se ejecutan las instrucciones en una máquina von Neumann clásica, con extensiones para hilos, sincronización de multiprocesadores y ejecución SIMD [Describir]
3. Evaluar un diagrama de ejemplo con paralelismo a nivel de instrucción y riesgos para describir cómo se manejan en las canalizaciones de procesadores típicos [Evaluar (valorar)]

KACore:

4. Discutir cómo se representan las instrucciones a nivel de máquina y en el contexto de un ensamblador simbólico [Debatir]

5. Mapear un ejemplo de patrones de lenguaje de alto nivel a notaciones de lenguaje ensamblador/máquina [Mapear/Mapa conceptual]
6. Contrastar diferentes formatos de instrucción considerando aspectos como direcciones por instrucción y formatos de longitud variable vs longitud fija [Contrastar]
7. Analizar un diagrama de subrutina para comentar cómo se manejan las llamadas de subrutina a nivel de ensamblador [Analizar]
8. Describir conceptos básicos de interrupciones y operaciones de E/S [Describir]
9. Escribir un programa simple en lenguaje ensamblador para procesamiento y manipulación de cadenas/arreglos [Escribir]

3.3.4. AR/Jerarquía de Memoria

Temas:
CS Core

- Jerarquía de memoria: la importancia de la localidad temporal y espacial
- Organización y operaciones de memoria principal
- Memoria persistente (por ejemplo, SSD, discos estándar)
- Latencia, tiempo de ciclo, ancho de banda e intercalado
- Memorias caché: 1. Mapeo de direcciones 2. Tamaño de bloque 3. Política de reemplazo y almacenamiento 4. Prebúsqueda
- Coherencia de caché multiprocesador
- Memoria virtual (soporte de hardware)
- Manejo de fallas y confiabilidad
- Confiabilidad: 1. Codificación de error 2. Compresión de datos 3. Integridad de datos

KA Core

- Procesamiento en Memoria (PIM)

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Usando un diagrama del sistema de memoria, identificar los principales tipos de tecnología de memoria (por ejemplo, SRAM, DRAM) y su costo y rendimiento relativos [Analizar]
2. Medir el efecto de la latencia de memoria en el tiempo de ejecución [Evaluar]
3. Enumerar las funciones de un sistema con gestión de memoria virtual [Enumerar]
4. Calcular el tiempo promedio de acceso a memoria bajo varias configuraciones de caché y memoria y mezclas de referencias de instrucciones y datos [Computar/Calcular]

3.3.5. AR/Interconexión y Comunicación

Temas:
CS Core

- Fundamentos de E/S: 1. Protocolo de enlace y almacenamiento en búfer 2. E/S programada 3. E/S basada en interrupciones
- Estructuras de interrupción: vectorizadas y priorizadas, acuse de recibo de interrupción
- Dispositivos de E/S (por ejemplo, ratón, teclado, pantalla, cámara, sensores, actuadores)
- Almacenamiento externo, organización física y unidades
- Fundamentos de buses: 1. Protocolos de bus 2. Arbitraje 3. Acceso directo a memoria (DMA)

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Analizar un diagrama de control de interrupciones para comentar cómo se usan las interrupciones para implementar control de E/S y transferencias de datos [Analizar]
2. Enumerar varios tipos de buses en un sistema informático [Enumerar]
3. Enumerar las ventajas de los discos magnéticos y contrastarlas con las de los discos de estado sólido [Listar/Enumerar]

3.3.6. AR/Organización Funcional

Temas:
KA Core

- Implementación de rutas de datos simples, incluyendo segmentación de instrucciones, detección de riesgos y resolución (por ejemplo, paradas, reenvío)
- Unidad de control: 1. Implementación cableada 2. Realización microprogramada
- Segmentación de instrucciones
- Introducción al paralelismo a nivel de instrucción (ILP)

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Comparar implementaciones alternativas de rutas de datos en arquitecturas de computadoras modernas [Comparar]
2. Producir un conjunto de señales de control para sumar dos enteros usando implementaciones cableadas y microprogramadas [Producir]
3. Discutir el paralelismo a nivel de instrucción usando segmentación y los riesgos significativos que pueden ocurrir [Debatir]
4. Diseñar un procesador completo, incluyendo ruta de datos y control [Diseñar]
5. Calcular los ciclos promedio por instrucción para una implementación dada de procesador y sistema de memoria [Computar/Calcular]

3.3.7. AR/Rendimiento y Eficiencia Energética

Temas:
KA Core

- Evaluación de rendimiento-energía (introducción): rendimiento, consumo de energía, costos de memoria y comunicación
- Predicción de bifurcaciones, ejecución especulativa, ejecución fuera de orden, algoritmo de Tomasulo
- Mejoras para procesadores vectoriales y GPUs
- Soporte de hardware para multihilos: 1. Condiciones de carrera 2. Implementaciones de bloqueo 3. Sincronización punto a punto 4. Implementación de barreras
- Escalabilidad
- Arquitecturas alternativas incluyendo VLIW/EPIC, aceleradores y otros procesadores de propósito especial
- Escalado dinámico de voltaje y frecuencia (DVFS)
- Silicio Oscuro

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Discutir métricas de evaluación de rendimiento y eficiencia energética [Debatir]
2. Analizar un diagrama de ejecución especulativa y escribir sobre las decisiones que se pueden tomar [Analizar]
3. Crear un diagrama de evaluación comparativa de rendimiento-vatio de GPU [Crear]
4. Escribir un programa multihilo que sume (en paralelo) elementos de dos vectores enteros [Escribir]
5. Recomendar un conjunto de decisiones de diseño para arquitecturas de computadoras alternativas [Recomendar]
6. Enumerar conceptos clave asociados con el escalado dinámico de voltaje y frecuencia [Enumerar]
7. Medir la mejora en ahorro de energía para una cuantización de entero de 8 bits comparada con una cuantización de 32 bits [Evaluar]

3.3.8. AR/Arquitecturas Heterogéneas

Temas:
KA Core

- Arquitecturas SIMD y MIMD (por ejemplo, GPUs de Propósito General, TPUs y NPUs)

- Sistemas de memoria heterogéneos: 1. Memoria compartida versus memoria distribuida 2. Memoria volátil vs no volátil 3. Protocolos de coherencia
- Arquitecturas Específicas de Dominio (DSAs): 1. Acelerador de Aprendizaje Automático 2. Computación en red 3. Sistemas embebidos para aplicaciones emergentes 4. Computación neuromórfica 5. Dispositivos de computación perimetral
- Soluciones de empaquetado e integración como 3DIC y chiplets
- Aprendizaje automático en diseño de arquitectura: 1. Algoritmos de IA para análisis de carga de trabajo 2. Optimización de configuraciones de arquitectura para rendimiento y eficiencia energética

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Analizar un diagrama de sistema con arquitecturas paralelas alternativas, por ejemplo, SIMD y MIMD, e identificar las diferencias clave [Analizar]
2. Discutir qué problemas de gestión de memoria se encuentran en multiprocesadores que no están presentes en uniprocesadores y cómo estos problemas podrían resolverse [Debatir]
3. Indicar las diferencias entre placa base de memoria, interconexión de memoria del procesador y memoria remota a través de redes, sus implicaciones para la latencia de acceso y su impacto en el rendimiento del programa [Diferenciar]
4. Discutir cómo determinarías cuándo usar un acelerador específico de dominio en lugar de una CPU de propósito general [Debatir]
5. Enumerar diferencias clave en los principios de diseño arquitectónico entre una unidad de procesamiento basada en vector y basada en escalar [Enumerar]
6. Enumerar las ventajas y desventajas de una arquitectura PIM [Listar/Enumerar]

3.3.9. AR/Arquitecturas de Procesador Seguras

Temas:
KA Core

- Principios de Hardware Seguro: 1. Análisis de Riesgos de Seguridad, Protección de Activos y Modelo de Amenazas 2. Aceleración Criptográfica con Hardware 3. Soporte para virtualización (por ejemplo, aislamiento de SO)
- Raíces de confianza en hardware, Funciones Físicamente No Clonables (PUF)
- Generadores de Números Aleatorios por Hardware
- Extensiones de protección de memoria: 1. Verificación de límites de puntero en tiempo de ejecución (por ejemplo, desbordamiento de búfer) 2. Protección a nivel microarquitectónico 3. Protección a nivel de ISA
- Entorno de Ejecución Confiable (TEE): 1. Protecciones de Base de Computadora Confiable 2. Protección de máquinas virtuales 3. Protección de contenedores 4. Módulos de software confiables (Enclaves)
- Cifrado homomórfico para procesamiento de datos que preserva la privacidad

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Discutir principios de hardware seguro, explorando un marco para análisis de riesgos y protección de activos [Debatir]
2. Resumir cómo las Funciones Físicamente No Clonables (PUF) pueden ser un identificador único de dispositivo en aplicaciones de seguridad [Resumir]
3. Distinguir un generador de números aleatorios con soporte de hardware dedicado de generadores sin hardware dedicado a generar entropía [Distinguir]
4. Enumerar las ventajas y desventajas de la protección de memoria a nivel de ISA [Listar/Enumerar]
5. Describir problemas de diseño clave de un entorno de ejecución confiable (TEE) para soportar máquinas virtuales [Describir]

3.3.10. AR/Arquitecturas Cuánticas

Temas:
KA Core

- Principios: 1. El principio de dualidad onda-partícula 2. El principio de incertidumbre en el experimento de doble rendija 3. ¿Qué es un Qubit? Superposición, interferencia y medición. Fotones como qubits 4. Sistemas de dos qubits, Entrelazamiento, estados de Bell, El teorema de No Señalización
- Axiomas de la MC: principio de superposición, axioma de medición, evolución unitaria
- Compuertas de un solo qubit para el modelo de circuito de computación cuántica: X, Z, H
- Compuertas de dos qubits y productos tensoriales, trabajando con matrices
- El Teorema de No Clonación. El protocolo de Teleportación Cuántica
- Algoritmos: 1. Algoritmos cuánticos simples: Bernstein-Vazirani, algoritmo de Simon 2. Implementar Deutsch-Josza con Interferómetros Mach-Zehnder 3. Factorización cuántica (Algoritmo de Shor) 4. Búsqueda cuántica (Algoritmo de Grover)
- Aspectos de implementación: 1. La implementación física de qubits 2. Control clásico de una Unidad de Procesamiento Cuántico (QPU) 3. Mitigación y control de errores, NISQ y más allá 4. Enfoques de medición
- Aplicaciones Emergentes: 1. Cifrado post-cuántico 2. La Internet Cuántica 3. Computación cuántica adiabática (AQC) y recocido cuántico

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Discutir cómo un objeto cuántico producido como partícula se propaga como una onda y se detecta como partícula con una distribución de probabilidad correspondiente a la onda [Debatir]
2. Discutir la naturaleza a nivel cuántico que es inherentemente probabilística [Debatir]
3. Expresar tu punto de vista sobre el entrelazamiento que puede usarse para crear correlaciones no clásicas, pero no hay forma de usar el entrelazamiento cuántico para enviar mensajes más rápido que la velocidad de la luz [Evaluar]
4. Describir el paralelismo cuántico y el papel de la interferencia constructiva vs destructiva en algoritmos cuánticos dada la naturaleza probabilística de las mediciones [Describir]
5. Analizar un fragmento de código proporcionando el papel de la transformada cuántica de Fourier (QFT) en el algoritmo de Shor [Analizar]
6. Escribir un programa para implementar el algoritmo de Shor en un simulador, destacando los componentes clásicos y aspectos del algoritmo de Shor [Escribir]
7. Enumerar los detalles de cada modalidad de qubit (por ejemplo, ion atrapado, superconductor, espín de silicio, fotónico, punto cuántico, átomo neutro, topológico, centro de color, electrón sobre helio) [Enumerar]
8. Contrastar AQC con el modelo de compuertas de computación cuántica y los problemas para los que cada uno es más adecuado resolver [Contrastar]

3.3.11. AR/Problemas de Sostenibilidad

Temas:
Non Core

- Impactos ambientales de decisiones de implementación: 1. Objetivos de sostenibilidad, consumo de recursos y viabilidad económica 2. Huella de carbono, desechos electrónicos de hardware 3. La huella energética de los centros de datos en varias cargas de trabajo (por ejemplo, entrenamiento y uso de modelos de IA) 4. Directrices para estándares de diseño sostenible

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Evaluar los impactos ambientales del despliegue de un proyecto dado (por ejemplo, el consumo de energía de CPUs y GPUs, contribución a desechos electrónicos, y efecto de la virtualización de hardware en centros de datos) [Evaluar (valorar)]

3.4. Gestión de Datos (DM)

Desde mediados de la década de 1970, el estudio de la Gestión de Datos (DM) ha significado un estudio casi exclusivo de los sistemas de bases de datos relacionales. Dependiendo del contexto institucional, los estudiantes han estudiado, en proporciones variables, lo siguiente.

- Modelado de datos y diseño de bases de datos: por ejemplo, modelo de datos E-R, modelo relacional, teoría de normalización
- Construcción de consultas: por ejemplo, álgebra relacional, SQL
- Procesamiento de consultas: por ejemplo, índices (árbol B+, hash), algoritmos (por ejemplo, ordenamiento externo, selección, proyección, unión), optimización de consultas (transformaciones, selección de índices)
- Internos de DBMS: por ejemplo, concurrencia/bloqueos, gestión de transacciones, gestión de búfer

Se espera que los graduados de hoy posean habilidades de usuario (en lugar de implementador) de DBMS. Estas incluyen principalmente modelado de datos y construcción de consultas; capacidad para tomar una colección no organizada de datos, organizarla usando un DBMS y acceder/actualizar la colección mediante consultas.

Además, los estudiantes necesitan estudiar lo siguiente:

- El rol que juegan los datos en una organización. Esto incluye el Ciclo de Vida de los Datos: Creación-Procesamiento-Revisión/Reporte-Retención/Recuperación-Destrucción.
- Los aspectos sociales/legales de la recolección de datos: por ejemplo, escala, privacidad de datos, privacidad de base de datos (cumplimiento) por diseño, desidentificación, propiedad, confiabilidad, seguridad de base de datos y aplicaciones intencionales y no intencionales.
- Tecnologías emergentes y avanzadas que están aumentando/reemplazando los sistemas relacionales tradicionales, particularmente aquellas usadas para soportar análisis de datos (grandes), incluyendo NoSQL (por ejemplo, JSON, XML, bases de datos de almacén clave-valor), bases de datos en la nube, MapReduce y dataframes.
- Los roles existentes y emergentes para aquellos involucrados con la gestión de datos, que incluyen los siguientes.
  - Profesionales de características de producto: aquellos que usan tanto bases de datos operativas SQL como NoSQL.
  - Profesionales analíticos/Profesionales en datos: aquellos que escriben código SQL, Python y Scala analítico para construir activos de datos para grupos de negocios.
  - Analistas de negocio: aquellos que construyen/gestionan datos más frecuentemente con hojas de cálculo Excel.
  - Profesionales de infraestructura de datos: aquellos que implementan un sistema de gestión de datos en una variedad de aplicaciones de datos (por ejemplo, OLTP).
  - **Todos** los que producen o consumen datos deben comprender los problemas sociales, éticos y profesionales asociados.

Un rol que trasciende todas las categorías anteriores es el de custodio de datos. Anteriormente, los datos eran vistos como un recurso para ser gestionado (Gestión de Sistemas de Información) al igual que otros recursos empresariales. Hoy, los datos son vistos en un contexto más amplio. Los datos sobre clientes ahora pueden verse como pertenecientes a (o en algunos contextos nacionales, como propiedad de) esos clientes. Ahora existe una comprensión aceptada de que el almacenamiento y uso seguro y ético de los datos institucionales es parte de ser un custodio de datos responsable.

Además, reconocemos la tensión entre un enfoque curricular en preparación profesional versus el estudio de un área de conocimiento como un esfuerzo científico. Esto es particularmente cierto con la Gestión de Datos. Por ejemplo, demostrar (o al menos conocer) la completitud de los Axiomas de Armstrong es fundamental en la teoría de dependencias funcionales. Sin embargo, la mayoría de los graduados en informática nunca utilizarán este concepto durante sus carreras profesionales. Lo mismo puede decirse de muchos otros temas en el canon de Gestión de Datos. Por el contrario, si nuestros graduados solo pueden normalizar datos en forma normal de Boyce-Codd (usando una herramienta automatizada) y escribir consultas SQL, sin comprender el rol que juegan los índices en la ejecución eficiente de consultas, les hemos hecho un perjuicio a ellos y a la sociedad.

Con este fin, el número de horas de Núcleo CS es relativamente pequeño en relación con las horas de Núcleo KA. Este enfoque está diseñado para permitir que las instituciones con contextos diferentes personalicen sus planes de estudio apropiadamente. Una institución que se enfoque en implementación OLTP, por ejemplo, priorizaría el almacenamiento eficiente y el acceso a datos, mientras que una institución que se enfoque en características de producto priorizaría el acceso programático a bases de datos existentes.

Sin embargo, como una institución maneje esta tensión, deseamos dar voz a una de las ironías de los planes de estudio de informática. Los estudiantes típicamente pasan gran parte de su vida educativa leyendo (y escribiendo) datos de un archivo o interactivamente, mientras que fuera de la academia los datos predominantes provienen de bases de datos a las que se accede programáticamente. Quizás en un futuro no muy lejano los estudiantes aprenderán acceso programático a bases de datos desde temprano y luego continuarán esta práctica a medida que avanzan en su plan de estudios.

Finalmente, entendemos que aunque el KA de Gestión de Datos puede ser ortogonal a los KAs de SEC (Seguridad) y SEP (Sociedad, Ética y la Profesión), también es punto cero para estas (y otras) áreas de conocimiento. Al diseñar almacenes de datos persistentes, la pregunta de qué debe almacenarse debe examinarse desde perspectivas legales y éticas. ¿Hay preocupaciones de privacidad? Y tan importante como eso, ¿qué tan bien protegidos están los datos?

Unidad de Conocimiento (KU)	**Core Tier1**	**Core Tier2**	**Alternativo**
3.4.1 El Rol de los Datos y el Ciclo de Vida de los Datos (Pág. 58)			No
3.4.2 Conceptos Fundamentales de Sistemas de Bases de Datos (Pág. 58)			No
3.4.3 Modelado de Datos (Pág. 59)			No
3.4.4 Bases de Datos Relacionales (Pág. 59)			No
3.4.5 Construcción de Consultas (Pág. 60)			No
3.4.6 Procesamiento de Consultas (Pág. 61)			No
3.4.7 Internos de DBMS (Pág. 61)			No
3.4.8 Sistemas NoSQL (Pág. 62)			No
3.4.9 Seguridad y Privacidad de Datos (Pág. 62)			No
3.4.10 Análisis de Datos (Pág. 63)			No
3.4.11 Bases de Datos Distribuidas/Computación en la Nube (Pág. 63)			No
3.4.12 Bases de Datos Semiestructuradas y No Estructuradas (Pág. 63)			No
3.4.13 Sociedad, Ética y Profesión (Pág. 64)			No

3.4.1. DM/El Rol de los Datos y el Ciclo de Vida de los Datos

Temas:
CS Core

- Ciclo de Vida de los Datos: Creación-Procesamiento-Revisión/Reporte-Retención/Recuperación-Destrucción

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Identificar las cinco etapas del Ciclo de Vida de los Datos [Analizar]

3.4.2. DM/Conceptos Fundamentales de Sistemas de Bases de Datos

Temas:
CS Core

- Propósito y ventajas de los sistemas de bases de datos
- Componentes de los sistemas de bases de datos
- Diseño de funciones fundamentales de DBMS (por ejemplo, mecanismos de consulta, gestión de transacciones, gestión de búfer, métodos de acceso)
- Arquitectura de base de datos, independencia de datos y abstracción de datos
- Gestión de transacciones
- Normalización
- Enfoques para gestionar grandes volúmenes de datos (por ejemplo, sistemas de bases de datos NoSQL, uso de MapReduce)
- Cómo soportar aplicaciones solo de CRUD
- Bases de datos distribuidas/sistemas basados en la nube

- Datos estructurados, semiestructurados y no estructurados
- Uso de un lenguaje de consulta declarativo

KA Core

- Sistemas que soportan contenido estructurado y/o de flujo

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Identificar al menos cuatro ventajas que proporciona el uso de un sistema de base de datos [Analizar]
2. Enumerar los componentes de un sistema de base de datos (relacional) [Enumerar]
3. Seguir una consulta a medida que es procesada por los componentes de un sistema de base de datos (relacional) [Analizar]
4. Defender el valor de la independencia de datos [Defender]
5. Componer una consulta simple de selección-proyección-unión en SQL [Componer]
6. Enumerar las cuatro propiedades de un gestor de transacciones correcto [Enumerar]
7. Describir las ventajas de eliminar datos duplicados repetidos [Describir]
8. Esbozar cómo MapReduce usa paralelismo para procesar datos eficientemente [Esquematizar/Esbozar]
9. Evaluar las diferencias entre bases de datos estructuradas y semiestructuradas/no estructuradas [Evaluar]

3.4.3. DM/Modelado de Datos

Temas:

CS Core

- Modelado de datos
- Modelo de datos relacional

KA Core

- Modelos conceptuales (por ejemplo, entidad-relación, diagramas UML)
- Modelos de datos semiestructurados (expresados usando DTD, XML o JSON Schema, por ejemplo)

Non Core

- Modelos de hojas de cálculo
- Modelos orientados a objetos: 1. GraphQL
- Nuevas características en SQL
- Temas de Modelado de Datos Especializados: 1. Datos de series temporales (agregación, unión) 2. Datos de grafos (recorrido de enlaces) 3. Técnicas para evitar acceso ineficiente a datos crudos (por ejemplo, "precio promedio diario"): vistas materializadas y estructuras de datos especiales (por ejemplo, Hyperloglog, mapa de bits) 4. Datos Geoespaciales (por ejemplo, bases de datos GIS)

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Describir los componentes del modelo de datos relacional [Describir]
2. Modelar relaciones 1:1, 1:n y n:m usando el modelo de datos relacional [Usar]

KACore:

3. Describir los componentes del modelo E-R (o algún otro modelo no relacional) [Describir]
4. Modelar un entorno dado usando un modelo de datos conceptual [Analizar]
5. Modelar un entorno dado usando el modelo de datos basado en documentos o basado en almacén clave-valor [Aplicar]

3.4.4. DM/Bases de Datos Relacionales

Temas:

CS Core

- Integridad de entidad y referencial: Clave candidata, superclaves
- Diseño de base de datos relacional

KA Core

- Mapeo de esquema conceptual a un esquema relacional
- Diseño físico de base de datos: estructuras de archivos y almacenamiento
- Introducción a la teoría de dependencias funcionales
- Teoría de Normalización: 1. Descomposición de un esquema; propiedades de unión sin pérdida y preservación de dependencias de una descomposición 2. Formas normales (BCNF) 3. Desnormalización (para eficiencia)

Non Core

- Teoría de dependencias funcionales: 1. Clausura de un conjunto de atributos 2. Cubierta canónica
- Teoría de normalización: 1. Dependencia multivaluada (4NF) 2. Dependencia de unión (PJNF, 5NF) 3. Teoría de representación

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Describir las características definitorias detrás del modelo de datos relacional [Describir]
2. Comentar sobre la diferencia entre una clave foránea y una superclave [Explicar]
3. Enumerar los diferentes tipos de restricciones de integridad [Enumerar]

KACore:

4. Componer un esquema relacional a partir de un esquema conceptual que contenga relaciones 1:1, 1:n y n:m [Componer]
5. Mapear estructura de archivo apropiada a relaciones e índices [Mapear/Mapa conceptual]
6. Describir cómo la teoría de dependencias funcionales generaliza la noción de clave [Describir]
7. Defender una descomposición dada como sin pérdida y/o que preserva dependencias [Defender]
8. Detectar qué forma normal produce una descomposición dada [Detectar]
9. Comentar sobre las razones para desnormalizar una relación [Interpretar]

3.4.5. DM/Construcción de Consultas

Temas:
CS Core

- Formación de Consultas SQL: 1. Ejecución interactiva de SQL 2. Ejecución programática de una consulta SQL

KA Core

- Álgebra Relacional
- SQL: 1. Definición de datos incluyendo especificaciones de integridad y otras restricciones 2. Sublenguaje de actualización

Non Core

- Cálculo Relacional
- QBE y entornos de 4ta generación
- Diferentes formas de invocar consultas no procedimentales en lenguajes convencionales
- Introducción a otros lenguajes de consulta principales (por ejemplo, XPATH, SPARQL)
- Procedimientos almacenados

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Componer consultas SQL que incorporen selección, proyección, unión, intersección, diferencia de conjuntos y división de conjuntos [Componer]
2. Determinar cuándo una consulta SQL anidada está correlacionada o no [Determinar]
3. Iterar sobre datos recuperados programáticamente de una base de datos a través de una consulta SQL [Crear]

KACore:

4. Definir, en SQL, un esquema de relación, incluyendo todas las restricciones de integridad y activadores de eliminación/actualización [Definir]
5. Componer una consulta SQL para actualizar una tupla en una relación [Componer]

3.4.6. DM/Procesamiento de Consultas

Temas:
KA Core

- Estructuras de página
- Estructuras de índice: 1. Árboles B+ 2. Índices hash: estáticos y dinámicos 3. Creación de índices en SQL
- Estructuras de archivos: 1. Archivos de montón (*heap*) 2. Archivos hash
- Algoritmos para operadores de consulta: 1. Ordenamiento externo 2. Selección 3. Proyección; con y sin eliminación de duplicados 4. Uniones naturales: Bucle anidado, Mezcla-ordenada, Unión hash 5. Análisis de eficiencia de algoritmos
- Transformaciones de consultas
- Optimización de consultas: 1. Rutas de acceso 2. Construcción de plan de consulta 3. Estimación de selectividad 4. Planes solo de índice
- Procesamiento de Consultas en Paralelo (por ejemplo, exploración paralela, unión paralela, agregación paralela)
- Afinamiento/rendimiento de base de datos: 1. Selección de índices 2. Impacto de índices en el rendimiento de consultas 3. Desnormalización

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Describir el propósito y organización de las estructuras de índice tanto de árbol B+ como hash [Describir]
2. Componer un comando SQL para crear un índice (de cualquier tipo) [Componer]
3. Especificar los pasos para los diversos algoritmos de operadores de consulta: ordenamiento externo, proyección con eliminación de duplicados, unión por mezcla-ordenada, unión hash, unión por bucle anidado por bloques [Analizar]
4. Derivar el tiempo de ejecución (en solicitudes de E/S) para cada uno de los algoritmos anteriores [Analizar]
5. Transformar una consulta en álgebra relacional a su equivalente apropiado para una ejecución canalizada de profundidad izquierda [Rediseñar]
6. Calcular estimaciones de selectividad para una operación de selección y/o unión dada [Computar/Calcular]
7. Describir cómo modificar una estructura de índice para facilitar una operación solo de índice para una relación dada [Describir]
8. Para un escenario dado decidir qué índices soportar para la ejecución eficiente de un conjunto de consultas [Evaluar (valorar)]
9. Describir cómo los DBMS aprovechan el paralelismo para acelerar el procesamiento de consultas dividiendo el trabajo entre múltiples procesadores o nodos [Describir]

3.4.7. DM/Internos de DBMS

Temas:
KA Core

- Gestión de Búfer de DB
- Gestión de Transacciones: 1. Niveles de Aislamiento 2. ACID 3. Serializabilidad 4. Transacciones Distribuidas
- Control de Concurrencia: 1. Bloqueo en 2 Fases 2. Estrategias de manejo de bloqueos mutuos 3. Modelos de consistencia basados en quórum
- Gestor de Recuperación: 1. Relación con el Gestor de Búfer

Non Core

- Control de Concurrencia: 1. Control de concurrencia optimista 2. Control de concurrencia por marca de tiempo
- Gestor de Recuperación: 1. Registro por escritura anticipada 2. Sistema de recuperación ARIES (Análisis, REDO, UNDO)

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Describir cómo un DBMS gestiona su Grupo de Búferes [Describir]

2. Describir las cuatro propiedades para un gestor de transacciones correcto [Describir]
3. Esbozar el principio de serializabilidad [Esquematizar/Esbozar]

3.4.8. DM/Sistemas NoSQL

Temas:
KA Core

- ¿Por qué NoSQL? (por ejemplo, Desajuste de impedancia entre Aplicación [CRUD] y RDBMS)
- Modelo de datos Clave-Valor y Documento

Non Core

- Sistemas de almacenamiento (por ejemplo, sistemas Clave-Valor, Lagos de Datos)
- Modelos de Distribución (Fragmentación y Replicación)
- Bases de Datos de Grafos
- Modelos de Consistencia (Actualización y Lectura, consistencia de quórum, teorema CAP)
- Modelo de procesamiento (por ejemplo, Map-Reduce, map-reduce multi-etapa, map-reduce incremental)
- Estudios de Caso: Sistema de almacenamiento en la nube (por ejemplo, S3); Bases de datos de grafos; "Cuándo no usar NoSQL"

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Desarrollar un caso de uso para el uso de NoSQL sobre RDBMS [Crear]
2. Describir las características definitorias detrás de los modelos de datos basados en Clave-Valor y Documentos [Describir]

3.4.9. DM/Seguridad y Privacidad de Datos

Temas:
CS Core

- Diferencias entre seguridad de datos y privacidad de datos
- Proteger datos y sistemas de bases de datos de ataques, incluyendo ataques de inyección como inyección SQL
- Información de identificación personal (PII) y su protección
- Consideraciones éticas para asegurar la seguridad y privacidad de los datos

KA Core

- Necesidad de, y diferentes enfoques para asegurar datos en reposo, en tránsito y durante el procesamiento
- Auditoría de base de datos y su rol en forensía digital
- Inferencia de datos y prevención de ataques
- Leyes y regulaciones que gobiernan la seguridad de datos y la privacidad de datos

Non Core

- Factores de riesgo típicos y medidas de prevención para asegurar la integridad de datos
- Ransomware y prevención de pérdida y destrucción de datos

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Describir las diferencias en los objetivos para seguridad de datos y privacidad de datos [Describir]
2. Identificar y mitigar riesgos asociados con diferentes enfoques para proteger datos [Analizar]
3. Describir consideraciones legales y éticas de seguridad y privacidad de datos de extremo a extremo [Describir]

KACore:

4. Desarrollar un sistema de auditoría de base de datos dadas consideraciones de riesgo [Crear]
5. Aplicar varios enfoques de exploración de datos para comprender conjuntos de datos no familiares [Aplicar]

3.4.10. DM/Análisis de Datos

Temas:
KA Core

- Técnicas exploratorias de datos (motivación, representación, estadísticas descriptivas, visualizaciones)
- Ciclo de vida de ciencia de datos: comprensión del negocio, comprensión de datos, preparación de datos, modelado, evaluación, despliegue y aceptación del usuario
- Algoritmos de minería de datos y aprendizaje automático: por ejemplo, clasificación, agrupamiento, asociación, regresión
- Adquisición y gobernanza de datos
- Consideraciones de seguridad y privacidad de datos
- Equidad y sesgo de datos
- Técnicas de visualización de datos y su uso en análisis de datos
- Resolución de Entidades

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Describir varios enfoques de exploración de datos, incluyendo visualización, para comprender conjuntos de datos no familiares [Describir]
2. Aplicar varios enfoques de exploración de datos para comprender conjuntos de datos no familiares [Aplicar]
3. Describir algoritmos básicos de aprendizaje automático/minería de datos y cuándo son apropiados para su uso [Describir]
4. Aplicar varios algoritmos de aprendizaje automático/minería de datos [Aplicar]
5. Describir consideraciones legales y éticas en la adquisición, uso y modificación de conjuntos de datos [Describir]
6. Describir problemas de equidad y sesgo en la recolección y uso de datos [Describir]

3.4.11. DM/Bases de Datos Distribuidas/Computación en la Nube

Temas:
Non Core

- DBMS Distribuido: 1. Almacenamiento de datos distribuido 2. Procesamiento de consultas distribuido 3. Modelo de transacción distribuida 4. Soluciones homogéneas y heterogéneas 5. Bases de datos distribuidas cliente-servidor
- DBMS Paralelo: 1. Arquitecturas de DBMS paralelo: memoria compartida, disco compartido, nada compartido 2. Aceleración y escalamiento, por ejemplo, uso del modelo de procesamiento MapReduce 3. Replicación de datos y modelos de consistencia débil

Aprendizaje esperado (*Learning Outcomes*):

3.4.12. DM/Bases de Datos Semiestructuradas y No Estructuradas

Temas:
Non Core

- Datos no estructurados vectorizados (texto, video, audio, etc.) y almacenamiento vectorial: 1. Vectorizador TF-IDF con n-grama 2. Word2Vec 3. Base de datos de arreglos o manejo de tipo de datos arreglo
- Bases de datos semiestructuradas (por ejemplo, JSON): 1. Almacenamiento: *a*) Codificación y compresión de tipos de datos anidados 2. Indexación: *a*) Árbol B, índice de salto, filtro Bloom *b*) Índice invertido y compresión de mapa de bits *c*) Indexación por curva de llenado de espacio para datos geo-semiestructurados 3. Procesamiento de consultas para casos de uso OLTP y OLAP: *a*) Compromisos de inserción, selección, actualización/eliminación *b*) Estudios de caso sobre Postgres/JSON, MongoDB y Snowflake/JSON

Aprendizaje esperado (*Learning Outcomes*):

3.4.13. DM/Sociedad, Ética y Profesión

Temas:
CS Core

- Problemas relacionados con la escala
- Privacidad de datos en general: 1. Privacidad por diseño y por defecto
- Anonimato de datos
- Propiedad/custodia de datos
- Aplicaciones intencionales y no intencionales de los datos almacenados

KA Core

- Fiabilidad de los datos
- Procedencia, linaje de datos y gestión de metadatos
- Seguridad de datos

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Enumerar tres problemas sociales y tres legales relacionados con grandes colecciones de datos [Enumerar]
2. Describir el valor de la privacidad de datos [Describir]
3. Identificar las partes interesadas en competencia con respecto a la propiedad de los datos [Analizar]
4. Enumerar tres consecuencias negativas no intencionales de una aplicación centrada en datos (bien conocida) dada (por ejemplo, Facebook, LastPass, Ashley Madison) [Enumerar]

KACore:

5. Describir el significado de procedencia y linaje de datos [Describir]
6. Identificar cómo una base de datos podría contribuir a la seguridad de los datos, así como cómo puede introducir inseguridades [Analizar]

3.5. Fundamentos de los Lenguajes de Programación (FPL)

Los fundamentos de los lenguajes de programación están enraizados en las matemáticas discretas, la lógica y los lenguajes formales, y proporcionan una base para la comprensión de los complejos lenguajes de programación modernos. Aunque los lenguajes de programación varían según el paradigma del lenguaje y el dominio del problema, y evolucionan en respuesta tanto a las necesidades sociales como al avance tecnológico, comparten un modelo abstracto subyacente de computación y desarrollo de programas. Esto sigue siendo cierto incluso a medida que el hardware del procesador y su interfaz con las herramientas de programación se entrelazan y complejizan progresivamente. Una comprensión de las abstracciones y paradigmas de programación comunes permite un aprendizaje más rápido de los lenguajes de programación.

El área de conocimiento de Fundamentos de los Lenguajes de Programación se ocupa de articular los conceptos y principios subyacentes de los lenguajes de programación, la especificación formal de un lenguaje de programación y el comportamiento de un programa, explicar cómo se implementan los lenguajes de programación, comparar las fortalezas y debilidades de varios paradigmas de programación, y describir cómo los lenguajes de programación interactúan con entidades como sistemas operativos y hardware. Los conceptos aquí cubiertos son aplicables a varios lenguajes y comprender estos principios ayuda al aprendiz a pasar fácilmente de un lenguaje a otro, así como a seleccionar un paradigma y lenguaje de programación que mejor se adapte al problema en cuestión.

Los lenguajes de programación son el medio a través del cual los programadores describen conceptos con precisión, formulan algoritmos y razonan sobre soluciones. A lo largo de su carrera, un científico de la computación aprenderá y trabajará con muchos lenguajes diferentes, por separado o juntos. Los desarrolladores de software deben entender diferentes modelos de programación, características y construcciones de programación, y conceptos subyacentes para tomar decisiones de diseño informadas entre lenguajes que admiten múltiples enfoques complementarios. Sería útil saber cómo se definen, componen e implementan las características de los lenguajes de programación para mejorar la eficiencia de ejecución y el mantenimiento a largo plazo del software desarrollado. También es útil un conocimiento básico de la traducción de lenguajes, el análisis de programas, el comportamiento en

tiempo de ejecución, la gestión de memoria y la interacción de procesos concurrentes que se comunican entre sí mediante paso de mensajes, memoria compartida y sincronización. Finalmente, algunos desarrolladores e investigadores necesitarán diseñar nuevos lenguajes, un ejercicio que requiere una mayor familiaridad con los principios básicos.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
3.5.1 Programación Orientada a Objetos (Pág. 65)			No
3.5.2 Programación Funcional (Pág. 66)			No
3.5.3 Programación Lógica (Pág. 67)			No
3.5.4 Scripting de Shell (Pág. 68)			No
3.5.5 Programación Dirigida por Eventos y Reactiva (Pág. 68)			No
3.5.6 Computación Paralela y Distribuida (Pág. 68)			No
3.5.7 Programación Orientada a Aspectos (Pág. 69)			No
3.5.8 Sistemas de Tipos (Pág. 70)			No
3.5.9 Modelo de Ejecución y Memoria de Sistemas (Pág. 71)			No
3.5.10 Traducción y Ejecución de Lenguajes (Pág. 72)			No
3.5.11 Abstracción y Representación de Programas (Pág. 72)			No
3.5.12 Análisis Sintáctico (Pág. 73)			No
3.5.13 Análisis Semántico del Compilador (Pág. 74)			No
3.5.14 Análisis de Programas y Analizadores (Pág. 74)			No
3.5.15 Generación de Código (Pág. 75)			No
3.5.16 Comportamiento en Tiempo de Ejecución y Sistemas (Pág. 75)			No
3.5.17 Construcciones de Programación Avanzadas (Pág. 76)			No
3.5.18 Pragmática del Lenguaje (Pág. 77)			No
3.5.19 Semántica Formal (Pág. 77)			No
3.5.20 Metodologías de Desarrollo Formal (Pág. 77)			No
3.5.21 Principios de Diseño de Lenguajes de Programación (Pág. 78)			No
3.5.22 Sociedad, Ética y Profesión (Pág. 78)			No

3.5.1. FPL/Programación Orientada a Objetos

Temas:
CS Core

- Programación imperativa como subconjunto de la programación orientada a objetos.
- Diseño orientado a objetos: 1. Descomposición en objetos que poseen estado y tienen comportamiento. 2. Diseño de jerarquía de clases para modelado.
- Definición de clases: campos, métodos y constructores.
- Subclases, herencia (incluyendo herencia múltiple) y sobrescritura de métodos.
- Despacho dinámico: definición de llamada a método.
- Manejo de excepciones.
- Idiomas orientados a objetos para encapsulación: 1. Privacidad, ocultación de datos y visibilidad de miembros de clase. 2. Interfaces que revelan solo firmas de métodos. 3. Clases base abstractas, rasgos (*traits*) y mixins.
- Propiedades dinámicas vs estáticas.
- Composición vs herencia.
- Subtipado: 1. Polimorfismo de subtipos; conversiones ascendentes (*upcasts*) implícitas en lenguajes tipados. 2. Noción de reemplazo conductual: los subtipos actúan como el supertipo. 3. Relación entre subtipado y herencia.

KA Core

- Clases de colecciones, iteradores y otros componentes comunes de biblioteca.
- Metaprogramación y reflexión.

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Enumerar las diferencias entre los paradigmas de programación imperativa y orientada a objetos [Enumerar]
2. Componer una clase a través del diseño, implementación y prueba para cumplir con los requisitos de comportamiento [Componer]
3. Construir una jerarquía de clases simple utilizando subclases que permita reutilizar código para subclases distintas [Crear]
4. Predecir y validar el flujo de control en un programa usando despacho dinámico [Evaluar]
5. Comparar y contrastar cómo difieren las soluciones computacionales a un problema en los enfoques procedural, funcional y orientado a objetos [Comparar]
6. Comparar y contrastar los mecanismos para definir y proteger elementos de datos dentro de los enfoques procedural, funcional y orientado a objetos [Comparar]
7. Comparar y contrastar los beneficios y costos/impacto de usar herencia (subclases) y composición (específicamente, cómo basar la composición en funciones de orden superior) [Comparar]
8. Explicar la relación entre la herencia orientada a objetos (compartición de código y sobrescritura) y el subtipado (la idea de que un subtipo sea usable en un contexto que espera el supertipo) [Explicar]
9. Usar mecanismos de encapsulación orientados a objetos como interfaces y miembros privados [Usar]
10. Definir y usar iteradores y otras operaciones en agregados, incluyendo operaciones que toman funciones como argumentos, en múltiples lenguajes de programación, seleccionando los idiomas más naturales para cada lenguaje [Definir]
11. Comparar y contrastar cómo difieren las soluciones computacionales a un problema en los enfoques procedural, funcional y orientado a objetos [Comparar]
12. Comparar y contrastar los mecanismos para definir y proteger elementos de datos dentro de los enfoques procedural, funcional y orientado a objetos [Comparar]

KACore:

13. Usar clases de colecciones e iteradores efectivamente para resolver un problema [Usar]

3.5.2. FPL/Programación Funcional

Temas:

CS Core

- Expresiones lambda y evaluación: 1. Enlace (binding) de variables y reglas de ámbito (scope). 2. Paso de parámetros. 3. Expresiones lambda anidadas y orden de reducción.
- Programación libre de efectos secundarios: 1. Las llamadas a funciones no tienen efectos secundarios, facilitando el razonamiento composicional. 2. Variables inmutables y copia de datos vs reducción. 3. Uso de recursión vs bucles vs tuberías (map/reduce).
- Procesamiento de datos estructurados (por ejemplo, árboles) mediante funciones con casos para cada variante de datos: 1. Funciones definidas sobre datos compuestos en términos de funciones aplicadas a las piezas constituyentes. 2. Estructuras de datos persistentes.
- Usar funciones de orden superior (tomar, devolver y almacenar funciones).

KA Core

- Metaprogramación y reflexión.
- Clausuras (*closures*) de funciones (funciones que usan variables en el entorno léxico circundante): 1. Significado y definición básica: crear clausuras en tiempo de ejecución capturando el entorno. 2. Idiomas canónicos: devoluciones de llamada (*callbacks*), argumentos para iteradores, código reutilizable mediante argumentos de funciones. 3. Usar una clausura para encapsular datos en su entorno. 4. Evaluación diferida (*lazy*) vs ansiosa (*eager*).

Non Core

- Máquina de reducción de grafos (graph reduction) y llamada por necesidad (*call-by-need*).
- Implementar evaluación diferida.
- Integración con el paradigma de programación lógica usando conceptos como lógica ecuacional, estrechamiento (narrowing), residuación (residuation) y unificación semántica.
- Integración con otros paradigmas de programación como imperativo y orientado a objetos.

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Desarrollar algoritmos básicos que eviten asignar a estados mutables o considerar igualdad referencial [Crear]
2. Desarrollar funciones útiles que tomen y devuelvan otras funciones [Crear]

KACore:

3. Explicar un ejemplo simple de una expresión lambda implementada usando una máquina virtual, como una máquina SECD, mostrando el almacenamiento y recuperación del entorno [Explicar]
4. Interpretar correctamente las variables y el ámbito léxico en un programa que usa clausuras de funciones [Interpretar]
5. Usar mecanismos de encapsulación funcional como clausuras e interfaces modulares [Usar]
6. Comparar y contrastar ejecución con estado (stateful) vs sin estado (stateless) [Comparar]
7. Definir y usar iteradores y otras operaciones en agregados, incluyendo operaciones que toman funciones como argumentos, en múltiples lenguajes de programación, seleccionando los idiomas más naturales para cada lenguaje [Definir]

NonCore:

8. Ilustrar la reducción de grafos usando una expresión λ con una subexpresión compartida [Aplicar]
9. Ilustrar la ejecución de una expresión λ anidada simple usando una máquina abstracta, como una máquina ABC [Aplicar]
10. Ilustrar estrechamiento (narrowing), residuación y unificación semántica usando ejemplos ilustrativos simples [Aplicar]
11. Ilustrar las construcciones de concurrencia usando ejemplos de programación simples de conceptos conocidos, como un búfer siendo leído y escrito concurrentemente o secuencialmente [Aplicar]

3.5.3. FPL/Programación Lógica

Temas:

KA Core

- Cuantificadores universales vs existenciales.
- Lógica de predicados de primer orden vs lógica de orden superior.
- Expresar relaciones complejas usando conectivos lógicos y relaciones más simples.
- Definiciones de cláusula de Horn, hechos (facts), metas (goals) y submetas (subgoals).
- Unificación y algoritmo de unificación; unificación vs aserción vs evaluación de expresiones.
- Mezclar relaciones con funciones.
- Cortes (cuts), retroceso (backtracking) y no determinismo.
- Suposiciones de mundo cerrado (closed-world) vs mundo abierto (open-world).

Non Core

- Sobrecarga de memoria por copia de variables al manejar programas iterativos.
- Construcciones de programación para almacenar cómputo parcial y podar árboles de búsqueda.
- Mezclar programación funcional y programación lógica usando conceptos como lógica ecuacional, estrechamiento (narrowing), residuación y unificación semántica.
- Programación lógica de orden superior, con restricciones (constraint) e inductiva.
- Integración con otros paradigmas de programación como la programación orientada a objetos.
- Construcciones de programación avanzadas como listas de diferencia (difference-lists), creación de estructuras de datos definidas por el usuario, conjunto de (set of), etc.

Aprendizaje esperado (*Learning Outcomes*):

KACore:

1. Usar un lenguaje lógico para implementar un algoritmo convencional [Usar]
2. Usar un lenguaje lógico para implementar un algoritmo que emplee búsqueda implícita usando cláusulas, relaciones y cortes (cuts) [Usar]
3. Usar un ejemplo ilustrativo simple para mostrar la correspondencia entre la Lógica de Predicados de Primer Orden (FOPL) y los programas lógicos que usan cláusulas de Horn [Usar]
4. Usar ejemplos para ilustrar el algoritmo de unificación y su papel en el paso de parámetros en la reducción de consultas [Usar]
5. Usar programas lógicos simples que entrelazan relaciones, funciones y programación recursiva, como números factoriales y de Fibonacci, y relaciones complejas simples entre entidades, e

ilustrar la ejecución y paso de parámetros usando unificación y retroceso [Usar]

NonCore:

6. Ilustrar el cómputo de programas simples como Fibonacci y mostrar la sobrecarga por recomputación, y luego mostrar cómo mejorar la sobrecarga de ejecución [Aplicar]

3.5.4. FPL/Scripting de Shell

Temas:

CS Core

- Manejo de errores/excepciones
- Tuberías (piping)
- Comandos del sistema: 1. Interfaz con sistemas operativos
- Variables de entorno
- Abstracción de archivo y operadores
- Estructuras de datos, como arreglos y listas.
- Expresiones regulares
- Programas y procesos
- Flujo de trabajo

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Crear y ejecutar scripts automatizados para gestionar varias tareas del sistema [Crear]
2. Resolver varios problemas de procesamiento de texto mediante scripting [Resolver]

3.5.5. FPL/Programación Dirigida por Eventos y Reactiva

Temas:

CS Core

- Programación procedural vs programación reactiva: ventajas de la programación reactiva para capturar eventos.
- Componentes de la programación reactiva: fuente de eventos, señales de eventos, escuchas (listeners) y despachadores (dispatchers), objetos de evento, adaptadores, manejadores de eventos (event-handlers).
- Modelos sin estado (stateless) y de transición de estado (state-transition) de programación basada en eventos.
- Usos canónicos como interfaces gráficas de usuario (GUIs), dispositivos móviles, robots, servidores.

KA Core

- Usar un framework reactivo: 1. Definir manejadores/escuchas de eventos 2. Parametrización de emisores de eventos y argumentos de eventos 3. Eventos generados externamente y eventos generados por el programa
- Separación de modelo, vista y controlador
- Programas dirigidos por eventos y reactivos como sistemas de transición de estado

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Implementar manejadores de eventos para usar en sistemas reactivos, como GUIs [Implementar]
2. Examinar por qué un estilo de programación dirigida por eventos es natural en dominios donde los programas reaccionan a eventos externos [Examinar]

KACore:

3. Definir y usar un framework reactivo [Definir]
4. Describir un sistema interactivo en términos de un modelo, una vista y un controlador [Describir]

3.5.6. FPL/Computación Paralela y Distribuida

Temas:

CS Core

- Seguridad (safety) y vivacidad (liveness): 1. Condiciones de carrera (race conditions) 2. Dependencias/precondiciones 3. Modelos de fallos (fault models) 4. Terminación
- Modelos de programación: 1. Modelo de actores (*actor models*) 2. Modelos procedurales y reactivos 3. Modelos de programación síncrona/asíncrona 4. Paralelismo de datos (*data parallelism*)

- Propiedades: 1. Propiedades basadas en orden: *a*) Conmutatividad *b*) Independencia 2. Propiedades basadas en consistencia: *a*) Atomicidad *b*) Consenso
- Control de ejecución: 1. Espera asíncrona *Async await* 2. Promesas (*promises*) 3. Hilos (*threads*)

- Comunicación y coordinación: 1. Mutexes 2. Paso de mensajes (*message-passing*) 3. Memoria compartida (*shared memory*) 4. Cobegin-coend 5. Monitores 6. Canales (*channels*) 7. Hilos (*threads*) 8. Guardas (*guards*)

KA Core

- Futuros (futures)
- Soporte del lenguaje para paralelismo de datos como forall, desenrollado de bucles (loop unrolling), map/reduce
- Efecto de los modelos de consistencia de memoria en la semántica del lenguaje y la generación correcta de código
- Interfaces de Programación de Aplicaciones de Transferencia de Estado Representacional (REST APIs)
- Tecnologías y enfoques: computación en la nube (*cloud computing*), computación de alto rendimiento, computación cuántica, computación ubicua
- Sobrecarga del paso de mensajes
- Granularidad del programa para una explotación eficiente de la concurrencia
- Concurrencia y otros paradigmas de programación (por ejemplo, funcional)

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Explicar por qué los lenguajes de programación no garantizan la consistencia secuencial en presencia de condiciones de carrera (data races) y qué deben hacer los programadores como resultado [Explicar]
2. Implementar programas concurrentes correctos usando múltiples modelos de programación, como memoria compartida, actores, futuros, construcciones de sincronización y primitivas de paralelismo de datos [Implementar]
3. Usar un modelo de paso de mensajes para analizar un protocolo de comunicación [Usar]
4. Usar construcciones de sincronización como métodos monitorizados/sincronizados en un programa simple [Usar]
5. Modelar dependencia de datos usando construcciones de programación simples que involucran variables, lectura y escritura [Implementar]
6. Modelar dependencia de control usando construcciones simples como selección e iteración [Implementar]

KACore:

7. Explicar cómo las APIs REST integran aplicaciones y automatizan procesos [Explicar]
8. Explicar beneficios, limitaciones y desafíos relacionados con la computación distribuida y paralela [Explicar]

3.5.7. FPL/Programación Orientada a Aspectos

Temas:

Non Core

- Aspectos
- Puntos de unión (join points)
- Consejos (advice): 1. Antes (*before*) 2. Después (*after*) (como *finally*, *returning* o *throwing*) 3. Alrededor (*around*)
- Cortes de punto (point cuts): 1. Designadores
- Tejido (weaving) - estático y dinámico
- Alternativas incluyendo anotaciones y IDEs

Aprendizaje esperado (*Learning Outcomes*):

3.5.8. FPL/Sistemas de Tipos

Temas:
CS Core

- Un tipo como un conjunto de valores junto con un conjunto de operaciones: 1. Tipos primitivos (por ejemplo, números, booleanos) 2. Tipos compuestos construidos a partir de otros tipos (por ejemplo, registros/estructuras, uniones, arreglos, listas, funciones, referencias usando operaciones de conjuntos)
- Asociación de tipos a variables, argumentos, resultados y campos
- Seguridad de tipos (type safety) como un aspecto de la corrección del programa
- Seguridad de tipos y errores causados por usar valores inconsistentemente dados sus tipos pretendidos
- Objetivos y limitaciones del tipado estático y dinámico: detectar y eliminar errores lo antes posible.
- Tipos genéricos (polimorfismo paramétrico): 1. Definición y ventajas del polimorfismo: paramétrico, de subtipado (subtyping), de sobrecarga (overloading) y de coerción 2. Comparación de tipos monomórficos y polimórficos 3. Comparación con polimorfismo ad-hoc (sobrecarga) y polimorfismo de subtipos 4. Parámetros genéricos y tipado 5. Uso de bibliotecas genéricas como colecciones 6. Comparación con polimorfismo ad hoc (sobrecarga) y polimorfismo de subtipos 7. Polimorfismo prescriptivo vs descriptivo 8. Modelos de implementación de tipos polimórficos 9. Subtipado

KA Core

- Equivalencia de tipos: equivalencia estructural vs nominal
- Beneficios complementarios del tipado estático y dinámico: 1. Errores tempranos vs errores tardíos/evitados 2. Hacer cumplir invariantes durante el desarrollo y mantenimiento del código vs posponer decisiones de tipado mientras se prototipa y permitir convenientemente patrones de codificación flexibles como colecciones heterogéneas. 3. Reglas de tipado para tipos de función, producto y suma 4. Evitar mal uso del código vs permitir más reutilización de código 5. Detectar programas incompletos vs permitir que programas incompletos se ejecuten 6. Relación con el análisis estático 7. Decidibilidad

Non Core

- Constructores de tipos composicionales, como tipos producto (para agregados), tipos suma (para uniones), tipos función, tipos cuantificados y tipos recursivos
- Verificación de tipos (type checking)
- Subtipado: 1. Polimorfismo de subtipos; conversiones ascendentes implícitas en lenguajes tipados 2. Noción de reemplazo conductual: los subtipos actúan como el supertipo 3. Relación entre subtipado y herencia
- Seguridad de tipos como preservación más progreso (preservation plus progress)
- Inferencia de tipos
- Sobrecarga estática
- Proposiciones como tipos (implicación como función, conjunción como producto, disyunción como suma)
- Tipos dependientes (cuantificación universal como función dependiente, cuantificación existencial como producto dependiente).

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Describir, tanto para un tipo primitivo como para uno compuesto, los valores que tienen ese tipo [Describir]
2. Describir, para un lenguaje con un sistema de tipos estático, las operaciones que están prohibidas estáticamente, como pasar el tipo incorrecto de valor a una función o método [Describir]
3. Describir ejemplos de errores de programa detectados por un sistema de tipos [Describir]
4. Identificar propiedades del programa, para múltiples lenguajes de programación, que se verifican estáticamente y propiedades del programa que se verifican dinámicamente [Analizar]

5. Describir un programa de ejemplo que no verifica tipos en un lenguaje particular y que sin embargo no tendría error si se ejecutara [Describir]
6. Usar tipos y mensajes de error de tipos para escribir y depurar programas [Usar]

KACore:

7. Explicar cómo las reglas de tipado definen el conjunto de operaciones que son legales para un tipo [Explicar]
8. Enumerar las reglas de tipo que gobiernan el uso de un tipo compuesto particular [Listar/Enumerar]
9. Explicar por qué la indecidibilidad requiere que los sistemas de tipos aproximen conservadoramente el comportamiento del programa [Explicar]
10. Definir y usar piezas de programa (como funciones, clases, métodos) que usen tipos genéricos, incluyendo para colecciones [Definir]
11. Discutir las diferencias entre genéricos, subtipado y sobrecarga [Debatir]
12. Explicar múltiples beneficios y limitaciones del tipado estático al escribir, mantener y depurar software [Explicar]

NonCore:

13. Definir un sistema de tipos de manera precisa y composicional [Definir]
14. Para varios constructores de tipos fundamentales, identificar los valores que describen y los invariantes que hacen cumplir [Analizar]
15. Describir precisamente los invariantes preservados por un sistema de tipos sano (sound) [Describir]
16. Demostrar seguridad de tipos para un lenguaje simple en términos de teoremas de preservación y progreso [Probar/Demostrar]
17. Implementar un algoritmo de inferencia de tipos basado en unificación para un lenguaje simple [Implementar]
18. Explicar cómo la sobrecarga estática y los algoritmos de resolución asociados influyen en el comportamiento dinámico de los programas [Explicar]

3.5.9. FPL/Modelo de Ejecución y Memoria de Sistemas

Temas:

CS Core

- Estructuras de datos para traducción, ejecución, traducción y movilidad de código como pila, montón (heap), alias (compartición usando punteros), secuencia indexada y cadena
- Acceso directo, indirecto e indexado a ubicación de memoria
- Representación en tiempo de ejecución de abstracciones de datos como variables, arreglos, vectores, registros, elementos de datos basados en punteros como listas enlazadas y árboles, y objetos
- Máquina de bajo nivel abstracta con instrucción simple, pila y montón para explicar traducción y ejecución
- Distribución en memoria en tiempo de ejecución: registro de activación (con varios punteros), datos estáticos, pila de llamadas, montón: 1. Traducción de construcciones de selección e iterativas a diagramas de flujo de control 2. Traducción de diagramas de flujo de control a código abstracto de bajo nivel 3. Implementación de bucles, recursión y llamadas de cola (tail calls) 4. Traducción de llamadas a funciones/procedimientos y retorno de llamadas, incluyendo diferentes mecanismos de paso de parámetros usando una máquina abstracta
- Gestión de memoria: 1. Asignación y acceso de bajo nivel a estructuras de datos de alto nivel como tipos de datos básicos, arreglos n-dimensionales, vectores, registros y objetos 2. Retorno de procedimiento como mecanismo de desasignación automática para elementos de datos locales en la pila 3. Gestión manual de memoria: asignar, desasignar y reutilizar memoria del montón 4. Gestión automática de memoria: recolección de basura (garbage collection) como técnica automatizada usando la noción de alcanzabilidad (reachability)
- Computación verde (green computing).

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Explicar cómo se ejecuta una construcción central del lenguaje, como abstracciones de datos y abstracciones de control [Explicar]

2. Explicar cómo las implementaciones de lenguajes de programación típicamente organizan la memoria en secciones de datos globales, texto, montón y pila, y cómo características como recursión y gestión de memoria se mapean a este modelo de memoria [Explicar]
3. Explicar por qué ocurren las fugas de memoria (memory leaks) y los problemas de punteros colgantes (dangling pointers), y qué puede hacer un programador para evitarlos/corregirlos [Explicar]

3.5.10. FPL/Traducción y Ejecución de Lenguajes

Temas:
CS Core
- Modelos de ejecución para JIT (Just-In-Time), compilador, intérprete
- Uso de código intermedio, por ejemplo, bytecode
- Limitaciones y beneficios de JIT, compilador e intérprete
- Compiladores/transpiladores cruzados (cross compilers/transpilers)
- Representación BNF y BNF extendida de gramática libre de contexto
- Árbol de análisis (parse tree) usando una oración simple como expresión aritmética o sentencia if-then-else
- Ejecución como código nativo o dentro de una máquina virtual
- Canalización de traducción de lenguaje: análisis sintáctico, análisis (parsing), verificación de tipos opcional, generación de código/traducción y optimización, enlace (linking), carga (loading), ejecución

KA Core
- Representación en tiempo de ejecución de construcciones centrales del lenguaje como objetos (tablas de métodos) y funciones que pueden pasarse como parámetros a y devolverse desde funciones (clausuras)
- Desarrollo seguro de compiladores

Aprendizaje esperado (*Learning Outcomes*):
CSCore:
1. Explicar y comprender las diferencias entre implementaciones de lenguaje compilado, JIT e interpretado, incluyendo los beneficios y limitaciones de cada una [Explicar]
2. Diferenciar sintaxis y análisis (parsing) de semántica y evaluación [Diferenciar]
3. Usar BNF y BNF extendida para especificar la sintaxis de construcciones simples como if-then-else, declaración de tipo y construcciones iterativas para lenguajes conocidos como C++ o Python [Usar]
4. Ilustrar el árbol de análisis usando una oración/expresión aritmética simple [Aplicar]
5. Ilustrar traducción de diagramas de sintaxis a BNF/BNF extendida para construcciones simples como if-then-else, declaración de tipo, construcciones iterativas, etc [Aplicar]
6. Ilustrar ambigüedad en el análisis usando if-then-else anidado/expresión aritmética y mostrar resolución usando orden de precedencia [Aplicar]

KACore:
7. Discutir los beneficios y limitaciones de la recolección de basura, incluyendo la noción de alcanzabilidad [Debatir]

3.5.11. FPL/Abstracción y Representación de Programas

Temas:
KA Core
- BNF y expresiones regulares
- Programas que toman (otros) programas como entrada, como intérpretes, compiladores, verificadores de tipos, generadores de documentación
- Componentes de un lenguaje: 1. Definiciones de alfabetos, delimitadores, oraciones, sintaxis y semántica 2. Sintaxis vs semántica
- Programa como un conjunto de oraciones significativas no ambiguas
- Abstracciones básicas de programación: constantes, variables, declaraciones (incluyendo declaraciones anidadas), comando, expresión, asignación, selección, iteración definida e indefinida,

iteradores, función, procedimiento, módulos, manejo de excepciones
- Variables mutables vs inmutables: ventajas y desventajas de reutilizar ubicación de memoria existente vs ventajas de copiar y mantener valores antiguos; almacenar cómputo parcial vs recomputación
- Tipos de variables: estáticas, locales, no locales (*nonlocal*), globales; necesidad y problemas con variables no locales y globales.
- Reglas de ámbito (*scope*): estático vs dinámico; visibilidad de variables; efectos secundarios (*side-effects*).
- Efectos secundarios inducidos por variables no locales, variables globales y variables con alias (*aliased*).

Non Core
- Valores-L (*L-values*) y Valores-R (*R-values*): mapear nombre de variable mutable a Valores-L; mapear nombres de variables inmutables a Valores-R
- Entorno (*environment*) vs almacén (*store*) y sus propiedades
- Abstracción de datos y de control
- Mecanismos para intercambio de información entre unidades de programa como procedimientos, funciones y módulos: variables no locales, variables globales, paso de parámetros, importación-exportación entre módulos
- Estructuras de datos para representar código para ejecución, traducción o transmisión.
- Representación de instrucciones de bajo nivel como instrucciones de máquina virtual, lenguaje ensamblador y representación binaria
- Cálculo lambda, enlace de variables (variable binding) y renombrado de variables.
- Tipos de semántica: operacional, axiomática, denotacional, conductual; definir y usar árboles de sintaxis abstracta; contrastar con sintaxis concreta.

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Ilustrar el ámbito de variables y visibilidad usando programas simples [Aplicar]
2. Ilustrar diferentes tipos de paso de parámetros usando un lenguaje de programación pseudo simple [Aplicar]
3. Explicar el efecto secundario usando variables globales y no locales y cómo corregir tales programas [Explicar]
4. Explicar cómo los programas que procesan otros programas tratan a los otros programas como sus datos de entrada [Explicar]
5. Describir una gramática y un árbol de sintaxis abstracta para un lenguaje pequeño [Describir]
6. Describir los beneficios de tener representaciones de programas distintas de cadenas de código fuente [Describir]
7. Implementar un programa para procesar alguna representación de código para algún propósito, como un intérprete, un optimizador de expresiones o un generador de documentación [Implementar]

3.5.12. FPL/Análisis Sintáctico

Temas:
Non Core
- Gramáticas regulares vs gramáticas libres de contexto
- Escaneo (*scanning*) y análisis (*parsing*) basados en especificaciones de lenguaje
- Análisis léxico usando expresiones regulares
- Tokens y su uso
- Estrategias de análisis (*parsing*) incluyendo técnicas descendentes (*top-down*) (por ejemplo, descenso recursivo o LL) y ascendentes (*bottom-up*) (por ejemplo, LR o GLR): 1. Tablas de anticipación (*lookahead*) y su aplicación al análisis
- Teoría de lenguajes: 1. Jerarquía de Chomsky 2. Derivación más a la izquierda/más a la derecha y ambigüedad 3. Transformación de gramáticas
- Mecanismos de recuperación de errores del analizador (parser)
- Generar escáneres y analizadores a partir de especificaciones declarativas

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Usar gramáticas formales para especificar la sintaxis de lenguajes [Usar]
2. Ilustrar el papel de las tablas de anticipación en el análisis [Aplicar]
3. Usar herramientas declarativas para generar analizadores y escáneres [Usar]
4. Reconocer problemas clave en definiciones de sintaxis: ambigüedad, asociatividad, precedencia [Reconocer]

3.5.13. FPL/Análisis Semántico del Compilador

Temas:
Non Core

- Árboles de sintaxis abstracta; contraste con sintaxis concreta
- Definir, recorrer y modificar representaciones de programa de alto nivel
- Ámbito (scope) y resolución de enlaces (binding)
- Semántica estática: 1. Verificación de tipos. 2. Definir antes de usar 3. Anotaciones y entornos (*frameworks*) de verificación estática extendida (*extended static checking*).
- Valores-L/Valores-R
- Semántica de llamada
- Tipos de paso de parámetros con ilustraciones simples y comparación: llamada por valor (call by value), llamada por referencia (call by reference), llamada por valor-resultado (call by value-result), llamada por nombre (call by name), llamada por necesidad (call by need) y sus variaciones
- Especificaciones declarativas como gramáticas atribuidas (attribute grammars) y sus aplicaciones en manejo de gramáticas sensibles al contexto limitadas

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Dibujar el árbol de sintaxis abstracta para un lenguaje pequeño [Diagramar]
2. Implementar análisis estáticos sensibles al contexto, a nivel de fuente, como verificadores de tipos o resolver identificadores para identificar sus ocurrencias de enlace (binding) [Implementar]
3. Describir análisis semánticos usando una gramática atribuida [Describir]

3.5.14. FPL/Análisis de Programas y Analizadores

Temas:
Non Core

- Representaciones de programa relevantes, como bloques básicos (basic blocks), grafos de flujo de control (control-flow graphs), cadenas definición-uso (def-use chains) y asignación única estática (static single assignment)
- Indecidibilidad y consecuencias para el análisis de programas
- Análisis insensible al flujo (flow-insensitive), como verificación de tipos y análisis de punteros y alias escalables
- Análisis sensible al flujo (flow-sensitive), como análisis de flujo de datos (dataflow) hacia adelante y hacia atrás
- Análisis sensible a la ruta (path-sensitive), como verificación de modelos de software (software model checking) y verificación de software
- Herramientas y frameworks para implementar analizadores
- Papel del análisis estático en la optimización de programas y el análisis de dependencia de datos durante la explotación de concurrencia
- Papel del análisis de programas en la verificación (parcial) y la búsqueda de errores (bug-finding)
- Paralelización: 1. Análisis para auto-paralelización 2. Análisis para detectar errores de concurrencia

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Explicar la diferencia entre grafo de flujo de datos (dataflow graph) y grafo de flujo de control [Explicar]
2. Explicar por qué los análisis de programa no triviales y sólidos (sound) deben ser aproximados [Explicar]
3. Argumentar por qué un análisis es correcto (sound y terminante) [Argumentar]
4. Explicar por qué el alias potencial limita el análisis de programa sólido y cómo el análisis de alias puede ayudar [Explicar]
5. Usar los resultados de un análisis de programa para optimización de programas y/o corrección parcial del programa [Usar]

3.5.15. FPL/Generación de Código

Temas:
Non Core

- Conjuntos de instrucciones
- Flujo de control
- Gestión de memoria
- Llamadas a procedimientos y despacho de métodos (method dispatching)
- Compilación separada; enlace (linking)
- Selección de instrucciones
- Planificación de instrucciones (por ejemplo, encadenamiento - pipelining)
- Asignación de registros
- Optimización de código como una forma de análisis de programas
- Generación de programas mediante IA generativa

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Identificar todos los pasos esenciales para convertir automáticamente código fuente en lenguaje ensamblador u otros lenguajes de bajo nivel [Analizar]
2. Explicar el código de bajo nivel necesario para llamar a funciones/métodos en lenguajes modernos [Explicar]
3. Discutir por qué la compilación separada requiere convenciones de llamada uniformes [Debatir]
4. Discutir por qué la compilación separada limita la optimización debido a efectos desconocidos de las llamadas [Debatir]
5. Discutir oportunidades para la optimización introducidas por la traducción ingenua y enfoques para lograr la optimización, como selección de instrucciones, planificación de instrucciones, asignación de registros y optimización de mirilla (peephole optimization) [Debatir]

3.5.16. FPL/Comportamiento en Tiempo de Ejecución y Sistemas

Temas:
Non Core

- Modelos de proceso usando pilas y montones para asignar y desasignar registros de activación y recuperar entornos usando punteros de marco (frame pointers) y direcciones de retorno durante una llamada a procedimiento, incluyendo ejemplos de paso de parámetros
- Esquemas de búsqueda de código usando tablas hash para métodos en implementaciones de programas orientados a objetos
- Distribución de datos para objetos y registros de activación
- Asignación de objetos en el montón (heap)
- Implementar entidades virtuales y métodos virtuales; tablas de métodos virtuales (virtual method tables) y su aplicación
- Comportamiento en tiempo de ejecución de programas orientados a objetos
- Comparar y contrastar la asignación de memoria durante el intercambio de información usando paso de parámetros y variables no locales (usando cadena de enlaces estáticos).
- Enfoques y técnicas de gestión dinámica de memoria: malloc/free, recolección de basura (mark-sweep, copying, reference counting), regiones (también conocidas como arenas o zonas)
- Compilación justo a tiempo (just-in-time) y recompilación dinámica

- Interfaz con el sistema operativo (por ejemplo, para inicialización del programa)
- Interoperabilidad entre lenguajes de programación incluyendo mecanismos de paso de parámetros y representación de datos: 1. *Big endian*, *little endian* 2. Distribución de datos de tipos de datos compuestos como arreglos
- Otras características comunes de las máquinas virtuales, como carga de clases (class loading), hilos (threads) y verificación de seguridad
- Sandboxing

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Discutir beneficios y limitaciones de la gestión automática de memoria [Debatir]
2. Explicar el uso de metadatos en representaciones en tiempo de ejecución de objetos y registros de activación, como punteros de clase, longitudes de arreglo, direcciones de retorno y punteros de marco (frame pointers) [Explicar]
3. Comparar y contrastar asignación estática vs asignación basada en pila vs asignación basada en montón de elementos de datos [Comparar]
4. Explicar por qué algunos elementos de datos no pueden desasignarse automáticamente al final de una llamada a procedimiento/método (necesidad de recolección de basura) [Explicar]
5. Discutir ventajas, desventajas y dificultades de la compilación justo a tiempo y la recompilación dinámica [Debatir]
6. Discutir el uso del sandboxing en código móvil [Debatir]
7. Identificar los servicios proporcionados por los sistemas en tiempo de ejecución de lenguajes modernos [Analizar]

3.5.17. FPL/Construcciones de Programación Avanzadas

Temas:
Non Core

- Mecanismos de encapsulación
- Evaluación diferida (delayed evaluation) y flujos infinitos (infinite streams)
- Comparar y contrastar evaluación diferida vs evaluación ansiosa (eager)
- Unificación vs aserción vs evaluación de expresiones
- Abstracciones de control: manejo de excepciones, continuaciones (continuations), mónadas (monads).
- Abstracciones orientadas a objetos: herencia múltiple, mixins, rasgos (traits), multimétodos (multimethods)
- Metaprogramación: macros, programación generativa, desarrollo basado en modelos (model-based development)
- Manipulación de cadenas mediante coincidencia de patrones (pattern-matching) (expresiones regulares)
- Evaluación dinámica de código ("eval")
- Soporte del lenguaje para verificar aserciones, invariantes y pre/post-condiciones
- Lenguajes específicos del dominio, como lenguajes de bases de datos, lenguajes de ciencia de datos, lenguajes de computación embebida, lenguajes síncronos, lenguajes de interfaz de hardware

- Modelos y lenguajes de computación de alto rendimiento masivamente paralelos

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Usar varias construcciones e idiomas de programación avanzados correctamente [Usar]
2. Discutir cómo varias construcciones de programación avanzadas tienen como objetivo mejorar la estructura del programa, la calidad del software y la productividad del programador [Debatir]
3. Discutir cómo varias construcciones de programación avanzadas interactúan con la definición e implementación de otras características del lenguaje [Debatir]

3.5.18. FPL/Pragmática del Lenguaje

Temas:
Non Core

- Efecto de las necesidades tecnológicas y los requisitos de software en el desarrollo y evolución de los lenguajes de programación
- Dominios de problemas y paradigma de programación
- Criterios para un buen diseño de lenguaje de programación: 1. Principios de diseño de lenguajes como la ortogonalidad 2. Definir construcciones de control e iteración 3. Modularización de software grande
- Orden de evaluación, precedencia y asociatividad
- Evaluación ansiosa vs diferida
- Definir construcciones de control e iteración
- Llamadas externas y bibliotecas del sistema

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Discutir el papel de conceptos como la ortogonalidad y valores por defecto bien elegidos en el diseño de lenguajes [Debatir]
2. Evaluar objetivamente y justificar decisiones de diseño de lenguaje [Evaluar]
3. Implementar un programa de ejemplo cuyo resultado pueda diferir bajo diferentes reglas para el orden de evaluación, precedencia o asociatividad [Implementar]
4. Ilustrar usos de la evaluación diferida, como abstracciones de control definidas por el usuario [Aplicar]
5. Discutir la necesidad de permitir llamadas a llamadas externas y bibliotecas del sistema y las consecuencias para la implementación del lenguaje [Debatir]

3.5.19. FPL/Semántica Formal

Temas:
Non Core

- Sintaxis vs semántica
- Enfoques de semántica: axiomática, operacional, denotacional, basada en tipos
- Semántica axiomática de construcciones abstractas como asignación, selección, iteración usando precondición, postcondiciones e invariante de bucle
- Análisis de semántica operacional de construcciones abstractas y secuencia de tales como asignación, evaluación de expresión, selección, iteración usando entorno y almacén: 1. Ejecución simbólica (symbolic execution) 2. Verificadores de restricciones (constraint checkers)
- Semántica denotacional: 1. Cálculo Lambda.
- Demostraciones por inducción sobre la semántica del lenguaje
- Definiciones y demostraciones formales para sistemas de tipos: 1. Proposiciones como tipos (implicación como función, conjunción como producto, disyunción como suma) 2. Tipos dependientes (cuantificación universal como función dependiente, cuantificación existencial como producto dependiente) 3. Parametricidad

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Construir semántica formal para un lenguaje pequeño [Crear]
2. Escribir un programa de cálculo lambda y mostrar su evaluación a una forma normal [Escribir]
3. Discutir los diferentes enfoques de la semántica operacional, denotacional y axiomática [Debatir]
4. Usar inducción para demostrar propiedades de todos los programas en un lenguaje [Usar]
5. Usar inducción para demostrar propiedades de todos los programas en un lenguaje que está bien tipado según un sistema de tipos definido formalmente [Usar]
6. Usar parametricidad para establecer el comportamiento del código dado solo su tipo [Usar]

3.5.20. FPL/Metodologías de Desarrollo Formal

Temas:
Non Core

- Lenguajes y metodologías de especificación formal
- Demostradores de teoremas (theorem provers), asistentes de prueba (proof assistants) y lógicas
- Verificadores de restricciones (constraint checkers)
- Tipos dependientes (cuantificación universal como función dependiente, cuantificación existencial como producto dependiente)
- Especificación y descarga de pruebas (proof discharge) para sistemas de software completamente verificados usando pre/post condiciones, tipos de refinamiento (refinement types), etc.
- Modelado formal e implementación/refinamiento manual de sistemas de software.
- Uso de pruebas simbólicas (symbolic testing) y fuzzing en el desarrollo de software.
- Verificación de modelos (model checking).
- Comprensión de situaciones donde los métodos formales pueden aplicarse efectivamente y cómo estructurar el desarrollo para maximizar su valor.

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Usar técnicas de modelado formal para desarrollar y validar arquitecturas [Usar]
2. Usar lenguajes de programación asistidos por pruebas (proof assisted) para desarrollar artefactos de software completamente especificados y verificados [Usar]
3. Usar soporte de verificador y especificación en lenguajes de programación para validar formalmente propiedades del sistema [Usar]
4. Integrar herramientas de validación simbólica en un flujo de trabajo de programación [Integrar]
5. Discutir cuándo y cómo los métodos formales pueden usarse efectivamente en el proceso de desarrollo [Debatir]

3.5.21. FPL/Principios de Diseño de Lenguajes de Programación

Temas:
Non Core

- Principios de diseño de lenguajes: 1. Simplicidad 2. Seguridad 3. Traducción rápida 4. Código objeto eficiente 5. Ortogonalidad 6. Legibilidad 7. Completitud 8. Estrategias de implementación

- Diseñar un lenguaje para ajustarse a un dominio o problema específico
- Interoperabilidad entre lenguajes de programación
- Portabilidad del lenguaje
- Descripción formal de un lenguaje de programación
- Principios de computación verde (green computing)

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Entender qué constituye un buen diseño de lenguaje y aplicar ese conocimiento para evaluar un lenguaje de programación real [Evaluar]

3.5.22. FPL/Sociedad, Ética y Profesión

Temas:
Non Core

- Impacto de los lenguajes de programación centrados en inglés
- Mejorar la accesibilidad e inclusión para personas con discapacidades - Compatibilidad con tecnologías de asistencia
- Factores humanos relacionados con los lenguajes de programación y la usabilidad: 1. Impacto de la sintaxis en la accesibilidad 2. Compatibilidad con diferencias culturales (por ejemplo, moneda, decimales, fechas) 3. Neurodiversidad
- Etimología de términos como ”clase” (class), ”maestro” (master) y ”esclavo” (slave) en lenguajes de programación
- Aumentar la accesibilidad mediante el soporte de múltiples idiomas dentro de las aplicaciones (UTF)

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Diseñar conscientemente lenguajes de programación para que sean inclusivos y no ofensivos [Diseñar]

3.6. Gráficos y Técnicas Interactivas (GIT)

Los Gráficos y Técnicas Interactivas (GIT) engloban los principios, algoritmos y tecnologías involucrados en el renderizado, modelado, animación e interacción con contenido visual e interactivo. Esta área de conocimiento integra conceptos de gráficos por computadora, interacción humano-computadora, visualización y procesamiento de imágenes para crear experiencias digitales inmersivas y efectivas. Con la proliferación de aplicaciones interactivas en entretenimiento, educación, visualización científica e interfaces de usuario, GIT juega un papel crucial en la computación moderna. Los estudiantes que estudian GIT aprenden a diseñar, implementar y evaluar sistemas que generan contenido visual, facilitan la interacción del usuario y aprovechan tecnologías emergentes como la realidad virtual, la realidad aumentada y la computación tangible. El área enfatiza tanto los fundamentos teóricos como las habilidades prácticas, preparando a los graduados para contribuir en diversos campos, incluyendo el desarrollo de videojuegos, simulación, visualización de datos y medios interactivos.

Unidad de Conocimiento (KU)	**Core Tier1**	**Core Tier2**	**Alternativo**
3.6.1 Conceptos Fundamentales de Gráficos y Técnicas Interactivas (Pág. 79)			No
3.6.2 Visualización (Pág. 80)			No
3.6.3 Renderizado Aplicado y Técnicas (Pág. 80)			No
3.6.4 Modelado Geométrico (Pág. 80)			No
3.6.5 Sombreado y Renderizado Avanzado (Pág. 81)			No
3.6.6 Animación por Computadora (Pág. 81)			No
3.6.7 Simulación (Pág. 81)			No
3.6.8 Inmersión (Pág. 81)			No
3.6.9 Interacción (Pág. 82)			No
3.6.10 Procesamiento de Imágenes (Pág. 82)			No
3.6.11 Computación Tangible/Física (Pág. 82)			No
3.6.12 Sociedad, Ética y Profesión (Pág. 83)			No

3.6.1. GIT/Conceptos Fundamentales de Gráficos y Técnicas Interactivas

Temas:
CS Core

- Descripción general del pipeline de gráficos por computadora y sistemas interactivos
- Conceptos básicos de renderizado: rasterización, trazado de rayos (*ray tracing*), modelos de sombreado
- Teoría del color y espacios de color
- Sistemas de coordenadas y transformaciones
- Técnicas básicas de interacción: dispositivos de entrada, manejo de eventos

KA Core

- Conceptos avanzados de renderizado: iluminación global (*global illumination*), renderizado basado en física
- Hardware gráfico y técnicas de aceleración
- Percepción humana y cognición visual

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Explicar las etapas básicas del pipeline de gráficos por computadora [Explicar]
2. Describir técnicas básicas de renderizado como rasterización y trazado de rayos [Describir]
3. Aplicar teoría del color para diseñar gráficos visualmente efectivos [Aplicar]
4. Implementar transformaciones de coordenadas para gráficos 2D y 3D [Implementar]
5. Diseñar aplicaciones interactivas básicas utilizando dispositivos de entrada comunes y manejo de eventos [Diseñar]

KACore:

6. Comparar técnicas avanzadas de renderizado como iluminación global y renderizado basado en física [Comparar]
7. Analizar el rol del hardware gráfico en acelerar tareas de renderizado [Analizar]
8. Explicar cómo la percepción humana influye en el diseño de gráficos e interacción [Explicar]

3.6.2. GIT/Visualización

Temas:

KA Core

- Principios de visualización de datos: percepción, cognición y codificación efectiva
- Tipos de visualizaciones: científicas, de información, geográficas
- Técnicas de visualización para datos multivariados, temporales y jerárquicos
- Visualización interactiva y exploración
- Evaluación de la efectividad de la visualización

Aprendizaje esperado (*Learning Outcomes*):

KACore:

1. Aplicar principios de percepción y cognición para diseñar visualizaciones efectivas [Aplicar]
2. Seleccionar técnicas de visualización apropiadas para diferentes tipos de datos y tareas [Evaluar]
3. Implementar visualizaciones interactivas para exploración de datos [Implementar]
4. Evaluar la efectividad de una visualización usando métodos empíricos [Evaluar]

3.6.3. GIT/Renderizado Aplicado y Técnicas

Temas:

KA Core

- Algoritmos de rasterización: conversión por barrido (*scan conversion*), antialiasing
- Trazado de rayos (*ray tracing*): estructuras de aceleración, muestreo (*sampling*), sombreado
- Mapeo de texturas (*texture mapping*) y filtrado
- Modelos de sombreado: Lambertiano, Phong, BRDF
- Técnicas avanzadas: mapeo de sombras (*shadow mapping*), mapeo ambiental (*environment mapping*), renderizado diferido (*deferred rendering*)

Aprendizaje esperado (*Learning Outcomes*):

KACore:

1. Implementar algoritmos de rasterización para renderizado de primitivas [Implementar]
2. Implementar trazado de rayos básico con estructuras de aceleración [Implementar]
3. Aplicar técnicas de mapeo de texturas y filtrado para mejorar el realismo visual [Aplicar]
4. Comparar diferentes modelos de sombreado y sus aplicaciones [Comparar]
5. Implementar técnicas avanzadas de renderizado como mapeo de sombras o renderizado diferido [Implementar]

3.6.4. GIT/Modelado Geométrico

Temas:

KA Core

- Representaciones geométricas: polígonos, curvas, superficies, formas implícitas
- Técnicas de modelado: curvas spline, superficies de subdivisión, geometría sólida constructiva
- Modelado procedural y generación
- Algoritmos geométricos: intersección, distancia, descomposición

Aprendizaje esperado (*Learning Outcomes*):

KACore:

1. Describir diferentes representaciones geométricas y sus compensaciones (trade-offs) [Describir]
2. Implementar técnicas de modelado como curvas spline o superficies de subdivisión [Implementar]
3. Crear modelos procedurales usando algoritmos o reglas [Crear]
4. Aplicar algoritmos geométricos para resolver problemas como pruebas de intersección o procesamiento de mallas (mesh) [Aplicar]

3.6.5. GIT/Sombreado y Renderizado Avanzado

Temas:
KA Core

- Sombreado avanzado: sombreado basado en física, dispersión subsuperficial (subsurface scattering)
- Iluminación global (global illumination): radiosidad, mapeo de fotones (photon mapping), trazado de caminos (path tracing)
- Técnicas de renderizado en tiempo real: nivel de detalle (level of detail), culling, oclusión
- Programación GPU y sombreadores (shaders)

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Implementar modelos de sombreado avanzados para materiales realistas [Implementar]
2. Comparar algoritmos de iluminación global y sus compensaciones [Comparar]
3. Optimizar el renderizado en tiempo real usando técnicas como nivel de detalle y *culling* [Crear]
4. Desarrollar sombreadores GPU para efectos de renderizado personalizados [Crear]

3.6.6. GIT/Animación por Computadora

Temas:
KA Core

- Principios de animación: tiempo (timing), espaciado (spacing), anticipación, seguimiento (follow-through)
- Animación por fotogramas clave (keyframe) e interpolación
- Animación procedural: sistemas de partículas, comportamiento de bandadas (flocking), basada en física
- Animación de personajes: animación esquelética, skinning, cinemática inversa
- Captura de movimiento (motion capture) y retargeting

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Aplicar principios de animación para crear movimiento creíble [Aplicar]
2. Implementar animación por fotogramas clave con técnicas de interpolación [Implementar]
3. Crear animaciones procedurales como sistemas de partículas o simulaciones basadas en física [Crear]
4. Implementar animación de personajes usando sistemas esqueléticos y skinning [Implementar]
5. Procesar datos de captura de movimiento para retargeting de animación [Implementar]

3.6.7. GIT/Simulación

Temas:
KA Core

- Simulación basada en física: cuerpos rígidos, fluidos, tela
- Métodos numéricos para simulación: integración, detección de colisiones
- Restricciones y optimizaciones de simulación en tiempo real
- Aplicaciones en videojuegos, efectos visuales y visualización científica

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Implementar simulaciones basadas en física para cuerpos rígidos o fluidos [Implementar]
2. Aplicar métodos de integración numérica para simulaciones dinámicas [Aplicar]
3. Optimizar simulaciones para rendimiento en tiempo real [Crear]
4. Desarrollar simulaciones para aplicaciones específicas como videojuegos o visualización científica [Crear]

3.6.8. GIT/Inmersión

Temas:
KA Core

- Conceptos de realidad virtual (VR) y realidad aumentada (AR)
- Tecnologías de visualización inmersiva: pantallas montadas en la cabeza (head-mounted displays), CAVEs
- Seguimiento (tracking) y entrada para entornos inmersivos
- Presencia y comodidad en sistemas inmersivos

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Explicar los conceptos centrales y diferencias entre VR y AR [Explicar]
2. Evaluar tecnologías de visualización inmersiva y sus aplicaciones [Evaluar]
3. Implementar soluciones de seguimiento y entrada para entornos inmersivos [Implementar]
4. Diseñar experiencias inmersivas que maximicen la presencia y minimicen la incomodidad [Diseñar]

3.6.9. GIT/Interacción

Temas:
KA Core

- Paradigmas de interacción: manipulación directa, basada en gestos, tangible
- Diseño de interfaz de usuario para aplicaciones gráficas
- Interacción multimodal: combinando retroalimentación visual, auditiva, háptica
- Evaluación de técnicas de interacción: usabilidad, experiencia de usuario

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Comparar diferentes paradigmas de interacción y su idoneidad para varias tareas [Comparar]
2. Diseñar interfaces de usuario para aplicaciones intensivas en gráficos [Diseñar]
3. Implementar sistemas de interacción multimodal incorporando múltiples canales de retroalimentación [Implementar]
4. Realizar evaluaciones de usabilidad de técnicas de interacción [Evaluar]

3.6.10. GIT/Procesamiento de Imágenes

Temas:
KA Core

- Representación y formatos de imagen
- Filtrado y mejora: convolución, dominio de frecuencia
- Segmentación de imágenes y detección de características
- Operaciones morfológicas
- Aplicaciones en gráficos, visión y multimedia

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Manipular imágenes usando diferentes representaciones y formatos [Procesar]
2. Aplicar técnicas de filtrado para mejora y análisis de imágenes [Aplicar]
3. Implementar algoritmos de segmentación de imágenes para extracción de características [Implementar]
4. Usar operaciones morfológicas para tareas de procesamiento de imágenes [Usar]
5. Desarrollar aplicaciones que integren procesamiento de imágenes con sistemas de gráficos o visión [Crear]

3.6.11. GIT/Computación Tangible/Física

Temas:
KA Core

- Interfaces tangibles y computación física
- Sensores y actuadores para sistemas interactivos
- Sistemas embebidos para arte y diseño interactivo
- Instalaciones y entornos interactivos

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Diseñar interfaces tangibles que unan interacciones físicas y digitales [Diseñar]
2. Integrar sensores y actuadores para crear sistemas responsivos [Integrar]
3. Desarrollar sistemas embebidos para proyectos de arte o diseño interactivo [Crear]
4. Crear instalaciones interactivas que involucren a los usuarios en espacios físicos [Crear]

3.6.12. GIT/Sociedad, Ética y Profesión

Temas:
KA Core

- Consideraciones éticas en gráficos e interacción: representación, accesibilidad, privacidad
- Impacto social de las tecnologías inmersivas y la visualización
- Práctica profesional en industrias de gráficos e interactivas

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Analizar problemas éticos relacionados con representación, accesibilidad y privacidad en gráficos e interacción [Analizar]
2. Discutir las implicaciones sociales de las tecnologías inmersivas y la visualización de datos [Debatir]
3. Describir prácticas y estándares profesionales en industrias de gráficos e interactivas [Describir]

3.7. Interacción Humano-Computadora (HCI)

La Interacción Humano-Computadora (HCI) es un campo multidisciplinario que se centra en el diseño, evaluación e implementación de sistemas informáticos interactivos para uso humano, y el estudio de los principales fenómenos que los rodean. HCI integra conocimientos y métodos de la informática, psicología, diseño, sociología y otros campos para crear tecnologías que sean útiles, usables y satisfactorias para las personas. El objetivo central de HCI es mejorar las interacciones entre usuarios y computadoras haciendo los sistemas más usables, accesibles y eficientes, considerando al mismo tiempo las capacidades y limitaciones humanas.

En el mundo actual, donde la tecnología informática impregna todos los aspectos de la vida diaria—desde teléfonos inteligentes y sitios web hasta dispositivos médicos y sistemas de hogares inteligentes—los principios de HCI son esenciales para crear sistemas que las personas puedan usar de manera efectiva y segura. HCI enfatiza procesos de diseño centrados en el usuario, prototipado iterativo y evaluación empírica para asegurar que los sistemas satisfagan las necesidades reales de los usuarios. El campo aborda diversos temas que incluyen usabilidad, experiencia de usuario (UX), accesibilidad, diseño de interacción, métodos de investigación de usuarios y las implicaciones sociales de la tecnología.

HCI prepara a los estudiantes para diseñar, implementar y evaluar sistemas interactivos que no solo sean funcionales sino que también consideren factores humanos, implicaciones éticas e impacto social. Los graduados con experiencia en HCI trabajan en roles como diseñadores de UX, diseñadores de interacción, ingenieros de usabilidad y gerentes de productos en diversas industrias, incluyendo desarrollo de software, atención médica, educación, entretenimiento y políticas públicas.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
3.7.1 Comprender al Usuario: Objetivos individuales e interacciones con otros (Pág. 84)			No
3.7.2 Responsabilidad y Rendición de Cuentas en el Diseño (Pág. 84)			No
3.7.3 Accesibilidad y Diseño Inclusivo (Pág. 84)			No
3.7.4 Evaluación del Diseño (Pág. 85)			No
3.7.5 Diseño de Sistemas (Pág. 85)			No
3.7.6 Sociedad, Ética y Profesión (Pág. 86)			No

3.7.1. HCI/Comprender al Usuario: Objetivos individuales e interacciones con otros

Temas:
CS Core
- Capacidades y limitaciones humanas: percepción, cognición, habilidades motoras
- Características del usuario: demografía, experiencia, contexto cultural
- Objetivos, tareas y contextos de uso del usuario
- Aspectos sociales y organizacionales de la interacción

KA Core
- Modelos y teorías cognitivas relevantes para HCI
- Aspectos emocionales y afectivos de la interacción
- Métodos de investigación de usuarios: entrevistas, encuestas, observaciones

Aprendizaje esperado (*Learning Outcomes*):
CSCore:
1. Describir las capacidades y limitaciones humanas que afectan el diseño de interacción [Describir]
2. Analizar cómo las características del usuario influyen en la interacción con sistemas informáticos [Analizar]
3. Identificar objetivos y tareas del usuario en contextos de uso específicos [Analizar]
4. Explicar factores sociales y organizacionales que impactan la adopción y uso de la tecnología [Explicar]

KACore:
5. Aplicar modelos cognitivos para predecir y explicar el comportamiento del usuario [Aplicar]
6. Considerar factores emocionales y afectivos en el diseño de interacción [Analizar]
7. Realizar investigación de usuarios usando métodos apropiados como entrevistas u observaciones [Evaluar]

3.7.2. HCI/Responsabilidad y Rendición de Cuentas en el Diseño

Temas:
CS Core
- Consideraciones éticas en HCI: equidad, transparencia, rendición de cuentas
- Diseño para valores y diseño sensible a valores
- Innovación responsable y ética del diseño

KA Core
- Rendición de cuentas algorítmica y sesgo en sistemas interactivos
- Responsabilidad profesional en la práctica de HCI
- Consideraciones legales y regulatorias para sistemas interactivos

Aprendizaje esperado (*Learning Outcomes*):
CSCore:
1. Identificar problemas éticos en el diseño e implementación de HCI [Analizar]
2. Aplicar enfoques de diseño sensible a valores al desarrollo tecnológico [Aplicar]
3. Discutir prácticas de innovación responsable en HCI [Debatir]

KACore:
4. Analizar la rendición de cuentas algorítmica y el sesgo en sistemas interactivos [Analizar]
5. Describir responsabilidades profesionales en la práctica de HCI [Describir]
6. Explicar consideraciones legales y regulatorias relevantes para sistemas interactivos [Explicar]

3.7.3. HCI/Accesibilidad y Diseño Inclusivo

Temas:
CS Core
- Principios y pautas de accesibilidad
- Discapacidades y tecnologías de asistencia
- Diseño inclusivo y principios de diseño universal

KA Core
- Métodos de evaluación para accesibilidad
- Diseñar para diversas capacidades y contextos

- Estándares de accesibilidad y cumplimiento

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Aplicar principios de accesibilidad al diseño de sistemas interactivos [Aplicar]
2. Describir cómo diferentes discapacidades afectan la interacción con la tecnología [Describir]
3. Diseñar sistemas inclusivos usando principios de diseño universal [Diseñar]

KACore:

4. Realizar evaluaciones de accesibilidad de sistemas interactivos [Evaluar]
5. Crear diseños que acomoden diversas capacidades y contextos de uso [Crear]
6. Garantizar el cumplimiento con estándares y regulaciones de accesibilidad [Evaluar]

3.7.4. HCI/Evaluación del Diseño

Temas:
CS Core

- Principios y heurísticas de usabilidad
- Métodos de evaluación: evaluación heurística, recorrido cognitivo (cognitive walkthrough)
- Pruebas con usuarios: planificación, ejecución, análisis

KA Core

- Métodos avanzados de evaluación: experimentos controlados, estudios de campo
- Métricas y medición de UX
- Métodos de evaluación analítica: modelado y simulación

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Aplicar principios y heurísticas de usabilidad para evaluar diseños [Aplicar]
2. Realizar evaluaciones heurísticas y recorridos cognitivos [Evaluar]
3. Planificar, conducir y analizar sesiones de pruebas con usuarios [Planear]

KACore:

4. Diseñar y conducir experimentos controlados para investigación en HCI [Diseñar]
5. Medir la experiencia de usuario usando métricas apropiadas [Evaluar]
6. Usar métodos analíticos para modelar y predecir el desempeño del usuario [Usar]

3.7.5. HCI/Diseño de Sistemas

Temas:
CS Core

- Proceso de diseño: diseño centrado en el usuario, diseño iterativo
- Diseño de interacción: conceptos, principios, patrones
- Técnicas de prototipado: baja fidelidad, alta fidelidad, interactivo
- Arquitectura de información y diseño de navegación

KA Core

- Técnicas avanzadas de interacción: gestos, voz, multimodal
- Diseño para dominios específicos: móvil, web, computación ubicua
- Sistemas de computación colaborativa y social

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Aplicar procesos de diseño centrado en el usuario para desarrollar sistemas interactivos [Aplicar]
2. Diseñar interacciones usando principios y patrones establecidos [Diseñar]
3. Crear prototipos en diferentes niveles de fidelidad para exploración de diseño [Crear]
4. Estructurar información y navegación para una interacción efectiva del usuario [Estructurar]

KACore:

5. Implementar técnicas avanzadas de interacción como interfaces de gestos o voz [Implementar]
6. Diseñar sistemas interactivos para dominios y plataformas específicos [Diseñar]
7. Diseñar sistemas colaborativos que apoyen la interacción grupal [Diseñar]

3.7.6. HCI/Sociedad, Ética y Profesión

Temas:
CS Core
- Impacto social de las tecnologías interactivas
- Problemas éticos en la investigación y práctica de HCI
- Práctica profesional en HCI

KA Core
- Consideraciones culturales e interculturales en HCI
- Sostenibilidad y HCI
- Tendencias futuras y desafíos en HCI

Aprendizaje esperado (*Learning Outcomes*):
CSCore:
1. Analizar el impacto social de las tecnologías interactivas en individuos y sociedad [Analizar]
2. Abordar problemas éticos en proyectos de investigación y diseño de HCI [Evaluar]
3. Describir prácticas profesionales y trayectorias profesionales en HCI [Describir]

KACore:
4. Diseñar sistemas que tomen en cuenta diferencias y contextos culturales [Diseñar]
5. Incorporar consideraciones de sostenibilidad en el diseño de HCI [Integrar]
6. Discutir tendencias emergentes y desafíos futuros en HCI [Debatir]

3.8. Fundamentos Matemáticos y Estadísticos (MSF)

Los fundamentos matemáticos y estadísticos de la informática proporcionan las bases teóricas para la disciplina. Incluyen matemáticas discretas, probabilidad, estadística, álgebra lineal y cálculo. Estas áreas son esenciales para comprender algoritmos, estructuras de datos, aprendizaje automático, gráficos, criptografía y muchos otros temas en informática.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
3.8.1 Matemáticas Discretas (Pág. 86)			No
3.8.2 Probabilidad (Pág. 87)			No
3.8.3 Estadística (Pág. 87)			No
3.8.4 Álgebra Lineal (Pág. 88)			No
3.8.5 Cálculo (Pág. 89)			No

3.8.1. MSF/Matemáticas Discretas

Temas:
CS Core
- Conjuntos: operaciones, cardinalidad, conjuntos potencia, productos cartesianos
- Relaciones: propiedades, relaciones de equivalencia, órdenes parciales
- Funciones: inyectivas, sobreyectivas, biyectivas, composición, inversa
- Lógica: lógica proposicional y de predicados, cuantificadores, demostraciones
- Grafos: definiciones, caminos, ciclos, árboles, conectividad
- Conteo: permutaciones, combinaciones, teorema del binomio
- Relaciones de recurrencia: resolver recurrencias simples

KA Core
- Teoría avanzada de conjuntos: conjuntos infinitos, argumento diagonal de Cantor
- Relaciones avanzadas: clausuras, retículos
- Funciones avanzadas: funciones recursivas, computabilidad
- Lógica avanzada: sistemas de demostración, completitud, decidibilidad
- Teoría avanzada de grafos: planaridad, coloración, redes
- Conteo avanzado: funciones generadoras, principio de inclusión-exclusión
- Recurrencia avanzada: funciones generadoras, comportamiento asintótico

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Definir operaciones y relaciones básicas de conjuntos [Explicar]
2. Aplicar operaciones de conjuntos para resolver problemas [Aplicar]
3. Definir propiedades de las relaciones [Explicar]
4. Determinar si una relación es una relación de equivalencia [Aplicar]
5. Definir tipos de funciones [Explicar]
6. Componer funciones y encontrar inversas [Aplicar]
7. Explicar lógica proposicional y de predicados [Explicar]
8. Construir demostraciones lógicas simples [Aplicar]
9. Definir conceptos básicos de grafos [Explicar]
10. Resolver problemas básicos de grafos [Aplicar]
11. Explicar permutaciones y combinaciones [Explicar]
12. Aplicar técnicas de conteo para resolver problemas [Aplicar]
13. Definir relaciones de recurrencia [Explicar]
14. Resolver relaciones de recurrencia simples [Aplicar]

KACore:

15. Explicar conceptos de conjuntos infinitos y cardinalidad [Explicar]
16. Aplicar operaciones de clausura a relaciones [Aplicar]
17. Analizar propiedades avanzadas de grafos [Evaluar]
18. Desarrollar funciones generadoras para problemas de conteo [Crear]

3.8.2. MSF/Probabilidad

Temas:
CS Core

- Probabilidad básica: espacios muestrales, eventos, axiomas
- Probabilidad condicional, independencia, teorema de Bayes
- Variables aleatorias: discretas y continuas, distribuciones
- Esperanza, varianza, desviación estándar
- Distribuciones comunes: uniforme, binomial, normal

KA Core

- Distribuciones conjuntas, covarianza, correlación
- Teoremas límite: ley de los grandes números, teorema del límite central
- Procesos estocásticos: cadenas de Markov
- Teoría de la información: entropía, información mutua

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Definir conceptos básicos de probabilidad [Explicar]
2. Calcular probabilidades de eventos [Aplicar]
3. Definir probabilidad condicional e independencia [Explicar]
4. Aplicar el teorema de Bayes para actualizar probabilidades [Aplicar]
5. Definir variables aleatorias y distribuciones [Explicar]
6. Calcular esperanza y varianza [Aplicar]
7. Describir distribuciones de probabilidad comunes [Explicar]
8. Aplicar distribuciones comunes para modelar fenómenos del mundo real [Aplicar]

KACore:

9. Explicar distribuciones conjuntas y dependencia [Explicar]
10. Calcular covarianza y correlación [Aplicar]
11. Analizar las implicaciones de los teoremas límite [Evaluar]
12. Modelar sistemas simples usando cadenas de Markov [Crear]

3.8.3. MSF/Estadística

Temas:
CS Core

- Estadística descriptiva: medidas de centro y dispersión
- Visualización de datos: histogramas, diagramas de caja, diagramas de dispersión
- Distribuciones muestrales: variabilidad muestral, teorema del límite central
- Estimación: estimaciones puntuales, intervalos de confianza
- Prueba de hipótesis: hipótesis nula y alternativa, valores p

KA Core

- Regresión: regresión lineal simple y múltiple
- Análisis de varianza (ANOVA)
- Métodos no paramétricos
- Diseño experimental: aleatorización, control, enmascaramiento

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Definir medidas de estadística descriptiva [Explicar]
2. Calcular estadísticas descriptivas para un conjunto de datos [Aplicar]
3. Explicar técnicas comunes de visualización de datos [Explicar]
4. Crear visualizaciones apropiadas para datos dados [Aplicar]
5. Definir distribuciones muestrales y variabilidad [Explicar]
6. Aplicar el teorema del límite central para aproximar distribuciones [Aplicar]
7. Definir estimaciones puntuales e intervalos de confianza [Explicar]
8. Calcular intervalos de confianza para medias y proporciones [Aplicar]
9. Definir conceptos de prueba de hipótesis [Explicar]
10. Realizar una prueba de hipótesis simple e interpretar valores p [Aplicar]

KACore:

11. Explicar regresión lineal simple y múltiple [Explicar]
12. Ajustar un modelo de regresión e interpretar coeficientes [Aplicar]
13. Analizar varianza usando ANOVA [Evaluar]
14. Diseñar un experimento simple con controles apropiados [Crear]

3.8.4. MSF/Álgebra Lineal

Temas:

CS Core

- Vectores: operaciones, producto punto, producto cruz
- Matrices: operaciones, inversas, determinantes
- Sistemas de ecuaciones lineales: eliminación gaussiana, forma matricial
- Espacios vectoriales: subespacios, base, dimensión
- Transformaciones lineales: representación matricial, núcleo, imagen

KA Core

- Valores propios y vectores propios: diagonalización
- Ortogonalidad: bases ortonormales, Gram-Schmidt
- Descomposición en valores singulares (SVD)
- Aplicaciones: mínimos cuadrados, gráficos por computadora, aprendizaje automático

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Definir operaciones y propiedades de vectores [Explicar]
2. Calcular operaciones y productos vectoriales [Aplicar]
3. Definir operaciones y propiedades matriciales [Explicar]
4. Calcular operaciones matriciales, inversas y determinantes [Aplicar]
5. Explicar sistemas de ecuaciones lineales y métodos de solución [Explicar]
6. Resolver sistemas de ecuaciones lineales usando eliminación [Aplicar]
7. Definir espacios vectoriales, subespacios, base y dimensión [Explicar]
8. Determinar si un conjunto de vectores forma una base [Aplicar]
9. Definir transformaciones lineales y sus representaciones [Explicar]
10. Encontrar el núcleo y la imagen de una transformación lineal [Aplicar]

KACore:

11. Explicar valores propios y vectores propios [Explicar]

12. Diagonalizar una matriz cuando sea posible [Aplicar]
13. Analizar ortogonalidad y bases ortonormales [Evaluar]
14. Aplicar descomposición en valores singulares a una matriz [Crear]

3.8.5. MSF/Cálculo

Temas:
CS Core
- Límites y continuidad
- Derivadas: definición, reglas, aplicaciones
- Integrales: definidas e indefinidas, teorema fundamental
- Series: convergencia, series de Taylor

KA Core
- Cálculo multivariable: derivadas parciales, gradientes
- Optimización: extremos locales y globales, multiplicadores de Lagrange
- Ecuaciones diferenciales: técnicas básicas de solución
- Métodos numéricos: integración, diferenciación

Aprendizaje esperado (*Learning Outcomes*):
CSCore:
1. Definir límites y continuidad [Explicar]
2. Calcular límites de funciones [Aplicar]
3. Definir derivadas y reglas de diferenciación [Explicar]
4. Calcular derivadas de funciones [Aplicar]
5. Definir integrales y el teorema fundamental del cálculo [Explicar]
6. Calcular integrales definidas e indefinidas [Aplicar]
7. Definir series y convergencia [Explicar]
8. Calcular expansiones en series de Taylor [Aplicar]

KACore:
9. Explicar derivadas parciales y gradientes [Explicar]
10. Calcular derivadas parciales y gradientes [Aplicar]
11. Analizar problemas de optimización usando cálculo [Evaluar]
12. Resolver ecuaciones diferenciales simples [Crear]

3.9. Redes y Comunicaciones (NC)

Las redes y comunicaciones juegan un papel central en los sistemas informáticos interconectados que están transformando la vida diaria de miles de millones de personas. Internet público proporciona conectividad para aplicaciones en red que sirven a un número cada vez mayor de individuos y organizaciones en todo el mundo. Complementando al sector público, las principales redes propietarias aprovechan su alcance global para apoyar la computación, el almacenamiento y la entrega de contenido distribuidos rentables. Los avances en las redes satelitales amplían la conectividad a las zonas rurales. La comunicación dispositivo a dispositivo subyace al emergente Internet de las Cosas.

Esta área de conocimiento trata conceptos clave en redes y comunicaciones, así como sus instanciaciones representativas en Internet y otras redes informáticas. Además de los principios básicos de conmutación y estratificación, el área en su núcleo proporciona conocimiento sobre denominación, direccionamiento, fiabilidad, control de errores, control de flujo, control de congestión, jerarquía de dominios, enrutamiento, reenvío, modulación, codificación, entramado y control de acceso. El área también cubre unidades de conocimiento en seguridad de red y movilidad, como amenazas de seguridad, contramedidas, comunicación dispositivo a dispositivo y redes inalámbricas multiescalón. Además de los principios fundamentales, el área incluye su realización específica en Internet, así como habilidades prácticas en la implementación de conceptos de redes y comunicaciones. Finalmente, el área comprende temas emergentes como la virtualización de red y las redes cuánticas.

Como resultado de aprendizaje principal, los estudiantes desarrollan una comprensión exhaustiva del papel y funcionamiento de las redes y comunicaciones en sistemas informáticos en red. Aprenden cómo la estructura de red y los protocolos de comunicación afectan el comportamiento de las

aplicaciones distribuidas. El área puede usarse para educar no solo los principios clave sino también sus instanciaciones específicas en Internet y equipar al estudiante con habilidades prácticas de implementación. Si bien las tecnologías de sistemas informáticos, redes y comunicaciones avanzan a un ritmo acelerado, el conocimiento fundamental adquirido permite al estudiante aplicar fácilmente los conceptos en nuevos entornos tecnológicos.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
3.9.1 Fundamentos de Redes y Comunicaciones (Pág. 90)			No
3.9.2 Aplicaciones en Red (Pág. 90)			No
3.9.3 Soporte de Fiabilidad (Pág. 91)			No
3.9.4 Enrutamiento y Reenvío (Pág. 91)			No
3.9.5 Comunicación de Un Solo Salto (Pág. 91)			No
3.9.6 Seguridad de Red (Pág. 92)			No
3.9.7 Soporte de Movilidad (Pág. 92)			No
3.9.8 Temas Emergentes (Pág. 92)			No

3.9.1. NC/Fundamentos de Redes y Comunicaciones

Temas:
CS Core

- Importancia de las redes en la informática contemporánea, y desafíos asociados. Ver también: Contexto Social, Privacidad y Libertades Civiles
- Organización de Internet (por ejemplo, usuarios, Proveedores de Servicios de Internet, sistemas autónomos, proveedores de contenido, redes de entrega de contenido)
- Técnicas de conmutación (por ejemplo, circuito y paquete)
- Capas y sus roles (aplicación, transporte, red, enlace de datos y física)
- Principios de estratificación (por ejemplo, encapsulación y modelo de reloj de arena). Ver también: Conceptos Básicos
- Elementos de red (por ejemplo, routers, switches, hubs, puntos de acceso y hosts)
- Conceptos básicos de colas (por ejemplo, relación con latencia, congestión, niveles de servicio, etc.)

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Articular la organización de Internet [Articular]
2. Listar y definir la terminología de red apropiada [Listar/Enumerar]
3. Describir la estructura en capas de una arquitectura de red típica [Describir]
4. Identificar los diferentes tipos de complejidad en una red (bordes, núcleo, etc.) [Analizar]

3.9.2. NC/Aplicaciones en Red

Temas:
CS Core

- Esquemas de denominación y direccionamiento (por ejemplo, DNS e Identificadores Uniformes de Recursos)
- Paradigmas de aplicaciones distribuidas (por ejemplo, cliente/servidor, peer-to-peer, nube, edge y fog). Ver también: Comunicación, Coordinación
- Diversidad de demandas de aplicaciones en red (por ejemplo, latencia, ancho de banda y tolerancia a pérdidas). Ver también: Comunicación, Sostenibilidad, Contexto Social
- Cobertura de protocolos de capa de aplicación (por ejemplo, HTTP)
- Interacciones con APIs de TCP, UDP y Socket. Ver también: Programas

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Definir los principios de denominación, direccionamiento y localización de recursos [Definir]
2. Analizar las necesidades de demandas específicas de aplicaciones en red [Analizar]
3. Describir los detalles de un protocolo de capa de aplicación [Describir]

4. Implementar una aplicación simple cliente-servidor basada en sockets [Implementar]

3.9.3. NC/Soporte de Fiabilidad

Temas:
KA Core

- Entrega no fiable (por ejemplo, UDP)
- Principios de fiabilidad (por ejemplo, entrega sin pérdidas, duplicación o fuera de orden). Ver también: Confiabilidad del Sistema
- Control de errores (por ejemplo, retransmisión, corrección de errores)
- Control de flujo (por ejemplo, parar y esperar, basado en ventana)
- Control de congestión (por ejemplo, notificación implícita y explícita de congestión)
- TCP y problemas de rendimiento (por ejemplo, Tahoe, Reno, Vegas, Cubic)

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Describir el funcionamiento de protocolos de entrega fiable [Describir]
2. Listar los factores que afectan el rendimiento de los protocolos de entrega fiable [Listar/Enumerar]
3. Describir algunos problemas de diseño de fiabilidad de TCP [Describir]
4. Diseñar e implementar un protocolo fiable simple [Diseñar]

3.9.4. NC/Enrutamiento y Reenvío

Temas:
KA Core

- Paradigmas y jerarquía de enrutamiento (por ejemplo, intra/inter dominio, centralizado y descentralizado, enrutamiento de origen, circuitos virtuales, QoS)
- Métodos de reenvío (por ejemplo, tablas de reenvío y algoritmos de coincidencia)
- IP y problemas de escalabilidad (por ejemplo, NAT, CIDR, BGP, diferentes versiones de IP)

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Describir varios paradigmas y jerarquías de enrutamiento [Describir]
2. Describir cómo se reenvían los paquetes en una red IP [Describir]
3. Describir cómo Internet aborda los desafíos de escalabilidad [Describir]

3.9.5. NC/Comunicación de Un Solo Salto

Temas:
KA Core

- Introducción a modulación, ancho de banda y medios de comunicación
- Codificación y Entramado
- Control de Acceso al Medio (MAC) (por ejemplo, acceso aleatorio y acceso programado)
- Ethernet y WiFi
- Conmutación (por ejemplo, árboles de expansión, VLANs).
- Topologías de Red de Área Local (por ejemplo, centro de datos, redes de campus).

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Describir algunos aspectos básicos de modulación, ancho de banda y medios de comunicación [Describir]
2. Describir en detalle un protocolo MAC [Describir]
3. Demostrar comprensión de las compensaciones (trade-offs) de soluciones de codificación y entramado [Demostrar]
4. Describir detalles de la implementación de Ethernet [Describir]
5. Describir cómo funciona la conmutación [Describir]
6. Describir un tipo de topología de LAN [Describir]

3.9.6. NC/Seguridad de Red

Temas:
KA Core

- Introducción general sobre seguridad (Amenazas, vulnerabilidades y contramedidas). Ver también: Políticas de Seguridad, Leyes y Delitos Informáticos, Fundamentos de Seguridad, Análisis, Diseño e Ingeniería de la Seguridad
- Amenazas específicas de red y tipos de ataque (por ejemplo, denegación de servicio, suplantación, rastreo y redireccionamiento de tráfico, atacante en el medio, ataques de integridad de mensajes, ataques de enrutamiento, ransomware y análisis de tráfico). Ver también: Fundamentos de Seguridad, Análisis, Diseño e Ingeniería de la Seguridad
- Contramedidas: 1. Criptografía (por ejemplo, SSL, TLS, simétrica/asimétrica) 2. Arquitecturas para redes seguras (por ejemplo, canales seguros, protocolos de enrutamiento seguro, DNS seguro, VPNs, DMZ, Zero Trust Network Access, hiper seguridad de red, protocolos de comunicación anónima, aislamiento) 3. Monitoreo de red, detección de intrusiones, firewalls, protección contra suplantación y DoS, honeypots, trazabilidad, BGP Sec, RPKI

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Describir algunos de los modelos de amenaza de seguridad de red [Describir]
2. Describir contramedidas específicas basadas en red [Describir]
3. Analizar varios aspectos de seguridad de red a partir de un estudio de caso [Analizar]

3.9.7. NC/Soporte de Movilidad

Temas:
KA Core

- Principios de comunicación celular (por ejemplo, 4G, 5G)
- Principios de LANs inalámbricas (principalmente 802.11)
- Comunicación dispositivo a dispositivo (por ejemplo, comunicación IoT)
- Redes inalámbricas multiescalón (por ejemplo, redes ad hoc, oportunistas, tolerantes a retrasos)

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Describir algunos aspectos de comunicación celular como el registro [Describir]
2. Describir cómo 802.11 soporta usuarios móviles [Describir]
3. Describir usos prácticos de comunicación dispositivo a dispositivo, así como multiescalón [Describir]
4. Describir un tipo de red móvil como ad hoc [Describir]

3.9.8. NC/Temas Emergentes

Temas:
KA Core

- Middleboxes (por ejemplo, avances en el uso de IA, redes basadas en intención, filtrado, inspección profunda de paquetes, balanceo de carga, NAT, CDN)
- Virtualización de Red (por ejemplo, SDN, Redes de Centros de Datos)
- Redes Cuánticas (por ejemplo, Introducción al dominio, teleportación, seguridad, Internet Cuántica)
- Satélite, mmWave, Luz Visible

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Describir el valor de los avances en middleboxes en redes [Describir]
2. Describir la importancia de las Redes Definidas por Software [Describir]
3. Describir algunos de los valores añadidos logrados al usar Redes Cuánticas [Describir]

3.10. Sistemas Operativos (OS)

El sistema operativo es una colección de servicios necesarios para interconectar de manera segura el hardware con las aplicaciones. Los temas centrales se centran en los mecanismos y políticas necesarios para virtualizar cómputo, memoria y entrada/salida (I/O). Los temas generales que se reutilizan en muchos niveles de los sistemas informáticos están bien ilustrados en los sistemas operativos (por ejemplo, sondeo vs interrupciones, caché, flexibilidad vs costos, enfoques de planificación de procesos, reemplazo de páginas, etc.). El área de conocimiento de Sistemas Operativos contiene los conceptos subyacentes clave para otras áreas de conocimiento: límites de confianza, concurrencia, persistencia y extensibilidad segura.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
3.10.1 Rol y Propósito de los Sistemas Operativos (Pág. 93)			No
3.10.2 Principios del Sistema Operativo (Pág. 94)			No
3.10.3 Concurrencia (Pág. 95)			No
3.10.4 Protección y Seguridad (Pág. 95)			No
3.10.5 Planificación (Pág. 96)			No
3.10.6 Modelo de Proceso (Pág. 96)			No
3.10.7 Gestión de Memoria (Pág. 97)			No
3.10.8 Gestión de Dispositivos (Pág. 97)			No
3.10.9 API de Sistemas de Archivos e Implementación (Pág. 98)			No
3.10.10 Sistemas de Archivos Avanzados (Pág. 98)			No
3.10.11 Virtualización (Pág. 99)			No
3.10.12 Sistemas en Tiempo Real y Embebidos (Pág. 100)			No
3.10.13 Tolerancia a fallos (Pág. 100)			No
3.10.14 Sociedad, Ética y Profesión (Pág. 101)			No

3.10.1. OS/Rol y Propósito de los Sistemas Operativos

Temas:
CS Core

- Los sistemas operativos median entre el hardware de propósito general y el software específico de la aplicación.
- Funciones universales del sistema operativo (por ejemplo, proceso, interfaces de usuario y dispositivos, persistencia de datos)
- Funciones extendidas y/o especializadas del sistema operativo (por ejemplo, sistemas embebidos, tipos de servidor como archivos, web, multimedia, gestores de arranque y seguridad de arranque)

- Problemas de diseño (por ejemplo, eficiencia, robustez, flexibilidad, portabilidad, seguridad, compatibilidad, energía, seguridad, compensaciones entre verificación de errores y rendimiento, flexibilidad y rendimiento, y seguridad y rendimiento). Ver también: Análisis, Diseño e Ingeniería de la Seguridad
- Influencias de seguridad, redes, multimedia, computación paralela y distribuida
- Preocupación general de seguridad/protección: Omitir considerar la seguridad en cada capa crea una oportunidad para acceder de manera inapropiada a los recursos. 1. Se puede lograr acceso no autorizado a archivos en una unidad no cifrada moviendo el medio a otra computadora. 2. La seguridad impuesta por los sistemas operativos puede ser derrotada infiltrando la capa de arranque antes de que se cargue el sistema operativo. 3. El aislamiento de procesos puede ser subvertido por una verificación de autorización inadecuada en los límites de la API. 4. Las vulnerabilidades en el firmware del sistema pueden proporcionar vectores de ataque que eviten por completo el sistema operativo. 5. El aislamiento inadecuado de la memoria, cómputo y hardware de la máquina virtual puede exponer el sistema anfitrión a ataques desde sistemas invitados. 6. El sistema operativo puede necesitar mitigar la explotación de vulnerabilidades de hardware y firmware, lo que lleva a posibles reducciones de rendimiento (por ejemplo, mitigaciones de Spectre y Meltdown).

- Exposición de funciones del sistema operativo en shells y programación de sistemas. Ver también: Scripting de Shell

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Comprender los objetivos y funciones de los sistemas operativos modernos [Explicar]
2. Evaluar los problemas de diseño en diferentes escenarios de uso (por ejemplo, sistema operativo en tiempo real, móvil, servidor) [Evaluar]
3. Comprender las funciones de un sistema operativo contemporáneo con respecto a conveniencia, eficiencia y capacidad de evolucionar [Explicar]
4. Comprender cómo la evolución y la estabilidad son deseables y mutuamente antagónicas en las funciones del sistema operativo [Explicar]

3.10.2. OS/Principios del Sistema Operativo

Temas:
CS Core

- Diseño y enfoques de software del sistema operativo (por ejemplo, monolítico, en capas, modular, micro-kernel, unikernel)
- Abstracciones, procesos y recursos
- Concepto de llamadas al sistema y enlaces a interfaces de programas de aplicación (por ejemplo, Win32, Java, Posix). Ver también: Organización de Máquina a Nivel de Ensamblador
- La evolución del vínculo entre la arquitectura del hardware y las funciones del sistema operativo
- Protección de recursos significa proteger algunas instrucciones/funciones de la máquina. Ver también: Organización de Máquina a Nivel de Ensamblador 1. Las aplicaciones no pueden acceder arbitrariamente a ubicaciones de memoria o direcciones de dispositivos de almacenamiento de archivos. 2. Protección de coprocesadores y dispositivos de red
- Aprovechamiento de interrupciones desde el nivel de hardware: rutinas de servicio e implementaciones. Ver también: Organización de Máquina a Nivel de Ensamblador 1. Interrupciones de temporizador para implementar segmentos de tiempo 2. Interrupciones de E/S para poner hilos bloqueados en espera sin sondeo
- Concepto de estado usuario/sistema y protección, transición al modo kernel usando llamadas al sistema. Ver también: Organización de Máquina a Nivel de Ensamblador
- Mecanismo para invocar llamadas al sistema, el cambio de modo y contexto correspondiente y retorno desde interrupción. Ver también: Organización de Máquina a Nivel de Ensamblador
- Costos de rendimiento de cambios de contexto y vaciados de caché asociados al realizar cambios de proceso en entornos mitigados para Spectre.

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Comprender cómo la aplicación de enfoques de diseño de software al diseño/implementación de sistemas operativos (por ejemplo, en capas, modular, etc.) afecta la robustez y mantenibilidad de un sistema operativo [Explicar]
2. Categorizar llamadas al sistema por propósito [Categorizar]
3. Comprender la dinámica de invocar una llamada al sistema (por ejemplo, pasar parámetros, cambio de modo) [Explicar]
4. Evaluar si una función puede implementarse en la capa de aplicación o solo puede realizarse mediante llamadas al sistema [Evaluar]
5. Aplicar técnicas de SO para aislamiento, protección y rendimiento a través de las funciones del SO (por ejemplo, similitudes de inanición en planificación de procesos, planificación de solicitudes de disco, semáforos, etc.) y más allá [Aplicar]
6. Comprender cómo la separación en modo kernel y usuario afecta la seguridad y el rendimiento [Explicar]
7. Comprender las ventajas y desventajas de usar procesamiento de interrupciones para habilitar la multiprogramación [Explicar]
8. Analizar vectores potenciales de ataque a través de los sistemas operativos y las características de seguridad diseñadas para proteger contra ellos [Analizar]

3.10.3. OS/Concurrencia

Temas:
CS Core
- Abstracción de hilos relativa a la concurrencia
- Condiciones de carrera, regiones críticas (rol de interrupciones, si es necesario). Ver también: Programas
- Interbloqueos (deadlocks) e inanición. Ver también: Coordinación
- Problemas de multiprocesador (bloqueos de giro (spin-locks), reentrada).
- Concurrencia multiproceso vs multihilo

KA Core
- Creación de hilos, estados, estructuras. Ver también: Conceptos Básicos
- APIs de hilos
- Interbloqueos e inanición (condiciones necesarias/mitigaciones). Ver también: Coordinación
- Implementar código seguro para hilos (semáforos, bloqueos mutex, variables de condición). Ver también: Rendimiento y Eficiencia Energética, Evaluación de Rendimiento, Evaluación
- Condiciones de carrera en memoria compartida. Ver también: Coordinación

Non Core
- Gestionar acceso atómico a objetos del SO (por ejemplo, gran bloqueo del kernel vs muchos bloqueos pequeños vs estructuras de datos sin bloqueo como listas)

Aprendizaje esperado (*Learning Outcomes*):
CSCore:
1. Comprender las ventajas y desventajas de la concurrencia como funciones inseparables dentro del marco del sistema operativo [Explicar]
2. Comprender cómo la implementación a nivel de arquitectura resulta en problemas de concurrencia incluyendo condiciones de carrera [Explicar]
3. Comprender problemas de concurrencia en sistemas multiprocesador [Explicar]

KACore:
4. Comprender el rango de mecanismos que pueden emplearse a nivel del sistema operativo para realizar sistemas concurrentes y describir los beneficios de cada uno [Explicar]
5. Comprender técnicas para lograr sincronización en un sistema operativo (por ejemplo, describir cómo puede implementarse un semáforo usando primitivas del SO) incluyendo control de intra-concurrencia y uso de atómicos de hardware [Explicar]
6. Analizar con precisión código para identificar condiciones de carrera y soluciones apropiadas para abordarlas [Analizar]

3.10.4. OS/Protección y Seguridad

Temas:
CS Core
- Descripción general de mecanismos de seguridad del sistema operativo. Ver también: Fundamentos de Seguridad
- Ataques y antagonismo (planificación, etc.). Ver también: Fundamentos de Seguridad
- Revisión de vulnerabilidades principales en sistemas operativos reales. Ver también: Fundamentos de Seguridad
- Estrategias de mitigación del sistema operativo como copias de seguridad. Ver también: Confiabilidad del Sistema

KA Core
- Separación política/mecanismo. Ver también: Gobierno de la Seguridad
- Métodos y dispositivos de seguridad. Ver también: Fundamentos de Seguridad 1. Anillos de protección (historia desde Multics hasta x86 virtualizado) 2. Anillos -1 y -2 de x86_64 (hipervisor y ME/PSP)
- Protección, control de acceso y autenticación. Ver también: Fundamentos de Seguridad, Criptografía

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Comprender el requisito de mecanismos de protección y seguridad en sistemas operativos [Explicar]
2. Listar y describir los vectores de ataque que aprovechan vulnerabilidades del SO [Listar/Enumerar]
3. Comprender los mecanismos disponibles en un SO para controlar el acceso a recursos [Explicar]

KACore:

4. Resumir las características y limitaciones de un sistema operativo que impactan la protección y seguridad [Resumir]

3.10.5. OS/Planificación

Temas:

KA Core

- Planificación con expropiación y sin expropiación
- Planificadores y políticas (por ejemplo, primero en llegar primero en ser servido, trabajo más corto primero, prioridad, round robin, multinivel). Ver también: Gestión de Recursos
- Conceptos de planificación de multiprocesador simétrico (SMP) y coherencia de caché Ver también: Jerarquía de Memoria
- Temporizadores (por ejemplo, construir muchos temporizadores a partir de temporizadores de hardware finitos). Ver también: Organización de Máquina a Nivel de Ensamblador
- Justicia e inanición

Non Core

- Subtemas de sistemas operativos como planificación consciente de energía y planificación en tiempo real. Ver también: Rendimiento y Eficiencia Energética, Plataformas Embebidas, Plataformas Móviles
- Planificación cooperativa, como futexes de Linux y planificación en espacio de usuario.

Aprendizaje esperado (*Learning Outcomes*):

KACore:

1. Comparar y contrastar los algoritmos comunes utilizados para planificación con y sin expropiación de tareas en sistemas operativos, como prioridad, comparación de rendimiento y esquemas de participación justa [Comparar]
2. Explicar las relaciones entre algoritmos de planificación y dominios de aplicación [Explicar]
3. Explicar las distinciones entre tipos de planificadores de procesador como corto plazo, mediano plazo, largo plazo y E/S [Explicar]
4. Evaluar un problema o solución para determinar la idoneidad para multiprocesamiento asimétrico y/o simétrico [Evaluar]
5. Evaluar un problema o solución para determinar la idoneidad como proceso vs hilos [Evaluar]
6. Listar algunos contextos que se benefician de la planificación con expropiación y con plazo límite [Listar/Enumerar]

NonCore:

7. Explicar las formas en que la lógica incorporada en los algoritmos de planificación es aplicable a otros mecanismos de sistemas operativos, como primero en llegar primero en ser servido o prioridad para E/S de disco, planificación de red, planificación de proyectos y problemas más allá de la computación [Explicar]

3.10.6. OS/Modelo de Proceso

Temas:

KA Core

- Procesos e hilos relativos a virtualización de memoria protegida, estado del proceso, aislamiento de memoria, etc.
- Huella de memoria/segmentación (por ejemplo, pila, montón, etc.). Ver también: Organización de Máquina a Nivel de Ensamblador
- Creación y carga de ejecutables, bibliotecas compartidas y enlace dinámico. Ver también: Traducción y Ejecución de Lenguajes
- Despacho y cambio de contexto. Ver también: Organización de Máquina a Nivel de Ensamblador

- Comunicación entre procesos (por ejemplo, memoria compartida, paso de mensajes, señales, variables de entorno). Ver también: Comunicación

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Comprender cómo los procesos e hilos usan características de concurrencia para virtualizar el control [Explicar]
2. Comprender las razones para usar interrupciones, despacho y cambio de contexto para apoyar concurrencia y virtualización en un sistema operativo [Explicar]
3. Comprender los diferentes estados por los que puede pasar una tarea, y las estructuras de datos necesarias para apoyar la gestión de muchas tareas [Explicar]
4. Comprender las diferentes formas de asignar memoria a tareas, citando los méritos relativos de cada una [Explicar]
5. Aplicar el mecanismo de comunicación entre procesos apropiado para un propósito específico en un artefacto de software programado [Aplicar]

3.10.7. OS/Gestión de Memoria

Temas:
KA Core

- Revisión de memoria física, traducción de direcciones y hardware de gestión de memoria. Ver también: Jerarquía de Memoria, Matemáticas Discretas
- Impacto de la jerarquía de memoria incluyendo concepto de caché, búsqueda en caché y almacenamiento en caché por CPU en mecanismos y políticas del sistema operativo Ver también: Jerarquía de Memoria, Rendimiento del Sistema
- Direccionamiento lógico y físico, virtualización de espacio de direcciones. Ver también: Jerarquía de Memoria, Matemáticas Discretas
- Conceptos de paginación, reemplazo de páginas, trashing y asignación de páginas y marcos
- Técnicas de asignación/desasignación/almacenamiento (algoritmos y estructura de datos) rendimiento y flexibilidad 1. Arenas, asignadores de losas (slab allocators), listas libres, clases de tamaño, páginas de tamaño heterogéneo (páginas enormes)
- Almacenamiento en caché de memoria y coherencia de caché y el efecto de vaciar la caché para evitar vulnerabilidades de ejecución especulativaVer también: Organización Funcional, Jerarquía de Memoria, Rendimiento del Sistema)
- Mecanismos y conceptos de seguridad en gestión de memoria incluyendo sandboxing, protección, aislamiento y vectores relevantes de ataque. Ver también: Fundamentos de Seguridad

Non Core

- Memoria virtual: aprovechando el hardware de memoria virtual para servicios del SO y eficiencia

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Explicar jerarquía de memoria y compensaciones costo-rendimiento [Explicar]
2. Resumir los principios de memoria virtual aplicados a almacenamiento en caché y paginación [Resumir]
3. Evaluar las compensaciones en términos de tamaño de memoria (memoria principal, memoria caché, memoria auxiliar) y velocidad del procesador [Evaluar]
4. Describir la razón y el uso de la memoria caché (rendimiento y proximidad, cómo las cachés complican el aislamiento y la abstracción de máquina virtual) [Describir]
5. Codificar/Desarrollar programas eficientes que consideren los efectos del reemplazo de páginas y asignación de marcos en el rendimiento de un proceso y el sistema en el que se ejecuta [Crear]

NonCore:

6. Explicar cómo se utiliza el hardware para una virtualización eficiente [Explicar]

3.10.8. OS/Gestión de Dispositivos

Temas:
KA Core

- Estrategias de almacenamiento en búfer. Ver también: Interconexión y Comunicación
- Acceso Directo a Memoria (DMA) y E/S sondeada, E/S mapeada en memoria. Ver también: Interconexión y Comunicación 1. Protocolos de comunicación DMA (por ejemplo, búfers en anillo, etc.)
- Histórico y contextual - Gestión de dispositivos de almacenamiento persistente. Por ejemplo: Magnético, Dispositivo de Estado Sólido (SSD). Ver también: Historia de la Informática

Non Core

- Abstracciones de interfaz de dispositivo, capa de abstracción de hardware
- Propósito del controlador de dispositivo, abstracción, implementación y desafíos de prueba
- Tolerancia a fallos de alto nivel en comunicación de dispositivos

Aprendizaje esperado (*Learning Outcomes*):

KACore:

1. Explicar la implementación de control de dispositivos a nivel de arquitectura y vincular mecanismos y políticas relevantes del sistema operativo (por ejemplo, estrategias de almacenamiento en búfer, acceso directo a memoria) [Explicar]
2. Explicar las capas de gestión de dispositivos del SO y la arquitectura (por ejemplo, controlador de dispositivo, controlador de dispositivo, abstracción de dispositivo) [Explicar]
3. Explicar la relación entre el hardware físico y los dispositivos virtuales mantenidos por el sistema operativo [Explicar]
4. Explicar el almacenamiento en búfer de datos de E/S y describir estrategias para implementarlo [Explicar]
5. Describir las ventajas y desventajas del acceso directo a memoria y discutir las circunstancias en las que su uso está justificado [Describir]

NonCore:

6. Describir la complejidad y mejores prácticas para la creación de controladores de dispositivos [Describir]

3.10.9. OS/API de Sistemas de Archivos e Implementación

Temas:

KA Core

- Concepto de archivo incluyendo datos, metadatos, operaciones y modo de acceso
- Montaje de sistema de archivos
- Control de acceso a archivos
- Compartición de archivos
- Métodos básicos de asignación de archivos, incluida la tabla de asignación enlazada
- Estructuras del sistema de archivos que comprenden asignación de archivos incluyendo varias estructuras de directorio y métodos para identificar archivos de manera única (por ejemplo, nombre, identificador o ubicación de almacenamiento de metadatos)
- Técnicas de asignación/desasignación/almacenamiento (algoritmos y estructura de datos) impacto en rendimiento y flexibilidad (es decir, fragmentación interna y externa y compactación)
- Gestión de espacio libre como usar tablas de bits vs enlazado
- Implementación de directorios para segmentar y rastrear ubicación de archivos

Aprendizaje esperado (*Learning Outcomes*):

KACore:

1. Explicar las elecciones que deben tomarse en el diseño de sistemas de archivos [Explicar]
2. Evaluar diferentes enfoques para organización de archivos, reconociendo las fortalezas y debilidades de cada uno [Evaluar]
3. Aplicar constructores de software apropiadamente dado el conocimiento de la implementación del sistema de archivos [Aplicar]

3.10.10. OS/Sistemas de Archivos Avanzados

Temas:

KA Core

- Sistemas de archivos: particionamiento, montar/desmontar, sistemas de archivos virtuales
- Técnicas de implementación en profundidad
- Archivos mapeados en memoria. Ver también: Interconexión y Comunicación
- Sistemas de archivos de propósito especial
- Denominación, búsqueda, acceso, copias de seguridad
- Sistemas de archivos con registro (journaling) y estructurados en registro (log-structured). Ver también: Confiabilidad del Sistema

Non Core

- Sistemas de archivos distribuidos
- Sistemas de archivos cifrados
- Tolerancia a fallos

Aprendizaje esperado (*Learning Outcomes*):

KACore:

1. Explicar cómo los desarrollos de hardware han llevado a cambios en las prioridades para el diseño y la gestión de sistemas de archivos [Explicar]
2. Mapear abstracciones de archivo a una lista de dispositivos e interfaces relevantes [Mapear/Mapa conceptual]
3. Identificar y categorizar diferentes tipos de montaje [Analizar]
4. Explicar requisitos específicos de sistemas de archivos y las características de sistemas de archivos especializados que los satisfacen [Explicar]
5. Explicar el uso de registro y cómo los sistemas de archivos estructurados en registro mejoran la tolerancia a fallos [Explicar]

NonCore:

6. Explicar propósito y complejidad de los sistemas de archivos distribuidos [Explicar]
7. Listar ejemplos de protocolos de sistemas de archivos distribuidos [Listar/Enumerar]
8. Explicar mecanismos en sistemas de archivos para mejorar la tolerancia a fallos [Explicar]

3.10.11. OS/Virtualización

Temas:

KA Core

- Usar virtualización y aislamiento para lograr protección y rendimiento predecible. Ver también: Rendimiento del Sistema
- Paginación avanzada y memoria virtual. Ver también: Rendimiento del Sistema
- Sistemas de archivos virtuales y dispositivos virtuales.
- Contenedores y su comparación con máquinas virtuales.
- Trashing (por ejemplo, requisitos de Popek y Goldberg para sistemas recursivamente virtualizables).

Non Core

- Tipos de virtualización (incluyendo hardware/software, SO, servidor, servicio, red). Ver también: Rendimiento del Sistema
- Virtualización portátil; emulación vs aislamiento. Ver también: Rendimiento del Sistema
- Costo de virtualización. Ver también: Rendimiento del Sistema
- Máquinas virtuales y escapes de contenedores, peligros desde una perspectiva de seguridad. Ver también: Análisis, Diseño e Ingeniería de la Seguridad
- Hipervisores - extensiones de máquina virtual de hardware, anfitriones con soporte de kernel, QEMU KVM

Aprendizaje esperado (*Learning Outcomes*):

KACore:

1. Explicar cómo la arquitectura de hardware proporciona soporte y eficiencias para la virtualización [Explicar]
2. Explicar la diferencia entre emulación y aislamiento [Explicar]
3. Evaluar las compensaciones de la virtualización [Evaluar]

NonCore:

4. Explicar hipervisores y la necesidad de ellos junto con diferentes tipos de hipervisores [Explicar]

3.10.12. OS/Sistemas en Tiempo Real y Embebidos

Temas:
KA Core
- Planificación de procesos y tareas.
- Plazos límite y problemas de tiempo real. Ver también: Plataformas Embebidas
- Baja latencia vs "tiempo real suave" vs "tiempo real duro". Ver también: Plataformas Embebidas, Programación Dirigida por Eventos y Reactiva

Non Core
- Requisitos de gestión de memoria/disco en un entorno de tiempo real.
- Fallas, riesgos y recuperación.
- Preocupaciones especiales en sistemas de tiempo real (seguridad).

Aprendizaje esperado (*Learning Outcomes*):
KACore:
1. Explicar qué hace que un sistema sea un sistema de tiempo real [Explicar]
2. Explicar la latencia y sus fuentes en sistemas de software y sus características [Explicar]
3. Explicar las preocupaciones especiales que presentan los sistemas de tiempo real, incluido el riesgo, y cómo se abordan estas preocupaciones [Explicar]

NonCore:
4. Explicar características y mecanismos específicos de sistemas operativos de tiempo real [Explicar]

3.10.13. OS/Tolerancia a fallos

Temas:
KA Core
- Sistemas confiables y disponibles. Ver también: Confiabilidad del Sistema
- Enfoques de software y hardware para abordar la tolerancia (RAID). Ver también: Confiabilidad del Sistema

Non Core
- Redundancia espacial y temporal. Ver también: Confiabilidad del Sistema
- Métodos utilizados para implementar tolerancia a fallos. Ver también: Confiabilidad del Sistema
- Mecanismos de identificación y corrección de errores, sumas de comprobación (checksums) de memoria volátil en RAM. Ver también: Jerarquía de Memoria
- Verificación de consistencia y recuperación del sistema de archivos.
- Sistemas de archivos con registro y estructurados en registro. Ver también: Confiabilidad del Sistema
- Casos de uso para tolerancia a fallos (bases de datos, críticos para la seguridad). Ver también: Confiabilidad del Sistema
- Ejemplos de mecanismos del SO para detección, recuperación, reinicio para implementar tolerancia a fallos, uso de estas técnicas para los servicios propios del SO. Ver también: Confiabilidad del Sistema

Aprendizaje esperado (*Learning Outcomes*):
KACore:
1. Explicar cómo los sistemas operativos pueden facilitar la tolerancia a fallos, confiabilidad y disponibilidad [Explicar]
2. Explicar el rango de métodos para implementar tolerancia a fallos en un sistema operativo [Explicar]
3. Explicar cómo un sistema operativo puede continuar funcionando después de que ocurra una falla [Explicar]
4. Explicar las compensaciones de rendimiento y flexibilidad que impactan el uso de tolerancia a fallos [Explicar]

NonCore:
5. Describir en detalle los problemas y mecanismos de tolerancia a fallos del sistema operativo [Describir]

3.10.14. OS/Sociedad, Ética y Profesión

Temas:
KA Core

- Código abierto en sistemas operativos. Ver también: Propiedad Intelectual 1. Identificación de vulnerabilidades en kernels de código abierto, 2. Sistemas operativos invitados de código abierto, 3. Sistemas operativos anfitriones de código abierto, y 4. Cambios en monetización (actualizaciones pagas vs gratuitas).
- Problemas de fin de vida con la retirada de sistemas operativos. Concepto ejemplo: Implicaciones de privacidad del uso de sistemas operativos/entornos operativos propietarios, incluyendo telemetría, escaneo automatizado de datos personales, publicidad incorporada e integración automática en la nube.

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Explicar ventajas y desventajas de encontrar y abordar errores en kernels de código abierto [Explicar]
2. Contextualizar la historia e impacto positivo y negativo de Linux como producto de código abierto [Contextualizar]
3. Listar complicaciones con la dependencia de sistemas operativos pasados de fin de vida [Listar/Enumerar]
4. Comprender diferencias en encontrar y abordar errores para varios modelos de pago de sistemas operativos [Explicar]

3.11. Computación Paralela y Distribuida (PDC)

La programación paralela y distribuida organiza, coordina y controla múltiples cálculos que ocurren al mismo tiempo en diferentes lugares. La ubicuidad del paralelismo y la distribución son consecuencias inevitables del creciente número de puertas en los procesadores, procesadores en las computadoras y computadoras en todas partes que pueden usarse para mejorar el rendimiento en comparación con los programas secuenciales, al mismo tiempo que se hace frente a la interconectividad intrínseca del mundo y la posibilidad de que algunos componentes o conexiones fallen o se comporten de manera maliciosa. La programación paralela y distribuida elimina las restricciones de la programación secuencial que requieren que los pasos computacionales ocurran en un orden serial en un solo lugar, revelando distinciones, técnicas y análisis adicionales que se aplican en cada capa de los sistemas informáticos.

En la mayoría de los usos convencionales, la programación "paralela" se centra en establecer y coordinar múltiples actividades que pueden ocurrir al mismo tiempo, la programación "distribuida" se centra en establecer y coordinar actividades que pueden ocurrir en diferentes lugares, y la programación "concurrente" se centra en las interacciones de las actividades en curso entre sí y con el entorno. Sin embargo, los tres términos pueden aplicarse en la mayoría de los contextos. El paralelismo generalmente implica alguna forma de distribución porque múltiples actividades que ocurren sin restricciones de orden secuencial suceden en múltiples lugares físicos (a menos que dependan de cambios de contexto o efectos cuánticos). Por el contrario, las acciones en diferentes lugares no necesitan tener un orden secuencial específico entre sí en ausencia de restricciones de comunicación.

Las técnicas de programación paralela, distribuida y concurrente forman el núcleo de la Computación de Alto Rendimiento (HPC), los sistemas distribuidos y, cada vez más, casi todas las aplicaciones informáticas. El área de conocimiento PDC ha evolucionado desde un conjunto diverso de temas avanzados hasta un cuerpo central de conocimiento y práctica, que impregna casi todos los demás aspectos de la informática. El crecimiento del campo ha ocurrido de manera irregular en los diferentes subcampos de la informática, a veces con diferentes objetivos, terminología y prácticas, ocultando la superposición considerable de ideas y habilidades básicas que son el foco principal de esta área de conocimiento. Casi todos los problemas con una solución secuencial también admiten soluciones paralelas y/o distribuidas; surgen problemas y soluciones adicionales solo en el contexto de la concurrencia. Casi todos los dominios de aplicación de la computación paralela y distribuida son un área de estudio y/o ingeniería bien desarrollada, demasiado grande para enumerarla.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
3.11.1 Programas (Pág. 102)			No
3.11.2 Comunicación (Pág. 103)			No
3.11.3 Coordinación (Pág. 104)			No
3.11.4 Evaluación (Pág. 105)			No
3.11.5 Algoritmos (Pág. 106)			No

3.11.1. PDC/Programas

Temas:
CS Core

- Paralelismo 1. Paralelismo declarativo: Determinar qué acciones pueden, o no deben, realizarse en paralelo, a nivel de instrucciones, funciones, clausuras, acciones compuestas, sesiones, tareas y servicios es la idea principal que subyace a los algoritmos PDC; no hacerlo es la principal fuente de errores. Ver también: Algoritmos 2. Definir orden: por ejemplo, usando relaciones de "sucede antes" o grafos acíclicos dirigidos serie/paralelo que representan programas. 3. Independencia: determinar cuándo el orden no importa, en términos de conmutatividad, dependencias, precondiciones. 4. Garantizar el orden entre acciones que de otro modo serían paralelas cuando sea necesario, incluyendo bloqueos, publicación segura; e imponer comunicación - enviar un mensaje sucede antes de recibirlo; y relajarlo cuando no sea necesario.
- Distribución 1. Definir lugares, como dispositivos que ejecutan acciones, incluyendo componentes de hardware, hosts remotos; también pueden incluir dispositivos, hosts y usuarios externos no controlados. Ver también: Interconexión y Comunicación 2. Un dispositivo puede dividir el tiempo o emular múltiples acciones paralelas mediante menos procesadores mediante planificación y virtualización. Ver también: Planificación 3. Nombrar o identificar lugares (por ejemplo, IDs de dispositivo) y acciones como partes (por ejemplo, IDs de hilo). 4. Las actividades a través de lugares pueden comunicarse a través de medios. Ver también: Comunicación
- Iniciar actividades 1. Las opciones que permiten que las acciones se realicen (eventualmente) en lugares van desde el cableado directo hasta scripts de configuración; también establecer comunicación y gestión de recursos; estos se expresan de manera diferente en los lenguajes y contextos, generalmente confiando en el aprovisionamiento y gestión automatizados por las plataformas. Ver también: Gestión de Recursos 2. Procedimental: Permitir que múltiples acciones comiencen en un punto de programa dado; por ejemplo, iniciar nuevos hilos, posiblemente delimitando su alcance u organizándolos en grupos jerárquicos. 3. Reactiva: Habilitar al ocurrir un evento instalando un manejador de eventos, con menos control de cuándo comienzan o terminan las acciones, y puede aplicarse incluso en monoprocesadores. 4. Dependiente: Habilitar al completarse otras; por ejemplo, secuenciando conjuntos de acciones paralelas. Ver también: Coordinación. 5. Granularidad: El costo de ejecución de los cuerpos de las acciones debe superar la sobrecarga de organizarlos.
- Propiedades de Ejecución 1. Ejecución no determinista de acciones sin orden. 2. Consistencia: Garantizar el acuerdo entre partes sobre valores y predicados cuando sea necesario para evitar carreras, mantener seguridad y atomicidad, o llegar a consenso. 3. Tolerancia a fallos: Manejar fallos en las partes o la comunicación, incluyendo (Bizantino) mal comportamiento debido a partes y protocolos no confiables, cuando sea necesario para mantener el progreso o la disponibilidad. Ver también: Confiabilidad del Sistema. 4. Los compromisos son un foco de evaluación. Ver también: Evaluación.

KA Core

- Una o más de las siguientes asignaciones y mecanismos a través de sistemas en capas: 1. Paralelismo a nivel de instrucción y datos en la CPU. Ver también: Organización Funcional. 2. SIMD y paralelismo de datos heterogéneo. Ver también: Arquitecturas Heterogéneas. 3. Concurrencia planificada en multinúcleo, tareas, actores. Ver también: Planificación. 4. Clústeres, nubes; aprovisionamiento elástico. Ver también: Aspectos Comunes, Preocupaciones Compartidas. 5. Sistemas distribuidos en red. Ver también: Aplicaciones en Red. 6. Tecnologías emergentes como la computación cuántica y la computación molecular.

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Mostrar gráficamente (como un Grafo Acíclico Dirigido - *Directed Acyclic Graph* (DAG)) cómo paralelizar una expresión numérica compuesta; por ejemplo, $a = (b + c) * (d + e)$ [Diseñar]
2. Explicar por qué los conceptos de consistencia y tolerancia a fallos no surgen en programas puramente secuenciales [Explicar]

KACore:

3. Escribir una función que cuente eventos, como recepciones de paquetes de red, de manera eficiente [Escribir]
4. Escribir un programa de filtro/mapeo/reducción en múltiples estilos [Escribir]
5. Escribir un servicio que cree un hilo (u otra forma de activación procedimental) para devolver una página web solicitada a cada nuevo cliente [Escribir]

3.11.2. PDC/Comunicación

Temas:

CS Core

- Medios 1. Variedades: canales (paso de mensajes o E/S), memoria compartida, heterogéneos, almacenes de datos. 2. Dependencia de la disponibilidad y naturaleza del hardware subyacente, conectividad y protocolos; soporte del lenguaje, emulación. Ver también: Interconexión y Comunicación.
- Canales 1. Medios de comunicación de parte a parte explícitos (generalmente nombrados). 2. APIs: Sockets, constructos arquitectónicos, basados en lenguaje y de kits de herramientas, como Message Passing Interface (MPI), y constructos en capas como Llamada a Procedimiento Remoto (RPC). Ver también: Fundamentos de Redes y Comunicaciones. 3. APIs de canales de E/S.
- Memoria 1. Arquitecturas de memoria compartida en las que las partes se comunican directamente solo con la memoria en direcciones dadas, con extensiones a memoria heterogénea que admite múltiples almacenes de memoria con transferencia de datos explícita entre ellos; por ejemplo, memoria local y compartida de GPU, Acceso Directo a Memoria (DMA). 2. Jerarquías de memoria: Múltiples capas de dominios, alcances y cachés compartidos; localidad: latencia, falso uso compartido. 3. Propiedades de consistencia: Límites de atomicidad a nivel de bits, coherencia, orden local.
- Almacenes de Datos 1. Estructuras de datos mantenidas cooperativamente que implementan mapas y TADs relacionados. 2. Variedades: Propiedad exclusiva, compartida, fragmentada, replicada, inmutable, versionada.

KA Core

- Una o más de las siguientes propiedades y extensiones: 1. Topologías: Unicast, Multicast, Buzones, Conmutadores; Enrutamiento a través de redes de interconexión de hardware y software. 2. Propiedades de concurrencia de medios: Orden, consistencia, idempotencia, superposición de comunicación con computación. 3. Rendimiento del medio: Latencia, ancho de banda (rendimiento), contención (congestión), capacidad de respuesta (vivacidad), confiabilidad (tasas de error y pérdida), progreso basado en protocolo (acks, tiempos de espera, mediación). 4. Propiedades de seguridad del medio: integridad, privacidad, autenticación, autorización. Ver también: Codificación Segura. 5. Formatos de datos: Serialización, validación, cifrado, compresión. 6. Políticas de canal: Puntos finales, sesiones, almacenamiento en búfer, respuesta a saturación (esperar vs. descartar), control de velocidad. 7. Multiplexación y demultiplexación de muchos dispositivos o partes de E/S relativamente lentos; técnicas basadas en finalización y basadas en planificador; APIs async-await, select y polling. 8. Formalización y análisis de comunicación por canales; por ejemplo, CSP. 9. Aplicaciones de la teoría de colas para modelar y predecir el rendimiento. 10. Modelos de memoria: consistencia secuencial y de liberación/adquisición. 11. Gestión de memoria; incluyendo la reclamación de datos compartidos; conteo de referencias y alternativas. 12. Colocación y transferencia masiva de datos; reducción del tráfico de mensajes y mejora de la localidad; superposición de transferencia de datos y computación; impacto del diseño de datos como array de estructuras vs. estructura de arrays. 13. Emular memoria compartida: memoria compartida distribuida, Acceso Directo a Memoria Remota (RDMA). 14. Consistencia de almacenes de datos: Atomicidad, linealizabilidad, transaccionalidad, coherencia, orden causal,

resolución de conflictos, consistencia eventual, cadenas de bloques. 15. Fallos, particionamiento y fallos parciales; votación; protocolos como Paxos y Raft. 16. Compromisos de diseño entre consistencia, disponibilidad, tolerancia a partición (fallos); imposibilidad de cumplir con todos a la vez. 17. Seguridad y confianza: Fallos bizantinos, prueba de trabajo y alternativas.

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Explicar las similitudes y diferencias entre: (1) La parte A envía un mensaje en el canal X con contenido 1 recibido por la parte B (2) A establece la variable compartida X a 1, leída por B (3) A establece "X=1" en un mapa compartido distribuido al que accede B [Explicar]

KACore:

2. Escribir un programa que distribuya diferentes segmentos de un conjunto de datos a múltiples trabajadores y recoja los resultados (por ejemplo, sumar segmentos de un arreglo) [Escribir]
3. Escribir un programa paralelo que solicite datos de múltiples sitios y los resuma usando alguna forma de reducción [Escribir]
4. Comparar el rendimiento de versiones con y sin búfer de un programa productor-consumidor [Comparar]
5. Determinar si un esquema de comunicación dado proporciona propiedades de seguridad suficientes para un uso dado [Determinar]
6. Dar un ejemplo de un orden de accesos entre actividades concurrentes (por ejemplo, un programa con una carrera de datos) que no sea secuencialmente consistente [Crear]
7. Dar un ejemplo de un escenario en el que los envíos de mensajes bloqueantes pueden causar un punto muerto (deadlock) [Crear]
8. Describir al menos una técnica de diseño para evitar fallos de vivacidad en programas que usan múltiples bloqueos [Describir]
9. Escribir un programa que ilustre el reordenamiento de acceso a memoria o de mensajes [Escribir]
10. Describir los méritos relativos del control de concurrencia optimista versus conservador bajo diferentes tasas de contención entre actualizaciones [Describir]
11. Dar un ejemplo de un escenario en el que un intento de actualización optimista podría nunca completarse [Crear]
12. Modificar un sistema concurrente para usar un almacén de datos más escalable, confiable o disponible [Crear]
13. Usando una plataforma existente que admita almacenes de datos replicados, escribir un programa que mantenga un mapeo clave-valor incluso cuando uno o más hosts fallen [Escribir]

3.11.3. PDC/Coordinación

Temas:

CS Core

- Dependencias 1. La iniciación o progreso de una actividad puede depender de otras actividades, para evitar condiciones de carrera, garantizar la terminación o cumplir otros requisitos. 2. Garantizar el progreso evitando ciclos de dependencia, usando condiciones monótonas, eliminando dependencias no esenciales.
- Constructos de control y patrones de diseño: 1. Basados en finalización: Barreras, reuniones (joins), incluido el control de terminación. 2. Habilitados por datos: Colas, diseños productor-consumidor. 3. Basados en condición: Polling, reintentos, retrocesos, ayuda, suspensión, señalización, tiempos de espera. 4. Reactivos: Habilitar y activar continuaciones.
- Atomicidad 1. Instrucciones atómicas, ordenamientos de acceso local impuestos. 2. Bloqueos y exclusión mutua; granularidad de bloqueo. 3. Uso de bloqueos en un lenguaje específico; mantener la vivacidad sin introducir carreras. 4. Evitar punto muerto (*deadlock*): Ordenamiento, mayor granularidad, reintentos aleatorios; retrocesos, encapsulación mediante administradores de bloqueos. 5. Errores comunes: No bloquear o desbloquear cuando es necesario, mantener bloqueos mientras se invocan operaciones desconocidas. 6. Evitar bloqueos: replicación, solo lectura, propiedad, y construcciones no bloqueantes.

KA Core

- Una o más de las siguientes propiedades y extensiones: 1. Propiedades de progreso incluyendo libre de bloqueo (*lock-free*), libre de espera (*wait-free*), equidad (*fairness*), planificación por

prioridad, interacciones con consistencia, confiabilidad. 2. Rendimiento con respecto a contención, granularidad, convoy, escalabilidad. 3. Estructuras de datos y algoritmos no bloqueantes. 4. Propiedad y control de recursos. 5. Variantes y alternativas de bloqueos: bloqueos de secuencia, bloqueos de lectura-escritura; Actualizar-Copiando-Lectura (RCU), reentrada; tickets; control del ciclo activo (*spinning*) versus bloqueo. 6. Control basado en transacciones: Optimista y conservador. 7. Bloqueo distribuido: confiabilidad. 8. Alternativas a barreras: Relojes; contadores, relojes virtuales; flujo de datos y continuaciones; futuros y RPC; basadas en consenso, recolección de resultados con reductores y colectores. 9. Especulación, selección, cancelación; consecuencias en observabilidad y seguridad. 10. Control de recursos usando semáforos y variables de condición. 11. Flujo de control: Planificación de computaciones, bucles serie-paralelo con líderes (posiblemente elegidos), tuberías y flujos, paralelismo anidado. 12. Excepciones y fallos. Manejadores, detección, tiempos de espera, tolerancia a fallos, votación.

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Mostrar cómo evitar o reparar un error de carrera en un programa dado [Analizar]
2. Mostrar cómo garantizar que un programa termine correctamente cuando todas las tareas concurrentes de un conjunto hayan finalizado [Diseñar]

KACore:

3. Escribir una función que cuente eventos, como entradas de sensores o recepciones de paquetes de red, de manera eficiente [Escribir]
4. Escribir un programa de filtro/mapeo/reducción en múltiples estilos [Escribir]
5. Escribir un programa en el que la terminación de un conjunto de acciones paralelas sea seguida por otra [Escribir]
6. Escribir un programa que busque especulativamente una solución mediante múltiples actividades, terminando las demás cuando se encuentre una [Escribir]
7. Escribir un programa en el que una excepción numérica (como división por cero) en una actividad cause la terminación de las demás [Escribir]
8. Escribir un programa para que múltiples partes acuerden la hora actual del día; discutir sus limitaciones en comparación con protocolos como el protocolo de transferencia de red (NTP) [Escribir]
9. Escribir un servicio que cree un hilo (u otra forma de activación procedimental) para devolver una página web solicitada a cada nuevo cliente [Escribir]

3.11.4. PDC/Evaluación

Temas:

CS Core

- Requisitos de seguridad (safety) y vivacidad (*liveness*) en términos de constructos de lógica temporal para expresar ”siempre” y ”eventualmente” Ver también: Computación Paralela y Distribuida.
- Identificar, probar y reparar violaciones, incluyendo formas comunes de errores como no garantizar el orden necesario (errores de carrera), atomicidad (incluyendo errores de ”verificar luego actuar”) y terminación (bloqueo activo).
- Métricas de requisitos de rendimiento para rendimiento (*throughput*), capacidad de respuesta, latencia, disponibilidad, consumo de energía, escalabilidad, uso de recursos, costos de comunicación, espera y control de velocidad, equidad; acuerdos de nivel de servicio. Ver también: Rendimiento del Sistema.
- Impacto en el rendimiento de las opciones de diseño e implementación, incluyendo granularidad, sobrecarga, costos de consenso y consumo de energía. Ver también: Sostenibilidad.
- Estimar limitaciones de escalabilidad, por ejemplo usando la Ley de Amdahl o la Ley de Escalabilidad Universal. Ver también: Evaluación de Rendimiento.

KA Core

- Uno o más de los siguientes métodos y herramientas: 1. Extensiones a requisitos formales secuenciales como la linealizabilidad. 2. Especificaciones de protocolo, sesión y transacciones. 3. Uso de herramientas como Lenguaje Unificado de Modelado (UML), Lógica Temporal de Acciones (TLA), lógicas de programa. 4. Análisis de seguridad: seguridad y vivacidad en presencia de

comportamientos hostiles o con errores de otras partes; propiedades requeridas de mecanismos de comunicación (por ejemplo, ausencia de fugas entre capas), filtrado de entrada, limitación de velocidad. Ver también: Fundamentos de Seguridad. 5. Análisis estático aplicado a corrección, rendimiento (*throughput*), latencia, recursos, energía. Ver también: Sostenibilidad. 6. Análisis del modelo de Grafo Acíclico Dirigido (DAG) de eficiencia algorítmica (trabajo, span, caminos críticos). 7. Pruebas y depuración; herramientas como detectores de carreras, fuzzers, verificadores de dependencia de bloqueos, pruebas de unidad/esfuerzo/tortura, visualizaciones, integración continua, despliegue continuo, y generadores de pruebas. 8. Medir y comparar rendimiento (*throughput*), sobrecarga, espera, contención, comunicación, movimiento de datos, localidad, uso de recursos, comportamiento en presencia de números excesivos de eventos, clientes o hilos. Ver también: Evaluación de Rendimiento. 9. Análisis específicos del dominio de aplicación y técnicas de evaluación.

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Revisar una especificación para habilitar el paralelismo y la distribución sin violar otras propiedades o características esenciales [Rediseñar]
2. Explicar cómo las nociones concurrentes de seguridad (safety) y vivacidad (liveness) extienden sus contrapartes secuenciales [Explicar]
3. Especificar un conjunto de invariantes que deben mantenerse en cada paso de cómputo de paralelismo masivo [Analizar]
4. Escribir un programa de prueba que pueda revelar un error de carrera de datos; por ejemplo, perder una actualización cuando dos actividades intentan incrementar una variable [Escribir]
5. En un contexto dado, explicar hasta qué punto se esperaría que introducir paralelismo en un programa por lo demás secuencial mejoraría el rendimiento (*throughput*) y/o reduciría la latencia, y cómo podría afectar la eficiencia energética [Explicar]
6. Mostrar cómo cambian la escalabilidad y la eficiencia para problemas de muestra con y sin el supuesto de que el tamaño del problema cambia con el número de procesadores; además, explicar si y cómo cambiaría la escalabilidad bajo relajaciones de dependencias secuenciales [Diseñar]

KACore:

7. Especificar y medir el comportamiento cuando un servicio es solicitado por un número inesperadamente grande de clientes [Analizar]
8. Identificar y reparar un problema de rendimiento debido a cuellos de botella secuenciales [Analizar]
9. Comparar empíricamente el rendimiento (*throughput*) de dos implementaciones de un diseño común (quizás usando un marco de pruebas existente) [Comparar]
10. Identificar y reparar un problema de rendimiento debido a latencia de comunicación o datos [Analizar]
11. Identificar y reparar un problema de rendimiento debido a sobrecarga en la gestión de recursos [Analizar]
12. Identificar y reparar un problema de confiabilidad o disponibilidad [Analizar]

3.11.5. PDC/Algoritmos

Temas:
CS Core

- Expresar e implementar algoritmos en lenguajes y marcos de trabajo dados, para iniciar actividades (por ejemplo hilos), usar constructos de memoria compartida, y APIs de canales, sockets y/o llamada a procedimiento remoto (RPC). Ver también: Computación Paralela y Distribuida. 1. Ejemplos de datos paralelos incluyendo mapeo/reducción. 2. Uso de APIs de canal, socket y/o RPC en un lenguaje dado, con control del programa para enviar (generalmente procedimental) vs recibir (generalmente reactivo o basado en RPC). 3. Uso de bloqueos, barreras y/o sincronizadores para mantener la vivacidad sin introducir carreras.
- Panorama de dominios de aplicación comunes en multinúcleo, reactivo, datos paralelos, clúster, nube, sistemas distribuidos abiertos y marcos de trabajo (con referencia a la siguiente tabla). 1. Multinúcleo: Agentes de ejecución típicos: Hilos. Mecanismos de comunicación típicos: Memoria compartida, Atómicos, bloqueos. Dominios algorítmicos típicos: Gestión de recursos,

procesamiento de datos. Objetivos de ingeniería típicos: Rendimiento (*throughput*), latencia, energía. 2. Reactivo: Agentes de ejecución típicos: Manejadores, hilos. Mecanismos de comunicación típicos: Canales de E/S. Dominios algorítmicos típicos: Servicios, tiempo real. Objetivos de ingeniería típicos: Latencia. 3. Datos paralelos: Agentes de ejecución típicos: GPU, SIMD, aceleradores, híbridos. Mecanismos de comunicación típicos: Memoria heterogénea. Dominios algorítmicos típicos: Álgebra lineal, gráficos, análisis de datos. Objetivos de ingeniería típicos: Rendimiento (*throughput*), energía. 4. Clúster: Agentes de ejecución típicos: Hosts gestionados. Mecanismos de comunicación típicos: Sockets, canales. Dominios algorítmicos típicos: Simulación, análisis de datos. Objetivos de ingeniería típicos: Rendimiento (*throughput*). 5. Nube: Agentes de ejecución típicos: Hosts aprovisionados. Mecanismos de comunicación típicos: APIs de servicio. Dominios algorítmicos típicos: Aplicaciones web. Objetivos de ingeniería típicos: Escalabilidad. 6. Distribuido abierto: Agentes de ejecución típicos: Hosts autónomos. Mecanismos de comunicación típicos: Sockets, Almacenes de datos. Dominios algorítmicos típicos: Almacenes de datos y servicios tolerantes a fallos. Objetivos de ingeniería típicos: Confiabilidad.

KA Core

- Uno o más de los siguientes dominios algorítmicos. Ver también: Estrategias Algorítmicas: 1. Álgebra lineal: Operaciones con vectores y matrices, precisión/estabilidad numérica, aplicaciones en análisis de datos y aprendizaje automático. 2. Procesamiento de datos: ordenación, búsqueda y recuperación, estructuras de datos concurrentes. 3. Grafos, búsqueda y combinatoria: Marcado, paralelización de aristas, acotamiento, especulación, análisis basado en redes. 4. Modelado y simulación: ecuaciones diferenciales; aleatorización, problemas de N-cuerpos, algoritmos genéticos. 5. Lógica computacional: satisfactibilidad (SAT), programación lógica concurrente. 6. Gráficos y geometría computacional: Transformaciones, renderizado, trazado de rayos. 7. Gestión de recursos: Asignar, colocar, reciclar y planificar procesadores, memoria, canales y hosts; recursos exclusivos vs compartidos; algoritmos estáticos, dinámicos y elásticos; Restricciones de tiempo real; Lotes, priorización, partición; descentralización mediante robo de trabajo y técnicas relacionadas. 8. Servicios: Implementar APIs web, moneda electrónica, sistemas de transacción, juegos multijugador.

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Implementar un componente paralelo/distribuido basado en un algoritmo conocido [Implementar]
2. Escribir un programa de datos paralelos que, por ejemplo, calcule el promedio de un arreglo de números [Escribir]
3. Escribir un programa productor-consumidor en el que un componente genere números y otro calcule su promedio. Medir las aceleraciones cuando los números son escalares pequeños versus valores de multiprecisión grandes [Escribir]
4. Extender un programa secuencial dirigido por eventos estableciendo una nueva actividad en un manejador de eventos (por ejemplo, un nuevo hilo en un manejador de acciones de GUI) [Diseñar]
5. Mejorar el rendimiento de un componente secuencial introduciendo paralelismo y/o distribución [Crear]
6. Elegir entre diferentes diseños paralelos/distribuidos para componentes de un sistema dado [Evaluar (valorar)]

KACore:

7. Diseñar, implementar, analizar y evaluar un componente o aplicación para X que opere en un contexto dado, donde X esté en uno de los dominios listados, por ejemplo, un algoritmo genético para el diseño de una planta de fábrica [Diseñar]
8. Criticar el diseño e implementación de un componente o aplicación existente, o uno desarrollado por compañeros de clase [Críticar (análisis crítico)]
9. Comparar el rendimiento y la eficiencia energética de múltiples implementaciones de un diseño similar, por ejemplo, multinúcleo versus clúster versus GPU [Comparar]

3.12. Fundamentos de Desarrollo de Software (SDF)

La fluidez en el proceso de desarrollo de software es fundamental para el estudio de las ciencias de la computación. Para utilizar las computadoras de manera más efectiva para resolver problemas, los estudiantes deben ser competentes en la lectura y escritura de programas. Sin embargo, más allá de las habilidades de programación, deben poder seleccionar y utilizar estructuras de datos y algoritmos apropiados, así como herramientas modernas de desarrollo y prueba.

El área de conocimiento SDF reúne conceptos y habilidades fundamentales relacionados con el desarrollo de software, centrándose en aquellos que deben enseñarse al inicio de un programa de ciencias de la computación, típicamente en el primer año. Esto incluye conceptos de programación fundamentales y su uso efectivo para escribir programas, el uso de estructuras de datos fundamentales que pueden ser proporcionadas por el lenguaje de programación, conceptos básicos de buenas prácticas de programación para escribir programas de calidad, la lectura y comprensión de programas, y cierta comprensión del impacto de los algoritmos en el rendimiento de los programas. Las 43 horas de material en esta área de conocimiento pueden complementarse con material central de otras áreas a medida que los estudiantes avanzan a cursos de nivel intermedio y superior.

Esta área de conocimiento asume un lenguaje de programación contemporáneo con soporte incorporado para tipos de datos comunes, incluidos tipos de datos asociativos como diccionarios/mapas, como vehículo para introducir a los estudiantes a la programación (por ejemplo, Python, Java). Sin embargo, esto no pretende desalentar el uso de lenguajes más antiguos o de bajo nivel para SDF: las unidades de conocimiento a continuación pueden adaptarse adecuadamente al lenguaje real utilizado.

El surgimiento de la IA generativa y los Modelos de Lenguaje Grande (LLMs), que pueden generar programas para muchas tareas de programación, sin duda afectará a la profesión de programación y, en consecuencia, a la enseñanza de muchos temas de CS. Sin embargo, para poder utilizar efectivamente la IA generativa en tareas de programación, un programador debe tener una buena comprensión de los programas y, por lo tanto, aún debe aprender los fundamentos de la programación y desarrollar habilidades básicas de programación, que es el objetivo de SDF. En consecuencia, consideramos que los resultados deseados para SDF deberían permanecer iguales, aunque diferentes instructores ahora podrían dar más énfasis a la comprensión, documentación, especificaciones, análisis y pruebas de programas. (Esto es similar a enseñar a los estudiantes multiplicación, suma, etc., aunque se puedan usar calculadoras para hacerlo).

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
3.12.1 Conceptos y Prácticas Fundamentales de Programación (Pág. 108)			No
3.12.2 Estructuras de Datos Fundamentales (Pág. 109)			No
3.12.3 Algoritmos (Pág. 110)			No
3.12.4 Prácticas de Desarrollo de Software (Pág. 110)			No
3.12.5 Sociedad, Ética y la Profesión (Pág. 110)			No

3.12.1. SDF/Conceptos y Prácticas Fundamentales de Programación

Temas:
CS Core

- Conceptos básicos como variables, tipos de datos primitivos, expresiones y su evaluación
- Cómo funcionan los programas imperativos: estado y transiciones de estado en la ejecución de sentencias, flujo de control
- Construcciones básicas como sentencias de asignación, sentencias condicionales e iterativas, E/S básica
- Construcciones clave de modularidad como funciones (y métodos y clases, si son compatibles en el lenguaje) y conceptos relacionados como paso de parámetros, alcance, abstracción, encapsulamiento de datos Programación Orientada a Objetos
- Entrada y salida usando archivos y APIs
- Tipos de datos estructurados disponibles en el lenguaje de programación elegido como secuencias (por ejemplo, arreglos, listas), contenedores asociativos (por ejemplo, diccionarios, mapas), otros

(por ejemplo, conjuntos, tuplas) y cuándo y cómo usarlos Estructuras de Datos y Algoritmos Fundamentales
- Bibliotecas y frameworks proporcionados por el lenguaje (cuando/donde sea aplicable)
- Recursión
- Manejo de errores de tiempo de ejecución en programas (por ejemplo, manejo de excepciones).
- Conceptos básicos de errores de programación, pruebas y depuración Construcción de Software, Codificación Segura
- Documentar/comentar código a nivel de programa y de módulo. Construcción de Software
- Desarrollar una mentalidad de seguridad. Fundamentos de Seguridad

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Desarrollar programas que utilicen las construcciones fundamentales de programación: asignación y expresiones, E/S básica, sentencias condicionales e iterativas [Crear]
2. Desarrollar programas utilizando funciones con paso de parámetros [Crear]
3. Desarrollar programas que utilicen efectivamente los diferentes tipos de datos estructurados proporcionados en el lenguaje, como arreglos/listas, diccionarios y conjuntos [Crear]
4. Desarrollar programas que utilicen E/S de archivos para proporcionar persistencia de datos a través de múltiples ejecuciones [Crear]
5. Desarrollar programas que utilicen bibliotecas y frameworks proporcionados por el lenguaje (donde sea aplicable) [Crear]
6. Desarrollar programas que utilicen APIs para acceder o actualizar datos (por ejemplo, desde la web) [Crear]
7. Desarrollar programas que creen clases simples e instancien objetos de esas clases (si es compatible con el lenguaje) [Crear]
8. Explicar el concepto de recursión e identificar cuándo y cómo usarlo efectivamente [Explicar]
9. Desarrollar funciones recursivas [Crear]
10. Desarrollar programas que puedan manejar errores de tiempo de ejecución [Crear]
11. Leer un programa dado y explicar lo que hace [Explicar]
12. Escribir comentarios para un programa o módulo especificando lo que hace [Escribir]
13. Rastrear el flujo de control durante la ejecución de un programa [Trazar/Rastrear]
14. Usar la terminología apropiada para identificar elementos de un programa (por ejemplo, identificador, operador, operando) [Usar]

3.12.2. SDF/Estructuras de Datos Fundamentales

Temas:
CS Core

- Tipos abstractos de datos estándar como listas, pilas, colas, conjuntos y mapas/diccionarios, incluyendo operaciones sobre ellos.
- Seleccionar y usar estructuras de datos apropiadas.
- Implicaciones de rendimiento de la elección de estructura(s) de datos.
- Cadenas y procesamiento de cadenas.

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Escribir programas que utilicen cada uno de los tipos abstractos de datos clave proporcionados en el lenguaje (por ejemplo, arreglos, tuplas/registros/estructuras, listas, pilas, colas y tipos de datos asociativos como conjuntos, diccionarios/mapas) [Escribir]
2. Seleccionar la estructura de datos apropiada para un problema dado [Evaluar]
3. Explicar cómo puede cambiar el rendimiento de un programa al usar diferentes estructuras de datos u operaciones [Explicar]
4. Escribir programas que trabajen con texto utilizando las capacidades de procesamiento de cadenas proporcionadas por el lenguaje [Escribir]

3.12.3. SDF/Algoritmos

Temas:
CS Core

- Concepto de algoritmo y noción de eficiencia algorítmica
- Algunos algoritmos comunes (por ejemplo, ordenación, búsqueda, recorrido de árboles, recorrido de grafos)
- Impacto de los algoritmos en la eficiencia tiempo-espacio de los programas

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Explicar el rol de los algoritmos para escribir programas [Explicar]
2. Demostrar cómo un problema puede ser resuelto por diferentes algoritmos, cada uno con diferentes propiedades [Demostrar]
3. Explicar algunos algoritmos comunes (por ejemplo, ordenación, búsqueda, recorrido de árboles, recorrido de grafos) [Explicar]
4. Explicar el impacto en el rendimiento espacio/tiempo de algunos algoritmos [Explicar]

3.12.4. SDF/Prácticas de Desarrollo de Software

Temas:
CS Core

- Pruebas básicas, incluido el diseño de casos de prueba
- Uso de un IDE de propósito general, incluido su depurador
- Estilo de programación que mejora la legibilidad
- Especificar la funcionalidad de un módulo en lenguaje natural.

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Desarrollar pruebas para módulos y aplicar una variedad de estrategias para diseñar casos de prueba [Crear]
2. Explicar algunas limitaciones de las pruebas de programas [Explicar]
3. Compilar, ejecutar y depurar programas utilizando un IDE moderno y herramientas asociadas como depuradores visuales [Analizar]
4. Aplicar pautas básicas de estilo de programación para ayudar a la legibilidad de los programas, como comentarios, sangría, nombres adecuados de variables, etc [Aplicar]
5. Escribir especificaciones de un módulo como comentario de módulo que describa su funcionalidad [Escribir]

3.12.5. SDF/Sociedad, Ética y la Profesión

Temas:
CS Core

- Derechos de propiedad intelectual de los programadores para los programas que desarrollan.
- Plagio e integridad académica.
- Responsabilidad y obligación de los programadores con respecto al código que desarrollan para soluciones. Fundamentos de Seguridad
- Ética profesional básica del trabajo de los programadores.

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Explicar/comprender algunos de los problemas de propiedad intelectual relacionados con programas [Explicar]
2. Explicar/comprender cuándo se puede utilizar código desarrollado por otros y las formas adecuadas de revelar su uso [Explicar]
3. Explicar/comprender la responsabilidad de los programadores al desarrollar código para una solución general (que puede ser desarrollada por un equipo) [Explicar]
4. Explicar/comprender uno o más códigos de conducta aplicables a los programadores [Explicar]

3.13. Ingeniería de Software (SE)

Ya a principios de la década de 1970, el informático británico Brian Randell supuestamente dijo: "La ingeniería de software es la construcción multi-persona de programas multi-versión". Esta es una visión esencial: mientras que la programación es la habilidad que gobierna nuestra capacidad para escribir un programa, la ingeniería de software es distinta en dos dimensiones: tiempo y personas.

Primero, un proyecto de ingeniería de software es un esfuerzo de equipo; ser un experto en programación solitario es insuficiente. Los ingenieros de software calificados deben demostrar experiencia en comunicación y colaboración. La programación puede ser una actividad individual, pero la ingeniería de software es colaborativa, profundamente ligada a cuestiones de profesionalismo, trabajo en equipo y comunicación.

Segundo, un proyecto de ingeniería de software suele ser "multi-versión". Tiene una vida útil esperada; necesita funcionar correctamente durante meses, años o décadas. Las funcionalidades pueden agregarse o eliminarse para cumplir con los requisitos del producto. El propio equipo de ingeniería probablemente cambiará. El contexto tecnológico cambiará, a medida que evolucionen nuestras plataformas informáticas, cambien los lenguajes de programación, se actualicen las dependencias, etc. Esta exposición a cuestiones de tiempo y cambio es novedosa en comparación con un proyecto de programación: no basta con construir algo que funcione; debe funcionar y seguir funcionando. Muchos de los temas más desafiantes en tecnología comparten "el tiempo conducirá al cambio" como causa raíz: compatibilidad con versiones anteriores, desfase de versiones, gestión de dependencias, cambios de esquema, evolución de protocolos.

La ingeniería de software presenta un desafío particularmente difícil para el aprendizaje en un entorno académico. Dado que las principales diferencias entre la programación y la ingeniería de software son el tiempo y el trabajo en equipo, es difícil generar lecciones que requieran un trabajo en equipo exitoso y que presenten fielmente los desafíos del tiempo. Además, algunos temas de ingeniería de software serán más auténticos y relevantes si nuestros estudiantes experimentan proyectos colaborativos y a largo plazo in vivo en lugar de en el aula. Independientemente de que esto suceda como una pasantía, la participación en un proyecto de código abierto o un rol de ingeniería a tiempo completo, un mes de experiencia práctica a tiempo completo tiene más horas disponibles que el curso promedio de ingeniería de software.

Por lo tanto, un plan de estudios de ingeniería de software debe centrarse en los conceptos necesarios para la mayoría de los nuevos contratados graduados, y que son novedosos para aquellos que se capacitan principalmente como programadores, o que son conceptos abstractos que pueden no declararse/compartirse explícitamente en el trabajo. Tales temas incluyen, entre otros:

- Pruebas
- Trabajo en equipo, colaboración
- Comunicación
- Diseño
- Mantenimiento y evolución
- Herramientas de ingeniería de software

Todas las pruebas sugieren que el papel del software en nuestra sociedad seguirá creciendo en el futuro previsible. Además, la era de "dos programadores en un garaje" parece haber llegado a su fin. La mayoría del software importante hoy en día es un esfuerzo de equipo, que se basa en código existente y aprovecha la funcionalidad existente. El estudio de las habilidades de ingeniería de software es un contrapunto profundamente importante a la experiencia cotidiana de los estudiantes de informática: debemos impresionarles con la realidad de que pocos proyectos de software se gestionan escribiendo desde cero como un esfuerzo en solitario. La comunicación, el trabajo en equipo, la planificación, las pruebas y las herramientas son mucho más importantes a medida que nuestros estudiantes pasan del aula y dejan su huella en el mundo.

Aunque la mayoría de los graduados en CS ocuparán un puesto en la industria que requiere este material, los temas del CS Core presentados aquí son valiosos independientemente de si los graduados van a la industria o a la academia.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
3.13.1 Trabajo en Equipo (Pág. 112)			No
3.13.2 Herramientas y Entornos (Pág. 113)			No
3.13.3 Requisitos del Producto (Pág. 113)			No
3.13.4 Diseño de Software (Pág. 114)			No
3.13.5 Construcción de Software (Pág. 115)			No
3.13.6 Verificación y Validación de Software (Pág. 117)			No
3.13.7 Refactorización y Evolución del Código (Pág. 118)			No
3.13.8 Confiabilidad del Software (Pág. 119)			No
3.13.9 Métodos Formales (Pág. 120)			No

3.13.1. SE/Trabajo en Equipo

Temas:
CS Core

- Comunicación efectiva, incluida la oral y escrita, así como formal (correo electrónico, documentos, comentarios, presentaciones) e informal (chat de equipo, reuniones). Comunicación
- Causas comunes de conflicto en el equipo y enfoques para la resolución de conflictos.
- Programación cooperativa: 1. Programación por parejas (pair programming) o Enjambre (swarming) 2. Revisión de código 3. Colaboración mediante control de versiones
- Roles y responsabilidades en un equipo de software: Ética Profesional 1. Ventajas del trabajo en equipo 2. Riesgos y complejidad de dicha colaboración
- Procesos del equipo: responsabilidades para las tareas, estimación de esfuerzo, estructura de reuniones, cronograma de trabajo
- Importancia de la diversidad e inclusión en el equipo. Comunicación

KA Core

- Interfaz con las partes interesadas, como equipo: 1. Gestión y otros equipos no técnicos 2. Clientes 3. Usuarios
- Riesgos asociados con equipos físicos, distribuidos, híbridos y virtuales, incluyendo comunicación, percepción, estructura, puntos de falla, mitigación y recuperación, etc.

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Seguir prácticas efectivas de comunicación en equipo [Analizar]
2. Articular las fuentes, peligros y beneficios potenciales del conflicto en el equipo, especialmente enfocándose en el valor de estar en desacuerdo sobre ideas o propuestas sin insultar a las personas [Articular]
3. Facilitar una estrategia de resolución de conflictos y resolución de problemas en un entorno de equipo [Analizar]
4. Colaborar efectivamente en desarrollo/programación cooperativa [Analizar]
5. Proponer y delegar los roles y responsabilidades necesarios en un equipo de desarrollo de software [Proponer]
6. Redactar y seguir una agenda para una reunión de equipo [Componer]
7. Facilitar, a través de la participación en un proyecto de equipo, los elementos centrales de la formación de equipos, el establecimiento de una cultura de equipo saludable y la gestión de equipos, incluida la creación y ejecución de un plan de trabajo del equipo [Crear]
8. Promover la importancia y los beneficios que la diversidad y la inclusión aportan a un equipo de desarrollo de software [Defender]

KACore:

9. Referenciar, como equipo, la importancia de, y las estrategias para interactuar con partes interesadas fuera del equipo en niveles tanto técnicos como no técnicos [Referenciar]
10. Enumerar los riesgos asociados con equipos físicos, distribuidos, híbridos y virtuales y los posibles puntos de falla y cómo mitigarlos y recuperarse/aprender de los fallos [Enumerar]

3.13.2. SE/Herramientas y Entornos

Temas:
CS Core

- Gestión de configuración de software y control de versiones: Prácticas de Desarrollo de Software 1. Configuración en control de versiones, builds/configuración reproducibles. 2. Estrategias de ramificación en control de versiones. Ramas de desarrollo vs ramas de lanzamiento. Desarrollo basado en tronco (*trunk-based development*). 3. Estrategias de fusión/rebase, cuando sean relevantes.

KA Core

- Gestión de lanzamientos (*release management*).
- Herramientas de prueba, incluyendo herramientas de análisis estático y dinámico. Prácticas de Desarrollo de Software, Codificación Segura
- Automatización de procesos de software: 1. Sistemas de construcción (*build systems*): el valor de builds rápidos, herméticos y reproducibles, comparar/contrastar enfoques para construir un proyecto. 2. Integración Continua (CI): el uso de automatización y pruebas automatizadas para hacer una validación preliminar de que la revisión actual del tronco se construye y pasa las pruebas (básicas). 3. Gestión de dependencias: actualización de dependencias externas/aguas arriba, gestión de paquetes, SemVer.
- Herramientas de diseño y comunicación (documentos, diagramas, formas comunes de diagramas de diseño como UML).
- Conceptos y mecanismos de integración de herramientas. Prácticas de Desarrollo de Software
- Uso de las facilidades de un IDE moderno: depuración, refactorización, búsqueda/indexación, asistentes de código basados en ML, etc. Prácticas de Desarrollo de Software

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Describir la diferencia entre la gestión de configuración de software centralizada y distribuida [Describir]
2. Describir cómo el control de versiones puede usarse para ayudar en la gestión de lanzamientos de software [Describir]
3. Identificar elementos de configuración y usar una herramienta de control de código fuente en un proyecto pequeño basado en equipo [Analizar]

KACore:

4. Describir cómo las herramientas de prueba estáticas y dinámicas disponibles pueden integrarse en el entorno de desarrollo de software [Describir]
5. Comprender el uso de sistemas de CI como una fuente de verdad para el estado del código compartido del equipo (éxito en la construcción y pruebas) [Explicar]
6. Describir los problemas importantes al seleccionar un conjunto de herramientas para el desarrollo de un sistema de software específico, incluyendo herramientas para seguimiento de requisitos, modelado de diseño, implementación, automatización de construcción y pruebas [Describir]
7. Demostrar la capacidad de usar herramientas de software para apoyar el desarrollo de un producto de software de tamaño mediano [Demostrar]

3.13.3. SE/Requisitos del Producto

Temas:
KA Core

- Describir requisitos funcionales usando, por ejemplo, casos de uso o historias de usuario. 1. Usar al menos un método para documentar y estructurar requisitos funcionales. 2. Comprender cómo el método apoya el diseño y la implementación. 3. Fortalezas y debilidades de usar un enfoque específico.
- Propiedades de los requisitos, incluyendo consistencia, validez, completitud y viabilidad.
- Obtención (*elicitation*) de requisitos. 1. Fuentes de requisitos, por ejemplo, usuarios, administradores o personal de soporte. 2. Métodos de recopilación de requisitos, por ejemplo, encuestas, entrevistas o análisis de comportamiento.
- Requisitos no funcionales, por ejemplo, seguridad, usabilidad o rendimiento, también llamados Atributos de Calidad. Sostenibilidad

- Identificación y gestión de riesgos, incluyendo consideraciones éticas en torno al producto propuesto. Ética Profesional
- Comunicar y/o formalizar especificaciones de requisitos.

Non Core

- Prototipado de una herramienta tanto para obtener como para validar/confirmar requisitos.
- Evolución del producto: cuando cambian los requisitos, cómo entender qué efecto tiene eso y qué cambios deben realizarse.
- Estimación de esfuerzo: 1. Aprender técnicas para estimar mejor el esfuerzo requerido para completar una tarea; 2. Practicar la estimación y compararla con cuánto tiempo toman las tareas; 3. La estimación de esfuerzo es bastante difícil, por lo que es probable que los estudiantes se equivoquen en muchos casos, pero ver el proceso desarrollarse con su propio trabajo es valioso.

Aprendizaje esperado (*Learning Outcomes*):

KACore:

1. Comparar diferentes métodos de obtención de requisitos a lo largo de múltiples ejes [Comparar]
2. Identificar diferencias entre dos métodos de describir requisitos funcionales (por ejemplo, entrevistas con clientes, estudios de usuarios) y las situaciones donde se preferiría cada uno [Analizar]
3. Identificar qué comportamientos son requeridos, permitidos o prohibidos a partir de un conjunto dado de requisitos y una lista de comportamientos candidatos [Analizar]
4. Recopilar un conjunto de requisitos para un sistema de software simple [Analizar]
5. Identificar áreas de un sistema de software que deben cambiarse, dada una descripción del sistema y un conjunto de nuevos requisitos a implementar [Analizar]
6. Identificar los requisitos funcionales y no funcionales en un conjunto de requisitos [Analizar]

NonCore:

7. Crear un prototipo de un sistema de software para validar un conjunto de requisitos: construir una maqueta, MVP, etc [Crear]
8. Estimar el tiempo para completar un conjunto de tareas, luego comparar las estimaciones con el tiempo real tomado [Estimar]
9. Determinar una secuencia de implementación para un conjunto de tareas, respetando las dependencias entre ellas, con el objetivo de retirar el riesgo lo antes posible [Determinar]
10. Escribir una especificación de requisitos para un sistema de software simple [Escribir]

3.13.4. SE/Diseño de Software

Temas:

CS Core

- Principios de diseño de sistemas. Confiabilidad del Sistema 1. Niveles de abstracción (por ejemplo, diseño arquitectónico y diseño detallado) 2. Separación de preocupaciones (separation of concerns) 3. Ocultación de información (information hiding) 4. Acoplamiento y cohesión (coupling and cohesion)
- Arquitectura de software. Confiabilidad del Sistema 1. Paradigmas de diseño *a*) Descomposición funcional de arriba hacia abajo/diseño por capas *b*) Arquitectura orientada a datos *c*) Análisis y diseño orientado a objetos *d*) Diseño dirigido por eventos 2. Arquitecturas estándar (por ejemplo, cliente-servidor y arquitecturas de microservicios incluyendo discusiones sobre REST, n-capas, tuberías y filtros, Modelo Vista Controlador) 3. Identificar límites y dependencias de componentes
- Programación a gran escala vs programación a pequeña escala (programming in the large vs programming in the small). Confiabilidad del Sistema
- Malos olores del código (code smells) y otras indicaciones de la calidad del código, distintas de la corrección. Análisis, Diseño e Ingeniería de la Seguridad

KA Core

- Principios de diseño de API 1. Consistencia *a*) Las API consistentes son más fáciles de aprender y menos propensas a errores *b*) La consistencia es tanto interna (entre diferentes partes de la API) como externa (siguiendo patrones comunes de API) 2. Capacidad de composición (*composability*) 3. Documentación de contratos *a*) Las operaciones de la API deben describir su efecto en el sistema, pero generalmente no su implementación *b*) Precondiciones, postcondiciones

e invariantes 4. Capacidad de expansión (*expandability*) 5. Reporte de errores *a*) Los errores deben ser claros, predecibles y accionables *b*) La entrada que no coincida con el contrato debe producir un error *c*) Los errores que pueden gestionarse de manera confiable sin reportarse deben gestionarse
- Identificar y codificar invariantes de datos e invariantes temporales
- Modelos estructurales y de comportamiento de diseños de software
- Diseño de datos Modelado de Datos 1. Estructuras de datos 2. Sistemas de almacenamiento
- Trazabilidad de requisitos 1. Comprender qué requisitos son satisfechos por un diseño

Non Core
- Modelado de diseño, por ejemplo con diagramas de clases, diagramas de entidad-relación o diagramas de secuencia
- Medición y análisis de la calidad del diseño
- Principios de diseño y codificación seguros Análisis, Diseño e Ingeniería de la Seguridad 1. Principio de privilegio mínimo 2. Principio de valores predeterminados seguros ante fallos (fail-safe defaults) 3. Principio de aceptabilidad psicológica
- Evaluar compensaciones de diseño (tradeoffs) (por ejemplo, eficiencia vs confiabilidad, seguridad vs usabilidad)

Aprendizaje esperado (*Learning Outcomes*):
CSCore:
1. Identificar la arquitectura de software estándar de un diseño de alto nivel dado [Analizar]
2. Seleccionar y usar un paradigma de diseño apropiado para diseñar un sistema de software simple y explicar cómo se han aplicado los principios de diseño de sistemas en este diseño [Evaluar]
3. Adaptar un diseño de sistema defectuoso para que siga mejor principios como la separación de preocupaciones o la ocultación de información [Crear]
4. Identificar las dependencias entre un conjunto de componentes de software en un diseño arquitectónico [Analizar]

KACore:
5. Diseñar una API para un solo componente de un sistema de software grande, incluyendo la identificación y documentación de las invariantes, el contrato y las condiciones de error de cada operación [Diseñar]
6. Evaluar una descripción de API en términos de consistencia, capacidad de composición y capacidad de expansión [Evaluar]
7. Expandir un diseño existente para incluir una nueva funcionalidad [Crear]
8. Diseñar un conjunto de estructuras de datos para implementar una superficie de API proporcionada [Diseñar]
9. Identificar qué requisitos son satisfechos por un diseño de software proporcionado [Analizar]

NonCore:
10. Traducir un diseño de software en lenguaje natural a diagramas de clases [Traducir]
11. Adaptar un diseño de sistema defectuoso para que siga mejor los principios de privilegio mínimo y valores predeterminados seguros ante fallos [Crear]
12. Contrastar dos diseños de software a través de diferentes cualidades, como eficiencia o usabilidad [Contrastar]

3.13.5. SE/Construcción de Software

Temas:
CS Core
- Pruebas prácticas a pequeña escala Prácticas de Desarrollo de Software 1. Pruebas unitarias 2. Desarrollo guiado por pruebas (Test-driven development): esto es particularmente valioso para los estudiantes psicológicamente, ya que es mucho más fácil involucrarse constructivamente con el desafío de identificar entradas desafiantes para una API dada (casos límite, casos extremos) a priori. Si implementan primero, el instinto es a menudo evitar intentar hacer fallar su nueva creación, mientras que un enfoque de pruebas primero les da la satisfacción intelectual de detectar los casos problemáticos y luego ver cómo pasan más pruebas durante el proceso de desarrollo.
- Documentación Prácticas de Desarrollo de Software 1. Documentación de la interfaz: describir los requisitos de la interfaz, potencialmente incluyendo contratos (formales o informales), pre y post

condiciones, invariantes. 2. La documentación de la implementación debe centrarse en partes de código complicadas y no obvias, ya sea porque el código usa características avanzadas del lenguaje o porque el comportamiento del código es complejo. (No agregar comentarios que reafirmen operaciones comunes/obvias y características simples del lenguaje.) *a*) Aclarar el flujo de datos, cómputo, etc., centrándose en lo que el código es. *b*) Identificar partes sutiles/complicadas del código y refactorizar para que sean autoexplicativas si es posible o proporcionar comentarios apropiados para aclarar.

KA Core

- Estilo de codificación Prácticas de Desarrollo de Software 1. Guías de estilo 2. Comentarios 3. Nomenclatura (*naming*)
- "Mejores Prácticas" para la codificación: técnicas, patrones/idiomas, mecanismos para construir programas de calidad Codificación Segura, Prácticas de Desarrollo de Software 1. Prácticas de codificación defensiva 2. Prácticas y principios de codificación segura 3. Usar mecanismos de manejo de excepciones para hacer los programas más robustos, tolerantes a fallos
- Depuración (debugging) Prácticas de Desarrollo de Software
- Registro (logging)
- Uso de bibliotecas y frameworks desarrollados por otros Prácticas de Desarrollo de Software

Non Core

- Pruebas a mayor escala 1. Dobles de prueba (test doubles) (*stubs*, *mocks*, *fakes*) 2. Inyección de dependencias (*dependency injection*)
- Secuenciación del trabajo, incluyendo identificación de dependencias, hitos y retiro de riesgo 1. Identificación de dependencias: identificar las dependencias entre diferentes tareas 2. Hitos: una colección de tareas que sirve como marcador de progreso cuando se completa. Idealmente, el hito abarca una unidad útil de funcionalidad. 3. Retiro de riesgo: identificar qué elementos de un proyecto son riesgosos y priorizar la finalización de tareas que aborden esos riesgos.
- Problemas potenciales de seguridad en programas Codificación Segura 1. Desbordamientos de búfer y otros tipos de desbordamientos 2. Condiciones de carrera (race conditions) 3. Inicialización incorrecta, incluida la elección de privilegios 4. Validación de entrada (input validation)
- Documentación (autogenerada)
- Contexto de desarrollo: "campo verde" (*green field*) vs base de código existente 1. Análisis de impacto del cambio (*change impact analysis*) 2. Realización del cambio (*change actualization*)
- Gestión de lanzamientos (release management)
- Prácticas de DevOps

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Escribir pruebas unitarias apropiadas para un componente pequeño (varias funciones, un solo tipo, etc.) [Escribir]
2. Escribir comentarios apropiados de interfaz y (si es necesario) de implementación para un componente pequeño [Escribir]

KACore:

3. Describir técnicas, patrones de codificación y mecanismos para implementar diseños para lograr propiedades deseadas como confiabilidad, eficiencia y robustez [Describir]
4. Escribir código robusto utilizando mecanismos de manejo de excepciones [Escribir]
5. Describir prácticas de codificación segura y defensiva [Describir]
6. Seleccionar y usar un estándar de codificación definido en un proyecto de software pequeño [Usar]
7. Comparar y contrastar estrategias de integración, incluyendo integración de arriba hacia abajo (top-down), de abajo hacia arriba (bottom-up) y sándwich [Comparar]
8. Describir el proceso de analizar e implementar cambios en la base de código desarrollada para un proyecto específico [Describir]
9. Describir el proceso de analizar e implementar cambios en una base de código existente grande [Describir]

NonCore:

10. Reescribir un programa simple para eliminar vulnerabilidades comunes, como desbordamientos de búfer, desbordamientos de enteros y condiciones de carrera [Aplicar]

11. Escribir un componente de software que realice alguna tarea no trivial y sea resistente a errores de entrada y de tiempo de ejecución [Escribir]

3.13.6. SE/Verificación y Validación de Software

Temas:
CS Core

- Conceptos de verificación y validación 1. Verificación: ¿Estamos construyendo la cosa correctamente? 2. Validación: ¿Construimos la cosa correcta?
- Por qué importan las pruebas: ¿El componente sigue siendo funcional a medida que el código evoluciona?
- Objetivos de las pruebas 1. Usabilidad 2. Confiabilidad 3. Conformidad con la especificación 4. Rendimiento 5. Seguridad
- Tipos de pruebas 1. Unitarias 2. Integración 3. Validación 4. Sistema
- Diferencias estilísticas entre el código de prueba y el código de producción: DAMP vs DRY - se justifica más duplicación en el código de prueba.

KA Core

- Planificación y generación de pruebas 1. Generación de casos de prueba, a partir de modelos formales, especificaciones, etc. 2. Cobertura de pruebas *a*) Matrices de prueba *b*) Cobertura de código - ¿cuánto del código está probado? *c*) Cobertura de entorno - ¿cuántas arquitecturas de hardware, sistemas operativos, navegadores, etc. están probados? 3. Datos y entradas de prueba
- Desarrollo de pruebas 1. Desarrollo guiado por pruebas 2. Pruebas orientadas a objetos, uso de mocks e inyección de dependencias 3. Técnicas de prueba de caja opaca (anteriormente, caja negra) y caja transparente (anteriormente, caja blanca) 4. Herramientas de prueba, incluyendo cobertura de código, análisis estático y fuzzing
- Verificación y validación en el ciclo de desarrollo 1. Revisiones de código 2. Automatización de pruebas, incluyendo automatización de herramientas 3. Pruebas pre-commit y post-commit 4. Compensaciones entre cobertura de pruebas y rendimiento/latencia de las pruebas 5. Seguimiento y priorización de defectos: reproducibilidad de los defectos reportados
- Desafíos de verificación y validación específicos del dominio 1. Pruebas de rendimiento y evaluación comparativa (benchmarking) 2. Asincronía, paralelismo y concurrencia 3. Crítico para la seguridad (*safety-critical*) 4. Numérico

Non Core

- Herramientas y automatización de verificación y validación 1. Análisis estático 2. Cobertura de código 3. Fuzzing 4. Análisis dinámico y contención de fallos (sanitizadores, etc.) 5. Registro y seguimiento de fallos
- Planificación y generación de pruebas 1. Estimación de fallos y terminación de pruebas, incluyendo siembra de defectos 2. Uso de números aleatorios y pseudoaleatorios en las pruebas
- Pruebas de rendimiento y evaluación comparativa 1. Rendimiento (*throughput*) y latencia 2. Degradación bajo carga (pruebas de estrés, manejo FIFO vs LIFO de solicitudes) 3. Aceleración (*speedup*) y escalado (*scaling*) *a*) Ley de Amdahl *b*) Ley de Gustafson *c*) Escalado suave (*soft*) y débil (*weak*) 4. Identificar y medir figuras de mérito (*figures of merit*) 5. Cuellos de botella de rendimiento comunes *a*) Limitado por cómputo (*compute-bound*) *b*) Limitado por ancho de banda de memoria *c*) Limitado por latencia 6. Métodos estadísticos y mejores prácticas para la evaluación comparativa (benchmarking) *a*) Estimación de la incertidumbre *b*) Intervalos de confianza 7. Análisis y presentación (gráficos, etc.) 8. Técnicas de medición de tiempo (*timing*)
- Pruebas de sistemas asíncronos, paralelos y concurrentes
- Verificación y validación de artefactos que no son código (documentación, materiales de capacitación)

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Explicar por qué las pruebas son importantes [Explicar]
2. Distinguir entre validación y verificación de programas [Distinguir]

3. Describir diferentes objetivos de las pruebas [Describir]
4. Comparar y contrastar los diferentes tipos y niveles de pruebas (regresión, unitarias, integración, sistemas y aceptación) [Comparar]

KACore:

5. Describir técnicas para crear un plan de pruebas y generar casos de prueba [Describir]
6. Crear un plan de pruebas para un segmento de código de tamaño mediano que incluya una matriz de pruebas y generación de datos y entradas de prueba [Crear]
7. Implementar un plan de pruebas para un segmento de código de tamaño mediano [Implementar]
8. Identificar los principios fundamentales de los métodos de desarrollo guiado por pruebas y explicar el papel de las pruebas automatizadas en estos métodos [Analizar]
9. Discutir problemas relacionados con las pruebas de software orientado a objetos [Debatir]
10. Describir el uso de mocks y la inyección de dependencias y su aplicación [Describir]
11. Realizar, como parte de una actividad en equipo, una revisión de código de un segmento de código de tamaño mediano [Evaluar]
12. Describir el papel que las herramientas pueden desempeñar en la validación del software [Describir]
13. Automatizar las pruebas en un proyecto de software pequeño [Diseñar]
14. Explicar los roles, pros y contras de las pruebas pre-commit y post-commit [Explicar]
15. Discutir las compensaciones entre la cobertura de pruebas y el rendimiento/latencia de las pruebas y cómo esto puede afectar la verificación [Debatir]
16. Usar una herramienta de seguimiento de defectos para gestionar defectos de software en un proyecto de software pequeño [Usar]
17. Discutir las limitaciones de las pruebas en ciertos dominios [Debatir]

NonCore:

18. Describir y comparar diferentes herramientas para verificación y validación [Describir]
19. Automatizar el uso de diferentes herramientas en un proyecto de software pequeño [Diseñar]
20. Explicar cómo y cuándo se deben usar números aleatorios en las pruebas [Explicar]
21. Describir enfoques para la estimación de fallos [Describir]
22. Estimar el número de fallos en una aplicación de software pequeña basándose en la densidad de fallos y la siembra de fallos [Estimar]
23. Describir el rendimiento (*throughput*) y la latencia y proporcionar ejemplos de cada uno [Describir]
24. Explicar la aceleración (speedup) y las diferentes formas de escalado (scaling) y cómo se calculan [Explicar]
25. Describir cuellos de botella de rendimiento comunes [Describir]
26. Describir métodos estadísticos y mejores prácticas para la evaluación comparativa (benchmarking) de software [Describir]
27. Explicar técnicas y desafíos para medir el tiempo al construir una evaluación comparativa (benchmark) [Explicar]
28. Identificar las figuras de mérito, construir y ejecutar una evaluación comparativa (benchmark) y analizar estadísticamente y visualizar los resultados para un proyecto de software pequeño [Analizar]
29. Describir técnicas y problemas con las pruebas de software asíncrono, concurrente y paralelo [Describir]
30. Crear un plan de pruebas para un segmento de código de tamaño mediano que contenga código asíncrono, concurrente y/o paralelo, incluyendo una matriz de pruebas y generación de datos y entradas de prueba [Crear]
31. Describir técnicas para la verificación y validación de artefactos que no son código [Describir]

3.13.7. SE/Refactorización y Evolución del Código

Temas:

KA Core

- Ley de Hyrum / Ley de las Interfaces Implícitas
- Compatibilidad con versiones anteriores 1. La compatibilidad no es una propiedad de una sola entidad, es una propiedad de una relación. 2. La compatibilidad con versiones anteriores debe

evaluarse en términos de proveedor + consumidor(es) o con un modelo bien especificado de qué formas de compatibilidad aspira/promete un proveedor.
- Refactorización 1. Patrones de refactorización estándar (renombrar, integrar, extraer, etc.) 2. Uso de herramientas de refactorización en el IDE 3. Aplicación de herramientas de análisis estático (para identificar código que necesita refactorización, generar cambios, etc.) 4. Valor de la refactorización como remedio para la deuda técnica
- Control de versiones (versioning) 1. Versionado Semántico (SemVer) 2. Desarrollo basado en tronco (*trunk-based development*)

Non Core
- Refactorización "a Gran Escala": técnicas cuando un cambio de refactorización es demasiado grande para comprometerlo de forma segura (proyectos grandes), o cuando es imposible sincronizar el cambio entre proveedor + todos los consumidores (múltiples repositorios, consumidores con código privado). 1. Expresar tanto las API antiguas como las nuevas para que puedan coexistir. 2. Minimizar el tamaño de los cambios de comportamiento. 3. ¿Por qué se requieren estas técnicas?. Por ejemplo, "consumidores de API que puedo ver" vs "consumidores que no puedo ver").

Aprendizaje esperado (*Learning Outcomes*):
KACore:
1. Identificar tanto el comportamiento explícito como el implícito de una interfaz e identificar riesgos potenciales de la Ley de Hyrum [Analizar]
2. Considerar las entradas de herramientas de análisis estático y/o los principios de Diseño de Software para identificar código que necesita refactorización [Analizar]
3. Identificar cambios que pueden considerarse ampliamente "compatibles con versiones anteriores", potencialmente con declaraciones explícitas sobre qué uso está o no soportado [Analizar]
4. Refactorizar la implementación de una interfaz para mejorar el diseño, claridad, etc., con impacto mínimo/cero en los usuarios existentes [Crear]
5. Evaluar si un cambio propuesto es lo suficientemente seguro dada la metodología de control de versiones en uso para un proyecto dado [Evaluar]

NonCore:
6. Planificar una refactorización compleja de múltiples pasos para cambiar el comportamiento predeterminado de una API de manera segura [Planear]

3.13.8. SE/Confiabilidad del Software

Temas:
KA Core
- Concepto de confiabilidad como probabilidad de fallo o tiempo medio entre fallos, y fallas (faults) como causa de fallos (failures)
- Identificar requisitos de confiabilidad para diferentes tipos de software
- Fallos de software causados por defectos/errores (bugs), por lo que para alta confiabilidad el objetivo es tener un mínimo de defectos - inyectando menos defectos (mejor capacitación, educación, planificación) y eliminando la mayoría de los defectos inyectados (pruebas, revisión de código, etc.)
- Confiabilidad del software, confiabilidad del sistema y comportamiento de fallos
- Ciclo de inyección y eliminación de defectos, y diferentes enfoques para la eliminación de defectos
- Comparar el enfoque de "presupuesto de error" (error budget) para la confiabilidad con el enfoque "libre de errores" (error-free) e identificar dominios donde cada uno es relevante.

Non Core
- Modelos de confiabilidad de software
- Técnicas y modelos de tolerancia a fallos de software 1. Diferencias contextuales en la tolerancia a fallos (por ejemplo, hacer fallar un sistema crítico para la aviación se evita firmemente, hacer fallar un sistema de procesamiento de datos antes de que se escriban datos corruptos en el almacenamiento es muy valioso)
- Prácticas de ingeniería de confiabilidad de software - incluyendo revisiones, pruebas, verificación de modelos práctica (practical model checking)

- Identificación de dominios de fallos dependientes e independientes, y su impacto en la confiabilidad del sistema
- Análisis basado en mediciones de la confiabilidad del software - telemetría, monitorización y alertas, paneles de control, métricas de calificación de lanzamientos, etc.

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Describir cómo determinar el nivel de confiabilidad requerido por un sistema de software [Describir]
2. Explicar los problemas que existen para lograr niveles muy altos de confiabilidad [Explicar]
3. Comprender enfoques para minimizar fallas que pueden aplicarse en cada etapa del ciclo de vida del software [Explicar]

NonCore:

4. Demostrar la capacidad de aplicar múltiples métodos para desarrollar estimaciones de confiabilidad para un sistema de software [Demostrar]
5. Identificar métodos que conducirán a la realización de una arquitectura de software que logre un nivel especificado de confiabilidad [Analizar]
6. Identificar formas de aplicar redundancia para lograr tolerancia a fallos [Analizar]
7. Identificar dependencias de punto único de fallo (single-point-of-failure, SPF) en un diseño de sistema [Analizar]

3.13.9. SE/Métodos Formales

Temas:
Non Core

- Especificación formal de interfaces 1. Especificación de pre y post condiciones 2. Lenguajes formales para escribir y analizar pre y post condiciones.
- Áreas de problemas bien atendidas por los métodos formales 1. Programación libre de bloqueos (*lock-free*), condiciones de carrera de datos 2. Sistemas asíncronos y distribuidos, bloqueos (*deadlock*), bloqueos vivos (*livelock*), etc.
- Comparación con otras herramientas y técnicas para la detección de defectos 1. Pruebas 2. *Fuzzing*
- Enfoques formales para el modelado y análisis de software 1. Verificadores de modelos (model checkers) 2. Buscadores de modelos (model finders)

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Describir el papel que las técnicas de especificación y análisis formal pueden desempeñar en el desarrollo de software complejo y comparar su uso como técnicas de validación y verificación con las pruebas [Describir]
2. Aplicar técnicas de especificación y análisis formal a diseños y programas de software de baja complejidad [Aplicar]
3. Explicar los beneficios y desventajas potenciales del uso de lenguajes de especificación formal [Explicar]

3.14. Seguridad (SEC)

La informática sustenta casi cada faceta de la infraestructura crítica moderna: transporte, comunicación, salud, educación, generación y distribución de energía, por nombrar algunas. Con los ataques desenfrenados y las violaciones de esta infraestructura, los graduados en ciencias de la computación tienen un papel importante en el diseño, implementación y operación de sistemas de software que sean robustos, seguros y que protejan la seguridad (security).

El área de conocimiento de Seguridad (SEC) se centra en desarrollar una mentalidad de seguridad en la ética general de los graduados en ciencias de la computación, de modo que la seguridad esté incorporada en todos sus productos de trabajo. Los estudiantes de informática necesitan aprender sobre las vulnerabilidades del sistema y comprender las amenazas contra los sistemas informáticos. La elección del título Seguridad fue intencional para servir como un término general de una sola

palabra para esta área de conocimiento, que también incluye conceptos para apoyar la privacidad, la criptografía, los sistemas seguros, los datos seguros y el código seguro.

El área de conocimiento SEC se basa en conceptos compartidos omnipresentes en todas las demás áreas de CS2023. Identifica siete conceptos transversales de la ciberseguridad: confidencialidad, integridad, disponibilidad, evaluación de riesgos, pensamiento sistémico, pensamiento adversarial y pensamiento centrado en el humano. El séptimo concepto, pensamiento centrado en el humano, se añade a los seis conceptos transversales definidos originalmente en los Planes de Estudios de Ciberseguridad 2017 (CSEC2017). Esta adición refuerza a los estudiantes que los humanos también son un eslabón en la cadena general de seguridad. Los principios para proteger sistemas incluyen seguridad por diseño (security-by-design), privacidad por diseño (privacy-by-design), defensa en profundidad (defense-in-depth) y confianza cero (zero-trust).

Otro concepto es la noción de garantía (assurance), que es una atestación de que los mecanismos de seguridad deben cumplir con las políticas de seguridad que se han definido para los datos, procesos y sistemas. La garantía está ligada a los conceptos de verificación y validación. Las consideraciones de privacidad y seguridad de los datos se comparten con las áreas de conocimiento DM (aspectos técnicos) y SEP.

El área de conocimiento SEC se sitúa así sobre varias de las otras áreas de conocimiento de CS2023, al tiempo que incluye conceptos adicionales que no están presentes en esas áreas de conocimiento. CS2023 trata la seguridad como un componente crucial del conjunto de habilidades de cualquier graduado en CS, y las horas necesarias para la preparación en seguridad provienen de todas las otras 16 áreas de conocimiento de CS2023.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
3.14.1 Fundamentos de Seguridad (Pág. 121)			No
3.14.2 Sociedad, Ética y la Profesión (Pág. 122)			No
3.14.3 Codificación Segura (Pág. 123)			No
3.14.4 Criptografía (Pág. 124)			No
3.14.5 Análisis, Diseño e Ingeniería de la Seguridad (Pág. 125)			No
3.14.6 Informática Forense (Pág. 126)			No
3.14.7 Gobierno de la Seguridad (Pág. 127)			No

3.14.1. SEC/Fundamentos de Seguridad

Esta unidad de conocimiento introduce conceptos fundamentales de seguridad, principios transversales y el desarrollo de una mentalidad de seguridad. Cubre confidencialidad, integridad, disponibilidad, evaluación de riesgos, pensamiento sistémico, pensamiento adversarial y pensamiento centrado en el humano. También incluye conceptos básicos de autenticación, autorización, vulnerabilidades, amenazas, superficies de ataque, denegación de servicio, principios de protección y el impacto de la IA en la seguridad y la privacidad.

Temas:

CS Core

- Desarrollar una mentalidad de seguridad que incorpore conceptos transversales: confidencialidad, integridad, disponibilidad, evaluación de riesgos, pensamiento sistémico, pensamiento adversarial, pensamiento centrado en el humano.
- Conceptos básicos de autenticación y autorización/control de acceso.
- Vulnerabilidades, amenazas, superficies de ataque y vectores de ataque Protección y Seguridad.
- Denegación de Servicio (DoS) y Denegación de Servicio Distribuida (DDoS) Protección y Seguridad.
- Principios y prácticas de protección, por ejemplo, privilegio mínimo, diseño abierto, valores predeterminados seguros ante fallos, defensa en profundidad y confianza cero; y cómo pueden implementarse Principios del Sistema Operativo, Protección y Seguridad, Construcción de Software, Políticas de Seguridad, Leyes y Delitos Informáticos.
- Consideraciones de optimización entre seguridad, privacidad, rendimiento y otros objetivos de diseño Prácticas de Desarrollo de Software, Verificación y Validación de Software, Diseño de Sistemas.

- Impacto de la IA en la seguridad y la privacidad: usar la IA para reforzar las defensas y abordar las capacidades adversariales aumentadas debido a la IA Aplicaciones e Impacto Social, Diseño de Sistemas, Sociedad, Ética y Profesión.

KA Core

- Modelos de control de acceso (por ejemplo, discrecional, obligatorio, basado en roles y basado en atributos).
- Controles de seguridad.
- Conceptos de confianza (trust) y confiabilidad (trustworthiness).
- Aplicaciones de una mentalidad de seguridad: web, nube y dispositivos móviles Diseño del Sistema, Aspectos Comunes, Preocupaciones Compartidas.
- Protección de sistemas embebidos y ciberfísicos Plataformas Embebidas.
- Principios de seguridad usable y computación centrada en el humano Diseño de Sistemas, Políticas de Seguridad, Leyes y Delitos Informáticos.
- Seguridad y confianza en sistemas de IA/aprendizaje automático, por ejemplo, adecuados para el propósito, límites operativos éticos, fuentes de conocimiento autorizadas, datos de entrenamiento verificados, pruebas de evaluación del sistema repetibles, atestación del sistema, validación/certificación independiente; consecuencias no deseadas de: efecto adverso Introducción: Problemas Fundamentales, Aprendizaje Automático, Aplicaciones e Impacto Social, Políticas de Seguridad, Leyes y Delitos Informáticos.
- Riesgos de seguridad en la construcción y operación de sistemas de IA/aprendizaje automático (por ejemplo, sesgo algorítmico, sesgo del corpus de conocimiento, sesgo del corpus de entrenamiento, violación de derechos de autor) Introducción: Problemas Fundamentales, Aprendizaje Automático, Aplicaciones e Impacto Social.
- Consideraciones de hardware en seguridad, por ejemplo, principios de hardware seguro, arquitecturas de procesador seguro, aceleración criptográfica, compartimentación, interacción software-hardware Organización de Máquina a Nivel de Ensamblador, Representación de Datos a Nivel de Máquina, Rol y Propósito de los Sistemas Operativos.

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Evaluar un sistema para posibles ataques que pueda lanzar un adversario [Evaluar]
2. Diseñar y desarrollar enfoques para proteger un sistema de un conjunto de amenazas identificadas [Diseñar]

KACore:

3. Describir cómo se puede evitar el daño a la privacidad del usuario [Describir]
4. Desarrollar un sistema que incorpore varios principios de seguridad y privacidad [Crear]
5. Comparar los diferentes modelos de control de acceso en términos de funcionalidad y rendimiento [Comparar]
6. Mostrar cómo un adversario podría usar algoritmos de aprendizaje automático para reducir la seguridad de un sistema [Analizar]
7. Mostrar cómo un desarrollador podría mejorar la seguridad de un sistema usando algoritmos de aprendizaje automático [Analizar]
8. Describir vulnerabilidades de hardware (especialmente de la CPU) que pueden impactar al software [Describir]

3.14.2. SEC/Sociedad, Ética y la Profesión

Esta unidad cubre los aspectos sociales, éticos, legales y profesionales de la seguridad y la privacidad, incluyendo seguridad por diseño, privacidad por diseño, hacking ético y las implicaciones de las tecnologías emergentes.

Temas:

CS Core

- Principios y prácticas de privacidad Políticas de Seguridad, Leyes y Delitos Informáticos.
- Impactos sociales de las fallas en seguridad y privacidad Contexto Social, Privacidad y Libertades Civiles, Políticas de Seguridad, Leyes y Delitos Informáticos.
- Aplicabilidad de leyes y regulaciones sobre seguridad y privacidad Políticas de Seguridad, Leyes y Delitos Informáticos.

- Consideraciones éticas profesionales al diseñar sistemas seguros y mantener la privacidad; hacking ético Ética Profesional, Privacidad y Libertades Civiles, Políticas de Seguridad, Leyes y Delitos Informáticos.

KA Core

- Seguridad por diseño (security by design) Seguridad del Sistema, Diseño del Sistema.
- Privacidad por diseño (privacy by design) e ingeniería de la privacidad Privacidad y Libertades Civiles, Políticas de Seguridad, Leyes y Delitos Informáticos.
- Implicaciones de seguridad y privacidad de actores maliciosos de IA/aprendizaje automático, por ejemplo, identificación de deep fakes Introducción: Problemas Fundamentales, Aprendizaje Automático, Privacidad y Libertades Civiles, Políticas de Seguridad, Leyes y Delitos Informáticos.
- Impactos sociales de los dispositivos del Internet de las Cosas (IoT) y otras tecnologías emergentes en la seguridad y la privacidad Privacidad y Libertades Civiles, Políticas de Seguridad, Leyes y Delitos Informáticos.

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Calcular el impacto de una falla de seguridad en un sistema dado [Calcular]
2. Construir un sistema que se ajuste a las leyes de seguridad [Crear]
3. Aplicar un conjunto de regulaciones de privacidad para diseñar un sistema que proteja la privacidad [Aplicar]

KACore:

4. Evaluar las ramificaciones legales de un sistema que no se ajusta a las leyes y regulaciones aplicables [Evaluar]
5. Construir un sistema diseñado para evitar daños a la privacidad del usuario [Crear]

3.14.3. SEC/Codificación Segura

Esta unidad cubre las vulnerabilidades comunes de software, las prácticas de codificación segura, la programación defensiva y las herramientas para el análisis estático y dinámico para prevenir y mitigar fallos de seguridad.

Temas:

CS Core

- Vulnerabilidades y debilidades comunes.
- Inyección SQL y otros ataques de inyección.
- Técnicas de cross-site scripting y mitigaciones.
- Validación de entrada y sanitización de datos Protección y Seguridad, Conceptos y Prácticas Fundamentales de Programación, Verificación y Validación de Software.
- Seguridad de tipos (type safety) y lenguajes seguros en tipos Sistemas de Tipos, Modelo de Ejecución y Memoria de Sistemas, Protección y Seguridad, Conceptos y Prácticas Fundamentales de Programación, Verificación y Validación de Software.
- Desbordamientos de búfer (buffer overflows), aplastamiento de pila (*stack smashing*) y desbordamientos de enteros Organización de Máquina a Nivel de Ensamblador, Modelo de Ejecución y Memoria de Sistemas, Protección y Seguridad.
- Problemas de seguridad debido a condiciones de carrera (race conditions) Computación Paralela y Distribuida, Evaluación.

KA Core

- Principios de no interferencia (noninterference) y no deducibilidad (nondeducibility).
- Prevención de ataques de flujo de información.
- Técnicas de seguridad ofensiva como defensa.
- Técnicas de detección de malware asistidas por IA.
- Ransomware: creación, prevención y mitigación.
- Uso seguro de componentes de terceros Construcción de Software, Verificación y Validación de Software.
- Malware: variedades, creación, ingeniería inversa y defensa contra ellos Modelo de Ejecución y Memoria de Sistemas, Traducción y Ejecución de Lenguajes.

- Garantía (assurance): pruebas (incluyendo fuzzing y pruebas de penetración), verificación y validación Protección y Seguridad, Conceptos y Prácticas Fundamentales de Programación, Construcción de Software, Verificación y Validación de Software.
- Análisis estático y dinámico Análisis de Programas y Analizadores, Protección y Seguridad, Evaluación, Verificación y Validación de Software.
- Compiladores seguros y generación de código seguro Comportamiento en Tiempo de Ejecución y Sistemas, Traducción y Ejecución de Lenguajes.

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Identificar problemas subyacentes en ejemplos dados de una enumeración de debilidades comunes y explicar cómo pueden evitarse [Analizar]
2. Aplicar técnicas de validación de entrada y sanitización de datos para mejorar la seguridad de un programa [Aplicar]
3. Describir cómo la selección de un lenguaje de programación puede impactar la seguridad del sistema que se está construyendo [Describir]
4. Reescribir un programa en un lenguaje seguro en tipos (por ejemplo, Java o Rust) originalmente escrito en un lenguaje de programación no seguro (por ejemplo, C/C++) [Rediseñar]
5. Evaluar un programa en busca de posibles ataques de desbordamiento de búfer y reescribirlo para prevenir tales ataques [Evaluar]
6. Evaluar un conjunto de programas relacionados en busca de posibles condiciones de carrera y evitar que un adversario las explote [Evaluar]
7. Evaluar y prevenir ataques de inyección SQL en una aplicación de base de datos [Evaluar]
8. Evaluar y prevenir ataques de cross-site scripting contra un sitio web [Evaluar]

KACore:

9. Describir diferentes tipos de software malicioso [Describir]
10. Construir un programa que pruebe todos los errores de manejo de entrada [Crear]
11. Explicar los riesgos del mal uso de interfaces con código de terceros y cómo usar correctamente el código de terceros [Explicar]
12. Discutir la necesidad de actualizar el software para corregir vulnerabilidades de seguridad y la gestión del ciclo de vida de la corrección [Debatir]
13. Construir un sistema que esté protegido de flujos de información no autorizados [Crear]
14. Aplicar herramientas estáticas y dinámicas para identificar fallos de programación [Aplicar]
15. Evaluar un sistema para detectar la existencia de malware y eliminarlo [Evaluar]
16. Implementar técnicas preventivas para reducir la ocurrencia de ransomware [Implementar]

3.14.4. SEC/Criptografía

Esta unidad introduce principios, algoritmos, protocolos criptográficos y sus aplicaciones en la protección de datos y comunicaciones, incluyendo encriptación simétrica/asimétrica, funciones hash, firmas digitales y temas emergentes como la criptografía cuántica y poscuántica.
Temas:
CS Core

- Diferencias entre las vistas algorítmica, aplicada y matemática de la criptografía.
- Preliminares matemáticos: aritmética modular, algoritmo euclidiano, independencia probabilística, conceptos básicos de álgebra lineal, teoría de números, campos finitos, complejidad, análisis asintótico Matemáticas Discretas, Álgebra Lineal.
- Criptografía básica: criptografía de clave simétrica y de clave pública Estructuras de Datos y Algoritmos Fundamentales, Matemáticas Discretas.
- Bloques de construcción criptográficos básicos, incluyendo encriptación simétrica, encriptación asimétrica, funciones hash y autenticación de mensajes Matemáticas Discretas.
- Criptosistemas clásicos, como cifrados de desplazamiento, sustitución, transposición, libros de códigos y máquinas Matemáticas Discretas.
- Principio de Kerckhoff y uso de bibliotecas evaluadas (vetted) Construcción de Software.
- Uso de la criptografía en aplicaciones del mundo real, por ejemplo, dinero electrónico, canales seguros entre clientes y servidores, correo electrónico seguro, autenticación de entidades, emparejamiento de dispositivos, esteganografía y sistemas de votación Seguridad de Red, Procesamiento

de Imágenes.

KA Core

- Matemáticas adicionales: primalidad, factorización y criptografía de curva elíptica Matemáticas Discretas.
- Criptosistemas de clave privada: redes de sustitución-permutación, criptoanálisis lineal, criptoanálisis diferencial, DES y AES Matemáticas Discretas, Seguridad de Red.
- Criptosistemas de clave pública: Diffie-Hellman y RSA Matemáticas Discretas.
- Integridad y autenticación de datos: funciones hash y firmas digitales Matemáticas Discretas, Seguridad y Privacidad de Datos.
- Protocolos criptográficos: autenticación de desafío-respuesta, protocolos de conocimiento cero, compromisos (commitment), transferencia inconsciente, cómputo seguro de dos o múltiples partes, funciones hash, compartición de secretos y aplicaciones Matemáticas Discretas.
- Capacidades del atacante: ataque de mensaje elegido (para firmas), ataques de cumpleaños (birthday attacks), ataques de canal lateral y ataques de inyección de fallos Seguridad de Red.
- Criptografía cuántica; criptografía poscuántica/resistente a la cuántica Estructuras de Datos y Algoritmos Fundamentales, Matemáticas Discretas.
- Blockchain y criptomonedas Matemáticas Discretas, Comunicación.

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Explicar el papel de la criptografía en el soporte de la seguridad y la privacidad [Explicar]
2. Discutir los riesgos de inventar los propios métodos criptográficos [Debatir]
3. Discutir la importancia de los números primos en criptografía y explicar su uso en algoritmos criptográficos [Debatir]
4. Implementar y criptoanalizar cifrados clásicos [Implementar]

KACore:

5. Describir cómo se pueden gestionar las claves criptográficas de forma segura [Describir]
6. Comparar el rendimiento en espacio y tiempo de un conjunto dado de métodos criptográficos [Comparar]
7. Discutir cómo funcionan los criptosistemas modernos de clave privada y formas de criptoanalizarlos [Debatir]
8. Discutir cómo funcionan los criptosistemas modernos de clave pública y formas de criptoanalizarlos [Debatir]
9. Comparar diferentes algoritmos criptográficos en términos de seguridad [Comparar]
10. Explicar protocolos de intercambio de claves y mostrar enfoques para reducir su fallo [Explicar]
11. Describir aplicaciones reales de primitivas y protocolos criptográficos [Describir]
12. Discutir cómo funciona la criptografía cuántica y el impacto de la computación cuántica en los algoritmos criptográficos [Debatir]

3.14.5. SEC/Análisis, Diseño e Ingeniería de la Seguridad

Esta unidad cubre enfoques sistemáticos para la ingeniería de seguridad, incluyendo modelado de amenazas, análisis de riesgos, diseño de seguridad, dominios de ataque y computación confiable (trustworthy computing).

Temas:

CS Core

- Objetivos de la ingeniería de seguridad: construir sistemas que permanezcan confiables a pesar de errores, accidentes o adversarios maliciosos Construcción de Software, Verificación y Validación de Software, Políticas de Seguridad, Leyes y Delitos Informáticos.
- Objetivos de la ingeniería de la privacidad: construir sistemas que diseñen, implementen y desplieguen características y controles de privacidad Privacidad y Libertades Civiles.
- Análisis de problemas y análisis situacional para abordar la seguridad del sistema Verificación y Validación de Software.
- Análisis de compensaciones (tradeoff) de ingeniería basado en tiempo, costo, tolerancia al riesgo, aceptación de riesgo, retorno de la inversión, etc. Evaluación, Verificación y Validación de Software.

KA Core

- Diseño e ingeniería de seguridad, incluyendo requisitos funcionales, subsistemas de seguridad, protección de información, pruebas de seguridad, evaluación de seguridad y evaluación (*assessment*) Evaluación, Requisitos del Producto, Verificación y Validación de Software.
- Análisis de seguridad, que cubre análisis de requisitos de seguridad; análisis de controles de seguridad; análisis de amenazas; y análisis de vulnerabilidades Análisis de Programas y Analizadores, Evaluación.
- Dominios de ataque de seguridad y superficies de ataque, por ejemplo, comunicaciones y redes, hardware, físico, ingeniería social, software y cadena de suministro Seguridad de Red.
- Modos, técnicas y tácticas de ataque de seguridad, por ejemplo, abuso de autenticación; fuerza bruta; manipulación de búfer; inyección de código; inserción de contenido; denegación de servicio; escucha; omisión de función; suplantación; ataque de integridad; intercepción; phishing; análisis de protocolo; abuso de privilegio; suplantación (spoofing); e inyección de tráfico Seguridad de Red, Protección y Seguridad, Verificación y Validación de Software.
- Atestación (attestation) de productos de software con respecto a su especificación y adaptabilidad Requisitos del Producto, Verificación y Validación de Software.
- Diseño y desarrollo de sistemas ciberfísicos.
- Consideraciones para la computación confiable (trustworthy computing), por ejemplo, empaquetado resistente a la manipulación, arranque confiable (trusted boot), núcleo confiable (*trusted kernel*), raíz de confianza de hardware (hardware root of trust), firma y verificación de software, criptografía basada en hardware, virtualización y contenedores Construcción de Software, Verificación y Validación de Software.

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Crear un modelo de amenazas para un sistema o diseño de sistema [Crear]
2. Aplicar análisis situacional para desarrollar soluciones seguras bajo un escenario especificado [Aplicar]
3. Evaluar un escenario dado para análisis de compensaciones (tradeoff) para el rendimiento del sistema, evaluación de riesgos y costos [Evaluar]

KACore:

4. Diseñar un conjunto de controles de seguridad técnicos, contramedidas y protecciones de información para cumplir con los requisitos de seguridad y los objetivos de seguridad de un sistema [Diseñar]
5. Evaluar la efectividad de las funciones de seguridad, controles técnicos y componentes para un sistema [Evaluar]
6. Identificar y mitigar vulnerabilidades y debilidades de seguridad en un sistema [Analizar]
7. Evaluar y predecir comportamientos emergentes en áreas como Ciencia de Datos, IA y Aprendizaje Automático [Evaluar]

3.14.6. SEC/Informática Forense

Esta unidad cubre principios, metodologías y herramientas para la informática forense (digital forensics), incluyendo recolección de evidencia, análisis, respuesta a incidentes y consideraciones legales.
Temas:
KA Core

- Principios y metodologías básicas para la informática forense.
- Diseño de sistemas para informática forense.
- Informática forense en diferentes situaciones: sistemas operativos, sistemas de archivos, informática forense de aplicaciones, informática forense web, informática forense de red, informática forense de dispositivos móviles, uso de auditoría de bases de datos Seguridad de Red.
- Ataques a la informática forense y prevención de tales ataques.
- Procesos de manejo de incidentes.
- Reglas de la evidencia - conceptos generales y diferencias entre jurisdicciones Políticas de Seguridad, Leyes y Delitos Informáticos.
- Problemas legales: protección y gestión de evidencia digital, cadena de custodia, reporte, actuar como testigo experto Políticas de Seguridad, Leyes y Delitos Informáticos.

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Explicar qué es una investigación digital y cómo se puede implementar Políticas de Seguridad, Leyes y Delitos Informáticos [Explicar]
2. Diseñar e implementar software para apoyar la informática forense [Diseñar]
3. Describir los requisitos legales para el uso de datos incautados y su uso. Políticas de Seguridad, Leyes y Delitos Informáticos [Describir]
4. Describir e implementar una cadena de custodia de extremo a extremo desde la incautación inicial de evidencia digital hasta la disposición de la evidencia. Privacidad y Libertades Civiles, Políticas de Seguridad, Leyes y Delitos Informáticos [Describir]
5. Extraer datos de un disco duro para cumplir con la ley Políticas de Seguridad, Leyes y Delitos Informáticos [Extraer]
6. Discutir las responsabilidades y obligaciones profesionales de una persona cuando testifica como experto en informática forense Ética Profesional [Debatir]
7. Recuperar datos basados en un término de búsqueda dado desde un sistema del que se ha hecho una imagen (imaged) [Analizar]
8. Reconstruir datos y eventos a partir de un historial de aplicación, o un artefacto web, o una base de datos en la nube, o un dispositivo móvil. Plataformas Móviles, Plataformas Web [Aplicar]
9. Capturar y analizar tráfico de red. Seguridad de Red [Analizar]
10. Desarrollar enfoques para abordar los desafíos asociados con la informática forense de dispositivos móviles [Crear]
11. Aplicar herramientas de informática forense para investigar violaciones de seguridad [Aplicar]
12. Identificar y mitigar métodos anti-forenses [Analizar]

3.14.7. SEC/Gobierno de la Seguridad

Esta unidad cubre el gobierno organizacional de seguridad, la gestión de riesgos, el desarrollo de políticas, la gestión del ciclo de vida de los datos y la protección de activos críticos.
Temas:
KA Core

- Proteger activos críticos de amenazas.
- Gobierno de seguridad (security governance): objetivos organizacionales y evaluación general de riesgos.
- Gestión de seguridad (security management): lograr y mantener niveles apropiados de confidencialidad, integridad, disponibilidad, responsabilidad (accountability), autenticidad y confiabilidad Verificación y Validación de Software.
- Política de seguridad (security policy): políticas organizacionales, políticas específicas de temas, políticas específicas de sistemas.
- Enfoques para identificar y mitigar riesgos para la infraestructura informática.
- Políticas de gestión del ciclo de vida de los datos: recolección de datos, copias de seguridad y retención; almacenamiento y servicios en la nube; divulgación de violaciones (breach disclosure) Seguridad y Privacidad de Datos.

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Describir activos críticos y cómo pueden protegerse [Describir]
2. Diferenciar entre gobierno de seguridad, gestión y controles, dando ejemplos de cada uno [Diferenciar]
3. Describir un control técnico e implementarlo para mitigar amenazas específicas [Describir]
4. Identificar y evaluar el riesgo de que programas y aplicaciones de base de datos causen violaciones [Analizar]
5. Diseñar e implementar estrategias de copia de seguridad apropiadas que se ajusten a una política dada [Diseñar]
6. Discutir una política de divulgación de violaciones basada en requisitos legales e implementar la política [Debatir]
7. Identificar los riesgos y beneficios de la externalización a la nube [Analizar]

3.15. Sociedad, Ética y la Profesión (SEP)

El Código de Ética y Conducta Profesional de la ACM establece: ”Las acciones de los profesionales de la informática cambian el mundo. Para actuar de manera responsable, deben reflexionar sobre los impactos más amplios de su trabajo, apoyando consistentemente el bien público”. El Código de Ética del IEEE comienza reconociendo ”la importancia de nuestras tecnologías en la afectación de la calidad de vida en todo el mundo”. El Código de Ética Profesional y Conducta de la AAAI comienza con ”Las acciones de los profesionales de la informática, y en particular, los profesionales de la IA, cambian el mundo. Para actuar de manera responsable, deben reflexionar sobre los impactos más amplios de su trabajo, apoyando consistentemente el bien público”.

Si bien los problemas técnicos dominan el plan de estudios de informática, no constituyen un programa educativo completo en un contexto más amplio. Es más evidente hoy que nunca que los estudiantes también deben estar expuestos al contexto social más amplio de la informática para desarrollar una comprensión de los problemas y responsabilidades sociales, éticos, legales y profesionales críticos y relevantes. Esta necesidad de incorporar el estudio de estos problemas no técnicos en el plan de estudios de la ACM se reconoció formalmente en 1991.

Los educadores en ciencias de la computación pueden optar por impartir el material en esta área de conocimiento dentro de los contextos de cursos técnicos y teóricos tradicionales, en cursos dedicados y como parte de cursos de proyecto integrador, proyecto y práctica profesional. El material en esta área de conocimiento se cubre mejor mediante una combinación de todo lo anterior.

El Código de Ética y Conducta Profesional de la ACM, el Código de Ética del IEEE y el Código de Ética Profesional y Conducta de la AAAI proporcionan orientación que sirve como base para la conducta de todos los profesionales de la informática en su trabajo. El Código de la ACM enfatiza que el razonamiento ético no es un algoritmo a seguir, y se espera que los profesionales de la informática consideren cómo su trabajo impacta el bien público como consideración principal.

Un estudiante de ciencias de la computación no debe graduarse sin comprender cómo la sociedad y la ética influyen en la profesión de la informática. Tampoco debería ser posible completar un título en ciencias de la computación sin aprender cómo los profesionales de la informática influyen en la sociedad, las consideraciones éticas involucradas en dar forma a ese impacto, y el papel del estudiante convertido en graduado en estas relaciones, tanto como profesionales de la informática como miembros de la sociedad.

Unidad de Conocimiento (KU)	**Core Tier1**	**Core Tier2**	**Alternativo**
3.15.1 Contexto Social (Pág. 128)			No
3.15.2 Métodos para el Análisis Ético (Pág. 129)			No
3.15.3 Ética Profesional (Pág. 130)			No
3.15.4 Propiedad Intelectual (Pág. 131)			No
3.15.5 Privacidad y Libertades Civiles (Pág. 133)			No
3.15.6 Comunicación (Pág. 133)			No
3.15.7 Sostenibilidad (Pág. 134)			No
3.15.8 Historia de la Informática (Pág. 135)			No
3.15.9 Economías de la Informática (Pág. 136)			No
3.15.10 Políticas de Seguridad, Leyes y Delitos Informáticos (Pág. 137)			No
3.15.11 Diversidad, Equidad, Inclusión y Accesibilidad (Pág. 138)			No

3.15.1. SEP/Contexto Social

El Contexto Social proporciona la base para todas las demás unidades de conocimiento en SEP, particularmente Ética Profesional. Cubre las implicaciones sociales de la informática en un mundo hiperconectado, el impacto de las aplicaciones informáticas en el bienestar y la seguridad individual, las consecuencias de involucrar tecnologías informáticas en la vida cívica y cómo los déficits en diversidad y accesibilidad en la informática afectan a la sociedad.

Temas:

CS Core

- Implicaciones sociales (por ejemplo, ideologías políticas y culturales) en un mundo hiperconectado donde las capacidades y el impacto de las redes sociales, la inteligencia artificial y la informática en general están evolucionando rápidamente.
- Impacto de las aplicaciones informáticas (por ejemplo, redes sociales, aplicaciones de inteligencia artificial) en el bienestar individual y la seguridad de todo tipo (por ejemplo, física, emocional, económica).
- Consecuencias de involucrar tecnologías informáticas, particularmente inteligencia artificial, tecnologías biométricas y sistemas de toma de decisiones algorítmicos, en la vida cívica (por ejemplo, tecnología de reconocimiento facial, etiquetas biométricas, algoritmos de distribución de recursos, software policial) y cómo la agencia y supervisión humana son cruciales.
- Cómo los déficits en diversidad y accesibilidad en la informática afectan a la sociedad y qué pasos se pueden tomar para mejorar la equidad en la informática.

KA Core

- Crecimiento y control de internet, datos, informática e inteligencia artificial.
- A menudo referido como la brecha digital, diferencias en el acceso a recursos de tecnología digital y sus ramificaciones resultantes para género, clase, etnia, geografía y/o países en desarrollo, incluyendo la consideración de la responsabilidad hacia aquellos que podrían ser menos ricos, bajo amenaza o que lucharían para que sus voces sean escuchadas.
- Problemas de accesibilidad, incluyendo requisitos legales como las Pautas de Accesibilidad al Contenido Web (www.w3.org/TR/WCAG21).
- Computación consciente del contexto (context-aware computing).

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Describir las diferentes formas en que la tecnología informática (redes, computación móvil, inteligencia artificial) media la interacción social a nivel personal y colectivo [Describir]
2. Identificar los supuestos y valores de los desarrolladores incorporados en el diseño de hardware y software, especialmente en lo que respecta a la usabilidad para poblaciones diversas, incluidos los desatendidos y aquellos con discapacidades [Analizar]
3. Interpretar el contexto social de un diseño dado y su implementación [Interpretar]
4. Analizar la eficacia de un diseño e implementación dados utilizando datos empíricos [Analizar]
5. Comprender las implicaciones del uso de la tecnología (por ejemplo, redes sociales) para diferentes identidades, culturas y comunidades [Explicar]

KACore:

6. Describir el papel de internet en facilitar la comunicación entre ciudadanos, gobiernos y entre sí [Describir]
7. Analizar los efectos de la dependencia de la informática en la implementación de la democracia (por ejemplo, entrega de servicios sociales, votación electrónica) [Analizar]
8. Describir el impacto de la falta de representación apropiada de personas de poblaciones históricamente minorizadas en la profesión de la informática (por ejemplo, cultura de la industria, diversidad de productos) [Describir]
9. Discutir las implicaciones de la conciencia del contexto en sistemas de computación ubicua [Debatir]
10. Expresar cómo el acceso a internet y a las tecnologías informáticas afecta a diferentes sociedades [Explicar]
11. Identificar por qué/cómo el acceso a internet puede ser visto como un derecho humano [Analizar]

3.15.2. SEP/Métodos para el Análisis Ético

Las teorías y principios éticos son los fundamentos del análisis ético porque son los puntos de vista que pueden proporcionar orientación a lo largo del camino hacia una decisión. Cada teoría enfatiza diferentes supuestos y métodos para determinar la ética de una acción dada. Es importante que los estudiantes reconozcan que las decisiones en diferentes contextos pueden requerir diferentes teorías éticas (incluyendo combinaciones) para llegar a resultados éticamente aceptables, y lo que constituye 'aceptable' depende de una variedad de factores como el contexto cultural. Aplicar métodos para el análisis ético requiere tanto una comprensión de los principios y supuestos subyacentes que guían una herramienta dada como una conciencia del contexto social para esa decisión. Los marcos éticos

tradicionales proporcionados por la filosofía occidental pueden ser útiles, pero no son inclusivos. Se debe hacer un esfuerzo para incluir perspectivas éticas decoloniales, indígenas e históricamente marginadas siempre que sea posible.

Temas:

CS Core

- Evitar falacias y tergiversaciones en la argumentación.
- Teorías éticas y toma de decisiones (marcos filosóficos y sociales, por ejemplo [1]).
- Reconocimiento del papel que juega la cultura en nuestra comprensión, adopción, diseño y uso de la tecnología informática.
- Por qué la ética es importante en la informática y cómo la ética es similar y diferente de las leyes y normas sociales.

KA Core

- Listas de verificación (checklists) profesionales.
- Rúbricas de evaluación.
- Análisis de partes interesadas (stakeholder analysis).
- Teoría del punto de vista (standpoint theory).
- Introducción a los marcos éticos (por ejemplo, consecuencialismo como el utilitarismo, no consecuencialismo como el deber, derechos o justicia, centrado en el agente como la virtud o el feminismo, contractualismo, ética del cuidado) y su uso para analizar un dilema ético.

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Describir cómo un contexto cultural dado impacta la toma de decisiones [Describir]
2. Expresar el uso de ejemplos y analogías en la argumentación ética [Analizar]
3. Analizar (y evitar) falacias lógicas básicas en un argumento [Analizar]
4. Analizar un argumento para identificar premisas y conclusión [Analizar]
5. Evaluar cómo y por qué la ética es tan importante en la informática y cómo se relaciona con las normas culturales, valores y leyes [Evaluar]
6. Justificar una decisión tomada sobre bases éticas [Justificar]

KACore:

7. Distinguir todas las posiciones de las partes interesadas en relación con su contexto cultural en una situación dada [Distinguir]
8. Analizar el potencial de introducir o perpetuar deuda ética (consideración diferida de impactos o implicaciones éticas) en decisiones técnicas [Analizar]
9. Discutir las ventajas y desventajas de los marcos éticos tradicionales [Debatir]
10. Analizar dilemas éticos relacionados con la creación y uso de tecnología desde múltiples perspectivas utilizando marcos éticos [Analizar]

3.15.3. SEP/Ética Profesional

La ética informática es una rama de la filosofía práctica que trata sobre cómo los profesionales de la informática deben tomar decisiones con respecto a la conducta profesional y social. Hay tres influencias principales: 1) el propio código ético personal del individuo, 2) cualquier regulación/decreto/etc. informal o formal de comportamiento ético existente en el lugar de trabajo, licencias, certificaciones o leyes aplicables, y 3) la exposición a códigos formales de ética y marcos éticos.

Temas:

CS Core

- Valores de la comunidad y las leyes por las que vivimos.
- La naturaleza de ser un profesional, incluyendo cuidado, atención, disciplina, responsabilidad fiduciaria y tutoría (mentoring).
- Mantenerse actualizado como profesional de la informática en términos de familiaridad, herramientas, habilidades, marcos legales y profesionales, así como la capacidad y responsabilidad de autoevaluarse y progresar en el campo de la informática.
- Certificación profesional, códigos de ética, conducta y práctica, como los de la ACM, IEEE, AAAI y otras sociedades internacionales.
- Responsabilidad (accountability), obligación (responsibility) y responsabilidad legal (liability) (por ejemplo, corrección del software, confiabilidad y seguridad, garantía, negligencia, responsa-

bilidad estricta, enfoques éticos para la divulgación de vulnerabilidades de seguridad), incluyendo si un producto/servicio debe construirse, no solo hacerlo porque es técnicamente posible.
- Introducción a teorías que describen la creación y uso humano de la tecnología, incluyendo instrumentalismo, sociología de los sistemas tecnológicos, justicia de la discapacidad (disability justice), tesis de la neutralidad, pragmatismo y modelos decoloniales, incluyendo desarrollar y usar tecnología para corregir errores y hacer el bien.
- Estrategias para reconocer y reportar diseños, sistemas, software y conducta profesional (o sus resultados) que puedan violar la ley o códigos profesionales de ética.

KA Core
- El papel del profesional de la informática y las sociedades profesionales en la política pública.
- Mantener la conciencia de las consecuencias.
- Disidencia ética y denuncia de irregularidades (whistleblowing).
- La relación entre la cultura regional y los dilemas éticos.
- Tratar con acoso y discriminación.
- Formas de acreditación profesional.
- Ergonomía y entornos informáticos saludables.
- Tiempo de comercialización (time-to-market) y consideraciones de costo versus estándares profesionales de calidad.

Aprendizaje esperado (*Learning Outcomes*):
CSCore:
1. Identificar problemas éticos que surgen en el diseño de software, prácticas de desarrollo y despliegue de software [Analizar]
2. Discutir cómo abordar problemas éticos en situaciones específicas [Debatir]
3. Expresar la responsabilidad ética de garantizar la corrección, confiabilidad y seguridad del software, incluyendo de dónde surge esta responsabilidad (por ejemplo, Códigos de Ética de ACM/IEEE/AAAI, leyes y regulaciones, políticas organizacionales) [Analizar]
4. Describir los mecanismos que típicamente existen para que un profesional se mantenga actualizado en asuntos éticos [Describir]
5. Describir las fortalezas y debilidades de los códigos profesionales relevantes como expresiones de ser un profesional y guías para la toma de decisiones [Describir]
6. Analizar un problema global de informática, observando el papel de los profesionales y funcionarios gubernamentales en la gestión de este problema [Analizar]

KACore:
7. Describir formas en que los profesionales y las organizaciones profesionales pueden contribuir a la política pública [Describir]
8. Describir las consecuencias del comportamiento profesional inapropiado [Describir]
9. Estar familiarizado con la denuncia de irregularidades (*whistleblowing*) y tener acceso a conocimiento para guiarlo a través de un incidente [Evaluar (valorar)]
10. Identificar ejemplos de cómo la cultura regional interactúa con dilemas éticos [Analizar]
11. Describir formas de acoso y discriminación y vías de asistencia [Describir]
12. Evaluar varias formas de acreditación profesional [Evaluar (valorar)]
13. Establecer la relación entre la ergonomía en los entornos informáticos y la salud de las personas [Enunciar/Establecer]
14. Describir problemas asociados con el impulso de las industrias para centrarse en el tiempo de comercialización (time-to-market) versus la aplicación de estándares profesionales de calidad [Describir]

3.15.4. SEP/Propiedad Intelectual

La propiedad intelectual se refiere a un rango de derechos intangibles de propiedad sobre cualquier producto del intelecto humano, como un programa de software. Las leyes, que varían según la localidad, proporcionan diferentes métodos para proteger estos derechos de propiedad basados en su tipo. Idealmente, las leyes de propiedad intelectual equilibran los intereses de los creadores y usuarios de la propiedad. Hay cuatro tipos de derechos de propiedad intelectual relevantes para el software: patentes, derechos de autor (copyrights), secretos comerciales y marcas registradas. Además, los derechos de propiedad a menudo están protegidos por licencias de usuario. Cada uno proporciona un tipo diferente

de protección legal.
Temas:
CS Core

- Derechos de propiedad intelectual.
- Propiedad intelectual digital intangible (IDIP).
- Fundamentos legales para la protección de la propiedad intelectual.
- Licencias de software comunes (por ejemplo, MIT, GPL y sus variantes, Apache, Mozilla, Creative Commons).
- Plagio y autoría.

KA Core

- Fundamentos filosóficos de la propiedad intelectual.
- Formas de propiedad intelectual (por ejemplo, derechos de autor, patentes, secretos comerciales, marcas registradas) y los derechos que protegen.
- Limitaciones a las protecciones de derechos de autor, incluyendo uso justo (fair use) y la doctrina de la primera venta (first sale doctrine).
- Leyes y tratados de propiedad intelectual que impactan la aplicación de los derechos de autor.
- Piratería de software y métodos técnicos para hacer cumplir los derechos de propiedad intelectual, como la gestión de derechos digitales (digital rights management) y el software de código cerrado como secreto comercial.
- Fundamentos morales y legales del movimiento de código abierto.
- Sistemas que usan datos de otros (por ejemplo, modelos de lenguaje grande).

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Describir y criticar la legislación y los precedentes destinados a las infracciones de derechos de autor digitales [Describir]
2. Identificar ejemplos contemporáneos de propiedad intelectual digital intangible [Analizar]
3. Seleccionar una licencia de software apropiada para un proyecto dado [Analizar]
4. Defender los usos legales y éticos de materiales con derechos de autor [Defender]
5. Interpretar la intención y implementación de las licencias de software [Interpretar]
6. Discutir si un uso de material con derechos de autor es probable que sea uso justo (fair use) [Debatir]
7. Analizar los problemas éticos inherentes en varios mecanismos de detección de plagio [Analizar]
8. Identificar múltiples formas de plagio más allá de la copia textual de texto o software (por ejemplo, paráfrasis intencional, tergiversación de autoría y atribución inadecuada) [Analizar]

KACore:

9. Discutir las bases filosóficas de la propiedad intelectual en un contexto apropiado (por ejemplo, país) [Debatir]
10. Distinguir los problemas conflictivos involucrados en la obtención de patentes de software [Distinguir]
11. Contrastar las protecciones y obligaciones de los derechos de autor, patentes, secretos comerciales y marcas registradas [Contrastar]
12. Describir la justificación para la protección legal de la propiedad intelectual en el contexto apropiado (por ejemplo, país) [Describir]
13. Analizar el uso de obras con derechos de autor bajo los conceptos de uso justo (fair use) y la doctrina de la primera venta (first sale doctrine) [Analizar]
14. Identificar los objetivos del movimiento de código abierto y su impacto en campos más allá de la informática, como el movimiento del derecho a reparar (right-to-repair) [Analizar]
15. Resumir la naturaleza global de la piratería de software [Resumir]
16. Criticar el uso de medidas técnicas de gestión de derechos digitales (por ejemplo, encriptación, marcas de agua, restricciones de copia y bloqueos regionales) desde las perspectivas de múltiples partes interesadas [Criticar]
17. Discutir la naturaleza de las leyes anti-elusión (anti-circumvention) en el contexto de la protección de derechos de autor [Debatir]

3.15.5. SEP/Privacidad y Libertades Civiles

El intercambio de información electrónica destaca la necesidad de equilibrar las protecciones de privacidad con el acceso a la información. La facilidad de acceso digital a muchos tipos de datos, además de copiar y distribuir estos datos, hace que los derechos de privacidad y las libertades civiles sean más complejos, especialmente dadas las diferencias culturales y legales en estas áreas. Complicando aún más las cosas, la privacidad también tiene componentes interpersonales, organizacionales, profesionales/comerciales y de gobernanza. Además, la naturaleza interconectada de las comunidades en línea plantea desafíos para gestionar expectativas y protecciones para la libertad de expresión en diversas culturas y naciones. Las compañías tecnológicas que proporcionan plataformas para contenido generado por usuarios están bajo una presión creciente para realizar tareas de gobernanza, enfrentando potencialmente responsabilidad por sus decisiones.

Temas:

CS Core

- Implicaciones de privacidad de la recopilación generalizada de datos, incluyendo pero no limitado a bases de datos transaccionales, almacenes de datos, sistemas de vigilancia, computación en la nube e inteligencia artificial.
- Concepciones de anonimato, seudonimidad e identidad.
- Soluciones basadas en tecnología para la protección de la privacidad (por ejemplo, encriptación de extremo a extremo y privacidad diferencial).
- Libertades civiles, derechos de privacidad y diferencias culturales.

KA Core

- Concepciones filosóficas y legales de la naturaleza de la privacidad, incluyendo el derecho a la privacidad.
- Fundamentos legales de la protección de la privacidad en jurisdicciones relevantes (por ejemplo, GDPR en la UE).
- Legislación de privacidad en áreas de práctica (por ejemplo, HIPAA en EE.UU., Ley de IA en la UE).
- Principios básicos de la investigación con sujetos humanos y principios más allá de lo que la ley requiere (por ejemplo, Informe Belmont, Declaración Universal de Derechos Humanos de la ONU y cómo esto se relaciona con la tecnología).
- Libertad de expresión y sus limitaciones.
- Contenido generado por usuarios, moderación de contenido y responsabilidad.

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Evaluar soluciones a las amenazas a la privacidad en bases de datos transaccionales y almacenes de datos [Evaluar]
2. Describir el papel de la recopilación de datos en la implementación de sistemas de vigilancia generalizada (por ejemplo, RFID, reconocimiento facial, cobro de peajes, computación móvil) [Describir]
3. Distinguir los conceptos y objetivos del anonimato y la seudonimidad [Distinguir]
4. Describir las ramificaciones de las protecciones de privacidad basadas en tecnología, incluyendo la privacidad diferencial y la encriptación de extremo a extremo [Describir]
5. Identificar diferencias culturales con respecto a la naturaleza y necesidad de la privacidad y otras libertades civiles [Analizar]

KACore:

6. Discutir la base filosófica para la protección legal de la privacidad personal en un contexto apropiado (por ejemplo, país) [Debatir]
7. Criticar la intención, valor potencial e implementación de varias formas de legislación de privacidad y principios más allá de lo que la ley requiere [Críticar (análisis crítico)]
8. Identificar estrategias para permitir una libertad de expresión apropiada [Analizar]

3.15.6. SEP/Comunicación

La informática es una disciplina inherentemente colaborativa y social, lo que hace de la comunicación un aspecto esencial de la profesión. Gran parte, pero no toda, de esta comunicación ocurre en un entorno profesional donde los estilos, expectativas y normas de comunicación difieren de otros

contextos donde se podría usar tecnología similar. Tanto la comunicación profesional como la informal transmiten información a varias audiencias que pueden tener diferentes objetivos y necesidades para esa información. Una buena comunicación también es necesaria para la transparencia y la confiabilidad. También es importante señalar que los profesionales de la informática no solo son comunicadores sino también oyentes que deben poder escuchar y utilizar reflexivamente los comentarios recibidos de diversas partes interesadas. Las habilidades de comunicación efectiva no son algo que uno 'simplemente sabe': se desarrollan y pueden aprenderse. Las habilidades de comunicación se enseñan mejor en contexto a lo largo del plan de estudios de pregrado.

Temas:

CS Core

- Comunicación oral, escrita y electrónica en equipo y grupo.
- Materiales de comunicación técnica (por ejemplo, código fuente y documentación, tutoriales, materiales de referencia, documentación de API).
- Comunicarse con diferentes partes interesadas, como clientes, liderazgo o el público.
- Colaboración en equipo (incluyendo herramientas) y resolución de conflictos.
- Requisitos de accesibilidad e inclusividad para dirigirse a audiencias profesionales.
- Competencia cultural en la comunicación, incluyendo considerar el impacto de la diferencia en el lenguaje natural.

KA Core

- Compensaciones (tradeoffs) en factores competitivos que afectan los canales y elecciones de comunicación.
- Comunicarse para resolver problemas o hacer recomendaciones en el lugar de trabajo, como plantear preocupaciones éticas o abordar problemas de accesibilidad.

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Comprender la importancia de escribir documentos técnicos concisos y precisos siguiendo estándares bien definidos para el formato y para incluir tablas, figuras y referencias apropiadas [Producir]
2. Analizar documentación técnica escrita para verificar precisión técnica, concisión, falta de ambigüedad y conciencia de la audiencia [Analizar]
3. Componer y entregar una presentación formal consciente de la audiencia, accesible y organizada [Componer]
4. Planificar interacciones (por ejemplo, virtuales, cara a cara, documentos compartidos) con otros de maneras que inviten a la participación inclusiva, modelen la consideración respetuosa de las contribuciones de otros y valoren explícitamente la diversidad de ideas [Planear]
5. Identificar y describir cualidades de la comunicación efectiva (por ejemplo, virtual, cara a cara, intragrupo, documentos compartidos) [Analizar]
6. Comprender cómo comunicarse efectiva y apropiadamente como miembro de un equipo, incluyendo técnicas de resolución de conflictos [Aplicar]
7. Discutir formas de influir en el desempeño y los resultados en equipos diversos y multiculturales [Debatir]

KACore:

8. Evaluar fortalezas y debilidades personales para trabajar de forma remota como parte de un equipo que proviene de diversos antecedentes y experiencias [Evaluar (valorar)]
9. Elegir una forma apropiada de comunicar preocupaciones éticas delicadas [Evaluar (valorar)]

3.15.7. SEP/Sostenibilidad

La sostenibilidad es definida por las Naciones Unidas como "el desarrollo que satisface las necesidades del presente sin comprometer la capacidad de las futuras generaciones para satisfacer sus propias necesidades". Alternativamente, es el "equilibrio entre el medio ambiente, la equidad y la economía". A medida que la informática se extiende a más y más aspectos de la existencia humana, ya estamos viendo estimaciones de que porcentajes de dos dígitos del uso global de electricidad son consumidos por actividades informáticas, que sin control probablemente crecerán. Además, la electrónica contribuye individualmente a la demanda de tierras raras, extracción mineral e innumerables preocupaciones sobre residuos electrónicos (e-waste). Los estudiantes deben obtener una base que reconozca estos costos

globales y ambientales y sus efectos potenciales a largo plazo en el medio ambiente y las comunidades locales.

Temas:

CS Core

- Impactos ambientales, sociales y culturales de las decisiones de implementación (por ejemplo, objetivos de sostenibilidad, sesgo/resultados algorítmicos, viabilidad económica y consumo de recursos).
- Impactos sociales y ambientales locales/regionales/globales de los sistemas informáticos y su uso (por ejemplo, huella de carbono, uso de recursos, residuos electrónicos) debido al hardware (por ejemplo, residuos electrónicos, centros de datos, utilización de elementos y recursos raros, reciclaje) y software (por ejemplo, servicios basados en la nube, blockchain, entrenamiento y uso de modelos de IA). Esto incluye el uso diario de hardware (hardware barato reemplazado con frecuencia) y software (navegación web, correo electrónico y otros servicios con demandas computacionales ocultas/remotas).
- Pautas para estándares de diseño sostenible.

KA Core

- Efectos sistémicos de tecnologías y fenómenos informáticos complejos (por ejemplo, IA generativa, centros de datos, redes sociales, externalización (offshoring), trabajo remoto).
- Computación ubicua (pervasive computing): procesamiento de información que se ha integrado en objetos y actividades cotidianas, como sistemas inteligentes de energía, redes sociales y sistemas de retroalimentación para promover comportamientos sostenibles, transporte, monitoreo ambiental, ciencia ciudadana y activismo.
- Cómo la sostenibilidad de los sistemas de software es interdependiente con los sistemas sociales, incluyendo el conocimiento y las habilidades de sus usuarios, los procesos y políticas organizacionales y su contexto social (por ejemplo, fuerzas del mercado, políticas gubernamentales).

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Identificar formas de ser un profesional sostenible en un área específica o con un proyecto específico [Analizar]
2. Evaluar los impactos ambientales del despliegue de un proyecto dado (por ejemplo, consumo de energía, contribución a los residuos electrónicos, impacto de la fabricación) [Evaluar (valorar)]
3. Describir los impactos sociales y ambientales globales del uso y disposición de computadoras [Describir]
4. Enumerar los efectos sostenibles de las prácticas y actividades modernas (por ejemplo, trabajo remoto, comercio electrónico, criptomonedas, modelos de IA, centros de datos) [Listar/Enumerar]

KACore:

5. Describir los impactos ambientales de las decisiones de diseño dentro del campo de la informática que se relacionan con el diseño de algoritmos, diseño de sistemas operativos, diseño de redes, diseño de bases de datos, etc [Describir]
6. Analizar los impactos sociales y ambientales de nuevos diseños de sistemas [Analizar]
7. Diseñar pautas para el diseño o despliegue de TI sostenible en áreas como sistemas inteligentes de energía, redes sociales, transporte, agricultura, sistemas de cadena de suministro, monitoreo ambiental y activismo ciudadano [Diseñar]
8. Evaluar aplicaciones informáticas en relación con problemas ambientales (por ejemplo, energía, contaminación, uso de recursos, reciclaje y reutilización, gestión y producción de alimentos) [Evaluar (valorar)]

3.15.8. SEP/Historia de la Informática

La historia es importante porque proporciona un mecanismo para comprender por qué nuestros sistemas informáticos operan como lo hacen, los contextos sociales en los que surgieron los enfoques actuales y cómo estos continúan resonando en la disciplina hoy. No solo la informática afecta a la sociedad, sino también viceversa, resultando en un contexto socio-técnico complejo que está en constante cambio, requiriendo la perspectiva de la historia para poner el presente, así como los futuros posibles, en la perspectiva apropiada. También informa decisiones basadas en éxitos y fracasos del

pasado, incluyendo el daño causado y cómo no repetirlos. La historia de la informática a menudo se enseña en contexto con conceptos fundamentales, como fundamentos de sistemas y fundamentos de desarrollo de software. Se debe poner un enfoque en aquellos que, debido a la marginación, no han figurado históricamente tan prominentemente como deberían.

Temas:

CS Core

- La historia de la informática: hardware, software y humano/organizacional.
- El papel de la historia en el presente, incluyendo dentro de diferentes contextos sociales, y la relevancia de esta historia en el futuro.

KA Core

- Edad I (Pre-digital): Computación analógica antigua (Stonehenge, mecanismo de Anticitera, reloj de la Catedral de Salisbury, etc.), tablas numéricas calculadas por humanos, Euclides, Lovelace, Babbage, Gödel, Church, Turing, hardware pre-electrónico (electromecánico y mecánico).
- Edad II (Computación moderna temprana): ENIAC, UNIVAC, Bombas (Bletchley Park y descifradores de códigos), compañías informáticas (por ejemplo, IBM), mainframes, etc.
- Edad III (Era de las PC): PCs, hardware y software informático moderno, Ley de Moore.
- Edad IV (Internet): Redes, arquitectura de internet, navegadores y su evolución, estándares, compañías y servicios nacidos en internet (por ejemplo, Google, Amazon, Microsoft, etc.), computación distribuida.
- Edad V (Móvil y Nube): Computación móvil y teléfonos inteligentes, computación en la nube y sus modelos (por ejemplo, SaaS), servidores remotos, seguridad y privacidad, redes sociales.
- Edad VI (IA): Sistemas de toma de decisiones, sistemas de recomendación, IA generativa y otras herramientas y tecnologías impulsadas por aprendizaje automático.

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Comprender la relevancia e impacto de la historia de la informática en eventos recientes, contexto presente y posibles resultados futuros, desde más de una perspectiva cultural [Evaluar (valorar)]
2. Discutir cómo las perspectivas sostenidas hoy han sido moldeadas por la historia, y que existen perspectivas alternativas (por ejemplo, temores de que la IA reemplace a los trabajadores humanos vs IA que aumenta el trabajo humano, varias visiones sobre la Ley de Moore) [Debatir]

KACore:

3. Identificar tendencias formativas y consecuentes en la historia del campo de la informática [Analizar]
4. Identificar las contribuciones de individuos u organizaciones pioneras (laboratorios de investigación, compañías informáticas, oficinas gubernamentales) en el campo de la informática [Analizar]
5. Discutir el contexto histórico para momentos importantes en la historia de la informática, como el cambio de tubos de vacío a transistores (TRADIC), sistemas operativos seminales tempranos (por ejemplo, OS 360), Xerox PARC y la primera computadora Apple con una GUI, la creación de paradigmas de lenguaje de programación específicos, el primer virus informático, la creación de internet, la creación de la WWW, la burbuja puntocom, Y2K, la introducción de teléfonos inteligentes, etc [Debatir]
6. Comparar la vida diaria antes y después del advenimiento de desarrollos históricos (por ejemplo, computadoras personales o internet) [Comparar]

3.15.9. SEP/Economías de la Informática

Las economías de la informática son importantes para aquellos que desarrollan y proporcionan recursos y servicios informáticos a otros, así como para la sociedad en general. Impactan a los usuarios de estos recursos y servicios, tanto profesionales como no profesionales. Los profesionales de la informática tienen el deber de conocer el impacto de estos temas en sus propios roles y actividades y cómo las elecciones hechas impactarán a los usuarios y a la sociedad.

Temas:

KA Core

- Modelos económicos: regulados y no regulados, monopolios, efectos de red (network effects) y mercado abierto; economías del conocimiento y de la atención (knowledge and attention econo-

mies).
- Estrategias de precios y despliegue: obsolescencia programada (planned obsolescence), suscripciones, freemium, licencias de software, código abierto (open-source), software libre (free software), adware.
- Impactos de las diferencias en el acceso a recursos informáticos y el efecto de la oferta y demanda de mano de obra calificada en la calidad de los productos informáticos.
- Automatización, IA y sus efectos en los mercados laborales, desarrolladores y usuarios.
- Preocupaciones éticas en torno a la economía de la atención (attention economy) y otras economías de la informática (por ejemplo, consentimiento informado, recopilación de datos, uso de jerga legal extensa en acuerdos de usuario).

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Resumir los efectos sociales de los modelos económicos (por ejemplo, las economías del conocimiento y de la atención) [Resumir]
2. Describir las diferencias y similitudes de estrategias competitivas (por ejemplo, suscripción vs freemium vs gratis) [Describir]
3. Discutir ejemplos de brechas digitales [Debatir]
4. Comprender los efectos de la automatización y la IA en la sociedad [Evaluar (valorar)]
5. Comprender las implicaciones éticas de las economías informáticas que dependen de la atención y los datos [Evaluar (valorar)]

3.15.10. SEP/Políticas de Seguridad, Leyes y Delitos Informáticos

Si bien las políticas de seguridad, las leyes y los delitos informáticos son temas importantes, es esencial que se vean con la base de otras unidades de conocimiento social y profesional, como Propiedad Intelectual, Privacidad y Libertades Civiles, Contexto Social y Ética Profesional. Las computadoras, internet y la inteligencia artificial, quizás más que cualquier otra tecnología, han transformado la sociedad en los últimos 75 años. Al mismo tiempo, estas tecnologías han contribuido a amenazas sin precedentes a la privacidad; nuevas categorías de delitos informáticos y comportamientos antisociales; grandes interrupciones a organizaciones; y la concentración a gran escala de riesgo en los sistemas de información.

Temas:
CS Core

- Delitos informáticos, recurso legal para criminales informáticos e impacto en víctimas y sociedad.
- Ingeniería social, fraude habilitado por la informática, robo de identidad y recuperación de estos.
- Terrorismo cibernético, hacking criminal y hacktivismo.
- Malware, virus, gusanos.
- Ataques a infraestructura crítica como redes eléctricas y oleoductos.
- Fundamentos no técnicos de la seguridad (por ejemplo, ingeniería humana, política, confidencialidad).

KA Core

- Beneficios y desafíos de las leyes de delitos informáticos existentes y propuestas.
- Políticas de seguridad y los desafíos del cambio y el cumplimiento.
- Responsabilidad por la seguridad a lo largo del ciclo de vida de la informática.
- Leyes internacionales y locales y cómo se cruzan.

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Enumerar ejemplos clásicos de delitos informáticos e incidentes de ingeniería social con impacto social [Listar/Enumerar]
2. Identificar problemas con las leyes que se aplican a los delitos informáticos [Analizar]
3. Describir la motivación y ramificaciones del terrorismo cibernético, robo de datos, hacktivismo, ransomware y otros ataques [Describir]
4. Examinar los problemas éticos y legales que rodean el mal uso del acceso y varias violaciones de seguridad [Examinar]

5. Discutir el papel del profesional en la seguridad y las compensaciones y desafíos involucrados [Debatir]

KACore:

6. Investigar medidas que pueden tomar tanto individuos como organizaciones, incluidos gobiernos, para prevenir o mitigar los efectos indeseables de los delitos informáticos [Investigar]
7. Diseñar una política de seguridad para toda la compañía, que incluya procedimientos para la gestión de contraseñas y monitoreo de empleados [Diseñar]
8. Comprender cómo la legislación de una región puede afectar las actividades en otra (por ejemplo, cómo el GDPR de la UE se aplica globalmente, cuando hay personas de la UE involucradas) [Evaluar (valorar)]

3.15.11. SEP/Diversidad, Equidad, Inclusión y Accesibilidad

A pesar de ser un campo creativo, a menudo altamente remunerado con innumerables oportunidades laborales (y otras), las inequidades raciales, de género y muchas otras en la representación son generalizadas en muchas regiones. Para demasiados estudiantes, su primer curso de ciencias de la computación es su último. Hay muchos factores, incluido el legado del racismo sistémico, el capacitismo (ableism), el sexismo, el clasismo y otras injusticias que contribuyen a la falta de identidades diversas dentro de las ciencias de la computación, y no hay una solución única o rápida.

Infundir explícitamente diversidad, equidad, inclusión y accesibilidad (DEIA) en todo el plan de estudios de ciencias de la computación demuestra su importancia para el departamento, la institución y nuestro campo, todos los cuales probablemente tengan una declaración y/o iniciativa(s) de DEIA. Este énfasis en DEIA es importante éticamente y es un indicador clave de nuestro tiempo. Muchos profesionales en informática ya reconocen la atención a DEIA como integral a la práctica disciplinaria. Independientemente del grado en que estos valores aparezcan en cualquier clase de ciencias de la computación, la investigación sugiere que la falta de atención a ellos resulta en diseños inferiores además de daño y su perpetuación. No solo los datos respaldan que los equipos diversos superan a los homogéneos, sino que los equipos diversos pueden prevenir mejor los fracasos tecnológicos flagrantes en titulares recientes como el mal uso del reconocimiento facial, lesiones y muertes por bolsas de aire y otros fracasos conocidos de la ciencia y la informática.

Temas:

CS Core

- Cómo la identidad impacta y es impactada por las tecnologías y entornos informáticos (académicos y profesionales).
- Los beneficios de los equipos de desarrollo diversos y los impactos de los equipos que no son diversos.
- Lenguaje inclusivo y terminología cargada, y por qué su uso importa.
- Comportamientos inclusivos y por qué importan.
- Diseñar y desarrollar tecnología con la accesibilidad en mente.
- Cómo los profesionales de la informática pueden influir e impactar la diversidad, equidad, inclusión y accesibilidad, incluyendo pero no solo a través del software que crean.

KA Core

- Expertos y sus prácticas que reflejan las identidades del aula y del mundo a través de principios prácticos de DEIA.
- Marginación histórica debido a mecanismos sociales sistémicos, supremacía tecnológica y desafíos de infraestructura global para la diversidad, equidad, inclusión y accesibilidad.
- Diferencias y necesidades interculturales en diversidad, equidad, inclusión y accesibilidad.

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Definir y distinguir equidad, igualdad, diversidad, inclusión y accesibilidad [Definir]
2. Identificar lenguaje, prácticas y comportamientos que pueden hacer que alguien se sienta incluido en un lugar de trabajo y/o equipo, y por qué es relevante. Evitar terminología cargada (véase Words Matter, www.acm.org/diversity-inclusion/words-matter); esto incluye identificar y acomodar a usuarios que a menudo son excluidos sin pensarlo y no son considerados en absoluto [Analizar]

3. Evaluar la accesibilidad de su aula o laboratorio. Evaluar la accesibilidad de su página web. (Véase www.w3.org/WAI y www.w3.org/TR/WCAG21) [Evaluar]
4. Demostrar colegialidad y respeto cuando se trabaja con miembros del equipo que no comparten su identidad. No es suficiente simplemente asignar proyectos en equipo. La facultad debe preparar a los estudiantes para el trabajo en equipo y monitorear, guiar y evaluar la efectividad de sus equipos estudiantiles a lo largo de un proyecto [Demostrar]
5. Comparar la demografía de los estudiantes de ciencias de la computación y STEM de su institución con la demografía institucional general. Si difieren, identificar factores que contribuyen al acceso, compromiso y logro inequitativo en ciencias de la computación entre grupos marginados. Si no, evaluar por qué no [Comparar]
6. Identificar los supuestos y valores de los desarrolladores incorporados en el diseño de hardware y software, especialmente en lo que respecta a la usabilidad por parte de poblaciones diversas [Analizar]
7. Identificar ejemplos de los beneficios que los equipos diversos pueden aportar a los productos de software y cómo la falta de diversidad tiene costos [Analizar]

KACore:

8. Analizar el trabajo de expertos que reflejan las identidades del aula y del mundo [Analizar]
9. Evaluar el impacto del poder y el privilegio en la profesión de la informática en lo que respecta a la cultura, industria, productos y sociedad [Evaluar (valorar)]
10. Desarrollar ejemplos de cambios sistémicos que podrían abordar positivamente la diversidad, equidad, inclusión y accesibilidad en un contexto familiar (es decir, en un curso introductorio de informática) y en un contexto no familiar, y cuándo estos podrían ser diferentes o iguales [Crear]
11. Comparar la demografía de su institución con la demografía general de la comunidad. Si difieren, identificar factores que contribuyen al acceso, compromiso y logro inequitativo entre grupos marginados. Si no, evaluar por qué no [Comparar]

3.16. Fundamentos de Sistemas (SF)

Un sistema informático es un conjunto de infraestructuras de hardware y software sobre las cuales se construyen aplicaciones. Los sistemas informáticos se han convertido en un pilar de la vida diaria de las personas. Como tal, es esencial que los estudiantes aprendan conocimientos sobre sistemas informáticos, adquieran las habilidades para usar y diseñar estos sistemas y comprendan la lógica y los principios fundamentales en los sistemas informáticos. Esto podría equipar a los estudiantes con la competencia necesaria para una carrera relacionada con las ciencias de la computación.

En el plan de estudios de Ciencias de la Computación, el estudio de los sistemas informáticos generalmente abarca múltiples áreas de conocimiento, incluyendo, entre otros, sistemas operativos, sistemas paralelos y distribuidos, redes de comunicaciones, arquitectura y organización de computadoras, e ingeniería de software. El área de conocimiento de Fundamentos de Sistemas, como sugiere su nombre, se centra en los conceptos fundamentales y principios de diseño en sistemas informáticos que son compartidos por estos cursos dentro de sus respectivos núcleos. El objetivo de esta área de conocimiento es presentar una visión integradora de estos conceptos fundamentales y principios de diseño de una manera unificada aunque simplificada, proporcionando una base común para los diferentes mecanismos y políticas especializados apropiados para el área de dominio específica. Los conceptos fundamentales en esta área de conocimiento incluyen una visión general de los sistemas informáticos, conceptos básicos como estado y transición de estado, asignación y planificación de recursos, y así sucesivamente. Además, esta área de conocimiento introduce principios básicos de diseño para mejorar la confiabilidad, disponibilidad, eficiencia y seguridad de los sistemas informáticos.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
3.16.1 Visión General de los Sistemas Informáticos (Pág. 140)			No
3.16.2 Conceptos Básicos (Pág. 140)			No
3.16.3 Gestión de Recursos (Pág. 141)			No
3.16.4 Rendimiento del Sistema (Pág. 142)			No
3.16.5 Evaluación de Rendimiento (Pág. 143)			No
3.16.6 Confiabilidad del Sistema (Pág. 143)			No
3.16.7 Seguridad del Sistema (Pág. 144)			No
3.16.8 Diseño del Sistema (Pág. 144)			No
3.16.9 Sociedad, Ética y la Profesión (Pág. 145)			No

3.16.1. SF/Visión General de los Sistemas Informáticos

Esta unidad de conocimiento introduce los bloques de construcción y componentes básicos de una computadora, el hardware como paradigma computacional, las abstracciones e interfaces de programación, la distinción entre servicios de aplicación y del sistema operativo, conceptos básicos de segmentación (pipelining) y el concepto de escalado (scaling).

Temas:

CS Core

- Bloques de construcción y componentes básicos de una computadora (compuertas, flip-flops, registros, interconexiones; ruta de datos (datapath) + control + memoria).
- Hardware como paradigma computacional: Bloques de construcción lógicos fundamentales; Expresiones lógicas, minimización, formas de suma de productos Lógica Digital y Sistemas Digitales.
- Abstracciones de programación, interfaces, uso de bibliotecas Programas.
- Distinción e interacción entre servicios de aplicación y del sistema operativo, llamada a procedimiento remoto (*remote procedure call*) Rol y Propósito de los Sistemas Operativos.
- Concepto básico de segmentación (pipelining), etapas de procesamiento superpuestas Organización Funcional.
- Concepto básico de escalado (scaling): rendimiento vs tamaño del problema.

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Describir los bloques de construcción básicos de las computadoras y su papel en el desarrollo histórico de la arquitectura de computadoras [Describir]
2. Diseñar un circuito lógico simple usando los bloques de construcción fundamentales del diseño lógico para resolver un problema simple (por ejemplo, un sumador) [Diseñar]
3. Describir cómo los sistemas informáticos se construyen de capa sobre capa, basados en la separación de preocupaciones, con interfaces bien definidas, ocultando detalles de las capas bajas a las capas más altas [Describir]
4. Describir que el hardware, el sistema operativo, la máquina virtual y la aplicación son capas adicionales de interpretación/procesamiento [Describir]
5. Describir los mecanismos de cómo los errores se detectan, se señalan hacia atrás y se manejan a través de las capas [Describir]
6. Construir un programa simple (por ejemplo, un cliente/servidor TCP) usando métodos de capas, detección y recuperación de errores, y reflejo del estado de error a través de las capas [Crear]
7. Identificar errores en un programa por capas usando herramientas para seguimiento de programas, ejecución paso a paso (single stepping) y depuración [Analizar]
8. Comprender el concepto de escalado fuerte (strong) vs débil (weak), es decir, cómo el rendimiento se ve afectado por la escala del problema vs la escala de los recursos para resolver el problema. Esto puede motivarse con ejemplos simples del mundo real [Evaluar (valorar)]

3.16.2. SF/Conceptos Básicos

Esta unidad de conocimiento cubre sistemas digitales vs analógicos, compuertas lógicas, estado y transiciones de estado, máquinas de estados finitos, lógica combinacional y secuencial, e introduce

procesamiento secuencial vs paralelo.
Temas:
CS Core

- Sistemas Digitales vs Analógicos/Discretos vs Continuos.
- Compuertas lógicas simples, expresiones lógicas, simplificación de lógica booleana.
- Relojes, Estado, Secuenciación.
- Estado y transición de estado (por ejemplo, estado inicial, estado final, ciclo de vida de estados) Modelos Computacionales y Lenguajes Formales.
- Máquinas de estados finitos (por ejemplo, NFA, DFA) Modelos Computacionales y Lenguajes Formales.
- Lógica Combinacional, Lógica Secuencial, Registros, Memorias Lógica Digital y Sistemas Digitales.
- Computadoras y Protocolos de Red como ejemplos de Máquinas de Estado Fundamentos de Redes y Comunicaciones.
- Procesamiento secuencial vs paralelo. Programas, Concurrencia.
- Procesamiento secuencial a nivel de aplicación: hilo único (single thread) Programas, Concurrencia.
- Procesamiento paralelo simple a nivel de aplicación: a nivel de solicitud (servicios web/cliente-servidor/distribuido), hilo único por servidor, múltiples hilos con múltiples servidores, segmentación (pipelining) Programas, Concurrencia.

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Describir las diferencias entre sistemas digitales y analógicos, y entre sistemas discretos y continuos. Puede dar ejemplos del mundo real de estos sistemas [Describir]
2. Describir las computaciones como un sistema caracterizado por un conjunto conocido de configuraciones con transiciones de una configuración única (estado) a otra (estado) [Describir]
3. Describir la distinción entre sistemas cuya salida es solo una función de su entrada (sin estado - stateless) y aquellos con memoria/historial (con estado - stateful) [Describir]
4. Desarrollar descripciones de máquinas de estado para soluciones de enunciados de problemas simples (por ejemplo, secuenciación de semáforos, reconocedores de patrones) [Crear]
5. Describir una computadora como una máquina de estado que interpreta instrucciones de máquina [Describir]
6. Explicar cómo un programa o protocolo de red también puede expresarse como una máquina de estado y que pueden existir representaciones alternativas para la misma computación [Explicar]
7. Derivar el comportamiento de serie temporal de una máquina de estado a partir de su representación de máquina de estado (por ejemplo, máquina de estado de gestión de conexión TCP) [Analizar]
8. Escribir un problema secuencial simple y una versión paralela simple del mismo programa [Escribir]
9. Evaluar el rendimiento de versiones secuenciales y paralelas simples de un programa con diferentes tamaños de problema y ser capaz de describir las aceleraciones (speed-ups) logradas [Evaluar]
10. Describir en una línea de tiempo de ejecución cómo los eventos y operaciones de paralelismo pueden tener lugar simultáneamente (es decir, al mismo tiempo). Explicar cómo el trabajo puede realizarse en menos tiempo transcurrido si esto puede explotarse [Describir]

3.16.3. SF/Gestión de Recursos

Esta unidad de conocimiento cubre diferentes tipos de recursos en sistemas informáticos y algoritmos comunes de asignación y planificación de recursos.
Temas:
CS Core

- Diferentes tipos de recursos (por ejemplo, participación del procesador, memoria, disco, ancho de banda de red) Planificación, Gestión de Memoria, API de Sistemas de Archivos e Implementación, Fundamentos de Redes y Comunicaciones.
- Algoritmos comunes de asignación/planificación de recursos (por ejemplo, primero en llegar

primero en ser servido, planificación basada en prioridad, planificación justa (fair scheduling) y planificación con desalojo (preemptive scheduling)) Planificación.

KA Core

- Ventajas y desventajas de los algoritmos de planificación comunes Planificación.

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Definir cómo los recursos finitos de computadora (por ejemplo, participación del procesador, memoria, almacenamiento y ancho de banda de red) se gestionan mediante su cuidadosa asignación a las entidades existentes [Definir]
2. Describir cómo funcionan los algoritmos comunes de asignación/planificación de recursos [Describir]
3. Desarrollar algoritmos de planificación comunes y evaluar sus rendimientos [Crear]

KACore:

4. Describir los pros y los contras de los algoritmos de planificación comunes [Describir]

3.16.4. SF/Rendimiento del Sistema

Esta unidad de conocimiento cubre las latencias en los sistemas informáticos, las memorias caché y la localidad, la coherencia de caché, la jerarquía de memoria, la virtualización y el aislamiento.

Temas:

CS Core

- Latencias en sistemas informáticos: 1. Velocidad de la luz y las computadoras (un pie por nanosegundo vs relojes de un GHz) Organización Funcional 2. Latencias de memoria vs disco vs memoria a través de la red Jerarquía de Memoria, Rendimiento y Eficiencia Energética
- Memorias caché y los efectos de la localidad espacial y temporal en el rendimiento en procesadores y sistemas Jerarquía de Memoria, Rendimiento y Eficiencia Energética, Gestión de Memoria.
- Memorias caché y coherencia de caché en bases de datos, sistemas operativos, sistemas distribuidos y arquitectura de computadoras Gestión de Memoria, Jerarquía de Memoria, Internos de DBMS.
- Introducción a la jerarquía de memoria del procesador Jerarquía de Memoria, Rendimiento y Eficiencia Energética.

KA Core

- La fórmula del tiempo promedio de acceso a memoria Jerarquía de Memoria.
- Justificación de la virtualización y el aislamiento: protección y rendimiento predecible Virtualización.
- Niveles de indirección, ilustrados por la memoria virtual para gestionar recursos de memoria física Virtualización.
- Métodos para implementar memoria virtual y máquinas virtuales Virtualización.

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Describir el desglose de la latencia de los sistemas informáticos en términos de memoria, disco y red [Describir]
2. Explicar la importancia de la localidad en la determinación del rendimiento del sistema [Explicar]
3. Calcular el tiempo promedio de acceso a memoria y describir las compensaciones (tradeoffs) en el rendimiento de la jerarquía de memoria en términos de capacidad, tasa de fallos/aciertos (miss/hit rate) y tiempo de acceso [Calcular]

KACore:

4. Explicar por qué es importante aislar y proteger la ejecución de programas individuales y entornos que comparten recursos subyacentes comunes [Explicar]
5. Describir cómo el concepto de indirección puede crear la ilusión de una máquina dedicada y sus recursos incluso cuando se comparten físicamente entre múltiples programas y entornos [Describir]
6. Evaluar el rendimiento de dos instancias de aplicación ejecutándose en máquinas virtuales separadas y determinar el efecto del aislamiento de rendimiento [Evaluar]

3.16.5. SF/Evaluación de Rendimiento

Esta unidad de conocimiento cubre figuras de mérito de rendimiento, cargas de trabajo y puntos de referencia (benchmarks), ecuación CPI, Ley de Amdahl, análisis de orden de magnitud y herramientas analíticas para evaluación cuantitativa.

Temas:

CS Core

- Figuras de mérito de rendimiento Rendimiento y Eficiencia Energética, Evaluación.
- Cargas de trabajo y puntos de referencia representativos (benchmarks), y métodos de recolección y análisis de figuras de mérito de rendimiento Rendimiento y Eficiencia Energética, Evaluación.
- Ecuación CPI (Ciclos por Instrucción) como herramienta para comprender las compensaciones (tradeoffs) en el diseño de conjuntos de instrucciones, tuberías de procesador (pipelines) y organizaciones del sistema de memoria Rendimiento y Eficiencia Energética, Evaluación.
- Ley de Amdahl: la parte de la computación que no se puede acelerar limita el efecto de las partes que sí se pueden Rendimiento y Eficiencia Energética, Evaluación.
- Análisis de orden de magnitud (notación Big O) Análisis de Complejidad.
- Análisis de caminos lentos y rápidos de un sistema Organización Funcional.
- Eventos y su efecto en el rendimiento (por ejemplo, detenciones de instrucciones (stalls), fallos de caché, fallos de página) Gestión de Memoria, Organización Funcional.

KA Core

- Herramientas analíticas para guiar la evaluación cuantitativa.
- Comprender sistemas por capas, cargas de trabajo y plataformas, sus implicaciones para el rendimiento y los desafíos que representan para la evaluación.
- Trampas de los micro puntos de referencia (microbenchmark pitfalls).

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Explicar cómo los componentes de la arquitectura del sistema contribuyen a mejorar su rendimiento [Explicar]
2. Explicar las circunstancias en las que una figura dada de una métrica de rendimiento del sistema es útil [Explicar]
3. Explicar el uso e insuficiencias de los puntos de referencia (benchmarks) como medida del rendimiento del sistema [Explicar]
4. Describir la Ley de Amdahl y discutir sus limitaciones [Describir]
5. Aplicar estudios de límite o cálculos simples para producir estimaciones de orden de magnitud para una métrica de rendimiento dada en un contexto dado [Aplicar]
6. Aplicar herramientas de software para perfilar y medir el rendimiento del programa [Aplicar]

KACore:

7. Diseñar y conducir un experimento orientado al rendimiento de un sistema común (por ejemplo, un sistema operativo y Spark) [Diseñar]
8. Diseñar un experimento de rendimiento en un sistema por capas para determinar el efecto de un parámetro del sistema en el rendimiento del sistema [Diseñar]

3.16.6. SF/Confiabilidad del Sistema

Esta unidad de conocimiento cubre la distinción entre errores (bugs), fallas (faults) y fallos (failures); confiabilidad vs disponibilidad; y confiabilidad mediante redundancia, incluyendo verificación y reintento, codificación redundante y duplicación.

Temas:

CS Core

- Distinción entre errores (bugs), fallas (faults) y fallos (failures) Coordinación, Confiabilidad del Software.
- Confiabilidad vs disponibilidad.
- Confiabilidad mediante redundancia: 1. verificación y reintento (*check and retry*) Tolerancia a fallos, Soporte de Fiabilidad 2. codificación redundante (códigos de corrección de errores, CRC, FEC, RAID) Jerarquía de Memoria, Soporte de Fiabilidad, Bases de Datos Distribui-

das/Computación en la Nube 3. duplicación/reflejo (*mirroring*)/réplicas Bases de Datos Distribuidas/Computación en la Nube

KA Core

- Otros enfoques para la confiabilidad (por ejemplo, registro por diario - journaling) Tolerancia a fallos, Soporte de Fiabilidad, Confiabilidad del Software.

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Explicar la distinción entre errores de programa, errores del sistema y fallas de hardware (por ejemplo, memoria corrupta) y excepciones (por ejemplo, intento de dividir por cero) [Explicar]
2. Articular la distinción entre detectar, manejar y recuperarse de fallas y los métodos para su implementación [Articular]
3. Describir el papel de los códigos de corrección de errores en proporcionar técnicas de verificación y corrección de errores en memorias, almacenamiento y redes [Describir]
4. Aplicar algoritmos simples para explotar información redundante con el propósito de corrección de datos [Aplicar]

KACore:

5. Comparar diferentes métodos de detección y corrección de errores por su sobrecarga de datos, complejidad de implementación y tiempo de ejecución relativo para codificar, detectar y corregir errores [Comparar]

3.16.7. SF/Seguridad del Sistema

Esta unidad de conocimiento cubre problemas comunes de seguridad del sistema y contramedidas, incluyendo criptografía y arquitectura de seguridad, así como sistemas de contramedida representativos, como sistemas de detección de intrusiones y sistemas antivirus.

Temas:

CS Core

- Problemas comunes de seguridad del sistema (por ejemplo, virus, ataques de denegación de servicio y escuchas - eavesdropping) Protección y Seguridad, Seguridad de Red, Fundamentos de Seguridad, Análisis, Diseño e Ingeniería de la Seguridad.
- Contramedidas Principios del Sistema Operativo, Protección y Seguridad, Seguridad de Red: 1. Criptografía Criptografía 2. Arquitectura de seguridad Análisis, Diseño e Ingeniería de la Seguridad

KA Core

- Sistemas de contramedida representativos: 1. Sistemas de detección de intrusiones, cortafuegos (firewalls) Seguridad de Red 2. Sistemas antivirus

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Describir algunos problemas comunes de seguridad del sistema y dar ejemplos [Describir]
2. Describir algunas contramedidas contra problemas de seguridad del sistema [Describir]

KACore:

3. Describir sistemas de contramedida representativos [Describir]

3.16.8. SF/Diseño del Sistema

Esta unidad de conocimiento cubre criterios comunes de diseño de sistemas (por ejemplo, vivacidad (liveness), seguridad (safety), robustez, escalabilidad y seguridad (security)) y los diseños de sistemas representativos.

Temas:

CS Core

- Criterios comunes de diseño de sistemas (por ejemplo, vivacidad (liveness), seguridad (safety), robustez, escalabilidad y seguridad (security)) Evaluación.

KA Core

- Diseños de sistemas representativos (por ejemplo, servidor web Apache, Spark y Linux).

Aprendizaje esperado (*Learning Outcomes*):

CSCore:

1. Describir criterios comunes de diseño de sistemas [Describir]
2. Dada la funcionalidad requerida de un sistema y sus criterios de diseño clave, proporcionar un diseño de alto nivel de este sistema [Generar]

KACore:

3. Describir el diseño de algunos sistemas representativos [Describir]

3.16.9. SF/Sociedad, Ética y la Profesión

Esta unidad de conocimiento cubre los derechos de propiedad intelectual de los sistemas informáticos, licencias de software comunes y delitos informáticos.

Temas:

KA Core

- Derechos de propiedad intelectual de los sistemas informáticos Propiedad Intelectual.
- Licencias de software comunes Propiedad Intelectual.
- Delitos informáticos Políticas de Seguridad, Leyes y Delitos Informáticos.

Aprendizaje esperado (*Learning Outcomes*):

KACore:

1. Describir los derechos de propiedad intelectual de los sistemas informáticos [Describir]
2. Enumerar licencias de software representativas y comparar sus diferencias [Listar/Enumerar]
3. Enumerar delitos informáticos representativos [Listar/Enumerar]

3.17. Desarrollo de Plataformas Especializadas (SPD)

El área de conocimiento de Desarrollo de Plataformas Especializadas (SPD) se refiere a los atributos que involucran la creación de software dirigido a plataformas de hardware no tradicionales. El desarrollo para cada plataforma especializada, por ejemplo, robots, sistemas móviles, sistemas basados en web y sistemas embebidos, típicamente involucra consideraciones únicas.

Las necesidades sociales e industriales han creado una alta demanda de desarrolladores en plataformas especializadas, como aplicaciones móviles, plataformas web, plataformas robóticas y sistemas embebidos. Algunas habilidades profesionales únicas relevantes para esta área de conocimiento incluyen las siguientes:

- Crear aplicaciones que proporcionen una experiencia de usuario consistente en varios dispositivos, tamaños de pantalla y sistemas operativos.
- Desarrollar interfaces de programación de aplicaciones (APIs) para soportar la funcionalidad de cada plataforma especializada.
- Gestionar desafíos relacionados con limitaciones de recursos como cómputo, memoria, almacenamiento, y redes y comunicaciones.
- Aplicar preocupaciones transversales como optimización, seguridad, mejores prácticas de desarrollo, etc.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
3.17.1 Aspectos Comunes, Preocupaciones Compartidas (Pág. 146)			No
3.17.2 Plataformas Web (Pág. 146)			No
3.17.3 Plataformas Móviles (Pág. 147)			No
3.17.4 Plataformas Robóticas (Pág. 147)			No
3.17.5 Plataformas Embebidas (Pág. 148)			No
3.17.6 Plataformas de Juegos (Pág. 149)			No
3.17.7 Plataformas de Computación Interactiva (Pág. 150)			No
3.17.8 SPD-SEP/Móvil (Pág. 151)			No
3.17.9 SPD-SEP Web (Pág. 152)			No
3.17.10 SPD-SEP/Juegos (Pág. 152)			No
3.17.11 SPD-SEP/Robótica (Pág. 153)			No
3.17.12 SPD-SEP/Interactiva (Pág. 154)			No

3.17.1. SPD/Aspectos Comunes, Preocupaciones Compartidas

Temas:
CS Core

- Visión general de las plataformas de desarrollo (es decir, web, móvil, juegos, robótica, embebida e interactiva). 1. Entrada/sensores/dispositivos de control/dispositivos hápticos 2. Limitaciones de recursos *a*) Computacional *b*) Almacenamiento de datos *c*) Memoria *d*) Comunicación 3. Requisitos - seguridad, disponibilidad de tiempo de actividad, tolerancia a fallos Confiabilidad del Software, Análisis, Diseño e Ingeniería de la Seguridad 4. Salida/actuadores/dispositivos hápticos
- Programación mediante Interfaces de Programación de Aplicaciones (API) específicas de la plataforma vs construcción de aplicaciones tradicional
- Visión general de los Lenguajes de plataforma (por ejemplo, Python, Swift, Lua, Kotlin)
- Programación bajo restricciones y requisitos de la plataforma (por ejemplo, herramientas de desarrollo disponibles, desarrollo, consideraciones de seguridad) Fundamentos de Seguridad
- Técnicas para aprender y dominar un lenguaje de programación específico de la plataforma

Aprendizaje esperado (*Learning Outcomes*):
CSCore:

1. Enumerar las restricciones de la programación móvil [Listar/Enumerar]
2. Enumerar las características de los lenguajes de scripting [Listar/Enumerar]
3. Describir el modelo de tres capas (three-tier) de la programación web [Describir]
4. Describir cómo se mantiene el estado en la programación web [Describir]

3.17.2. SPD/Plataformas Web

Temas:
KA Core

- Lenguajes de programación web (por ejemplo, HTML5, JavaScript, PHP, CSS)
- Plataformas, frameworks o meta-frameworks web 1. Servicios en la nube 2. API, Componentes Web (Web Components)
- Software como Servicio (SaaS)
- Estándares web como el modelo de objetos del documento (DOM), accesibilidad Accesibilidad y Diseño Inclusivo
- Consideraciones de Seguridad y Privacidad Políticas de Seguridad, Leyes y Delitos Informáticos

Non Core

- Análisis de requisitos para aplicaciones web
- Servicios de computación Sistemas NoSQL 1. Alojamiento en la Nube (Cloud Hosting) 2. Escalabilidad (por ejemplo, Autoescalado (*Autoscaling*), Clústeres) 3. Estimación de costos para servicios
- Gestión de datos Conceptos Fundamentales de Sistemas de Bases de Datos 1. Residencia de datos (data residency): dónde se ubican los datos y qué rutas se pueden tomar para acceder a ellos 2. Integridad de datos (data integrity): garantizar que los datos sean accesibles y que se eliminen cuando sea requerido
- Arquitectura 1. Monolitos vs Microservicios 2. Micro-frontends 3. Arquitecturas Dirigidas por Eventos (Event-Driven) vs RESTful: ventajas y desventajas 4. Sin servidor (Serverless), computación en la nube bajo demanda
- Soluciones de almacenamiento Bases de Datos Relacionales, Sistemas NoSQL 1. Bases de datos relacionales 2. Bases de datos NoSQL

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Diseñar e implementar una aplicación basada en web utilizando un diseño de arquitectura de microservicios [Diseñar]
2. Describir las restricciones, como alojamiento, servicios y escalabilidad, relacionadas con las plataformas web [Describir]
3. Comparar y contrastar la programación web con la programación de propósito general [Comparar]

4. Describir las diferencias entre Software-como-Servicio (SaaS) y productos de software tradicionales [Describir]
5. Discutir cómo los estándares web impactan el desarrollo de software [Debatir]
6. Evaluar una aplicación web existente contra los estándares web actuales [Evaluar]

3.17.3. SPD/Plataformas Móviles

Temas:
KA Core

- Desarrollo con 1. Lenguajes de programación móvil 2. Entornos de programación móvil
- Restricciones de la plataforma móvil 1. Diseño de interfaz de usuario Comprender al Usuario: Objetivos individuales e interacciones con otros 2. Seguridad
- Acceso 1. Acceso a datos a través de APIs Construcción de Consultas 2. Diseño de puntos finales de API (endpoints) para aplicaciones móviles: trampas y consideraciones de diseño 3. Interfaces de red y web Fundamentos de Redes y Comunicaciones, Modelado de Datos

Non Core

- Desarrollo 1. Desarrollo nativo (native) versus multiplataforma (cross-platform) 2. Patrones de diseño/arquitectura de software para aplicaciones móviles Diseño de Software
- Restricciones de la plataforma móvil 1. Diseño de interfaz de usuario responsivo (responsive) Accesibilidad y Diseño Inclusivo 2. Heterogeneidad y movilidad de los dispositivos 3. Diferencias en las experiencias de usuario (por ejemplo, entre aplicaciones móviles y basadas en web) 4. Compensación (tradeoff) entre energía y rendimiento
- Capacidades (*affordances*) de la computación móvil 1. Aplicaciones conscientes de la ubicación (*location-aware*) 2. Computación impulsada por sensores (por ejemplo, giroscopio, acelerómetro, datos de salud de un reloj) 3. Telefonía y mensajería instantánea 4. Realidad aumentada Inmersión
- Especificación y pruebas Prácticas de Desarrollo de Software, Verificación y Validación de Software
- Computación asíncrona Algoritmos 1. Diferencia con la programación síncrona tradicional 2. Manejo del éxito mediante devoluciones de llamada (*callbacks*) 3. Manejo de errores de forma asíncrona 4. Pruebas de código asíncrono y problemas típicos en las pruebas

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Comparar la programación móvil con la programación de propósito general [Comparar]
2. Desarrollar una aplicación móvil consciente de la ubicación con integración de API de datos [Crear]
3. Construir una aplicación móvil impulsada por sensores capaz de registrar datos en un servidor remoto [Crear]
4. Crear una aplicación de comunicación que incorpore telefonía y mensajería instantánea [Crear]
5. Evaluar los pros y los contras del desarrollo de aplicaciones móviles nativas y multiplataforma [Evaluar]

3.17.4. SPD/Plataformas Robóticas

Temas:
KA Core

- Tipos de plataformas y dispositivos robóticos Robótica
- Sensores, cómputo embebido y efectores (actuadores) Computación Tangible/Física
- Lenguajes y bibliotecas específicos de robótica Robótica
- Arquitectura de software robótico (por ejemplo, usando el Sistema Operativo de Robot (ROS - *Robot Operating System*))
- Restricciones y consideraciones de diseño de la plataforma robótica Robótica
- Interconexiones con sistemas físicos o simulados Computación Tangible/Física, Simulación
- Algoritmos Robóticos Robótica, Animación por Computadora 1. Cinemática directa (*forward kinematics*) 2. Cinemática inversa (*inverse kinematics*) 3. Dinámica 4. Navegación y planificación de rutas (*path planning*) 5. Agarre (*grasping*) y manipulación

- Consideraciones de seguridad e interacción Ética Profesional, Contexto Social

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Diseñar e implementar una aplicación en una plataforma robótica dada [Diseñar]
2. Integrar un kit de robot basado en Arduino y programarlo para navegar por un laberinto [Integrar]
3. Comparar lenguajes y técnicas específicos de robótica con aquellos utilizados para el desarrollo de software de propósito general [Comparar]
4. Explicar la lógica detrás del diseño de la plataforma robótica y sus interconexiones con sistemas físicos o simulados [Explicar]
5. Dada una aplicación de alto nivel, diseñar una arquitectura de software de robot usando ROS especificando todos los componentes e interconexiones (temas ROS - ROS topics) para lograr esa aplicación [Diseñar]
6. Discutir las restricciones que una plataforma robótica dada impone a los desarrolladores [Debatir]

3.17.5. SPD/Plataformas Embebidas

Temas:
KA Core

- Introducción a las características únicas de los sistemas embebidos 1. Sistemas de tiempo real (*real-time*) vs tiempo real suave (*soft real-time*) y no tiempo real 2. Restricciones de recursos, como perfiles de memoria y plazos (*deadlines*) Jerarquía de Memoria
- API para arquitecturas personalizadas 1. Tecnología GPU Arquitecturas Heterogéneas, Sombreado y Renderizado Avanzado 2. Arreglos de Compuertas Programables en Campo (FPGA - *Field Programmable Gate Arrays*) Lógica Digital y Sistemas Digitales 3. Sistemas multiplataforma (*cross-platform*)
- Sistemas Embebidos 1. Microcontroladores 2. Interrupciones y retroalimentación (*feedback*) 3. Manejadores de interrupciones en lenguajes de alto nivel Visión General de los Sistemas Informáticos 4. Interrupciones duras (*hard*) y suaves (*soft*) y salidas de trampa (*trap-exits*) Principios del Sistema Operativo 5. Interacción con hardware, actuadores y sensores 6. Eficiencia energética 7. Codificación débilmente temporizada (*loosely timed coding*) y sincronización 8. Adaptadores de software (software adapters)
- Programación embebida
- Sistemas de tiempo real duro (*hard*) vs sistemas de tiempo real suave (*soft*) Sistemas en Tiempo Real y Embebidos 1. Oportunidad (*timeliness*) 2. Sincronización/planificación del tiempo 3. Priorización 4. Latencia 5. Jitter de cómputo (*compute jitter*)
- Gestión de recursos en tiempo real
- Gestión de memoria 1. Mapeo de construcciones de programación (variable) a una ubicación de memoria Jerarquía de Memoria 2. Memoria compartida Gestión de Memoria 3. Gestión manual de memoria. 4. Recolección de basura (*garbage collection*) Traducción y Ejecución de Lenguajes
- Consideraciones de seguridad (*safety*) y análisis de seguridad Contexto Social, Ética Profesional
- Sensores y actuadores
- Análisis y verificación
- Diseño de aplicación

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Diseñar e implementar un pequeño sistema embebido para una plataforma dada (por ejemplo, un despertador inteligente o un dron) [Diseñar]
2. Describir las características únicas de los sistemas embebidos versus otros sistemas [Describir]
3. Interfaz con sensores/actuadores [Programar]
4. Depurar un problema con una plataforma embebida existente [Analizar]
5. Identificar diferentes tipos de arquitecturas embebidas [Analizar]
6. Evaluar qué arquitectura es mejor para un conjunto dado de requisitos [Evaluar]

7. Diseñar y desarrollar software para interactuar y controlar hardware [Diseñar]
8. Diseñar métodos para sistemas de tiempo real [Diseñar]
9. Evaluar la planificación en tiempo real y el análisis de planificabilidad (*schedulability*) [Evaluar]
10. Evaluar la especificación y verificación formal de restricciones y propiedades de tiempo [Evaluar]

3.17.6. SPD/Plataformas de Juegos

Temas:
KA Core

- Plataformas históricas y contemporáneas para juegos Lógica Digital y Sistemas Digitales 1. Evolución de las Plataformas de Juegos (por ejemplo, de Brown Box a Metaverso y más allá; Mejora en las Arquitecturas de Computación (CPU y GPU); Convergencia de Plataformas y Movilidad) 2. Plataformas de Juego Típicas (por ejemplo, Computadora Personal; Consola Doméstica; Consola Portátil; Máquina de Arcade; Televisión Interactiva; Teléfono Móvil; Tableta; Pantalla Montada en la Cabeza Integrada; Instalaciones Inmersivas y Simuladores; Dispositivos Habilitados para Internet de las Cosas; Sistemas CAVE; Navegadores Web; Sistemas de Transmisión Basados en la Nube) 3. Características y Restricciones de Diferentes Plataformas de Juegos (por ejemplo, Características (almacenamiento local, interconexión de redes, periféricos); Rendimiento en tiempo de ejecución (frecuencia GPU/CPU, número de núcleos); Conjuntos de chips (unidades de procesamiento de física, co-procesadores vectoriales); Ancho de banda de expansión (PCIe); Rendimiento de red (Ethernet); Tipos y capacidades de memoria (DDR/GDDR); Profundidad máxima de pila; Consumo de energía; Diseño térmico; Endian) 4. Sensores, Controladores y Actuadores Típicos (por ejemplo, diseños distintivos de sistemas de control - periféricos (ratón, teclado, joystick), controladores de juego, wearables, superficies interactivas; electrónica y hardware a medida; visión por computadora, seguimiento interno (*inside-out tracking*) y externo (*outside-in tracking*); electrónica y E/S habilitadas para IoT Interacción) 5. Ecosistemas de eSports (por ejemplo, evolución del gameplay a través de plataformas; juegos y eSports; eventos de juegos como torneos LAN/arcade y eventos internacionales como la Serie Olímpica de eSports; medios transmitidos (*streamed*) y espectadores; tecnologías multimedia y gestión de transmisión; juego profesional; datos y aprendizaje automático para entrenamiento y coaching)

- Sistemas de Simulación y Renderizado en Tiempo Real 1. Arquitecturas de CPU y GPU (por ejemplo, taxonomía de Flynn; paralelización; conjuntos de instrucciones; componentes estándar - arreglo de cómputo gráfico (*graphics compute array*), controlador de memoria gráfica (*graphics memory controller*), sistema básico de entrada/salida de matriz de gráficos de video (*video graphics array basic input/output system*); interfaz de bus; unidad de gestión de energía (*power management unit*); unidad de procesamiento de video (*video processing unit*); interfaz de pantalla (*display interface*)) Arquitecturas Heterogéneas 2. Tuberías (pipelines) para simulaciones físicas y renderizado gráfico: (por ejemplo, basado en mosaicos (*tile-based*), modo inmediato (*immediate-mode*)). Renderizado Aplicado y Técnicas 3. Contextos Comunes para Algoritmos, Estructuras de Datos y Funciones Matemáticas (por ejemplo, bucles de juego (*game loops*); partición espacial, eliminación de viewport (*culling*) y nivel de detalle (*level of detail*); detección y resolución de colisiones (*collision detection and resolution*); simulación física; comportamiento para agentes inteligentes; generación procedural de contenido (*procedural content generation*)) Matemáticas Discretas, Estructuras de Datos y Algoritmos Fundamentales 4. Representaciones de medios (por ejemplo, E/S y técnicas de cómputo para mundos virtuales: audio; música; sprites; modelos y texturas; texto; diálogo; multimedia (por ejemplo, olfato, táctil) Conceptos Fundamentales de Gráficos y Técnicas Interactivas)
- Herramientas y Técnicas de Desarrollo de Juegos 1. Lenguajes de Programación (por ejemplo, C++; C#; Lua; Python; JavaScript) 2. Lenguajes de Sombreado (*Shader Languages*) (por ejemplo, HLSL, GLSL; Shader Graph) 3. Bibliotecas y APIs Gráficas (por ejemplo, DirectX; SDL; OpenGL; Metal; Vulkan; WebGL). Renderizado Aplicado y Técnicas, Diseño de Sistemas 4. Herramientas y Entornos de Desarrollo Comunes (por ejemplo, IDEs; Depuradores (*Debuggers*); Perfiladores (*Profilers*); Sistemas de Control de Versiones incluyendo aquellos que manejan activos binarios; Kits de Desarrollo y Kits de Producción/Consumidor; Emuladores) Prácticas de Desarrollo de Software, Herramientas y Entornos

- Motores de Juego (*Game Engines*) 1. Motores de Juego Abiertos (por ejemplo, Unreal; Unity; Godot; CryEngine; Phyre; Source 2; Pygame y Ren'Py; Phaser; Twine; Spring RTS) 2. Técnicas (por ejemplo, Ideación, Prototipado, Diseño e Implementación Iterativos, Compilación de Ejecutables (*Compiling Executable Builds*), Operaciones de Desarrollo y Garantía de Calidad - Pruebas de Juego (*Play Testing*) y Pruebas Técnicas (*Technical Testing*), Perfilado (*Profiling*); Optimización, Portado (*Porting*); Internacionalización y Localización, Redes) Rendimiento y Eficiencia Energética, Requisitos del Producto
- Diseño de Juegos 1. Vocabulario (por ejemplo, definiciones de juego; modelo mecánica-dinámica-estética (*mechanics-dynamics-aesthetics model*); terminología de la industria; diseño de experiencia (*experience design*); modelos de experiencia y emoción) 2. Pensamiento de Diseño (*Design Thinking*) y Diseño de Experiencia Centrado en el Usuario (por ejemplo, métodos de diseño de juegos; iteración, incremento y el doble diamante (double-diamond); fases de pre- y post-producción; garantía de calidad, incluyendo pruebas alfa y beta; participación de partes interesadas y clientes; gestión de la comunidad) Diseño de Software 3. Géneros (por ejemplo, aventura; simulador de caminata (*walking simulator*); disparos en primera persona (first-person shooter); estrategia en tiempo real (real-time strategy); arena de batalla multijugador en línea (MOBA); juego de rol (rpg)) 4. Audiencias y Taxonomías de Jugadores (por ejemplo, personas que juegan videojuegos; diversidad y ampliación de la participación; placeres, tipos de jugadores y preferencias; Bartle, Yee) Comprender al Usuario: Objetivos individuales e interacciones con otros 5. Proliferación de tecnologías de juegos digitales a dominios más allá del entretenimiento (por ejemplo, Educación y Entrenamiento; Juegos Serios (*Serious Games*); Producción Virtual; eSports; Gamificación; Diseño de Experiencias Inmersivas; Práctica de la Industria Creativa; Práctica Artística; Retórica Procedural (Procedural Rhetoric)) Aplicaciones e Impacto Social

Aprendizaje esperado (*Learning Outcomes*):
KACore:

1. Recordar las características de las arquitecturas comunes de procesamiento gráfico de propósito general [Recordar/Evocar]
2. Identificar las etapas clave de la tubería de renderizado de modo inmediato (immediate-mode rendering pipeline) [Analizar]
3. Describir las restricciones clave que una plataforma de juego específica probablemente impondrá a los desarrolladores [Describir]
4. Explicar cómo los eSports se transmiten (streamed) a grandes audiencias a través de internet [Explicar]
5. Traducir funciones matemáticas complejas a código fuente de buen rendimiento [Traducir]
6. Usar una API gráfica estándar de la industria para renderizar un modelo 3D en una escena virtual [Usar]
7. Modificar un sombreador (shader) para cambiar un efecto visual de acuerdo con los requisitos establecidos [Crear]
8. Implementar un juego para una plataforma particular según la especificación [Implementar]
9. Optimizar una función para la detección de colisiones en un entorno simulado [Crear]
10. Evaluar el rendimiento en tiempo de ejecución y de memoria de un juego usando una herramienta y entorno de desarrollo estándar de la industria [Evaluar (valorar)]
11. Comparar las interfaces de diferentes plataformas de juegos, resaltando sus respectivas implicaciones para la interacción humano-computadora [Comparar]
12. Recomendar un conjunto apropiado de herramientas y técnicas de desarrollo para implementar un juego de un género particular para una plataforma dada [Recomendar]
13. Discutir los desafíos clave en hacer un juego digital que sea compatible multiplataforma (cross-platform) [Debatir]
14. Expresar cómo los desarrolladores de juegos pueden mejorar la accesibilidad de una interfaz de juego [Demostrar]
15. Crear formas novedosas de gameplay utilizando plataformas de juego de frontera [Crear]

3.17.7. SPD/Plataformas de Computación Interactiva

Temas:
Non Core

- Plataformas de Análisis de Datos 1. Cuadernos Jupyter; Google Colab; R; SPSS; Observable. 2. Plataformas de análisis de datos/SQL en la nube (por ejemplo, BigQuery) Construcción de Consultas *a*) Apache Spark *b*) Visualizaciones de Datos Visualización 3. Presentaciones interactivas respaldadas por datos 4. Herramientas de diseño que requieren bucles de retroalimentación de baja latencia (low-latency feedback loops) *a*) Herramientas de renderizado *b*) Herramientas de diseño gráfico
- Programación mediante prompts (Prompt programming) 1. IA generativa (por ejemplo, ChatGPT de OpenAI, Codex de OpenAI, Copilot de GitHub) y los Modelos de Lenguaje Grande (LLMs) son accedidos/interactuados
- Plataformas Cuánticas Arquitecturas Cuánticas 1. Programar operadores de lógica cuántica en máquinas cuánticas. 2. Usar API para servicios cuánticos disponibles 3. Análisis de señales/Análisis de Fourier/Procesamiento de señales (para composición musical, análisis de audio/RF) Procesamiento de Imágenes

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Analizar grandes conjuntos de datos de forma interactiva [Analizar]
2. Crear una pista de acompañamiento para una interpretación musical, como la codificación en vivo (*live coding*) [Crear]
3. Crear cuadernos computacionales convincentes que construyan una narrativa para un objetivo/historia periodística dada [Crear]
4. Implementar código interactivo que use un conjunto de datos y genere gráficos exploratorios [Implementar]
5. Crear un programa que realice una tarea usando sistemas de Modelos de Lenguaje Grande (LLM) [Crear]
6. Contrastar un programa desarrollado por una plataforma de IA y por un humano [Contrastar]
7. Implementar un sistema que interactúe con un humano sin usar una pantalla [Implementar]
8. Contextualizar los atributos de diferentes estilos de análisis de datos, como interactivo versus tubería (*pipeline*) de ingeniería [Contextualizar]
9. Escribir un programa usando una plataforma de computación de cuadernos (por ejemplo, búsqueda, ordenación o manipulación de grafos) [Escribir]
10. Demostrar un resultado de puerta cuántica (*quantum gate*) usando una plataforma cuántica [Demostrar]

3.17.8. SPD/SPD-SEP/Móvil

Temas:
Non Core

- Privacidad y protección de datos
- Accesibilidad en el diseño móvil
- Seguridad y ciberseguridad
- Impactos sociales de la tecnología móvil
- Uso ético de la IA y los algoritmos

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Comprender y defender las responsabilidades éticas para salvaguardar la privacidad del usuario y la protección de datos en aplicaciones móviles [Evaluar (valorar)]
2. Diseñar aplicaciones móviles con la accesibilidad en mente, asegurando un uso efectivo por personas con discapacidades [Diseñar]
3. Demostrar competencia en prácticas de codificación segura para mitigar riesgos asociados con diversas amenazas de seguridad en el desarrollo móvil [Demostrar]
4. Analizar los impactos sociales más amplios de la tecnología móvil, incluyendo su influencia en los patrones de comunicación, relaciones y salud mental [Analizar]
5. Comprender las consideraciones éticas del uso de IA en aplicaciones móviles, asegurando algoritmos imparciales y justos [Explicar]

3.17.9. SPD/SPD-SEP Web

Temas:
Non Core

- Preocupaciones de privacidad con las aplicaciones móviles
- Diseñar para la inclusividad y la accesibilidad
- Uso ético de la IA en aplicaciones móviles
- Desarrollo sostenible de aplicaciones y alojamiento de servidores
- Evitar spam o notificaciones intrusivas
- Abordar el ciberacoso y el acoso (*cyberbullying and harassment*)
- Promover comunidades en línea positivas
- Monetización y publicidad
- Uso ético de la gamificación

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Comprender cómo la computación móvil impacta las comunicaciones y el flujo de información dentro de la sociedad [Analizar]
2. Diseñar aplicaciones móviles que hayan hecho las tareas diarias más fáciles/rápidas [Diseñar]
3. Reconocer cómo la ubicuidad de la computación móvil ha afectado el equilibrio entre el trabajo y la vida personal [Reconocer]
4. Comprender cómo la computación móvil impacta el monitoreo de la salud y los servicios de atención médica [Analizar]
5. Definir cómo las aplicaciones móviles se usan para educar sobre y ayudar a lograr los objetivos de sostenibilidad de la ONU [Definir]

3.17.10. SPD/SPD-SEP/Juegos

Temas:
Non Core

- Derechos de Propiedad Intelectual en las Industrias Creativas 1. Propiedad de la Propiedad Intelectual: derechos de autor (*copyright*), marca registrada (*trademark*); derecho de diseño (*design right*), patente (*patent*), secreto comercial (*trade secret*), ley civil versus penal; acuerdos internacionales; generación procedural de contenido y las implicaciones de la inteligencia artificial generativa 2. Licenciamiento (Licensing): Excepciones de uso y uso justo (*fair usage*); acuerdos de licencia de código abierto (*open-source*); licenciamiento propietario y a medida (*bespoke*); cumplimiento (*enforcement*)
- Acceso Justo al Juego (*Fair Access to Play*) 1. Usabilidad de la Interfaz de Juego: requisitos del usuario, capacidades (*affordances*), diseño ergonómico, investigación de usuarios, medición de experiencia y métodos de evaluación heurística para juegos 2. Accesibilidad de la Interfaz de Juego: formas de discapacidad y deficiencia (impairment and disability); medios para facilitar el acceso al juego; diseño universal; requisitos legislados para plataformas de juegos; evaluación de cumplimiento; desafíos en mecánicas de juego y acceso
- Salud y Seguridad Relacionadas con los Juegos 1. Lesiones en el Juego: formas de mitigar lesiones comunes en la parte superior del cuerpo, como lesiones por esfuerzo repetitivo (repetitive strain injury); psicología del ejercicio y fisioterapia en eSports 2. Evaluación de Riesgos para Eventos y Fabricación: control de sustancias peligrosas para la salud (COSHH); seguridad contra incendios; seguridad eléctrica y electrónica; evaluación de riesgos para juegos y eventos de juegos; evaluación de riesgos para la fabricación 3. Salud Mental: motivación para jugar; gamificación y diseño lúdico (*gameful design*); psicología del juego - trastorno por juego en internet (*internet gaming disorder*)
- Cadena de Suministro y Sostenibilidad del Hardware de Plataforma 1. Ciclo de Vida de la Plataforma: composición de la plataforma - materiales, ensamblaje; excavación y procesamiento de minerales; uso de energía; reciclaje; obsolescencia programada (*planned obsolescence*). 2. Esclavitud Moderna (Modern Slavery): cadenas de suministro; trabajo forzado y derechos civiles; condiciones laborales; detección y remisión (*remission*); organismos de certificación y esfuerzos benéficos.

- Representación en los Medios y la Industria 1. Inclusión: identidad e identificación; inclusión de una amplia gama de personajes para audiencias diversas; representación en los medios y sus efectos; alfabetización mediática (*media literacy*); análisis de contenido; estereotipos; sexualización 2. Igualdad (Equality): historias y controversias, como gamergate, calidad de vida en la industria, discurso y conducta profesional en contextos empresariales, trayectorias hacia carreras en desarrollo de juegos, movilidad social, la experiencia de desarrolladores de diferentes orígenes e identidades, género y tecnología

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Discutir cómo los creadores pueden proteger su propiedad intelectual [Debatir]
2. Identificar trampas comunes en las interfaces de juego que excluyen a jugadores con visión deficiente o no funcional [Analizar]
3. Describir cómo la evaluación heurística puede usarse para identificar problemas de usabilidad en interfaces de juego [Describir]
4. Explicar por qué las lesiones en la parte superior del cuerpo son comunes en los eSports [Explicar]
5. Discutir cómo reformar personajes y diálogos en una escena para reducir la amenaza del estereotipo (*stereotype threat*) [Debatir]
6. Ilustrar cómo la representación de la raza en un juego puede influir en el riesgo de exclusión social en la comunidad en línea asociada al juego [Aplicar]
7. Modificar una política para un evento de LAN party para incluir mitigaciones que reduzcan el riesgo de incendio [Diseñar]
8. Diseñar una estrategia de gamificación para motivar el juego serio (*serious play*) para un juego de concientización [Diseñar]
9. Analizar el papel de las políticas de contratación de empresas y la defensa (*advocacy*) en la movilidad social [Analizar]
10. Evaluar la idoneidad de dos fabricantes para producir una nueva consola de juegos [Evaluar (valorar)]
11. Comparar opciones para el licenciamiento de código abierto de una herramienta de desarrollo de juegos [Comparar]
12. Recomendar cambios a una interfaz de juego específica para mejorar el acceso a jugadores que son sordos o cuya audición está de otra manera deficiente [Recomendar]
13. Discutir si los juegos son adictivos [Debatir]
14. Sugerir cómo la representación de las mujeres en los videojuegos influye en cómo los jugadores perciben a los miembros de esos grupos [Evaluar]
15. Crear un videojuego que defienda con éxito la ciencia del clima [Crear]

3.17.11. SPD/SPD-SEP/Robótica

Temas:
Non Core

- Imparcialidad (*fairness*), transparencia y rendición de cuentas (*accountability*) en algoritmos robóticos
- Mitigar sesgos (biases) en la toma de decisiones de robots
- Seguridad pública en espacios compartidos con robots
- Cumplimiento con las leyes de protección de datos
- Consentimiento del paciente y confianza en los robots médicos

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Identificar instancias de sesgo en algoritmos robóticos y proponer estrategias para mitigarlos [Analizar]
2. Evaluar y criticar sistemas robóticos por consideraciones éticas y de imparcialidad, sugiriendo mejoras donde sea necesario [Evaluar]
3. Analizar ejemplos del mundo real de sesgos en la toma de decisiones de robots y desarrollar estrategias para reducir el sesgo en sistemas robóticos [Analizar]
4. Evaluar los riesgos potenciales asociados con robots en espacios públicos y proponer medidas de seguridad para mitigar esos riesgos [Evaluar (valorar)]

5. Evaluar el impacto del consentimiento del paciente y la confianza en la efectividad de los despliegues de robots médicos en contextos de atención médica [Evaluar]

3.17.12. SPD/SPD-SEP/Interactiva

Temas:
Non Core

- Pautas éticas al usar modelos de IA para ayudar en el periodismo y la creación de contenido
- Rendición de cuentas (*accountability*) por las salidas generadas por IA
- Comportamiento entre programadores de prompts (*prompt programmers*) y desarrolladores de IA
- Confianza con el público al usar modelos de IA

Aprendizaje esperado (*Learning Outcomes*):
NonCore:

1. Indicar un marco para la rendición de cuentas (*accountability*) en el despliegue de modelos de IA, incluyendo documentación clara y atribución [Evaluar]
2. Discutir códigos de conducta éticos y estándares profesionales relevantes para la programación de prompts (*prompt programming*) y el desarrollo de IA [Debatir]
3. Crear planes y materiales de comunicación para educar al público sobre las capacidades, limitaciones y salvaguardas éticas de la IA [Crear]

Capítulo 4

Cuerpo del conocimiento de Matemática

Las 12 áreas de conocimiento en Matemática son:

4.1 Cálculo y Análisis (CAN) (Pág. 156)
- 4.1.1 Cálculo Diferencial e Integral (Pág. 156)
- 4.1.2 Cálculo Multivariable y Vectorial (Pág. 156)
- 4.1.3 Análisis Real e Integración de Lebesgue (Pág. 157)
- 4.1.4 Análisis Complejo y Mapeo Conforme (Pág. 157)
- 4.1.5 Análisis Funcional y Espacios de Banach/Hilbert (Pág. 157)
- 4.1.6 Análisis Armónico y Series de Fourier (Pág. 157)
- 4.1.7 Cálculo de Variaciones (Pág. 158)
- 4.1.8 Teoría de la Medida (Pág. 158)

4.2 Álgebra y Teoría de Números (ANT) (Pág. 158)
- 4.2.1 Álgebra Lineal Elemental y Avanzada (Pág. 158)
- 4.2.2 Teoría de Grupos y Simetría (Pág. 159)
- 4.2.3 Anillos, Cuerpos y Teoría de Galois (Pág. 159)
- 4.2.4 Teoría de Módulos y Álgebra Homológica (Pág. 159)
- 4.2.5 Teoría de Números Elemental y Analítica (Pág. 159)
- 4.2.6 Teoría de Números Algebraica (Pág. 160)
- 4.2.7 Teoría de Representación (Pág. 160)
- 4.2.8 Criptografía y Teoría de Codificación (Pág. 160)

4.3 Geometría y Topología (GTO) (Pág. 160)
- 4.3.1 Geometría Euclidiana y Proyectiva (Pág. 161)
- 4.3.2 Topología General (de Conjuntos de Puntos) (Pág. 161)
- 4.3.3 Geometría Diferencial de Curvas y Superficies (Pág. 161)
- 4.3.4 Geometría Riemanniana (Pág. 161)
- 4.3.5 Topología Algebraica (Pág. 161)
- 4.3.6 Geometría Simpléctica (Pág. 162)
- 4.3.7 Teoría Geométrica de Grupos (Pág. 162)
- 4.3.8 Topología Diferencial y Variedades (Pág. 162)

4.4 Matemáticas Discretas y Combinatoria (DMC) (Pág. 162)
- 4.4.1 Combinatoria Enumerativa (Pág. 163)
- 4.4.2 Teoría de Grafos y Análisis de Redes (Pág. 163)
- 4.4.3 Combinatoria Extremal y Probabilística (Pág. 163)
- 4.4.4 Optimización Discreta y Matroides (Pág. 164)
- 4.4.5 Teoría de Diseños (Pág. 164)
- 4.4.6 Lógica, Teoría de Conjuntos y Teoría de Modelos (Pág. 164)

4.5 Probabilidad y Estadística (PST) (Pág. 164)
- 4.5.1 Axiomas de Probabilidad y Variables Aleatorias (Pág. 165)
- 4.5.2 Teoremas de Límite (Pág. 165)
- 4.5.3 Procesos Estocásticos y Cadenas de Markov (Pág. 165)
- 4.5.4 Estadística Matemática y Teoría de la Estimación (Pág. 166)
- 4.5.5 Pruebas de Hipótesis y Modelos Lineales (Pág. 166)
- 4.5.6 Inferencia Bayesiana (Pág. 166)
- 4.5.7 Estadística de Alta Dimensión (Pág. 166)
- 4.5.8 Análisis de Series Temporales (Pág. 166)
- 4.5.9 Estadística No Paramétrica (Pág. 167)

4.6 Análisis Numérico y Científico (NSA) (Pág. 167)
- 4.6.1 Análisis de Errores y Aritmética de Punto Flotante (Pág. 167)
- 4.6.2 Álgebra Lineal Numérica (Pág. 167)
- 4.6.3 Teoría de Interpolación y Aproximación (Pág. 168)
- 4.6.4 Integración y Diferenciación Numérica (Pág. 168)
- 4.6.5 Métodos Numéricos para EDOs y EDPs (Pág. 168)
- 4.6.6 Métodos de Elementos Finitos y Volúmenes Finitos (Pág. 168)
- 4.6.7 Algoritmos de Optimización (Pág. 169)
- 4.6.8 Computación Paralela para Matemáticas (Pág. 169)

4.7 Modelado Matemático y Simulación (MMS) (Pág. 169)
- 4.7.1 Ecuaciones Diferenciales Ordinarias (EDO) (Pág. 169)
- 4.7.2 Ecuaciones Diferenciales Parciales (EDP) (Pág. 170)
- 4.7.3 Sistemas Dinámicos y Teoría de Bifurcaciones (Pág. 170)
- 4.7.4 Métodos de Perturbación y Análisis Asintótico (Pág. 170)
- 4.7.5 Dinámica de Fluidos y Mecánica de Medios Continuos (Pág. 171)
- 4.7.6 Teoría de Control y Control Óptimo (Pág. 171)
- 4.7.7 Ecuaciones Diferenciales Estocásticas (EDE) (Pág. 171)
- 4.7.8 Modelado Basado en Agentes (Pág. 171)

4.8 Fundamentos Matemáticos de la Ciencia de Datos (FDS) (Pág. 171)
- 4.8.1 Teoría del Aprendizaje Estadístico (Pág. 172)
- 4.8.2 Optimización para el Aprendizaje Automático (Pág. 172)
- 4.8.3 Análisis de Datos Topológicos (TDA) (Pág. 172)
- 4.8.4 Reducción de Dimensionalidad (Pág. 172)
- 4.8.5 Métodos de Núcleo (Kernel) y SVM (Pág. 173)
- 4.8.6 Teoría de la Información y Entropía (Pág. 173)
- 4.8.7 Teoría Matemática del Aprendizaje Profundo (Pág. 173)

4.9 Geometría y Topología Computacional (CGT) (Pág. 173)
- 4.9.1 Diagramas de Voronoi y Triangulaciones de Delaunay (Pág. 174)
- 4.9.2 Algoritmos de Envoltura Convexa (Pág. 174)
- 4.9.3 Topología Digital (Pág. 174)
- 4.9.4 Homología Aplicada y Números de Betti (Pág. 174)
- 4.9.5 Generación y Procesamiento de Mallas (Pág. 174)

4.10 Información y Computación Cuántica (QIC) (Pág. 175)
- 4.10.1 Fundamentos de Mecánica Cuántica para Matemáticos (Pág. 175)
- 4.10.2 Lógica y Circuitos Cuánticos (Pág. 175)
- 4.10.3 Algoritmos Cuánticos (Pág. 175)
- 4.10.4 Entrelazamiento y Desigualdades de Bell (Pág. 176)
- 4.10.5 Corrección de Errores Cuánticos (Pág. 176)
- 4.10.6 Redes de Tensores y Sistemas de Muchos Cuerpos (Pág. 176)

4.11 Biología Matemática y Ciencias de la Vida (MBL) (Pág. 176)
- 4.11.1 Dinámica de Poblaciones y Ecología (Pág. 177)
- 4.11.2 Epidemiología Matemática (Pág. 177)

- 4.11.3 Biología de Sistemas y Redes Metabólicas (Pág. 177)
- 4.11.4 Bioinformática y Alineamiento de Secuencias (Pág. 177)
- 4.11.5 Neurociencia Matemática (Pág. 178)

4.12 Matemáticas Financieras y Actuariales (FAM) (Pág. 178)

- 4.12.1 Teoría del Interés y Anualidades (Pág. 178)
- 4.12.2 Arbitraje y Completitud del Mercado (Pág. 178)
- 4.12.3 Teoría de Black-Scholes-Merton (Pág. 178)
- 4.12.4 Medidas de Riesgo (Pág. 179)
- 4.12.5 Modelos Actuariales y Contingencias de Vida (Pág. 179)
- 4.12.6 Finanzas Computacionales y Monte Carlo (Pág. 179)

4.1. Cálculo y Análisis (CAN)

Esta área cubre el estudio fundamental del cambio, los límites y las propiedades rigurosas de las funciones. Abarca desde la diferenciación e integración básica hasta la teoría de la medida avanzada y los espacios funcionales, proporcionando el conjunto de herramientas esencial para todo modelado matemático.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
4.1.1 Cálculo Diferencial e Integral (Pág. 156)			No
4.1.2 Cálculo Multivariable y Vectorial (Pág. 156)			No
4.1.3 Análisis Real e Integración de Lebesgue (Pág. 157)			No
4.1.4 Análisis Complejo y Mapeo Conforme (Pág. 157)			No
4.1.5 Análisis Funcional y Espacios de Banach/Hilbert (Pág. 157)			No
4.1.6 Análisis Armónico y Series de Fourier (Pág. 157)			No
4.1.7 Cálculo de Variaciones (Pág. 158)			No
4.1.8 Teoría de la Medida (Pág. 158)			No

4.1.1. CAN/Cálculo Diferencial e Integral

Estudio fundamental de límites, continuidad, derivadas e integrales para funciones de una sola variable.

Temas:

Core Tier1

- Límites y continuidad de funciones de una sola variable
- Derivadas y reglas de diferenciación
- El Teorema Fundamental del Cálculo
- Técnicas de integración incluyendo sustitución y partes

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Aplicar reglas de diferenciación para encontrar tasas de cambio y optimizar funciones [Usar]
2. Enunciar y explicar el Teorema Fundamental del Cálculo [Familiarizarse]

4.1.2. CAN/Cálculo Multivariable y Vectorial

Cálculo de funciones de varias variables, centrándose en derivadas parciales, integrales múltiples y teoremas de campos vectoriales.

Temas:

Core Tier1

- Derivadas parciales y el vector gradiente
- Integrales múltiples en varios sistemas de coordenadas

Core Tier2

- Campos vectoriales e integrales de línea y de superficie
- Teoremas integrales: Green, Stokes y Gauss

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular gradientes y derivadas direccionales para funciones de múltiples variables [Usar]

Core-Tier2:

2. Verificar el Teorema de la Divergencia y el Teorema de Stokes para campos vectoriales específicos [Evaluar]

4.1.3. CAN/Análisis Real e Integración de Lebesgue

Tratamiento riguroso del sistema de números reales, sucesiones, series y la integral de Lebesgue.

Temas:

Core Tier2

- Espacios métricos y propiedades topológicas de la recta real
- Medida de Lebesgue y conjuntos medibles
- Teoremas de convergencia: Convergencia Monótona y Dominada

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Probar la existencia de límites y continuidad usando definiciones épsilon-delta [Evaluar]
2. Contrastar la integral de Lebesgue con la integral de Riemann [Familiarizarse]

4.1.4. CAN/Análisis Complejo y Mapeo Conforme

Estudio de funciones de una variable compleja, analiticidad, integración de contorno y transformaciones geométricas.

Temas:

Core Tier2

- Funciones holomorfas y ecuaciones de Cauchy-Riemann
- Integración de contorno y el Teorema del Residuo

Alternativo

- Mapeo conforme y sus aplicaciones en física

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Evaluar integrales reales usando el Teorema del Residuo [Usar]
2. Determinar si una función compleja es analítica usando las ecuaciones de Cauchy-Riemann [Evaluar]

4.1.5. CAN/Análisis Funcional y Espacios de Banach/Hilbert

Estudio de espacios vectoriales dotados de estructuras relacionadas con límites, específicamente espacios de dimensiones infinitas.

Temas:

Alternativo

- Espacios vectoriales normados y espacios de Banach
- Espacios de Hilbert y bases ortonormales
- Operadores lineales acotados y compactos

Aprendizaje esperado (*Learning Outcomes*):

Elective:

1. Explicar el concepto de completitud en espacios de Banach y Hilbert [Familiarizarse]
2. Aplicar el Teorema de Representación de Riesz a problemas funcionales específicos [Usar]

4.1.6. CAN/Análisis Armónico y Series de Fourier

Estudio de la representación de funciones como ondas y las propiedades de las transformaciones de Fourier.

Temas:

Alternativo

- Series de Fourier y convergencia en espacios L2
- La transformada de Fourier y sus propiedades
- Introducción a la teoría de wavelets y análisis multirresolución

Aprendizaje esperado (*Learning Outcomes*):

Elective:

1. Descomponer funciones periódicas en sus componentes de Fourier [Usar]
2. Analizar el decaimiento de los coeficientes de Fourier en relación con la suavidad de la función [Evaluar]

4.1.7. CAN/Cálculo de Variaciones

Optimización de funcionales, tratando con la búsqueda de caminos y minimización de superficies.
Temas:
Alternativo
- Ecuaciones de Euler-Lagrange y extremales
- Restricciones y multiplicadores de Lagrange en espacios de funciones

Aprendizaje esperado (*Learning Outcomes*):
Elective:
1. Resolver problemas de braquistocrona y geodésicas usando principios variacionales [Evaluar]
2. Derivar las ecuaciones de movimiento para sistemas físicos usando funcionales [Usar]

4.1.8. CAN/Teoría de la Medida

Fundamentos de la integración y la probabilidad a través del estudio de sigma-álgebras y medidas.
Temas:
Core Tier2
- Sigma-álgebras y espacios medibles
- Medida abstracta y construcciones de medida exterior
- Teorema de Radon-Nikodym y continuidad absoluta

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier2:
1. Definir las propiedades requeridas para que una función de conjunto sea una medida [Familiarizarse]
2. Identificar conjuntos no medibles usando el axioma de elección [Evaluar]

4.2. Álgebra y Teoría de Números (ANT)

Propiedades estructurales de los sistemas matemáticos, incluyendo sistemas lineales, estructuras algebraicas abstractas y propiedades de los números enteros.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
4.2.1 Álgebra Lineal Elemental y Avanzada (Pág. 158)			No
4.2.2 Teoría de Grupos y Simetría (Pág. 159)			No
4.2.3 Anillos, Cuerpos y Teoría de Galois (Pág. 159)			No
4.2.4 Teoría de Módulos y Álgebra Homológica (Pág. 159)			No
4.2.5 Teoría de Números Elemental y Analítica (Pág. 159)			No
4.2.6 Teoría de Números Algebraica (Pág. 160)			No
4.2.7 Teoría de Representación (Pág. 160)			No
4.2.8 Criptografía y Teoría de Codificación (Pág. 160)			No

4.2.1. ANT/Álgebra Lineal Elemental y Avanzada

Espacios vectoriales, transformaciones lineales y teoría de matrices.
Temas:
Core Tier1
- Espacios vectoriales y dimensiones de subespacios
- Valores propios, vectores propios y diagonalización

Core Tier2
- Espacios con producto interno y ortogonalidad

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Resolver sistemas lineales usando factorizaciones de matrices [Usar]
2. Realizar la descomposición espectral en matrices simétricas [Evaluar]

4.2.2. ANT/Teoría de Grupos y Simetría

Estudio formal de la simetría a través de grupos, subgrupos e isomorfismos.

Temas:

Core Tier2

- Subgrupos, grupos cíclicos y teorema de Lagrange
- Subgrupos normales y grupos cociente
- Teoremas de Sylow y acciones de grupo

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Identificar simetrías usando la notación de la teoría de grupos [Familiarizarse]
2. Clasificar grupos de orden pequeño usando los teoremas de Sylow [Evaluar]

4.2.3. ANT/Anillos, Cuerpos y Teoría de Galois

Estructuras algebraicas avanzadas incluyendo polinomios, extensiones de cuerpos y resolubilidad.

Temas:

Core Tier2

- Anillos, ideales y anillos factoriales
- Extensiones de cuerpos y clausuras algebraicas

Alternativo

- Teoría de Galois y resolubilidad de polinomios

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Determinar la irreducibilidad de polinomios sobre diferentes cuerpos [Usar]

Elective:

2. Calcular el grupo de Galois de una extensión polinómica dada [Evaluar]

4.2.4. ANT/Teoría de Módulos y Álgebra Homológica

Generalización de espacios vectoriales sobre anillos y el estudio de sucesiones exactas.

Temas:

Alternativo

- Módulos libres y estructura de módulos finitamente generados
- Sucesiones exactas y persecución de diagramas (diagram chasing)
- Fundamentos de la teoría de categorías y funtores

Aprendizaje esperado (*Learning Outcomes*):

Elective:

1. Explicar la diferencia entre un espacio vectorial y un módulo [Familiarizarse]
2. Verificar la exactitud de una sucesión de módulos [Evaluar]

4.2.5. ANT/Teoría de Números Elemental y Analítica

Propiedades de los números enteros, divisibilidad y la distribución de los números primos.

Temas:

Core Tier1

- Congruencias y el Teorema del Resto Chino

Core Tier2

- Números primos y el Teorema de los Números Primos

Alternativo

- La función Zeta de Riemann y la Hipótesis de Riemann

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Resolver ecuaciones diofánticas lineales [Usar]

Core-Tier2:

2. Evaluar funciones aritméticas y sus productos de Dirichlet [Evaluar]

4.2.6. ANT/Teoría de Números Algebraica

Extensión de la teoría de números a cuerpos de números algebraicos y anillos de enteros.
Temas:
Alternativo

- Cuerpos de números algebraicos y sus anillos de enteros
- Grupos de clases de ideales y números de clases

Aprendizaje esperado (*Learning Outcomes*):
Elective:

1. Factorizar ideales en ideales primos en extensiones algebraicas [Usar]
2. Calcular el discriminante y la norma de un número algebraico [Evaluar]

4.2.7. ANT/Teoría de Representación

Estudio de estructuras algebraicas mediante la representación de sus elementos como transformaciones lineales de espacios vectoriales.
Temas:
Alternativo

- Representaciones de grupos finitos
- Teoría de caracteres y relaciones de ortogonalidad

Aprendizaje esperado (*Learning Outcomes*):
Elective:

1. Describir la descomposición de una representación en irreducibles [Familiarizarse]
2. Construir tablas de caracteres para grupos pequeños [Usar]

4.2.8. ANT/Criptografía y Teoría de Codificación

Aplicaciones del álgebra a la comunicación segura y la corrección de errores.
Temas:
Core Tier2

- Criptografía de clave pública: RSA y Diffie-Hellman
- Códigos correctores de errores: códigos de Hamming y Reed-Solomon

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier2:

1. Cifrar y descifrar mensajes usando el algoritmo RSA [Usar]
2. Evaluar la capacidad de detección de errores de un código lineal [Evaluar]

4.3. Geometría y Topología (GTO)

Exploración de las propiedades espaciales, desde las métricas del espacio euclidiano hasta las deformaciones continuas de la topología.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
4.3.1 Geometría Euclidiana y Proyectiva (Pág. 161)			No
4.3.2 Topología General (de Conjuntos de Puntos) (Pág. 161)			No
4.3.3 Geometría Diferencial de Curvas y Superficies (Pág. 161)			No
4.3.4 Geometría Riemanniana (Pág. 161)			No
4.3.5 Topología Algebraica (Pág. 161)			No
4.3.6 Geometría Simpléctica (Pág. 162)			No
4.3.7 Teoría Geométrica de Grupos (Pág. 162)			No
4.3.8 Topología Diferencial y Variedades (Pág. 162)			No

4.3.1. GTO/Geometría Euclidiana y Proyectiva

Sistemas axiomáticos y proyecciones en espacios geométricos.

Temas:

Core Tier1

- Sistemas axiomáticos y geometrías no euclidianas

Core Tier2

- Coordenadas proyectivas y dualidad

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Construir pruebas geométricas en varios marcos axiomáticos [Usar]

Core-Tier2:

2. Analizar propiedades de cónicas en el plano proyectivo [Evaluar]

4.3.2. GTO/Topología General (de Conjuntos de Puntos)

Conceptos fundamentales de continuidad, compacidad y conexidad.

Temas:

Core Tier2

- Espacios topológicos y conjuntos abiertos
- Compacidad y teorema de Tychonoff
- Conexidad y conexidad por caminos

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Determinar si un conjunto es compacto en una topología dada [Usar]
2. Diferenciar entre homeomorfismos y mapas continuos [Evaluar]

4.3.3. GTO/Geometría Diferencial de Curvas y Superficies

Propiedades locales y globales de curvas y superficies usando cálculo.

Temas:

Core Tier2

- Fórmulas de Frenet-Serret para curvas espaciales
- Curvatura gaussiana y media de superficies
- Theorema Egregium y geometría intrínseca

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Calcular curvaturas para una superficie parametrizada dada [Usar]
2. Explicar la importancia del teorema de Gauss-Bonnet [Familiarizarse]

4.3.4. GTO/Geometría Riemanniana

Estudio de variedades con métricas de Riemann y sus tensores de curvatura.

Temas:

Alternativo

- Métricas de Riemann y conexiones de Levi-Civita
- Geodésicas y mapas exponenciales
- Tensor de curvatura de Riemann y flujo de Ricci

Aprendizaje esperado (*Learning Outcomes*):

Elective:

1. Calcular los símbolos de Christoffel para una métrica dada [Evaluar]
2. Derivar las ecuaciones geodésicas para una variedad [Usar]

4.3.5. GTO/Topología Algebraica

Uso de estructuras algebraicas para estudiar espacios topológicos a través de grupos como el grupo fundamental.

Temas:
Alternativo
- Grupo fundamental y teoría de homotopía
- Teoría de homología simplicial y singular

Aprendizaje esperado (*Learning Outcomes*):
Elective:
1. Describir el grupo fundamental de un círculo y de un toro [Familiarizarse]
2. Aplicar la sucesión de Mayer-Vietoris para calcular grupos de homología [Usar]

4.3.6. GTO/Geometría Simpléctica

Geometría de formas simplécticas, motivada principalmente por la mecánica clásica.
Temas:
Alternativo
- Variedades simplécticas y teorema de Darboux
- Campos vectoriales hamiltonianos y dinámica

Aprendizaje esperado (*Learning Outcomes*):
Elective:
1. Explicar la relación entre la geometría simpléctica y el espacio de fases [Familiarizarse]
2. Verificar si una forma de grado 2 dada es una estructura simpléctica [Evaluar]

4.3.7. GTO/Teoría Geométrica de Grupos

Estudio de grupos finitamente generados mediante la exploración de la geometría de sus grafos de Cayley.
Temas:
Alternativo
- Grafos de Cayley y propiedades métricas de los grupos
- Grupos hiperbólicos en el sentido de Gromov

Aprendizaje esperado (*Learning Outcomes*):
Elective:
1. Dibujar el grafo de Cayley para una presentación de grupo simple [Usar]
2. Identificar si un grupo es hiperbólico usando criterios geométricos [Evaluar]

4.3.8. GTO/Topología Diferencial y Variedades

Propiedades de las variedades diferenciables y mapas suaves entre ellas.
Temas:
Core Tier2
- Variedades suaves y espacios tangentes
- Transversalidad y teoría de intersección

Alternativo
- Introducción a la teoría de Morse y puntos críticos

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier2:
1. Construir un atlas para una variedad dada como la n-esfera [Usar]
2. Evaluar el rango de un mapa suave usando el Jacobiano [Evaluar]

4.4. Matemáticas Discretas y Combinatoria (DMC)

Esta área trata con estructuras matemáticas finitas o contables. Es fundamental para la ciencia de la computación y la optimización, cubriendo el estudio de grafos, técnicas de conteo y los fundamentos lógicos de las matemáticas.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
4.4.1 Combinatoria Enumerativa (Pág. 163)			No
4.4.2 Teoría de Grafos y Análisis de Redes (Pág. 163)			No
4.4.3 Combinatoria Extremal y Probabilística (Pág. 163)			No
4.4.4 Optimización Discreta y Matroides (Pág. 164)			No
4.4.5 Teoría de Diseños (Pág. 164)			No
4.4.6 Lógica, Teoría de Conjuntos y Teoría de Modelos (Pág. 164)			No

4.4.1. DMC/Combinatoria Enumerativa

El arte de contar estructuras con propiedades específicas utilizando permutaciones, combinaciones y funciones generatrices.

Temas:

Core Tier1

- Principios básicos de conteo: suma, producto y el principio del palomar
- Permutaciones, combinaciones y coeficientes binomiales

Core Tier2

- Funciones generatrices ordinarias y exponenciales
- Relaciones de recurrencia lineales y sus soluciones

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Aplicar argumentos de conteo para resolver problemas que involucren conjuntos finitos [Usar]

Core-Tier2:

2. Resolver problemas de conteo complejos utilizando el principio de inclusión-exclusión [Evaluar]
3. Usar funciones generatrices para encontrar expresiones cerradas para sucesiones [Usar]

4.4.2. DMC/Teoría de Grafos y Análisis de Redes

Estudio de grafos, árboles y redes, centrándose en la conectividad, colorabilidad y trasversalidad.

Temas:

Core Tier1

- Grafos, caminos, ciclos y conectividad
- Árboles, árboles de expansión y sus propiedades

Core Tier2

- Grafos planares y problemas de coloración de grafos
- Flujos de red y el teorema del flujo máximo y corte mínimo

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Identificar caminos Eulerianos y Hamiltonianos en un grafo dado [Usar]

Core-Tier2:

2. Determinar el número cromático de varias familias de grafos [Evaluar]

4.4.3. DMC/Combinatoria Extremal y Probabilística

Estudio de qué tan grande o pequeña puede ser una colección de objetos bajo ciertas restricciones, a menudo utilizando métodos probabilísticos.

Temas:

Alternativo

- Teorema de Turán y teoría extremal de grafos
- Teoría de Ramsey para grafos y números enteros
- El método probabilístico y pruebas de existencia no constructivas

Aprendizaje esperado (*Learning Outcomes*):

Elective:

1. Explicar los límites proporcionados por el Teorema de Ramsey para casos pequeños [Familiarizarse]

2. Aplicar el método del primer momento para acotar probabilidades de propiedades de grafos [Evaluar]

4.4.4. DMC/Optimización Discreta y Matroides

Búsqueda de soluciones óptimas en estructuras finitas, incluyendo programación entera y teoría de matroides.

Temas:

Core Tier2

- Programación lineal entera y relajación

Alternativo

- Fundamentos de matroides: conjuntos independientes y bases
- Algoritmos ávidos (greedy) y optimización de matroides

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Formular problemas de programación de tareas del mundo real como programas enteros [Usar]

Elective:

2. Verificar si un sistema de conjuntos satisface los axiomas de intercambio de matroides [Evaluar]

4.4.5. DMC/Teoría de Diseños

Estudio de arreglos de conjuntos finitos que satisfacen condiciones de equilibrio y simetría.

Temas:

Alternativo

- Diseños de bloques incompletos balanceados (BIBDs)
- Cuadrados latinos y diseños ortogonales

Aprendizaje esperado (*Learning Outcomes*):

Elective:

1. Construir un Sistema Triple de Steiner para órdenes pequeños [Usar]
2. Analizar los parámetros de un diseño simétrico [Evaluar]

4.4.6. DMC/Lógica, Teoría de Conjuntos y Teoría de Modelos

Los fundamentos formales del razonamiento matemático y el estudio de estructuras matemáticas utilizando la lógica.

Temas:

Core Tier1

- Lógica proposicional y de predicados

Core Tier2

- Teoría de conjuntos axiomática (ZFC) y ordinales

Alternativo

- Teoremas de incompletitud de Gödel

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Formalizar enunciados matemáticos utilizando lógica de primer orden [Usar]

Core-Tier2:

2. Describir las implicaciones de la Hipótesis del Continuo [Familiarizarse]

4.5. Probabilidad y Estadística (PST)

Esta área cubre el modelado matemático de la incertidumbre y el análisis de datos. Abarca desde los fundamentos axiomáticos de la probabilidad hasta la inferencia estadística avanzada y el análisis de datos de alta dimensión.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
4.5.1 Axiomas de Probabilidad y Variables Aleatorias (Pág. 165)			No
4.5.2 Teoremas de Límite (Pág. 165)			No
4.5.3 Procesos Estocásticos y Cadenas de Markov (Pág. 165)			No
4.5.4 Estadística Matemática y Teoría de la Estimación (Pág. 166)			No
4.5.5 Pruebas de Hipótesis y Modelos Lineales (Pág. 166)			No
4.5.6 Inferencia Bayesiana (Pág. 166)			No
4.5.7 Estadística de Alta Dimensión (Pág. 166)			No
4.5.8 Análisis de Series Temporales (Pág. 166)			No
4.5.9 Estadística No Paramétrica (Pág. 167)			No

4.5.1. PST/Axiomas de Probabilidad y Variables Aleatorias

Fundamentos de la teoría de la probabilidad, incluyendo espacios muestrales, medidas de probabilidad y variables aleatorias.

Temas:

Core Tier1

- Espacios muestrales, eventos y axiomas de Kolmogorov
- Variables aleatorias discretas y continuas
- Esperanza, varianza y momentos

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular probabilidades condicionales usando el Teorema de Bayes [Usar]
2. Evaluar la función de densidad de probabilidad para variables aleatorias transformadas [Evaluar]

4.5.2. PST/Teoremas de Límite

Estudio del comportamiento asintótico de sumas de variables aleatorias.

Temas:

Core Tier2

- Ley de los Grandes Números: formas débil y fuerte
- Teorema del Límite Central y sus aplicaciones
- Funciones características y convergencia en distribución

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Explicar la importancia del Teorema del Límite Central en estadística [Familiarizarse]
2. Aplicar la Ley de los Grandes Números para estimar parámetros poblacionales [Usar]

4.5.3. PST/Procesos Estocásticos y Cadenas de Markov

Modelos para sistemas que evolucionan aleatoriamente a lo largo del tiempo.

Temas:

Core Tier2

- Cadenas de Markov en tiempo discreto y matrices de transición

Alternativo

- Procesos de Poisson y teoría de renovación
- Introducción al movimiento Browniano y martingalas

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Calcular distribuciones estacionarias para cadenas de Markov ergódicas [Evaluar]

Elective:

2. Modelar sistemas de colas utilizando procesos de Poisson [Usar]

4.5.4. PST/Estadística Matemática y Teoría de la Estimación

Métodos rigurosos para estimar parámetros poblacionales a partir de datos de muestra.

Temas:

Core Tier2

- Estimación puntual: Método de los Momentos y Máxima Verosimilitud
- Eficiencia, suficiencia y el teorema de Rao-Blackwell
- Construcción de intervalos de confianza

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Derivar estimadores de máxima verosimilitud para distribuciones comunes [Usar]
2. Verificar la insesgadez y consistencia de un estimador [Evaluar]

4.5.5. PST/Pruebas de Hipótesis y Modelos Lineales

Marcos para tomar decisiones basadas en datos y modelar relaciones entre variables.

Temas:

Core Tier2

- Lema de Neyman-Pearson y pruebas de razón de verosimilitud
- Regresión lineal simple y múltiple
- Modelos de Análisis de Varianza (ANOVA)

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Realizar pruebas t y pruebas F para evaluar hipótesis estadísticas [Usar]
2. Interpretar coeficientes de regresión y valores de R-cuadrado en modelos lineales [Evaluar]

4.5.6. PST/Inferencia Bayesiana

Métodos estadísticos que incorporan conocimiento previo y actualizan creencias utilizando datos.

Temas:

Alternativo

- Distribuciones a priori y a posteriori
- Distribuciones a priori conjugadas y actualización bayesiana
- Métodos de Monte Carlo por Cadenas de Markov (MCMC)

Aprendizaje esperado (*Learning Outcomes*):

Elective:

1. Calcular la distribución a posteriori para un modelo conjugado [Evaluar]
2. Implementar un algoritmo simple de Metropolis-Hastings [Usar]

4.5.7. PST/Estadística de Alta Dimensión

Métodos estadísticos para conjuntos de datos donde el número de variables excede el número de observaciones.

Temas:

Alternativo

- Técnicas de regularización: regresión Lasso y Ridge
- Dispersión (sparsity) y fundamentos de detección comprimida

Aprendizaje esperado (*Learning Outcomes*):

Elective:

1. Aplicar regresión Lasso para realizar selección de variables en altas dimensiones [Usar]
2. Comparar el rendimiento de Ridge y Lasso en presencia de multicolinealidad [Evaluar]

4.5.8. PST/Análisis de Series Temporales

Métodos para analizar puntos de datos recolectados o indexados en orden temporal.

Temas:

Alternativo

- Modelado ARMA y ARIMA
- Análisis espectral de series temporales

Aprendizaje esperado (*Learning Outcomes*):
Elective:

1. Pronosticar valores futuros utilizando un modelo ARIMA [Usar]
2. Verificar la estacionariedad en una serie temporal dada [Evaluar]

4.5.9. PST/Estadística No Paramétrica

Métodos estadísticos que no asumen una forma paramétrica específica para la distribución.
Temas:
Alternativo

- Pruebas basadas en rangos: Wilcoxon y Kruskal-Wallis
- Estimación de densidad por kernel

Aprendizaje esperado (*Learning Outcomes*):
Elective:

1. Realizar una prueba de suma de rangos de Wilcoxon en dos muestras independientes [Usar]
2. Estimar la densidad de una distribución utilizando métodos de kernel [Evaluar]

4.6. Análisis Numérico y Científico (NSA)

Esta área se centra en algoritmos para obtener soluciones numéricas a problemas matemáticos, esenciales para la simulación y la ingeniería. Cubre análisis de errores, álgebra lineal y ecuaciones diferenciales.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
4.6.1 Análisis de Errores y Aritmética de Punto Flotante (Pág. 167)			No
4.6.2 Álgebra Lineal Numérica (Pág. 167)			No
4.6.3 Teoría de Interpolación y Aproximación (Pág. 168)			No
4.6.4 Integración y Diferenciación Numérica (Pág. 168)			No
4.6.5 Métodos Numéricos para EDOs y EDPs (Pág. 168)			No
4.6.6 Métodos de Elementos Finitos y Volúmenes Finitos (Pág. 168)			No
4.6.7 Algoritmos de Optimización (Pág. 169)			No
4.6.8 Computación Paralela para Matemáticas (Pág. 169)			No

4.6.1. NSA/Análisis de Errores y Aritmética de Punto Flotante

Estudio de cómo se originan y propagan los errores en los cálculos numéricos.
Temas:
Core Tier1

- Representación de punto flotante y errores de redondeo
- Condicionamiento de problemas y estabilidad de algoritmos

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Explicar la pérdida de significancia en la resta de punto flotante [Familiarizarse]
2. Calcular el número de condición de una operación matemática [Evaluar]

4.6.2. NSA/Álgebra Lineal Numérica

Algoritmos numéricos eficientes para resolver sistemas lineales y problemas de valores propios.
Temas:
Core Tier1

- Descomposiciones LU y QR

Core Tier2

- Métodos iterativos: Jacobi, Gauss-Seidel y Gradiente Conjugado

- Descomposición en Valores Singulares (SVD) y aplicaciones

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Resolver sistemas lineales usando descomposición LU con pivoteo [Usar]

Core-Tier2:
2. Analizar la tasa de convergencia de solvers lineales iterativos [Evaluar]

4.6.3. NSA/Teoría de Interpolación y Aproximación

Técnicas para aproximar funciones utilizando polinomios o splines.
Temas:
Core Tier1
- Interpolación polinómica de Lagrange y Newton

Core Tier2
- Splines cúbicos y aproximación segmentaria (piecewise)
- Aproximación por mínimos cuadrados y polinomios ortogonales

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Construir un polinomio interpolador para un conjunto de puntos de datos [Usar]

Core-Tier2:
2. Evaluar el límite de error para la interpolación polinómica [Evaluar]

4.6.4. NSA/Integración y Diferenciación Numérica

Algoritmos para aproximar derivadas e integrales.
Temas:
Core Tier1
- Fórmulas de Newton-Cotes: reglas del Trapecio y de Simpson
- Fórmulas de diferencias finitas para derivadas

Core Tier2
- Cuadratura Gaussiana e integración de alto orden

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Calcular integrales definidas usando la regla de Simpson [Usar]

Core-Tier2:
2. Comparar la eficiencia de la cuadratura Gaussiana frente a las reglas trapeciales [Evaluar]

4.6.5. NSA/Métodos Numéricos para EDOs y EDPs

Resolución de ecuaciones diferenciales a través de la discretización.
Temas:
Core Tier2
- Métodos de Runge-Kutta y paso adaptativo
- Análisis de estabilidad para esquemas de paso de tiempo

Alternativo
- Métodos de disparo (shooting) para problemas de valor de contorno

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier2:
1. Implementar un solver de Runge-Kutta de cuarto orden [Usar]
2. Determinar la región de estabilidad absoluta para un esquema numérico [Evaluar]

4.6.6. NSA/Métodos de Elementos Finitos y Volúmenes Finitos

Técnicas numéricas avanzadas para resolver ecuaciones diferenciales parciales en geometrías complejas.
Temas:
Alternativo

- Formulaciones débiles y métodos de Galerkin
- Fundamentos de triangulación y generación de mallas

Aprendizaje esperado (*Learning Outcomes*):
Elective:

1. Describir los pasos del método de elementos finitos [Familiarizarse]
2. Aplicar el método de elementos finitos a una ecuación de calor 1D [Evaluar]

4.6.7. NSA/Algoritmos de Optimización

Métodos numéricos para encontrar mínimos y máximos de funciones.
Temas:
Core Tier2

- Descenso de gradiente y método de Newton

Alternativo

- Métodos Quasi-Newton (BFGS)
- Métodos de penalización y barrera para restricciones

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier2:

1. Implementar un algoritmo de descenso de gradiente con búsqueda de línea [Usar]

Elective:

2. Analizar la convergencia del método de Newton para formas cuadráticas [Evaluar]

4.6.8. NSA/Computación Paralela para Matemáticas

Aprovechamiento de arquitecturas paralelas para acelerar cálculos matemáticos.
Temas:
Alternativo

- Algoritmos de álgebra lineal paralela
- Aceleración por GPU y CUDA para matemáticas

Aprendizaje esperado (*Learning Outcomes*):
Elective:

1. Explicar la ley de Amdahl en el contexto del software matemático [Familiarizarse]
2. Usar librerías paralelas para resolver sistemas lineales a gran escala [Usar]

4.7. Modelado Matemático y Simulación (MMS)

Esta área une las matemáticas teóricas con las aplicaciones del mundo real. Se centra en formular, analizar y simular representaciones matemáticas de sistemas físicos, biológicos y sociales.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
4.7.1 Ecuaciones Diferenciales Ordinarias (EDO) (Pág. 169)			No
4.7.2 Ecuaciones Diferenciales Parciales (EDP) (Pág. 170)			No
4.7.3 Sistemas Dinámicos y Teoría de Bifurcaciones (Pág. 170)			No
4.7.4 Métodos de Perturbación y Análisis Asintótico (Pág. 170)			No
4.7.5 Dinámica de Fluidos y Mecánica de Medios Continuos (Pág. 171)			No
4.7.6 Teoría de Control y Control Óptimo (Pág. 171)			No
4.7.7 Ecuaciones Diferenciales Estocásticas (EDE) (Pág. 171)			No
4.7.8 Modelado Basado en Agentes (Pág. 171)			No

4.7.1. MMS/Ecuaciones Diferenciales Ordinarias (EDO)

Métodos para resolver y analizar ecuaciones que involucran derivadas de funciones de una sola variable.

Temas:
Core Tier1
- Ecuaciones de primer orden y separabilidad
- Ecuaciones lineales de orden superior con coeficientes constantes

Core Tier2
- Sistemas de EDO lineales de primer orden

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Resolver ecuaciones diferenciales lineales usando factores integrantes [Usar]

Core-Tier2:
2. Analizar la estabilidad de los puntos de equilibrio en un sistema de EDO [Evaluar]

4.7.2. MMS/Ecuaciones Diferenciales Parciales (EDP)

Ecuaciones que involucran múltiples variables, esenciales para describir el calor, la propagación de ondas y el flujo de fluidos.
Temas:
Core Tier2
- Separación de variables para las ecuaciones de calor y de onda
- Métodos de transformada de Laplace para EDP

Alternativo
- Funciones de Green y problemas de valor de contorno

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier2:
1. Aplicar la separación de variables para resolver la ecuación de calor 1D [Usar]

Elective:
2. Derivar la ecuación de onda a partir de principios físicos [Evaluar]

4.7.3. MMS/Sistemas Dinámicos y Teoría de Bifurcaciones

Comportamiento a largo plazo de sistemas en evolución, centrándose en la estabilidad, los ciclos y el caos.
Temas:
Core Tier2
- Retratos de fase y linealización

Alternativo
- Análisis de bifurcaciones: tridente, transcrítica y de Hopf
- Dinámica caótica y exponentes de Lyapunov

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier2:
1. Esbozar retratos de fase para sistemas autónomos bidimensionales [Usar]

Elective:
2. Identificar puntos de bifurcación variando los parámetros del sistema [Evaluar]

4.7.4. MMS/Métodos de Perturbación y Análisis Asintótico

Técnicas para encontrar soluciones aproximadas a problemas que contienen parámetros pequeños o grandes.
Temas:
Alternativo
- Teoría de perturbación regular para ecuaciones algebraicas
- Capas límite y expansiones asintóticas coincidentes

Aprendizaje esperado (*Learning Outcomes*):
Elective:
1. Encontrar raíces aproximadas de polinomios usando series de perturbación [Usar]
2. Aplicar la aproximación WKB a ecuaciones diferenciales oscilatorias [Evaluar]

4.7.5. MMS/Dinámica de Fluidos y Mecánica de Medios Continuos

Descripción matemática del movimiento de fluidos (líquidos y gases).

Temas:

Alternativo

- Ecuaciones de Navier-Stokes y viscosidad
- Flujo no viscoso y principio de Bernoulli

Aprendizaje esperado (*Learning Outcomes*):

Elective:

1. Describir el significado físico de cada término en las ecuaciones de Navier-Stokes [Familiarizarse]
2. Calcular la sustentación y la resistencia para perfiles geométricos simples [Usar]

4.7.6. MMS/Teoría de Control y Control Óptimo

Manipulación de las entradas de un sistema para lograr un comportamiento o estado deseado.

Temas:

Alternativo

- Control por retroalimentación y controladores PID
- Principio del Máximo de Pontryagin

Aprendizaje esperado (*Learning Outcomes*):

Elective:

1. Diseñar un controlador de retroalimentación de estado para un sistema lineal [Usar]
2. Evaluar la controlabilidad y observabilidad de un sistema [Evaluar]

4.7.7. MMS/Ecuaciones Diferenciales Estocásticas (EDE)

Ecuaciones diferenciales que incorporan ruido aleatorio, ampliamente utilizadas en finanzas y física.

Temas:

Alternativo

- Lema de Ito e integración estocástica
- Ecuación de Fokker-Planck y evolución de la probabilidad

Aprendizaje esperado (*Learning Outcomes*):

Elective:

1. Aplicar el Lema de Ito para diferenciar procesos estocásticos [Usar]
2. Resolver la EDE de Black-Scholes para la valoración de opciones [Evaluar]

4.7.8. MMS/Modelado Basado en Agentes

Simulación de las acciones e interacciones de agentes autónomos para evaluar sus efectos en el sistema en su conjunto.

Temas:

Alternativo

- Comportamiento emergente e inteligencia de enjambre
- Marcos de simulación y validación de Monte Carlo

Aprendizaje esperado (*Learning Outcomes*):

Elective:

1. Implementar una simulación en NetLogo o Python de un sistema depredador-presa [Usar]
2. Analizar cómo las reglas locales de los agentes conducen a patrones globales [Evaluar]

4.8. Fundamentos Matemáticos de la Ciencia de Datos (FDS)

Esta área proporciona los fundamentos matemáticos formales para extraer conocimiento de los datos, combinando geometría de alta dimensión, optimización y aprendizaje estadístico.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
4.8.1 Teoría del Aprendizaje Estadístico (Pág. 172)			No
4.8.2 Optimización para el Aprendizaje Automático (Pág. 172)			No
4.8.3 Análisis de Datos Topológicos (TDA) (Pág. 172)			No
4.8.4 Reducción de Dimensionalidad (Pág. 172)			No
4.8.5 Métodos de Núcleo (Kernel) y SVM (Pág. 173)			No
4.8.6 Teoría de la Información y Entropía (Pág. 173)			No
4.8.7 Teoría Matemática del Aprendizaje Profundo (Pág. 173)			No

4.8.1. FDS/Teoría del Aprendizaje Estadístico

Marco teórico para el aprendizaje automático, centrado en la generalización y el riesgo.

Temas:

Core Tier2

- Dimensión VC y minimización del riesgo estructural
- Compromiso entre sesgo y varianza y sobreajuste (overfitting)

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Explicar la conexión entre la complejidad del modelo y el error de generalización [Familiarizarse]
2. Calcular la dimensión VC para familias simples de clasificadores [Evaluar]

4.8.2. FDS/Optimización para el Aprendizaje Automático

Técnicas de optimización numérica adaptadas para el entrenamiento de modelos a gran escala.

Temas:

Core Tier2

- Descenso de Gradiente Estocástico (SGD) y variantes
- Optimización convexa y dualidad en SVM

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Implementar SGD con momento para un modelo de regresión logística [Usar]
2. Comparar la convergencia de la optimización por lotes (batch) frente a la estocástica [Evaluar]

4.8.3. FDS/Análisis de Datos Topológicos (TDA)

Uso de técnicas de topología algebraica para comprender la "forma" de los datos de alta dimensión.

Temas:

Alternativo

- Homología persistente y códigos de barras (barcodes)
- Complejos simpliciales a partir de nubes de puntos

Aprendizaje esperado (*Learning Outcomes*):

Elective:

1. Calcular el diagrama de persistencia para un conjunto de datos ruidosos [Usar]
2. Interpretar códigos de barras para identificar cúmulos (clusters) y bucles en los datos [Familiarizarse]

4.8.4. FDS/Reducción de Dimensionalidad

Reducción del número de variables bajo consideración preservando la información esencial.

Temas:

Core Tier2

- Análisis de Componentes Principales (PCA) y SVD

Alternativo

- Aprendizaje de variedades: t-SNE e Isomap

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier2:
1. Realizar PCA para reducir la dimensionalidad de un espacio de características [Usar]

Elective:
2. Evaluar la varianza explicada por un subconjunto de componentes principales [Evaluar]

4.8.5. FDS/Métodos de Núcleo (Kernel) y SVM

Uso de mapeo no lineal a través de núcleos para realizar clasificación lineal en espacios de alta dimensión.
Temas:
Core Tier2
- Teorema de Mercer y tipos de núcleos
- Formulación de Máquinas de Vectores de Soporte (SVM)

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier2:
1. Aplicar el truco del núcleo para transformar datos no lineales separables [Usar]
2. Elegir parámetros de núcleo adecuados mediante validación cruzada [Evaluar]

4.8.6. FDS/Teoría de la Información y Entropía

Cuantificación de la información, compresión y transmisión.
Temas:
Core Tier2
- Entropía de Shannon e información mutua
- Divergencia de Kullback-Leibler

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier2:
1. Calcular la entropía de una distribución de probabilidad discreta [Usar]
2. Minimizar la pérdida de entropía cruzada en una tarea de clasificación [Evaluar]

4.8.7. FDS/Teoría Matemática del Aprendizaje Profundo

Análisis de las redes neuronales como aproximadores universales y la dinámica de su entrenamiento.
Temas:
Alternativo
- Teoremas de Aproximación Universal
- Matemáticas de la retropropagación y desvanecimiento del gradiente

Aprendizaje esperado (*Learning Outcomes*):
Elective:
1. Explicar el papel de las funciones de activación no lineales en redes profundas [Familiarizarse]
2. Analizar el Jacobiano de una capa de red neuronal [Evaluar]

4.9. Geometría y Topología Computacional (CGT)

Se centra en el diseño, análisis e implementación de algoritmos para resolver problemas geométricos y topológicos de manera eficiente.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
4.9.1 Diagramas de Voronoi y Triangulaciones de Delaunay (Pág. 174)			No
4.9.2 Algoritmos de Envoltura Convexa (Pág. 174)			No
4.9.3 Topología Digital (Pág. 174)			No
4.9.4 Homología Aplicada y Números de Betti (Pág. 174)			No
4.9.5 Generación y Procesamiento de Mallas (Pág. 174)			No

4.9.1. CGT/Diagramas de Voronoi y Triangulaciones de Delaunay

Estructuras geométricas fundamentales para la partición espacial.

Temas:

Core Tier2

- Algoritmos de construcción de Voronoi (algoritmo de Fortune)
- Dualidad entre las estructuras de Voronoi y Delaunay

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Construir manualmente el diagrama de Voronoi para un conjunto pequeño de puntos [Usar]
2. Analizar la complejidad de las consultas de localización de puntos en un diagrama de Voronoi [Evaluar]

4.9.2. CGT/Algoritmos de Envoltura Convexa

Computación del conjunto convexo más pequeño que contiene un conjunto de puntos dado.

Temas:

Core Tier1

- Algoritmos de Graham scan y Jarvis march

Core Tier2

- Divide y vencerás para envolturas convexas en 3D

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Implementar el algoritmo Graham scan para encontrar la envoltura convexa de puntos en 2D [Usar]

Core-Tier2:

2. Comparar la complejidad temporal de diferentes algoritmos de envoltura convexa [Evaluar]

4.9.3. CGT/Topología Digital

Propiedades de conjuntos discretos en cuadrículas, esenciales para el procesamiento de imágenes.

Temas:

Alternativo

- Conectividad en cuadrículas 2D y 3D (conectividad 4/8 y 6/26)
- Característica de Euler discreta para imágenes digitales

Aprendizaje esperado (*Learning Outcomes*):

Elective:

1. Calcular el número de componentes conectados en una imagen binaria [Usar]
2. Verificar el Teorema de la Curva de Jordan en una cuadrícula discreta [Evaluar]

4.9.4. CGT/Homología Aplicada y Números de Betti

Computación de invariantes topológicos como agujeros y vacíos en estructuras de datos.

Temas:

Alternativo

- Matriz de borde y Forma Normal de Smith
- Algoritmos para calcular números de Betti

Aprendizaje esperado (*Learning Outcomes*):

Elective:

1. Calcular el primer número de Betti para un grafo o un complejo [Usar]
2. Analizar la conectividad de una red de sensores usando homología [Evaluar]

4.9.5. CGT/Generación y Procesamiento de Mallas

Creación y manipulación de representaciones discretas de formas geométricas.

Temas:

Alternativo

- Algoritmos de triangulación para polígonos
- Técnicas de simplificación y diezmado de superficies

Aprendizaje esperado (*Learning Outcomes*):
Elective:

1. Realizar una triangulación de Delaunay restringida de un polígono [Usar]
2. Evaluar la calidad de la malla usando la relación de aspecto y el ángulo mínimo [Evaluar]

4.10. Información y Computación Cuántica (QIC)

Esta área explora los fundamentos matemáticos de la mecánica cuántica aplicados a la teoría de la información. Involucra álgebra lineal en espacios de Hilbert complejos, teoría de operadores y el desarrollo de algoritmos que aprovechan la superposición y el entrelazamiento cuántico.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
4.10.1 Fundamentos de Mecánica Cuántica para Matemáticos (Pág. 175)			No
4.10.2 Lógica y Circuitos Cuánticos (Pág. 175)			No
4.10.3 Algoritmos Cuánticos (Pág. 175)			No
4.10.4 Entrelazamiento y Desigualdades de Bell (Pág. 176)			No
4.10.5 Corrección de Errores Cuánticos (Pág. 176)			No
4.10.6 Redes de Tensores y Sistemas de Muchos Cuerpos (Pág. 176)			No

4.10.1. QIC/Fundamentos de Mecánica Cuántica para Matemáticos

Axiomas matemáticos de la mecánica cuántica, incluyendo estados como vectores y observables como operadores.

Temas:
Core Tier2

- Postulados de la mecánica cuántica y vectores de estado
- Operadores autoadjuntos y observables

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier2:

1. Explicar el papel de los espacios de Hilbert complejos en la representación de estados cuánticos [Familiarizarse]
2. Calcular valores esperados para observables cuánticos dados [Evaluar]

4.10.2. QIC/Lógica y Circuitos Cuánticos

El estudio de las compuertas cuánticas y la arquitectura de los circuitos cuánticos.

Temas:
Core Tier2

- Compuertas unitarias y conjuntos de compuertas universales
- El modelo de circuito cuántico y la medición

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier2:

1. Construir un circuito para crear un estado de Bell utilizando compuertas Hadamard y CNOT [Usar]
2. Analizar la transformación del estado de un qubit a través de una serie de compuertas unitarias [Evaluar]

4.10.3. QIC/Algoritmos Cuánticos

Algoritmos que proporcionan aceleraciones sobre sus contrapartes clásicas, como los de Shor y Grover.

Temas:
Alternativo
- Algoritmo de Shor para la factorización de enteros
- Algoritmo de búsqueda de Grover y optimalidad
- La Transformada de Fourier Cuántica (QFT)

Aprendizaje esperado (*Learning Outcomes*):
Elective:
1. Implementar la Transformada de Fourier Cuántica para un número pequeño de qubits [Usar]
2. Evaluar la ganancia de complejidad del algoritmo de Grover sobre la búsqueda clásica [Evaluar]

4.10.4. QIC/Entrelazamiento y Desigualdades de Bell

Correlaciones no locales en sistemas cuánticos y su verificación matemática.
Temas:
Core Tier2
- Matrices de densidad y estados reducidos

Alternativo
- Teorema de Bell y desigualdad CHSH

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier2:
1. Determinar si un estado bipartito está entrelazado utilizando el criterio de la transpuesta parcial [Usar]

Elective:
2. Verificar la violación de la desigualdad de Bell para un estado dado [Evaluar]

4.10.5. QIC/Corrección de Errores Cuánticos

Esquemas matemáticos para proteger la información cuántica de la decoherencia.
Temas:
Alternativo
- Códigos estabilizadores y grupo de Pauli
- Introducción a los códigos de superficie y protección topológica

Aprendizaje esperado (*Learning Outcomes*):
Elective:
1. Describir el código de corrección de errores de inversión de bit de tres qubits [Familiarizarse]
2. Analizar el proceso de medición de síndrome para un código estabilizador [Evaluar]

4.10.6. QIC/Redes de Tensores y Sistemas de Muchos Cuerpos

Herramientas algebraicas avanzadas para describir sistemas cuánticos altamente correlacionados.
Temas:
Alternativo
- Estados de Producto de Matrices (MPS)
- Contracción de tensores y notación gráfica

Aprendizaje esperado (*Learning Outcomes*):
Elective:
1. Representar un tensor de alta dimensión como una red de tensores de menor orden [Usar]
2. Evaluar la dimensión de enlace requerida para niveles específicos de entrelazamiento [Evaluar]

4.11. Biología Matemática y Ciencias de la Vida (MBL)

Esta área aplica el modelado y análisis matemático a sistemas biológicos, cubriendo desde la dinámica de poblaciones y la propagación de enfermedades hasta la mecánica interna del cerebro y las secuencias genéticas.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
4.11.1 Dinámica de Poblaciones y Ecología (Pág. 177)			No
4.11.2 Epidemiología Matemática (Pág. 177)			No
4.11.3 Biología de Sistemas y Redes Metabólicas (Pág. 177)			No
4.11.4 Bioinformática y Alineamiento de Secuencias (Pág. 177)			No
4.11.5 Neurociencia Matemática (Pág. 178)			No

4.11.1. MBL/Dinámica de Poblaciones y Ecología

Modelos matemáticos de crecimiento, competencia y depredación en poblaciones biológicas.

Temas:

Core Tier2

- Ecuaciones de Lotka-Volterra para sistemas depredador-presa
- Crecimiento logístico y capacidad de carga

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Modelar un sistema de competencia de dos especies utilizando ecuaciones diferenciales [Usar]
2. Analizar la estabilidad de los estados estacionarios de coexistencia [Evaluar]

4.11.2. MBL/Epidemiología Matemática

Modelos para la propagación de enfermedades infecciosas en poblaciones.

Temas:

Core Tier2

- El modelo SIR (Susceptible-Infectado-Recuperado)
- El número básico de reproducción (R0)

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Calcular el umbral epidémico para una tasa de infección dada [Calcular]
2. Evaluar el impacto de la cobertura de vacunación en la inmunidad de grupo [Evaluar]

4.11.3. MBL/Biología de Sistemas y Redes Metabólicas

Modelado de reacciones bioquímicas y vías de señalización dentro de las células.

Temas:

Alternativo

- Cinética enzimática de Michaelis-Menten
- Análisis de Balance de Flujo (FBA) para redes metabólicas

Aprendizaje esperado (*Learning Outcomes*):

Elective:

1. Derivar las ecuaciones de tasa para una reacción enzimática de múltiples pasos [Usar]
2. Simular una red regulatoria genética simple [Evaluar]

4.11.4. MBL/Bioinformática y Alineamiento de Secuencias

Métodos computacionales y estadísticos para analizar secuencias de ADN, ARN y proteínas.

Temas:

Alternativo

- Programación dinámica para el alineamiento de secuencias (Needleman-Wunsch)
- Construcción matemática de árboles filogenéticos

Aprendizaje esperado (*Learning Outcomes*):

Elective:

1. Aplicar el algoritmo de Smith-Waterman para el alineamiento local de secuencias [Usar]
2. Analizar la significancia estadística de una coincidencia de secuencias [Evaluar]

4.11.5. MBL/Neurociencia Matemática

Modelos matemáticos de neuronas y redes neuronales.

Temas:

Alternativo

- El modelo de Hodgkin-Huxley de potenciales de acción
- Teoría de campos neuronales y formación de patrones en el cerebro

Aprendizaje esperado (*Learning Outcomes*):

Elective:

1. Explicar el significado fisiológico de las variables de compuerta en los modelos neuronales [Familiarizarse]
2. Simular el comportamiento de disparo de una neurona de tipo "integrar y disparar" (integrate-and-fire) [Evaluar]

4.12. Matemáticas Financieras y Actuariales (FAM)

Esta área se centra en la aplicación de la probabilidad, la estadística y el cálculo a las finanzas y los seguros, específicamente en la valoración de activos y la gestión de riesgos.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
4.12.1 Teoría del Interés y Anualidades (Pág. 178)			No
4.12.2 Arbitraje y Completitud del Mercado (Pág. 178)			No
4.12.3 Teoría de Black-Scholes-Merton (Pág. 178)			No
4.12.4 Medidas de Riesgo (Pág. 179)			No
4.12.5 Modelos Actuariales y Contingencias de Vida (Pág. 179)			No
4.12.6 Finanzas Computacionales y Monte Carlo (Pág. 179)			No

4.12.1. FAM/Teoría del Interés y Anualidades

Fundamentos matemáticos del valor del dinero en el tiempo.

Temas:

Core Tier1

- Interés compuesto y fuerza de interés
- Anualidades ciertas y amortización de préstamos

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular el valor presente de una serie de pagos futuros [Usar]
2. Desarrollar una tabla de amortización para un préstamo decreciente [Evaluar]

4.12.2. FAM/Arbitraje y Completitud del Mercado

Los teoremas fundamentales de la valoración de activos y el concepto de no arbitraje.

Temas:

Core Tier2

- Valoración libre de arbitraje en tiempo discreto
- Medidas de martingala equivalentes y neutralidad al riesgo

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Definir una estrategia de inversión autofinanciada [Familiarizarse]
2. Verificar la ley del precio único en un mercado completo [Evaluar]

4.12.3. FAM/Teoría de Black-Scholes-Merton

Modelado en tiempo continuo de derivados financieros utilizando cálculo estocástico.

Temas:

Core Tier2

- La ecuación diferencial parcial de Black-Scholes
- Las 'Griegas' y estrategias de cobertura delta (delta hedging)

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Valorar una opción de compra (call) europea utilizando la fórmula de Black-Scholes [Usar]
2. Analizar la sensibilidad del precio de una opción ante cambios en la volatilidad (Vega) [Evaluar]

4.12.4. FAM/Medidas de Riesgo

Cuantificación de pérdidas potenciales en carteras financieras.

Temas:

Core Tier2

- Valor en Riesgo (VaR) y VaR de cola (Tail VaR)
- Medidas de riesgo coherentes y sus propiedades

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Calcular el VaR para una cartera con distribución normal [Usar]
2. Comparar el VaR y el VaR Condicional en términos de subaditividad [Evaluar]

4.12.5. FAM/Modelos Actuariales y Contingencias de Vida

Matemáticas de los seguros de vida, pensiones y análisis de supervivencia.

Temas:

Core Tier2

- Funciones de supervivencia y tablas de vida
- Cálculo de primas netas y reservas

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Calcular la probabilidad de supervivencia utilizando una ley de Gompertz-Makeham [Usar]
2. Determinar el valor actuarial presente de una renta vitalicia [Evaluar]

4.12.6. FAM/Finanzas Computacionales y Monte Carlo

Técnicas de simulación numérica para instrumentos financieros complejos.

Temas:

Alternativo

- Métodos de Monte Carlo para opciones dependientes de la trayectoria
- Técnicas de reducción de varianza en simulaciones

Aprendizaje esperado (*Learning Outcomes*):

Elective:

1. Simular trayectorias de precios de activos utilizando el Movimiento Browniano Geométrico [Usar]
2. Estimar el precio de una opción americana utilizando el método de Longstaff-Schwartz [Evaluar]

Capítulo 5

Cuerpo del conocimiento de Física

Las 12 áreas de conocimiento en Física son:

5.1 Mecánica Clásica (CME) (Pág. 182)
- 5.1.1 Cinemática y Dinámica de Partículas (Pág. 182)
- 5.1.2 Trabajo, Energía y Fuerzas Conservativas (Pág. 183)
- 5.1.3 Sistemas de Partículas y Momento Lineal (Pág. 184)
- 5.1.4 Movimiento Rotacional y Momento Angular (Pág. 185)
- 5.1.5 Dinámica del Cuerpo Rígido en 3D (Pág. 185)
- 5.1.6 Marcos de Referencia no Inerciales y Fuerzas Ficticias (Pág. 186)
- 5.1.7 Mecánica Lagrangiana y Hamiltoniana (Pág. 186)
- 5.1.8 Movimiento de Fuerzas Centrales y Mecánica Orbital (Pág. 187)
- 5.1.9 Teoría de Pequeñas Oscilaciones y Osciladores Acoplados (Pág. 188)
- 5.1.10 Fundamentos de la Mecánica de Medios Continuos (Pág. 188)

5.2 Electromagnetismo (ELM) (Pág. 189)
- 5.2.1 Electrostática: Campo Eléctrico y Ley de Gauss (Pág. 189)
- 5.2.2 Potencial Eléctrico y Ecuaciones de Poisson/Laplace (Pág. 190)
- 5.2.3 Energía Electrostática y Dieléctricos (Pág. 190)
- 5.2.4 Magnetostática: Leyes de Biot-Savart y Ampère (Pág. 191)
- 5.2.5 Propiedades Magnéticas de la Materia (Pág. 191)
- 5.2.6 Electrodinámica: Ley de Faraday e Inducción (Pág. 192)
- 5.2.7 Ecuaciones de Maxwell y Transformaciones de Norma (Pág. 192)
- 5.2.8 Ondas Electromagnéticas y Propagación en Medios (Pág. 193)
- 5.2.9 Teoría de la Radiación y Radiación Dipolar (Pág. 193)
- 5.2.10 Relatividad Especial en Electromagnetismo (Pág. 194)

5.3 Termodinámica y Mecánica Estadística (TSM) (Pág. 194)
- 5.3.1 Leyes de la Termodinámica y Funciones de Estado (Pág. 194)
- 5.3.2 Teoría Cinética de los Gases (Pág. 195)
- 5.3.3 Máquinas Térmicas, Entropía y la Segunda Ley (Pág. 195)
- 5.3.4 Potenciales Termodinámicos y Relaciones de Maxwell (Pág. 196)
- 5.3.5 Transiciones de Fase y Fenómenos Críticos (Pág. 196)
- 5.3.6 Colectivos Microcanónico, Canónico y Gran Canónico (Pág. 196)
- 5.3.7 Estadística Cuántica: Fermi-Dirac y Bose-Einstein (Pág. 197)
- 5.3.8 Radiación de Cuerpo Negro y Fonones (Pág. 197)
- 5.3.9 Mecánica Estadística de No-Equilibrio (Pág. 197)
- 5.3.10 Aplicaciones a Sistemas Biológicos y Complejos (Pág. 198)

5.4 Mecánica Cuántica (QME) (Pág. 198)
- 5.4.1 Orígenes de la Teoría Cuántica y Dualidad Onda-Partícula (Pág. 199)
- 5.4.2 La Ecuación de Schrödinger y Funciones de Onda (Pág. 199)
- 5.4.3 Formalismo: Espacio de Hilbert y Notación de Dirac (Pág. 199)
- 5.4.4 Potenciales Unidimensionales (Pozos, Barreras, Tunelamiento) (Pág. 200)
- 5.4.5 El Oscilador Armónico Cuántico (Pág. 200)
- 5.4.6 Momento Angular y Física del Espín (Pág. 200)
- 5.4.7 El Átomo de Hidrógeno y Potenciales Centrales (Pág. 201)
- 5.4.8 Teoría de Perturbaciones Independiente y Dependiente del Tiempo (Pág. 201)
- 5.4.9 Partículas Idénticas y Simetría de Intercambio (Pág. 202)
- 5.4.10 Teoría de la Dispersión y Secciones Eficaces (Pág. 202)

5.5 Relatividad y Cosmología (RCO) (Pág. 202)
- 5.5.1 Relatividad Especial: Transformaciones de Lorentz y Cuadrivectores (Pág. 203)
- 5.5.2 Dinámica Relativista y Energía-Momento (Pág. 203)
- 5.5.3 Fundamentos de la Relatividad General: Principio de Equivalencia (Pág. 203)
- 5.5.4 Curvatura del Espacio-Tiempo y Ecuaciones de Campo de Einstein (Pág. 204)
- 5.5.5 Geometría de Schwarzschild y Agujeros Negros (Pág. 204)
- 5.5.6 Ondas Gravitacionales (Pág. 204)
- 5.5.7 Teoría del Big Bang y Ecuaciones de Friedmann (Pág. 205)
- 5.5.8 Fondo Cósmico de Microondas (CMB) (Pág. 205)
- 5.5.9 Materia Oscura y Energía Oscura (Pág. 205)
- 5.5.10 Inflación del Universo Temprano y Nucleosíntesis (Pág. 205)

5.6 Física de la Materia Condensada (CMP) (Pág. 206)
- 5.6.1 Estructuras Cristalinas y Redes Recíprocas (Pág. 206)
- 5.6.2 Difracción de Rayos X y Dinámica de Red (Pág. 206)
- 5.6.3 Modelo de Electrones Libres y Teoría de Bandas de Sólidos (Pág. 207)
- 5.6.4 Semiconductores, Aislantes y Metales (Pág. 207)
- 5.6.5 Propiedades Magnéticas: Para-, Dia- y Ferromagnetismo (Pág. 207)
- 5.6.6 Superconductividad: Teoría BCS y Alta Tc (Pág. 208)
- 5.6.7 Física de la Materia Blanda y Polímeros (Pág. 208)
- 5.6.8 Nanoestructuras y Sistemas de Baja Dimensionalidad (Pág. 208)
- 5.6.9 Espintrónica y Aislantes Topológicos (Pág. 209)
- 5.6.10 Sistemas de Electrones Fuertemente Correlacionados (Pág. 209)

5.7 Física Nuclear y de Partículas (NPP) (Pág. 209)
- 5.7.1 Estructura Nuclear, Energía de Enlace y Modelo de Capas (Pág. 210)
- 5.7.2 Radiactividad y Desintegraciones Nucleares (Alfa, Beta, Gamma) (Pág. 210)

- 5.7.3 Reacciones Nucleares, Fisión y Fusión (Pág. 211)
- 5.7.4 Aceleradores y Detectores de Partículas (Pág. 211)
- 5.7.5 El Modelo Estándar de la Física de Partículas (Pág. 211)
- 5.7.6 Quarks, Leptones y Bosones de Norma (Pág. 212)
- 5.7.7 Simetrías y Leyes de Conservación (CPT) (Pág. 212)
- 5.7.8 Interacciones Fundamentales y Diagramas de Feynman (Pág. 212)
- 5.7.9 Mecanismo de Higgs y más allá del Modelo Estándar (Pág. 213)
- 5.7.10 Física de Neutrinos y Oscilación (Pág. 213)

5.8 Información y Computación Cuántica (QIC) (Pág. 213)

- 5.8.1 Qubits, Superposición y Entrelazamiento (Pág. 214)
- 5.8.2 Comuertas y Circuitos Lógicos Cuánticos (Pág. 214)
- 5.8.3 Algoritmos Cuánticos (Shor, Grover) (Pág. 215)
- 5.8.4 Teletransportación y Criptografía Cuántica (Pág. 215)
- 5.8.5 Decoherencia y Corrección de Errores Cuánticos (Pág. 215)
- 5.8.6 Realizaciones Físicas de Computadoras Cuánticas (Pág. 216)
- 5.8.7 Sensores Cuánticos y Metrología (Pág. 216)
- 5.8.8 Redes Cuánticas y Comunicación (Pág. 216)

5.9 Óptica y Fotónica (OPH) (Pág. 217)

- 5.9.1 Óptica Geométrica e Instrumentos Ópticos (Pág. 217)
- 5.9.2 Óptica Ondulatoria: Interferencia y Difracción (Pág. 218)
- 5.9.3 Polarización y Óptica de Cristales (Pág. 218)
- 5.9.4 Física de Láseres: Principios y Aplicaciones (Pág. 218)
- 5.9.5 Óptica No Lineal y Conversión de Frecuencia (Pág. 219)
- 5.9.6 Fibra Óptica y Cristales Fotónicos (Pág. 219)
- 5.9.7 Óptica Ultrarrápida y Espectroscopía (Pág. 219)
- 5.9.8 Nano-óptica y Plasmónica (Pág. 220)

5.10 Física Computacional y Basada en Datos (CDP) (Pág. 220)

- 5.10.1 Métodos Numéricos para Ecuaciones Diferenciales en Física (Pág. 221)
- 5.10.2 Métodos y Simulaciones de Monte Carlo (Pág. 221)
- 5.10.3 Computación de Alto Rendimiento (HPC) para Simulaciones a Gran Escala (Pág. 222)
- 5.10.4 Aprendizaje Automático para el Descubrimiento Físico (Pág. 222)
- 5.10.5 Análisis de Datos y Cuantificación de la Incertidumbre (Pág. 222)
- 5.10.6 Modelado de Sistemas Adaptativos Complejos (Pág. 223)
- 5.10.7 Redes Neuronales para Sistemas Cuánticos (Pág. 223)
- 5.10.8 Redes Neuronales Informadas por la Física (PINNs) (Pág. 224)

5.11 Biofísica y Física Médica (BMP) (Pág. 224)

- 5.11.1 Física de las Macromoléculas Biológicas (Pág. 224)
- 5.11.2 Biomecánica y Fuerzas Celulares (Pág. 225)
- 5.11.3 Física Neuronal y Procesamiento de Información en el Cerebro (Pág. 225)
- 5.11.4 Física de Imágenes (MRI, CT, PET, Ultrasonido) (Pág. 225)
- 5.11.5 Radioterapia y Dosimetría (Pág. 226)
- 5.11.6 Mecánica Estadística de Redes Biológicas (Pág. 226)
- 5.11.7 Microfluídica y Lab-on-a-Chip (Pág. 227)

5.12 Astrofísica y Ciencias del Espacio (ASS) (Pág. 227)

- 5.12.1 Estructura y Evolución Estelar (Pág. 227)
- 5.12.2 Ciencia Planetaria y Exoplanetas (Pág. 228)
- 5.12.3 Dinámica y Estructura Galáctica (Pág. 228)
- 5.12.4 Astrofísica de Altas Energías (Rayos X, Rayos Gamma) (Pág. 228)
- 5.12.5 Técnicas Observacionales y Telescopios (Pág. 229)
- 5.12.6 Física Solar y Clima Espacial (Pág. 229)
- 5.12.7 Astronomía de Multimensajeros (Ondas Gravitacionales + Neutrinos) (Pág. 229)

5.1. Mecánica Clásica (CME)

La Mecánica Clásica proporciona los principios fundamentales que gobiernan el movimiento de objetos macroscópicos, desde partículas hasta cuerpos rígidos y medios continuos. Engloba dinámica newtoniana, leyes de conservación, formalismos Lagrangiano y Hamiltoniano, sistemas oscilatorios y mecánica celeste.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
5.1.1 Cinemática y Dinámica de Partículas (Pág. 182)			No
5.1.2 Trabajo, Energía y Fuerzas Conservativas (Pág. 183)			No
5.1.3 Sistemas de Partículas y Momento Lineal (Pág. 184)			No
5.1.4 Movimiento Rotacional y Momento Angular (Pág. 185)			No
5.1.5 Dinámica del Cuerpo Rígido en 3D (Pág. 185)			No
5.1.6 Marcos de Referencia no Inerciales y Fuerzas Ficticias (Pág. 186)			No
5.1.7 Mecánica Lagrangiana y Hamiltoniana (Pág. 186)			No
5.1.8 Movimiento de Fuerzas Centrales y Mecánica Orbital (Pág. 187)			No
5.1.9 Teoría de Pequeñas Oscilaciones y Osciladores Acoplados (Pág. 188)			No
5.1.10 Fundamentos de la Mecánica de Medios Continuos (Pág. 188)			No

5.1.1. CME/Cinemática y Dinámica de Partículas

Descripción matemática del movimiento (cinemática) y estudio de las fuerzas que causan o modifican el movimiento (dinámica) para partículas puntuales en varios sistemas coordenados y campos de

fuerza.

Temas:

Core Tier1

- Descripción vectorial de posición, velocidad y aceleración en coordenadas cartesianas, polares y esféricas
- Tres leyes de Newton del movimiento y marcos de referencia inerciales
- Formulación y solución de ecuaciones de movimiento para fuerzas constantes y variables
- Análisis del movimiento de proyectiles con y sin resistencia del aire ($F_{arrastre} = -bv$ o $-cv^2$)
- Fuerzas de fricción estática y cinética, incluyendo el ángulo de reposo
- Movimiento en medios resistivos incluyendo modelos de arrastre lineal y cuadrático
- Movimiento bajo restricciones como un péndulo o cuentas en alambres

Core Tier2

- Introducción a osciladores no lineales y análisis del espacio de fase
- Integración numérica de ecuaciones de movimiento usando métodos de Euler y Runge-Kutta

Alternativo

- Movimiento de partículas cargadas en campos electromagnéticos bajo la fuerza de Lorentz: $F = q(E + v \times B)$

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Formular las ecuaciones de movimiento para una partícula en una, dos y tres dimensiones dada una ley de fuerza [Usar]
2. Resolver analíticamente la trayectoria de un proyectil bajo gravedad constante despreciando la resistencia del aire [Usar]
3. Calcular la velocidad terminal de un objeto que cae a través de un fluido con arrastre cuadrático [Evaluar]
4. Determinar puntos de equilibrio y estabilidad para una partícula en un potencial unidimensional [Usar]
5. Identificar todas las fuerzas que actúan sobre una partícula en un escenario mecánico dado y dibujar un diagrama de cuerpo libre correcto [Familiarizarse]

Core-Tier2:

6. Implementar un algoritmo numérico simple como el método de Euler para computar el movimiento de un oscilador armónico amortiguado [Usar]
7. Construir e interpretar diagramas de espacio de fase (x vs. $\dot{x}$) para sistemas oscilatorios simples [Evaluar]

Elective:

8. Analizar el movimiento de una partícula cargada en campos eléctricos y magnéticos uniformes [Usar]
9. Aplicar fuerzas de restricción para resolver el movimiento de una partícula en un alambre curvo dado [Usar]

5.1.2. CME/Trabajo, Energía y Fuerzas Conservativas

Conceptos de energía en mecánica: teorema del trabajo y la energía, energía cinética y potencial, conservación de la energía mecánica y propiedades de los campos de fuerzas conservativas.

Temas:

Core Tier1

- Definición de trabajo como una integral de línea $W = \int \vec{F} \cdot d\vec{r}$
- Teorema del trabajo y la energía que relaciona el trabajo total con el cambio en la energía cinética: $W_{total} = \Delta K$
- Energía cinética para partículas individuales y sistemas de partículas
- Fuerzas conservativas e independencia de la trayectoria del trabajo
- Función de energía potencial $U(\vec{r})$ y la relación de fuerza $\vec{F} = -\nabla U$
- Conservación de la energía mecánica total $E = K + U$
- Diagramas de energía e interpretación de puntos de retorno y estabilidad del equilibrio

Core Tier2

- Trabajo realizado por fuerzas no conservativas y mecanismos de disipación de energía

- Superficies de energía potencial en Dos y Tres dimensiones

Alternativo

- Consideraciones energéticas para fuerzas dependientes de la velocidad, como las fuerzas magnéticas y de arrastre

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular el trabajo realizado por una fuerza a lo largo de una trayectoria especificada utilizando integrales de línea [Usar]
2. Determinar si un campo de fuerzas dado es conservativo y hallar su función de energía potencial [Usar]
3. Aplicar la conservación de la energía mecánica para resolver velocidades y posiciones en el movimiento unidimensional [Usar]
4. Analizar un diagrama de energía para identificar puntos de retorno, regiones permitidas y estabilidad del equilibrio [Evaluar]
5. Calcular la potencia instantánea entregada por una fuerza: $P = \vec{F} \cdot \vec{v}$ [Usar]

Core-Tier2:

6. Estimar la energía disipada por fuerzas no conservativas en un proceso mecánico [Evaluar]
7. Localizar y clasificar puntos críticos como máximos, mínimos y puntos de silla en una superficie de potencial bidimensional [Usar]

Elective:

8. Explicar por qué las fuerzas magnéticas no realizan trabajo sobre partículas cargadas a pesar de cambiar la dirección de su velocidad [Familiarizarse]

5.1.3. CME/Sistemas de Partículas y Momento Lineal

Dinámica de sistemas de múltiples partículas: movimiento del centro de masa, conservación del momento lineal, fuerzas internas y externas, y sistemas de masa variable.

Temas:

Core Tier1

- Definición de centro de masa: $\vec{R}_{CM} = \frac{1}{M}\sum m_i \vec{r}_i$
- Momento lineal de una partícula y de un sistema: $\vec{p} = m\vec{v}$ y $\vec{P} = M\vec{V}_{CM}$
- Segunda ley de Newton para sistemas: $\vec{F}_{ext} = \frac{d\vec{P}}{dt}$
- Conservación del momento lineal para sistemas aislados
- Distinción entre fuerzas internas y externas en un sistema
- Colisiones de dos cuerpos, incluyendo casos elásticos, inelásticos y completamente inelásticos
- Coeficiente de restitución y su significado físico en las colisiones

Core Tier2

- Sistemas con masa variable, como la ecuación del cohete: $M\frac{dV}{dt} = v_{ex}\frac{dM}{dt}$
- Sistemas de muchas partículas y su dinámica del centro de masa

Alternativo

- Distribuciones continuas de masa y el cálculo de su centro de masa mediante integración

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Localizar el centro de masa para un conjunto discreto de partículas en Una, Dos y Tres dimensiones [Usar]
2. Determinar el momento lineal total de un sistema a partir del movimiento de su centro de masa [Usar]
3. Aplicar la conservación del momento lineal para resolver problemas de colisión en Una y Dos dimensiones [Usar]
4. Clasificar colisiones como elásticas, inelásticas o completamente inelásticas basándose en la conservación de la energía cinética [Familiarizarse]
5. Calcular el coeficiente de restitución a partir de las velocidades antes y después de la colisión [Usar]

Core-Tier2:

6. Resolver la ecuación del cohete para determinar la velocidad en función de la masa expulsada [Usar]

7. Analizar cómo las fuerzas internas afectan a las partículas individuales pero no al momento total de un sistema [Evaluar]

Elective:

8. Integrar para hallar el centro de masa de cuerpos continuos simétricos como varillas, discos y esferas [Usar]

5.1.4. CME/Movimiento Rotacional y Momento Angular

Cinemática y dinámica rotacional, torque, momento de inercia, energía rotacional y conservación del momento angular para partículas y sistemas.

Temas:

Core Tier1

- Cinemática rotacional incluyendo desplazamiento angular, velocidad y aceleración (θ, ω, α)
- Definición de torque: $\vec{\tau} = \vec{r} \times \vec{F}$ y el cálculo de su magnitud
- Momento angular de una partícula: $\vec{L} = \vec{r} \times \vec{p}$
- Forma rotacional de la segunda ley de Newton: $\vec{\tau}_{net} = \frac{d\vec{L}}{dt}$
- Momento de inercia $I = \sum m_i r_i^2$ y el teorema de los ejes paralelos
- Energía cinética rotacional: $K_{rot} = \frac{1}{2} I\omega^2$
- Conservación del momento angular para sistemas aislados

Core Tier2

- Movimiento giroscópico y el fenómeno de la precesión
- Dinámica de rodadura sin deslizamiento: $v_{CM} = R\omega$

Alternativo

- El efecto Coriolis como manifestación de la dinámica rotacional en marcos no inerciales

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Relacionar las magnitudes cinemáticas lineales y angulares para el movimiento circular [Usar]
2. Calcular el torque respecto a un eje dado para una fuerza aplicada en un punto específico [Usar]
3. Determinar el momento de inercia para geometrías simples como una varilla, un aro, un disco o una esfera [Usar]
4. Aplicar la conservación del momento angular para analizar cambios en la velocidad de rotación cuando cambia el momento de inercia [Evaluar]
5. Calcular la energía cinética rotacional de un cuerpo rígido en rotación [Usar]

Core-Tier2:

6. Analizar el movimiento de objetos que ruedan sin deslizar por un plano inclinado [Evaluar]
7. Predecir la dirección de la precesión para una peonza o un giroscopio en rotación [Evaluar]

Elective:

8. Explicar el efecto Coriolis en términos de marcos de referencia rotatorios y su efecto sobre los proyectiles [Familiarizarse]

5.1.5. CME/Dinámica del Cuerpo Rígido en 3D

Movimiento tridimensional de cuerpos rígidos: tensor de inercia, ejes principales, ecuaciones de Euler y movimiento rotacional complejo.

Temas:

Core Tier2

- Tensor de inercia I_{ij} y su representación matricial
- Ejes principales y la diagonalización del tensor de inercia
- Ecuaciones de Euler para la rotación del cuerpo rígido: $I_1\dot{\omega}_1 = (I_2 - I_3)\omega_2\omega_3 + \tau_1$
- Rotación libre de peonzas asimétricas y análisis de estabilidad de los ejes de rotación
- Casos en los que el vector de momento angular no es paralelo al vector de velocidad angular
- Movimiento libre de torque y polhodias en la dinámica del cuerpo rígido
- Ángulos de Euler (ϕ, θ, ψ) y la descripción de la orientación en el espacio

Alternativo

- Rotación forzada y el fenómeno de la nutación
- Colisiones entre cuerpos rígidos en Tres dimensiones

- Simulación numérica de la dinámica compleja de cuerpos rígidos

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Construir el tensor de inercia para un sistema de masas discretas [Usar]
2. Diagonalizar un tensor de inercia dado para hallar los momentos principales y los ejes principales [Usar]
3. Aplicar las ecuaciones de Euler para analizar la rotación libre de torque de peonzas simétricas [Usar]
4. Interpretar por qué los vectores de momento angular y velocidad angular no son paralelos para cuerpos asimétricos [Evaluar]
5. Calcular los ángulos de Euler para describir la orientación de un cuerpo rígido en el espacio [Usar]

Elective:

6. Simular numéricamente el movimiento tridimensional de un cuerpo rígido bajo torques dados [Usar]
7. Analizar la nutación en una peonza simétrica pesada [Evaluar]

5.1.6. CME/Marcos de Referencia no Inerciales y Fuerzas Ficticias

Física en marcos de referencia acelerados: fuerzas centrífuga, de Coriolis y de Euler, con aplicaciones a la rotación de la Tierra.

Temas:

Core Tier2

- Transformaciones entre marcos de referencia inerciales y acelerados
- Fuerza centrífuga definida como $F_{cent} = -m\vec{\omega} \times (\vec{\omega} \times \vec{r}')$
- Fuerza de Coriolis definida como $F_{Cor} = -2m\vec{\omega} \times \vec{v}'$
- Fuerza de Euler para rotación no constante: $F_{Euler} = -m\frac{d\vec{\omega}}{dt} \times \vec{r}'$
- Gravedad efectiva en la Tierra en rotación: $g_{eff} = g - \vec{\omega} \times (\vec{\omega} \times \vec{R})$
- Péndulo de Foucault como demostración de la rotación de la Tierra
- Efecto Coriolis en los patrones climáticos globales y las corrientes oceánicas

Alternativo

- Precesión de Larmor en campos magnéticos vista como un efecto de marco rotatorio
- Fuerzas de marea como efectos gravitatorios diferenciales en marcos co-rotatorios
- Conexión con el principio de equivalencia de la Relatividad General

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Transformar la segunda ley de Newton a un marco de referencia uniformemente rotatorio [Usar]
2. Calcular las fuerzas centrífuga y de Coriolis para posiciones y velocidades dadas en un marco rotatorio [Usar]
3. Predecir la dirección de desviación de los proyectiles debido a la rotación de la Tierra mediante el efecto Coriolis [Evaluar]
4. Explicar la precesión de un péndulo de Foucault en términos de la rotación de la Tierra [Familiarizarse]
5. Calcular la aceleración gravitatoria efectiva en diferentes latitudes de la Tierra [Usar]

Elective:

6. Analizar la precesión de Larmor utilizando el concepto de marcos de referencia rotatorios [Evaluar]
7. Comparar las fuerzas de marea con las fuerzas centrífugas en sistemas co-rotatorios [Evaluar]

5.1.7. CME/Mecánica Lagrangiana y Hamiltoniana

Formulaciones avanzadas de la mecánica: principio de mínima acción, formulación lagrangiana, mecánica hamiltoniana y transformaciones canónicas.

Temas:

Core Tier2

- Principio de mínima acción y principio de acción estacionaria de Hamilton

- Definición de Lagrangiano: $L = T - V$ para sistemas conservativos
- Ecuaciones de movimiento de Euler-Lagrange: $\frac{d}{dt}\left(\frac{\partial L}{\partial \dot{q}_i}\right) = \frac{\partial L}{\partial q_i}$
- Coordenadas generalizadas y el tratamiento de restricciones holonómicas
- Coordenadas cíclicas y las leyes de conservación asociadas
- Transformada de Legendre del Lagrangiano al Hamiltoniano: $H = \sum p_i \dot{q}_i - L$
- Ecuaciones de Hamilton: $\dot{q}_i = \frac{\partial H}{\partial p_i}$ y $\dot{p}_i = -\frac{\partial H}{\partial q_i}$

Alternativo

- Paréntesis de Poisson: $\{f, g\} = \sum \left(\frac{\partial f}{\partial q_i}\frac{\partial g}{\partial p_i} - \frac{\partial f}{\partial p_i}\frac{\partial g}{\partial q_i}\right)$
- Transformaciones canónicas y sus funciones generatrices
- Variables de acción-ángulo para sistemas hamiltonianos periódicos

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Construir el Lagrangiano para sistemas mecánicos simples en coordenadas generalizadas adecuadas [Usar]
2. Deducir las ecuaciones de movimiento a partir de las ecuaciones de Euler-Lagrange [Usar]
3. Identificar coordenadas cíclicas y sus magnitudes conservadas asociadas en un Lagrangiano dado [Evaluar]
4. Formular el Hamiltoniano a partir de un sistema lagrangiano dado [Usar]
5. Resolver las ecuaciones de Hamilton para sistemas simples como el oscilador armónico [Usar]

Elective:

6. Calcular los paréntesis de Poisson para pares de variables dinámicas [Usar]
7. Aplicar transformaciones canónicas para simplificar la descripción de sistemas hamiltonianos [Usar]

5.1.8. CME/Movimiento de Fuerzas Centrales y Mecánica Orbital

Movimiento bajo fuerzas centrales: leyes de Kepler, dinámica orbital, potencial gravitatorio y el problema de los dos cuerpos.

Temas:

Core Tier2

- Fuerzas centrales $\vec{F} = F(r)\hat{r}$ y la conservación del momento angular
- Masa reducida y la reducción del problema de los dos cuerpos a un problema de un solo cuerpo
- Potencial efectivo: $U_{eff}(r) = U(r) + \frac{L^2}{2\mu r^2}$ y puntos de retorno
- Las Tres leyes de Kepler del movimiento planetario
- Ecuación de la órbita: $\frac{1}{r} = \frac{GM\mu^2}{L^2}(1 + e\cos\theta)$ para leyes del inverso del cuadrado
- Elementos orbitales incluyendo el semieje mayor, la excentricidad y la inclinación
- Clasificación energética de las órbitas en estados ligados ($E < 0$) y no ligados ($E \geq 0$)

Alternativo

- Teoría de la dispersión de Rutherford y el concepto de sección eficaz
- Precesión de órbitas en potenciales gravitatorios no Newtonianos
- Problema restringido de los Tres cuerpos y los cinco puntos de Lagrange

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Reducir un problema de fuerza central de dos cuerpos a un problema equivalente de Un solo cuerpo utilizando la masa reducida [Usar]
2. Analizar el potencial efectivo para determinar los puntos de retorno y los tipos de órbita permitidos [Evaluar]
3. Deducir la Tercera ley de Kepler $T^2 \propto a^3$ a partir de los principios de la gravitación newtoniana [Usar]
4. Calcular parámetros orbitales como el periodo y la energía a partir de las condiciones iniciales [Usar]
5. Clasificar una órbita como circular, elíptica, parabólica o hiperbólica según la energía total [Evaluar]

Elective:

6. Calcular ángulos de dispersión en experimentos de dispersión de Rutherford [Usar]

7. Localizar los cinco puntos de Lagrange en el problema circular restringido de los Tres cuerpos [Usar]

5.1.9. CME/Teoría de Pequeñas Oscilaciones y Osciladores Acoplados

Pequeñas oscilaciones alrededor del equilibrio: modos normales, autofrecuencias, amortiguamiento, forzamiento y sistemas acoplados.

Temas:

Core Tier2

- Análisis de estabilidad de los puntos de equilibrio utilizando las segundas derivadas de la energía potencial
- Aproximación de pequeñas oscilaciones mediante expansión de Taylor alrededor de un equilibrio estable
- Osciladores acoplados y la formulación de las ecuaciones de movimiento en forma matricial
- Modos normales y autofrecuencias derivadas del problema de autovalores
- Coordenadas normales que diagonalizan el sistema mecánico
- Osciladores amortiguados: regímenes subamortiguado, críticamente amortiguado y sobreamortiguado
- Oscilaciones forzadas y la física de los fenómenos de resonancia

Alternativo

- Sistemas continuos y la ecuación de onda como límite de muchos osciladores acoplados
- Oscilaciones no lineales y el estudio de los efectos anarmónicos
- Resonancia paramétrica e introducción a la ecuación de Mathieu

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Linealizar las ecuaciones de movimiento alrededor de un punto de equilibrio estable [Usar]
2. Hallar las frecuencias de los modos normales y los autovectores para sistemas de Dos osciladores acoplados [Usar]
3. Resolver la ecuación del oscilador armónico amortiguado para los Tres regímenes de amortiguamiento [Usar]
4. Analizar la amplitud y la respuesta de fase de un oscilador armónico forzado cerca de la resonancia [Evaluar]
5. Transformar a coordenadas normales para desacoplar las ecuaciones de movimiento de un sistema [Usar]

Elective:

6. Deducir la ecuación de onda a partir del límite continuo de muchos osciladores acoplados [Usar]
7. Investigar las correcciones anarmónicas al potencial del oscilador armónico simple [Evaluar]

5.1.10. CME/Fundamentos de la Mecánica de Medios Continuos

Introducción a los medios continuos: tensores de tensión y deformación, elasticidad, mecánica de fluidos y leyes de conservación.

Temas:

Alternativo

- Hipótesis del continuo y definición de campos de densidad
- Tensor de tensión σ_{ij} y el concepto de fuerza por unidad de área
- Tensor de deformación ϵ_{ij} como medida de la deformación local
- Ley de Hooke para materiales isotrópicos: $\sigma_{ij} = \lambda\epsilon_{kk}\delta_{ij} + 2\mu\epsilon_{ij}$
- Ecuación de Navier-Stokes para fluidos viscosos: $\rho\left(\frac{\partial\vec{v}}{\partial t} + \vec{v}\cdot\nabla\vec{v}\right) = -\nabla p + \mu\nabla^2\vec{v} + \vec{f}$
- Principio de Bernoulli para flujo de fluidos no viscosos e incompresibles
- Conservación de la masa y la ecuación de continuidad: $\frac{\partial\rho}{\partial t} + \nabla\cdot(\rho\vec{v}) = 0$
- Conservación del momento expresada en forma de mecánica de medios continuos
- Propagación de ondas en medios elásticos incluyendo ondas P y ondas S
- Viscosidad y reología de fluidos no Newtonianos

Aprendizaje esperado (*Learning Outcomes*):

Elective:

1. Interpretar el significado físico de las componentes del tensor de tensión [Familiarizarse]
2. Calcular las componentes del tensor de deformación para deformaciones simples [Usar]
3. Aplicar la ley de Hooke para relacionar tensión y deformación en materiales elásticos isotrópicos [Usar]
4. Simplificar la ecuación de Navier-Stokes para casos especiales como flujo estacionario o no viscoso [Usar]
5. Aplicar el principio de Bernoulli para calcular variaciones de presión en el flujo de fluidos [Usar]
6. Deducir la ecuación de continuidad a partir del principio de conservación de la masa [Usar]
7. Analizar diferentes modos de propagación de ondas en sólidos elásticos [Evaluar]

5.2. Electromagnetismo (ELM)

El electromagnetismo estudia el comportamiento de las cargas eléctricas en reposo y en movimiento, las propiedades de los campos eléctricos y magnéticos, y su interacción según lo descrito por las ecuaciones de Maxwell. Abarca la electrostática, magnetostática, electrodinámica y la propagación de ondas electromagnéticas en el vacío y la materia.

Unidad de Conocimiento (KU)	**Core Tier1**	**Core Tier2**	**Alternativo**
5.2.1 Electrostática: Campo Eléctrico y Ley de Gauss (Pág. 189)			No
5.2.2 Potencial Eléctrico y Ecuaciones de Poisson/Laplace (Pág. 190)			No
5.2.3 Energía Electrostática y Dieléctricos (Pág. 190)			No
5.2.4 Magnetostática: Leyes de Biot-Savart y Ampère (Pág. 191)			No
5.2.5 Propiedades Magnéticas de la Materia (Pág. 191)			No
5.2.6 Electrodinámica: Ley de Faraday e Inducción (Pág. 192)			No
5.2.7 Ecuaciones de Maxwell y Transformaciones de Norma (Pág. 192)			No
5.2.8 Ondas Electromagnéticas y Propagación en Medios (Pág. 193)			No
5.2.9 Teoría de la Radiación y Radiación Dipolar (Pág. 193)			No
5.2.10 Relatividad Especial en Electromagnetismo (Pág. 194)			No

5.2.1. ELM/Electrostática: Campo Eléctrico y Ley de Gauss

Análisis de cargas estacionarias, la naturaleza vectorial del campo eléctrico y la aplicación de la Ley de Gauss a sistemas simétricos.

Temas:

Core Tier1

- Ley de Coulomb para cargas puntuales y el principio de superposición lineal
- El vector de campo eléctrico $\vec{E}$ y el cálculo para distribuciones continuas de carga
- La Ley de Gauss en forma integral y su relación con la carga encerrada
- Forma diferencial de la Ley de Gauss: $\nabla \cdot \vec{E} = \rho/\epsilon_0$
- Definición de flujo eléctrico a través de superficies abiertas y cerradas
- Aplicaciones de la Ley de Gauss a esferas y cortezas
- Aplicaciones de la Ley de Gauss a líneas infinitas y cilindros
- Aplicaciones de la Ley de Gauss a láminas infinitas y placas paralelas

Core Tier2

- El Teorema de la Divergencia aplicado a vectores de campo electrostático
- Cargas puntuales representadas por la función delta de Dirac en 3D

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la fuerza electrostática neta sobre una carga a partir de una distribución discreta de cargas [Usar]
2. Aplicar la Ley de Gauss para hallar el campo eléctrico en sistemas con simetría esférica o cilíndrica [Usar]
3. Evaluar el flujo eléctrico a través de una superficie dada para un campo eléctrico no uniforme [Evaluar]

4. Determinar la densidad de carga volumétrica a partir de una expresión dada del campo eléctrico usando la forma diferencial de la Ley de Gauss [Usar]

Core-Tier2:

5. Usar la función delta de Dirac para expresar formalmente la densidad de carga de una carga puntual en el origen [Usar]

5.2.2. ELM/Potencial Eléctrico y Ecuaciones de Poisson/Laplace

La formulación del potencial escalar de la electrostática y la resolución de problemas de valores en la frontera.

Temas:

Core Tier1

- Definición del potencial escalar V y la relación $\vec{E} = -\nabla V$
- La naturaleza conservativa del campo electrostático: $\nabla \times \vec{E} = 0$
- Cálculo del potencial a partir de distribuciones de carga continuas de línea, superficie y volumen
- Propiedades de las superficies equipotenciales y su relación con las líneas de campo

Core Tier2

- Derivación de la ecuación de Poisson $\nabla^2 V = -\rho/\epsilon_0$
- Ecuación de Laplace $\nabla^2 V = 0$ en regiones libres de carga
- El primer y segundo teorema de unicidad para soluciones electrostáticas
- El método de las imágenes para conductores planos y esféricos conectados a tierra
- Solución de la ecuación de Laplace mediante separación de variables en coordenadas cartesianas y esféricas
- Expansión del potencial en términos de momentos monopolares, dipolares y cuadrupolares

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la energía potencial de una configuración de cargas puntuales [Usar]
2. Derivar el vector de campo eléctrico a partir de una función de potencial escalar dada [Usar]

Core-Tier2:

3. Resolver la ecuación de Laplace para una tubería rectangular con condiciones de frontera dadas [Evaluar]
4. Aplicar el método de las imágenes para encontrar la fuerza entre una carga puntual y un plano conductor infinito [Usar]
5. Analizar el término dominante de la expansión multipolar para una distribución de carga a grandes distancias [Evaluar]

5.2.3. ELM/Energía Electrostática y Dieléctricos

Estudio del almacenamiento de energía en campos, capacitores y el comportamiento macroscópico de materiales dieléctricos.

Temas:

Core Tier1

- Trabajo requerido para ensamblar una distribución de cargas
- Densidad de energía en el campo eléctrico: $u = \frac{1}{2}\epsilon_0 E^2$
- Definición de capacitancia $C = Q/V$ y cálculo para geometrías estándar
- Polarización atómica y el vector de polarización $\vec{P}$
- Cargas ligadas superficiales y volumétricas en medios polarizados
- El vector de desplazamiento eléctrico $\vec{D}$ y la Ley de Gauss en la materia
- Dieléctricos lineales, susceptibilidad eléctrica χ_e y permitividad relativa ϵ_r

Core Tier2

- Condiciones de frontera en la interfaz para los vectores $\vec{E}$ y $\vec{D}$
- Energía en sistemas dieléctricos: $W = \frac{1}{2}\int \vec{D} \cdot \vec{E} d\tau$

Alternativo

- Descripción microscópica de los dieléctricos y la relación de Clausius-Mossotti

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Calcular la capacitancia de un capacitor de placas paralelas lleno de un dieléctrico lineal [Usar]
2. Evaluar la distribución de carga ligada para una esfera dieléctrica con polarización uniforme [Evaluar]

Core-Tier2:
3. Resolver el campo eléctrico en ambos lados de una interfaz dieléctrica usando condiciones de frontera [Usar]
4. Analizar la fuerza ejercida sobre una placa dieléctrica al ser introducida en un capacitor [Evaluar]

5.2.4. ELM/Magnetostática: Leyes de Biot-Savart y Ampère

La física de las corrientes eléctricas constantes, los campos magnéticos y las fuerzas sobre cargas en movimiento.

Temas:
Core Tier1
- Ley de la fuerza de Lorentz para cargas puntuales: $\vec{F} = q(\vec{E} + \vec{v} \times \vec{B})$
- Fuerza magnética sobre cables y lazos conductores de corriente
- Ley de Biot-Savart para el campo magnético de un elemento de corriente
- Ley de Ampère en forma integral para corrientes de alta simetría
- Forma diferencial de la Ley de Ampère: $\nabla \times \vec{B} = \mu_0 \vec{J}$
- La naturaleza solenoidal de $\vec{B}$: $\nabla \cdot \vec{B} = 0$
- El potencial vectorial magnético $\vec{A}$ y su relación $\vec{B} = \nabla \times \vec{A}$

Core Tier2
- El calibre de Coulomb $\nabla \cdot \vec{A} = 0$ en magnetostática
- Momento dipolar magnético $\vec{m}$ de un lazo de corriente
- Expansión multipolar del potencial vectorial y el término dipolar

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Calcular el campo magnético de un cable recto infinito usando la Ley de Ampère [Usar]
2. Calcular el campo magnético en el eje de un lazo de corriente circular usando la Ley de Biot-Savart [Usar]
3. Verificar que un campo magnético dado satisface la condición solenoidal $\nabla \cdot \vec{B} = 0$ [Evaluar]

Core-Tier2:
4. Determinar el potencial vectorial magnético para un campo magnético uniforme [Usar]
5. Analizar el torque y la fuerza sobre un dipolo magnético en un campo magnético no uniforme [Evaluar]

5.2.5. ELM/Propiedades Magnéticas de la Materia

Magnetización de materiales, corrientes ligadas y clasificación de medios magnéticos.

Temas:
Core Tier1
- El vector de magnetización $\vec{M}$ como momento dipolar magnético por unidad de volumen
- Corrientes ligadas superficiales $\vec{K}_b$ y corrientes volumétricas $\vec{J}_b$
- La intensidad de campo magnético $\vec{H}$ y la Ley de Ampère en la materia
- Principios físicos del diamagnetismo y paramagnetismo

Core Tier2
- Ferromagnetismo, dominios magnéticos y el ciclo de histéresis
- Condiciones de frontera en la interfaz para los vectores $\vec{B}$ y $\vec{H}$
- Medios magnéticos lineales y permeabilidad magnética μ

Alternativo
- El potencial escalar magnético en regiones libres de corriente
- Introducción al diamagnetismo perfecto y el efecto Meissner

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Hallar la distribución de corriente ligada para un cilindro largo con magnetización longitudinal uniforme [Usar]
2. Clasificar materiales como diamagnéticos, paramagnéticos o ferromagnéticos según su susceptibilidad [Evaluar]

Core-Tier2:

3. Resolver los campos $\vec{B}$ y $\vec{H}$ en una esfera magnética lineal colocada en un campo uniforme [Usar]
4. Analizar la pérdida de energía por ciclo en un material ferromagnético a partir de su curva de histéresis [Evaluar]

5.2.6. ELM/Electrodinámica: Ley de Faraday e Inducción

Campos magnéticos dependientes del tiempo y la generación de fuerzas electromotrices.

Temas:

Core Tier1

- Ley de inducción de Faraday y Ley de Lenz para la dirección
- La regla del flujo para la fuerza electromotriz (FEM) en circuitos móviles
- La naturaleza no conservativa del campo eléctrico inducido: $\nabla \times \vec{E} = -\partial \vec{B}/\partial t$
- Inductancia mutua M entre dos circuitos
- Autoinductancia L y la fuerza contraelectromotriz en inductores
- Energía almacenada en el campo magnético: $W = \frac{1}{2\mu_0} \int B^2 d\tau$

Core Tier2

- Formación de corrientes de Foucault y frenado magnético
- Formalismo de la matriz de inductancia para sistemas de múltiples circuitos
- La fórmula de Neumann para calcular la inductancia mutua

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la FEM inducida en un lazo giratorio dentro de un campo magnético constante [Usar]
2. Aplicar la Ley de Lenz para determinar la dirección de la corriente inducida en un escenario físico dado [Evaluar]
3. Calcular la energía magnética total almacenada en un solenoide largo [Usar]

Core-Tier2:

4. Evaluar la inductancia mutua entre un cable largo y un lazo rectangular adyacente [Evaluar]

5.2.7. ELM/Ecuaciones de Maxwell y Transformaciones de Norma

La unificación de la electricidad y el magnetismo y la estructura matemática de las ecuaciones de campo.

Temas:

Core Tier1

- Corrección de Maxwell a la Ley de Ampère y la corriente de desplazamiento $\vec{J}_d$
- El conjunto completo de las cuatro ecuaciones de Maxwell en el vacío y la materia
- Conservación de la carga y la ecuación de continuidad: $\nabla \cdot \vec{J} + \partial \rho/\partial t = 0$

Core Tier2

- Formulación de las ecuaciones de Maxwell en términos de V y $\vec{A}$
- Concepto de invarianza de norma en electrodinámica
- La condición de calibre de Lorentz: $\nabla \cdot \vec{A} + \mu_0 \epsilon_0 \partial V/\partial t = 0$
- El teorema de Poynting y la conservación de la energía en los campos
- El vector de Poynting $\vec{S} = \vec{E} \times \vec{H}$ como densidad de flujo de energía
- Conservación del momento y el tensor de esfuerzos de Maxwell $\vec{T}$

Alternativo

- Potenciales retardados y ecuaciones de Jefimenko

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la corriente de desplazamiento en el espacio de un capacitor en proceso de carga [Usar]

2. Enunciar las cuatro ecuaciones de Maxwell tanto en forma integral como diferencial [Familiarizarse]

Core-Tier2:

3. Aplicar una transformación de norma a un conjunto de potenciales dado manteniendo los campos invariantes [Usar]
4. Analizar el flujo de energía y la disipación de potencia en una resistencia usando el vector de Poynting [Evaluar]
5. Calcular la presión de radiación ejercida por una onda plana sobre una superficie perfectamente reflectante [Usar]

5.2.8. ELM/Ondas Electromagnéticas y Propagación en Medios

La propagación de perturbaciones electromagnéticas en el vacío, dieléctricos y conductores.

Temas:

Core Tier1

- Derivación de la ecuación de onda electromagnética en el vacío
- Soluciones de ondas planas monocromáticas y sus propiedades
- Estados de polarización: lineal, circular y elíptica
- Densidad de energía, intensidad y momento de las ondas planas

Core Tier2

- Reflexión y refracción en incidencia normal sobre una interfaz
- Ecuaciones de Fresnel y ángulo de Brewster para incidencia oblicua
- Propagación de ondas en medios conductores y la profundidad de piel δ
- Dependencia de la permitividad con la frecuencia y la física de la dispersión
- Velocidad de fase frente a velocidad de grupo en medios dispersivos

Alternativo

- Modos TE y TM en guías de onda rectangulares

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Derivar la relación de dispersión y la velocidad de la luz a partir de las ecuaciones de Maxwell [Usar]
2. Determinar el estado de polarización de una onda a partir de sus componentes vectoriales [Evaluar]

Core-Tier2:

3. Aplicar las ecuaciones de Fresnel para encontrar el coeficiente de reflexión para las polarizaciones TE y TM [Usar]
4. Analizar cómo varía la profundidad de piel con la frecuencia para un conductor metálico dado [Evaluar]
5. Calcular el ángulo de Brewster para una interfaz entre aire y vidrio [Usar]

5.2.9. ELM/Teoría de la Radiación y Radiación Dipolar

Generación de radiación electromagnética por cargas aceleradas y dipolos oscilantes.

Temas:

Core Tier2

- Radiación de un dipolo eléctrico oscilante
- Radiación de un dipolo magnético oscilante
- Concepto de resistencia de radiación en antenas
- La fórmula de Larmor para la potencia total radiada por una carga puntual acelerada

Alternativo

- Los potenciales de Liénard-Wiechert para una carga puntual en movimiento
- Introducción a Bremsstrahlung y radiación sincrotrón

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Calcular la potencia total radiada por un dipolo eléctrico oscilante [Usar]
2. Analizar la distribución angular de la radiación de un dipolo eléctrico [Evaluar]

3. Aplicar la fórmula de Larmor para determinar la pérdida de potencia de un electrón en una órbita circular [Usar]

5.2.10. ELM/Relatividad Especial en Electromagnetismo

La covarianza relativista de las ecuaciones de Maxwell y la transformación de campos.

Temas:

Core Tier2

- La densidad de cuadricorriente J^μ
- El tensor de campo electromagnético $F^{\mu\nu}$ y sus propiedades
- Transformación de Lorentz de los campos $\vec{E}$ y $\vec{B}$
- Invariantes relativistas del campo electromagnético

Alternativo

- El tensor de campo dual y su papel en las ecuaciones de Maxwell

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Transformar los campos eléctricos y magnéticos de un sistema inercial a otro [Usar]
2. Verificar la covarianza de las ecuaciones de Maxwell usando la notación de cuadrivectores [Evaluar]
3. Explicar cómo un campo puramente eléctrico en un sistema puede aparecer como un campo magnético en otro [Familiarizarse]

5.3. Termodinámica y Mecánica Estadística (TSM)

La Termodinámica y la Mecánica Estadística cierran la brecha entre las propiedades microscópicas de las partículas individuales y los observables macroscópicos de la materia en conjunto. Esta área cubre las leyes de transferencia de energía, la entropía, los estados de equilibrio y los colectivos estadísticos utilizados para describir sistemas de muchos cuerpos en regímenes tanto clásicos como cuánticos.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
5.3.1 Leyes de la Termodinámica y Funciones de Estado (Pág. 194)			No
5.3.2 Teoría Cinética de los Gases (Pág. 195)			No
5.3.3 Máquinas Térmicas, Entropía y la Segunda Ley (Pág. 195)			No
5.3.4 Potenciales Termodinámicos y Relaciones de Maxwell (Pág. 196)			No
5.3.5 Transiciones de Fase y Fenómenos Críticos (Pág. 196)			No
5.3.6 Colectivos Microcanónico, Canónico y Gran Canónico (Pág. 196)			No
5.3.7 Estadística Cuántica: Fermi-Dirac y Bose-Einstein (Pág. 197)			No
5.3.8 Radiación de Cuerpo Negro y Fonones (Pág. 197)			No
5.3.9 Mecánica Estadística de No-Equilibrio (Pág. 197)			No
5.3.10 Aplicaciones a Sistemas Biológicos y Complejos (Pág. 198)			No

5.3.1. TSM/Leyes de la Termodinámica y Funciones de Estado

Principios fundamentales que rigen la energía, el calor y el trabajo, y la definición matemática de las variables de estado.

Temas:

Core Tier1

- Ley Cero de la Termodinámica y la definición de temperatura
- Primera Ley de la Termodinámica: $dU = dQ - dW$ y la conservación de la energía interna
- Distinción entre funciones de estado y variables dependientes de la trayectoria
- Propiedades matemáticas de las diferenciales exactas e inexactas en termodinámica
- Energía interna de los gases ideales y su dependencia de la temperatura

- Definición de entalpía $H = U + PV$ y su significado físico
- Capacidades caloríficas a volumen constante C_V y a presión constante C_P

Core Tier2

- Procesos adiabáticos y la relación $PV^\gamma = \text{constante}$
- Expansión de gases reales y el coeficiente de Joule-Thomson

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular el trabajo realizado por un gas durante expansiones isotérmicas y adiabáticas [Usar]
2. Evaluar el cambio en la energía interna para un proceso cíclico usando la Primera Ley [Evaluar]

Core-Tier2:

3. Aplicar la relación de Mayer para encontrar la diferencia entre C_P y C_V para un gas ideal [Usar]

5.3.2. TSM/Teoría Cinética de los Gases

Modelo microscópico de los gases basado en el movimiento molecular y las colisiones.

Temas:

Core Tier1

- Derivación de la presión macroscópica a partir de la transferencia de momento molecular
- Distribución de velocidades de Maxwell-Boltzmann y su dependencia de la temperatura
- Teorema de equipartición y grados de libertad para gases monoatómicos y diatómicos
- Velocidad cuadrática media, media y más probable de las moléculas
- Camino libre medio λ y su relación con la sección eficaz molecular y la densidad

Core Tier2

- Fenómenos de transporte: viscosidad, conductividad térmica y difusión en gases
- Efusión molecular y ley de Graham de la efusión
- Ecuación de estado de Van der Waals para gases reales: $(P + a/v^2)(v - b) = RT$

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la velocidad cuadrática media de las moléculas de nitrógeno a temperatura ambiente [Usar]
2. Interpretar el desplazamiento del pico de la distribución de Maxwell-Boltzmann al aumentar la temperatura [Evaluar]

Core-Tier2:

3. Calcular el camino libre medio de las partículas en una cámara de alto vacío [Usar]

5.3.3. TSM/Máquinas Térmicas, Entropía y la Segunda Ley

Análisis de la eficiencia térmica y la definición estadística/termodinámica de la entropía.

Temas:

Core Tier1

- Enunciados de Kelvin-Planck y Clausius de la Segunda Ley
- El ciclo de Carnot y la eficiencia teórica máxima $\eta = 1 - T_c/T_h$
- Definición de la escala de temperatura termodinámica absoluta
- Desigualdad de Clausius y la definición de entropía $dS \geq dQ/T$
- Principio del aumento de la entropía en sistemas aislados
- Análisis de ciclos termodinámicos usando diagramas Temperatura-Entropía $(T - S)$

Core Tier2

- Fórmula de la entropía de Boltzmann: $S = k_B \ln \Omega$
- Entropía de mezcla y la paradoja de Gibbs
- Tercera Ley de la Termodinámica y la inalcanzabilidad del cero absoluto

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la eficiencia máxima de una máquina térmica que opera entre dos temperaturas dadas [Usar]
2. Evaluar el cambio de entropía para un proceso irreversible como la mezcla de dos líquidos [Evaluar]

Core-Tier2:

3. Calcular la entropía de un sistema de diez espines que no interactúan [Usar]

5.3.4. TSM/Potenciales Termodinámicos y Relaciones de Maxwell

Las transformaciones de Legendre y las relaciones de simetría entre variables termodinámicas.

Temas:

Core Tier1

- Base matemática de las transformaciones de Legendre para potenciales termodinámicos
- Energía libre de Gibbs $G = H - TS$ y su papel en el equilibrio químico
- Energía libre de Helmholtz $A = U - TS$ y su relación con el trabajo máximo
- Derivación de las cuatro relaciones primarias de Maxwell a partir de diferenciales exactas
- La ecuación de Gibbs-Duhem y sus restricciones sobre las variables intensivas

Core Tier2

- Uso de cuadros termodinámicos para recordar relaciones entre potenciales
- Definición de potencial químico μ y equilibrio de intercambio de partículas

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Aplicar una relación de Maxwell para simplificar el cálculo de cambios de energía interna [Usar]
2. Determinar el potencial termodinámico apropiado a minimizar para un sistema a P y T constantes [Evaluar]

5.3.5. TSM/Transiciones de Fase y Fenómenos Críticos

Estudio de los cambios de estado de la materia y el comportamiento cerca de los puntos críticos.

Temas:

Core Tier1

- Diagramas de fase $P - T$ y $P - V$ para sistemas de un solo componente
- La ecuación de Clapeyron para las pendientes de los límites de fase
- Ecuación de Clausius-Clapeyron para transiciones líquido-vapor

Core Tier2

- Clasificación de Ehrenfest de las transiciones de fase de primer y segundo orden
- Calor latente de fusión, vaporización y sublimación
- Temperatura crítica, presión y opalescencia

Alternativo

- Introducción a la teoría de Landau de las transiciones de fase y parámetros de orden

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Estimar el cambio en el punto de ebullición del agua a altas altitudes usando la ecuación de Clausius-Clapeyron [Usar]

Core-Tier2:

2. Clasificar una transición dada como de primer o segundo orden basada en las discontinuidades de la capacidad calorífica [Evaluar]

5.3.6. TSM/Colectivos Microcanónico, Canónico y Gran Canónico

Marcos estadísticos para calcular propiedades macroscópicas a partir de probabilidades de microestados.

Temas:

Core Tier1

- Espacio de fase, celdas y el conteo de microestados
- El Colectivo Microcanónico (N, V, E): Postulado de igual probabilidad a priori
- El Colectivo Canónico (N, V, T) y el factor de Boltzmann $e^{-\beta E}$
- La función de partición Z y su relación con la energía libre de Helmholtz $A = -k_B T \ln Z$

Core Tier2

- El Colectivo Gran Canónico (μ, V, T) y el factor de Gibbs

- Fluctuaciones de energía y número de partículas en los colectivos
- Fórmula de la entropía de Gibbs y su relación con la teoría de la información

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la función de partición para un sistema de dos niveles y derivar su energía media [Usar]
2. Evaluar la presión y la entropía de un sistema partiendo de su función de partición [Evaluar]

Core-Tier2:

3. Calcular la gran función de partición para un sistema de partículas que no interactúan [Usar]

5.3.7. TSM/Estadística Cuántica: Fermi-Dirac y Bose-Einstein

Comportamiento estadístico de partículas idénticas gobernadas por el principio de exclusión de Pauli o la simetría de Bose.

Temas:

Core Tier1

- Partículas distinguibles vs. indistinguibles y simetría de la función de onda
- Distribución de Fermi-Dirac para fermiones y el nivel de Fermi ϵ_F
- Distribución de Bose-Einstein para bosones
- Densidad de estados $g(\epsilon)$ en el espacio de momento en TresD

Core Tier2

- Propiedades del gas de Fermi degenerado y presión de degeneración electrónica
- Fenómeno de la Condensación de Bose-Einstein (BEC) a bajas temperaturas
- El límite de Maxwell-Boltzmann de las estadísticas cuánticas a altas temperaturas

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la energía de Fermi para una densidad dada de electrones libres en un metal [Usar]

Core-Tier2:

2. Analizar las condiciones de temperatura y densidad requeridas para lograr la condensación de Bose-Einstein [Evaluar]
3. Determinar la presión de un gas de Fermi completamente degenerado a $T = 0$ [Usar]

5.3.8. TSM/Radiación de Cuerpo Negro y Fonones

Aplicación de las estadísticas cuánticas a los gases de fotones y vibraciones de red.

Temas:

Core Tier1

- Derivación de la ley de Planck para la densidad de energía espectral de la radiación de cuerpo negro
- Ley de Stefan-Boltzmann para la potencia total radiada: $P \propto T^4$
- Ley de desplazamiento de Wien para la longitud de onda máxima de emisión

Core Tier2

- Modelo de Einstein de calor específico basado en osciladores independientes
- Modelo de Debye de los sólidos y la ley T^3 para la capacidad calorífica a baja temperatura
- Los fonones como vibraciones de red cuantizadas y su densidad de estados

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la temperatura superficial de una estrella dada su longitud de onda de emisión máxima [Usar]

Core-Tier2:

2. Comparar los modelos de Einstein y Debye al explicar el calor específico experimental de los sólidos [Evaluar]

5.3.9. TSM/Mecánica Estadística de No-Equilibrio

Estudio de sistemas que se aproximan al equilibrio y coeficientes de transporte.

Temas:

Core Tier2

- Teoría de Einstein del movimiento browniano y el coeficiente de difusión
- Ecuación de Langevin para dinámica estocástica
- Introducción a la ecuación de Fokker-Planck para la evolución de la densidad de probabilidad
- El Teorema de Fluctuación-Disipación y su significado

Alternativo

- La ecuación de transporte de Boltzmann y la aproximación del tiempo de relajación

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Calcular la constante de difusión para una partícula esférica en un fluido viscoso [Usar]
2. Analizar el efecto de las fuerzas aleatorias en el desplazamiento cuadrático medio de una partícula [Evaluar]

5.3.10. TSM/Aplicaciones a Sistemas Biológicos y Complejos

Modelado termodinámico de macromoléculas, membranas y sistemas ricos en información.

Temas:

Core Tier2

- Mecánica estadística de cadenas poliméricas: Modelos de unión libre y tipo gusano
- Termodinámica del plegamiento de proteínas y desnaturalización del ADN
- Mecánica estadística de canales iónicos y potenciales de membrana

Alternativo

- Entropía de Shannon y su conexión con la entropía física
- Introducción a la autoorganización en sistemas alejados del equilibrio

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Modelar la curva fuerza-extensión de una molécula de ADN usando estadística de polímeros [Usar]
2. Evaluar la estabilidad del plegamiento de una proteína basada en el paisaje de energía libre [Evaluar]

5.4. Mecánica Cuántica (QME)

La Mecánica Cuántica proporciona el marco fundamental para comprender el comportamiento de la materia y la energía a la escala de átomos y partículas subatómicas. Sustituye el determinismo clásico por funciones de onda probabilísticas, operadores en espacios de Hilbert y los principios de superposición y entrelazamiento.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
5.4.1 Orígenes de la Teoría Cuántica y Dualidad Onda-Partícula (Pág. 199)			No
5.4.2 La Ecuación de Schrödinger y Funciones de Onda (Pág. 199)			No
5.4.3 Formalismo: Espacio de Hilbert y Notación de Dirac (Pág. 199)			No
5.4.4 Potenciales Unidimensionales (Pozos, Barreras, Tunelamiento) (Pág. 200)			No
5.4.5 El Oscilador Armónico Cuántico (Pág. 200)			No
5.4.6 Momento Angular y Física del Espín (Pág. 200)			No
5.4.7 El Átomo de Hidrógeno y Potenciales Centrales (Pág. 201)			No
5.4.8 Teoría de Perturbaciones Independiente y Dependiente del Tiempo (Pág. 201)			No
5.4.9 Partículas Idénticas y Simetría de Intercambio (Pág. 202)			No
5.4.10 Teoría de la Dispersión y Secciones Eficaces (Pág. 202)			No

5.4.1. QME/Orígenes de la Teoría Cuántica y Dualidad Onda-Partícula

Fundamentos históricos y la transición de la física clásica a la cuántica.

Temas:

Core Tier1

- Radiación de cuerpo negro y la cuantización de la energía de Planck
- El efecto fotoeléctrico y la hipótesis del fotón de Einstein
- Dispersión Compton y el comportamiento corpuscular de la luz
- El modelo de Bohr y la cuantización de las órbitas atómicas
- Hipótesis de De Broglie de las ondas de materia y la relación $\lambda = h/p$
- Experimento de Davisson-Germer y la difracción de electrones
- Principio de Incertidumbre de Heisenberg: $\Delta x \Delta p \geq \hbar/2$
- Principio de complementariedad de Bohr y la naturaleza de la medición

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la energía y el momento de un fotón dada su longitud de onda [Usar]
2. Evaluar la incertidumbre mínima en la posición para una partícula con una dispersión de momento conocida [Evaluar]
3. Explicar los resultados del experimento de la doble rendija tanto para la luz como para los electrones [Familiarizarse]

5.4.2. QME/La Ecuación de Schrödinger y Funciones de Onda

La ecuación fundamental de movimiento y la interpretación estadística de la función de onda.

Temas:

Core Tier1

- La ecuación de Schrödinger dependiente del tiempo: $i\hbar\frac{\partial\Psi}{\partial t} = \hat{H}\Psi$
- La ecuación de Schrödinger independiente del tiempo y estados estacionarios de energía
- Interpretación estadística de Born de la función de onda y densidad de probabilidad
- La condición de normalización $\int |\Psi|^2 dx = 1$
- Cálculo de valores de expectación para posición y momento
- Corriente de probabilidad J y la ecuación de continuidad

Core Tier2

- Teorema de Ehrenfest que relaciona los promedios cuánticos con las leyes clásicas

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Normalizar una función de onda matemática dada para una partícula en el espacio [Usar]
2. Calcular el valor de expectación de la energía cinética para un paquete de ondas gaussiano [Usar]

Core-Tier2:

3. Verificar que la probabilidad total se conserva en el tiempo usando la ecuación de Schrödinger [Evaluar]

5.4.3. QME/Formalismo: Espacio de Hilbert y Notación de Dirac

La estructura matemática rigurosa de la mecánica cuántica usando álgebra lineal.

Temas:

Core Tier1

- Vectores de estado como elementos de un espacio de Hilbert complejo
- Notación de Dirac: Bras, kets y productos internos
- Operadores hermíticos y su papel como observables físicos
- Ecuaciones de autovalores y la ortonormalidad de los autoestados
- La relación de completitud y operadores de proyección

Core Tier2

- Conmutadores y el principio de incertidumbre generalizado
- Representación matricial de operadores en una base discreta
- Transformaciones unitarias y cambio de base en el espacio de Hilbert

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Aplicar la notación de Dirac para calcular el producto interno y las amplitudes de probabilidad [Usar]
2. Verificar si un operador dado es hermítico examinando sus elementos de matriz [Evaluar]

Core-Tier2:
3. Resolver el problema de autovalores para una matriz hamiltoniana de Dos-por-Dos [Usar]

5.4.4. QME/Potenciales Unidimensionales (Pozos, Barreras, Tunelamiento)

Modelos solubles en UnaD que demuestran efectos cuánticos clave como estados ligados y tunelamiento.
Temas:
Core Tier1
- El pozo cuadrado infinito: niveles de energía y funciones de onda
- El pozo cuadrado finito y la existencia de estados ligados
- Dispersión en un escalón de potencial: coeficientes de reflexión y transmisión
- Tunelamiento cuántico a través de una barrera de potencial rectangular

Core Tier2
- Estados ligados y dispersión en un potencial de delta de Dirac
- La solución de partícula libre y la dispersión de paquetes de ondas

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Calcular la energía del estado fundamental para un electrón en un pozo cuadrado infinito [Usar]
2. Evaluar la probabilidad de transmisión para una partícula que tunela a través de una barrera delgada [Evaluar]

Core-Tier2:
3. Esbozar el comportamiento cualitativo de las funciones de onda para varios perfiles de potencial [Usar]

5.4.5. QME/El Oscilador Armónico Cuántico

El modelo más importante de la física, resuelto mediante métodos algebraicos y analíticos.
Temas:
Core Tier1
- El hamiltoniano del oscilador armónico y su límite clásico
- Método algebraico: operadores de escala (creación y aniquilación)
- Niveles de energía cuantizados: $E_n = (n + 1/2)\hbar\omega$
- Significado físico de la energía del punto cero

Core Tier2
- Método analítico: solución por serie de potencias y polinomios de Hermite

Alternativo
- Introducción a los estados coherentes y su evolución temporal

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Aplicar operadores de escala para encontrar el primer estado excitado a partir del estado fundamental [Usar]

Core-Tier2:
2. Analizar la probabilidad de encontrar una partícula osciladora en la región clásicamente prohibida [Evaluar]

5.4.6. QME/Momento Angular y Física del Espín

Simetría rotacional y el momento angular intrínseco de las partículas.
Temas:
Core Tier1

- Operadores de momento angular orbital y relaciones de conmutación
- Autofunciones del momento angular: armónicos esféricos Y_l^m
- Sistemas de espín un medio y el experimento de Stern-Gerlach
- Álgebra de las matrices de espín de Pauli

Core Tier2

- Adición de momento angular y coeficientes de Clebsch-Gordan
- Precesión del espín en un campo magnético uniforme

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular el valor de expectación de S_z para un estado de espinor dado [Usar]

Core-Tier2:

2. Evaluar los posibles valores del momento angular total para dos espines que interactúan [Evaluar]

5.4.7. QME/El Átomo de Hidrógeno y Potenciales Centrales

La solución de la mecánica cuántica para el átomo más simple y el movimiento en simetría esférica.

Temas:

Core Tier1

- Reducción de la ecuación de Schrödinger en 3D a la ecuación radial
- Término centrífugo y el potencial efectivo
- Espectro de energía del átomo de hidrógeno y la constante de Rydberg
- Números cuánticos principal, azimutal y magnético
- Estructura de las funciones de onda radiales y polinomios de Laguerre

Core Tier2

- Estructura de capas atómicas y las formas de los orbitales s, p, d

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Determinar el radio más probable para un electrón en el estado fundamental del hidrógeno [Usar]

Core-Tier2:

2. Analizar la degeneración de los niveles de energía en el átomo de hidrógeno despreciando el espín [Evaluar]

5.4.8. QME/Teoría de Perturbaciones Independiente y Dependiente del Tiempo

Métodos de aproximación para sistemas que no pueden resolverse exactamente.

Temas:

Core Tier2

- Correcciones de energía y de estado de primer orden para sistemas no degenerados
- Correcciones de energía de segundo orden y su significado
- Teoría de perturbaciones degenerada y la eliminación de degeneraciones
- Estructura fina del hidrógeno: correcciones relativistas y de espín-órbita
- El efecto Zeeman: átomos en campos magnéticos externos
- Teoría de perturbaciones dependiente del tiempo y amplitudes de transición
- Regla de Oro de Fermi para tasas de transición

Alternativo

- El principio variacional para estimar energías del estado fundamental
- La aproximación WKB para potenciales que varían lentamente

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Calcular el desplazamiento de energía de primer orden para un oscilador con una pequeña perturbación anharmónica [Usar]
2. Evaluar la tasa de transición entre dos estados bajo una perturbación armónica usando la Regla de Oro de Fermi [Evaluar]

5.4.9. QME/Partículas Idénticas y Simetría de Intercambio

La física de los sistemas que contienen múltiples partículas indistinguibles.
Temas:
Core Tier1

- El operador de intercambio y la simetría/antisimetría de los estados
- Distinción entre bosones y fermiones
- El principio de exclusión de Pauli y determinantes de Slater

Core Tier2

- El átomo de helio y el papel de la energía de intercambio
- Estados de espín singlete y triplete para sistemas de dos electrones

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Enunciar el principio de exclusión de Pauli y su impacto en la tabla periódica [Familiarizarse]

Core-Tier2:

2. Construir un determinante de Slater para un sistema de tres fermiones que no interactúan [Usar]

5.4.10. QME/Teoría de la Dispersión y Secciones Eficaces

Formalismo para describir colisiones e interacciones entre partículas.
Temas:
Core Tier2

- Definición de las secciones eficaces diferencial y total
- La amplitud de dispersión $f(\theta)$ y su relación con la sección eficaz
- La primera aproximación de Born para potenciales débiles
- Análisis de ondas parciales y desplazamientos de fase

Alternativo

- El teorema óptico que relaciona la dispersión hacia adelante con la sección eficaz total

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier2:

1. Aplicar la aproximación de Born para encontrar la amplitud de dispersión para un potencial de Yukawa [Usar]
2. Analizar el efecto de un potencial de esfera dura en el desplazamiento de fase de dispersión a baja energía [Evaluar]

5.5. Relatividad y Cosmología (RCO)

La Relatividad y la Cosmología exploran la naturaleza fundamental del espacio, el tiempo y la gravedad, desde el comportamiento local de partículas a altas velocidades hasta la evolución a gran escala del universo. Esta área cubre la transición de la relatividad especial a la general y los modelos matemáticos que describen la historia y composición del cosmos.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
5.5.1 Relatividad Especial: Transformaciones de Lorentz y Cuadrivectores (Pág. 203)			No
5.5.2 Dinámica Relativista y Energía-Momento (Pág. 203)			No
5.5.3 Fundamentos de la Relatividad General: Principio de Equivalencia (Pág. 203)			No
5.5.4 Curvatura del Espacio-Tiempo y Ecuaciones de Campo de Einstein (Pág. 204)			No
5.5.5 Geometría de Schwarzschild y Agujeros Negros (Pág. 204)			No
5.5.6 Ondas Gravitacionales (Pág. 204)			No
5.5.7 Teoría del Big Bang y Ecuaciones de Friedmann (Pág. 205)			No
5.5.8 Fondo Cósmico de Microondas (CMB) (Pág. 205)			No
5.5.9 Materia Oscura y Energía Oscura (Pág. 205)			No
5.5.10 Inflación del Universo Temprano y Nucleosíntesis (Pág. 205)			No

5.5.1. RCO/Relatividad Especial: Transformaciones de Lorentz y Cuadrivectores

El marco matemático del espacio-tiempo plano y la unificación del espacio y el tiempo.

Temas:

Core Tier1

- Ecuaciones de transformación de Lorentz para impulsos de coordenadas y velocidad
- El intervalo invariante ds^2 y la métrica de Minkowski
- Dilatación del tiempo, contracción de la longitud y tiempo propio τ
- Conos de luz, líneas de universo y estructura causal del espacio-tiempo

Core Tier2

- Definición general de cuadrivectores y el producto escalar
- Introducción a los tensores y notación de índices en el espacio de Minkowski

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular el tiempo transcurrido para un gemelo que viaja a velocidades relativistas [Usar]
2. Verificar la invarianza del intervalo espacio-temporal bajo un impulso de Lorentz [Evaluar]

5.5.2. RCO/Dinámica Relativista y Energía-Momento

Leyes de conservación y física de partículas dentro del régimen relativista.

Temas:

Core Tier1

- Definición del cuadrivector energía-momento P^μ
- Energía de masa en reposo y la relación $E = \gamma mc^2$
- La relación de capa de masa: $E^2 - (pc)^2 = (mc^2)^2$

Core Tier2

- Dinámica de colisiones de partículas y energías umbral de creación
- Fuerza de Minkowski y la versión relativista de la Segunda Ley de Newton

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular el momento de una partícula dada su energía total y masa en reposo [Usar]

Core-Tier2:

2. Analizar un proceso de desintegración de partículas usando la conservación del cuadrimomento [Evaluar]

5.5.3. RCO/Fundamentos de la Relatividad General: Principio de Equivalencia

El cambio conceptual de la gravedad como fuerza a la gravedad como geometría.

Temas:

Core Tier1

- El Principio de Equivalencia Débil y la universalidad de la caída libre
- Principio de Equivalencia de Einstein: física local en marcos de caída libre
- Derivación del corrimiento al rojo gravitacional a partir del principio de equivalencia

Core Tier2

- El Principio de Mach y su influencia en el desarrollo de la RG

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Explicar la equivalencia entre un campo gravitatorio y un marco acelerado [Familiarizarse]

5.5.4. RCO/Curvatura del Espacio-Tiempo y Ecuaciones de Campo de Einstein

El núcleo matemático de la RG: tensores, curvatura y las ecuaciones de campo.

Temas:

Core Tier2

- El tensor métrico $g_{\mu\nu}$ como descripción fundamental de la geometría
- Símbolos de Christoffel y la derivada covariante
- El tensor de curvatura de Riemann y las fuerzas de marea
- Tensor de Ricci, escalar de Ricci y el tensor de Einstein $G_{\mu\nu}$
- Las Ecuaciones de Campo de Einstein (EFE): $G_{\mu\nu} = \kappa T_{\mu\nu}$
- El tensor de energía-momento para fluidos perfectos

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Calcular los símbolos de Christoffel no nulos para una métrica simple dada [Usar]
2. Interpretar la relación entre la densidad de materia-energía y la curvatura del espacio-tiempo [Evaluar]

5.5.5. RCO/Geometría de Schwarzschild y Agujeros Negros

Soluciones de vacío de las EFE y la física de la gravedad extrema.

Temas:

Core Tier1

- La métrica de Schwarzschild para una masa estática con simetría esférica
- El radio de Schwarzschild R_s y el horizonte de sucesos

Core Tier2

- Órbitas de luz y partículas en la geometría de Schwarzschild
- Dilatación del tiempo gravitacional cerca de un cuerpo masivo
- Singularidades de coordenadas frente a singularidades físicas

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular el radio de Schwarzschild para el Sol y la Tierra [Usar]

Core-Tier2:

2. Analizar la deflexión de la luz por un objeto masivo en el límite de campo débil [Evaluar]

5.5.6. RCO/Ondas Gravitacionales

Ondulaciones propagadas en el tejido del espacio-tiempo.

Temas:

Core Tier2

- Gravedad linealizada y la ecuación de onda para perturbaciones del espacio-tiempo
- Modos de polarización Plus (+) y Cruz (×)
- La fórmula del cuadrupolo para la potencia de la radiación gravitacional
- Principios de detección mediante interferometría láser (LIGO/VIRGO)

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Estimar la amplitud adimensional de las ondas gravitacionales de un púlsar binario [Usar]

5.5.7. RCO/Teoría del Big Bang y Ecuaciones de Friedmann

El modelo cosmológico estándar y la expansión del universo.

Temas:

Core Tier1

- El Principio Cosmológico: Isotropía y Homogeneidad
- La métrica de Friedmann-Lemaître-Robertson-Walker (FLRW)
- La Ley de Hubble y la constante de Hubble H_0

Core Tier2

- Las ecuaciones de Friedmann para el factor de escala $a(t)$
- Densidad crítica y los parámetros de densidad $\Omega_m, \Omega_\Lambda, \Omega_k$
- Relación entre el corrimiento al rojo z y el factor de escala a

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Determinar la edad aproximada de un universo de flujo de Hubble dado H_0 [Usar]

Core-Tier2:

2. Analizar el destino del universo basado en el parámetro de densidad total [Evaluar]

5.5.8. RCO/Fondo Cósmico de Microondas (CMB)

La radiación relicta del universo temprano y caliente.

Temas:

Core Tier1

- Época de recombinación y desacoplamiento de fotones
- Espectro de cuerpo negro del CMB y temperatura actual ($T \approx 2{,}73$ K)

Core Tier2

- Anisotropías de temperatura y semillas de la estructura cósmica
- El espectro de potencia angular y los picos acústicos

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Describir el significado de la superficie de última dispersión [Familiarizarse]

5.5.9. RCO/Materia Oscura y Energía Oscura

Los misteriosos componentes que dominan la densidad de energía del universo.

Temas:

Core Tier1

- Evidencia de materia oscura a partir de las curvas de rotación galáctica
- La constante cosmológica Λ como energía oscura

Core Tier2

- Lentes gravitacionales como herramienta para mapear la masa de materia oscura
- Evidencia de expansión acelerada a partir de supernovas de tipo Ia
- El modelo estándar de cosmología ΛCDM

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Contrastar los roles de la materia oscura y la energía oscura en la evolución cósmica [Familiarizarse]

5.5.10. RCO/Inflación del Universo Temprano y Nucleosíntesis

Procesos de alta energía en los primeros minutos y fracciones de segundo.

Temas:

Core Tier2

- Teoría inflacionaria: solución a los problemas del horizonte y de planitud
- Nucleosíntesis del Big Bang: origen de los elementos ligeros (H, He, Li)
- Bariogénesis y la asimetría materia-antimateria

Alternativo

- Ruptura de simetría y transiciones de fase en el universo temprano

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Evaluar la abundancia predicha de helio-4 a partir de la teoría BBN [Evaluar]

5.6. Física de la Materia Condensada (CMP)

La Física de la Materia Condensada investiga las propiedades macroscópicas y microscópicas de la materia en sus fases sólida y líquida. Explora cómo las interacciones colectivas entre vastos números de átomos dan lugar a fenómenos emergentes como la superconductividad, el magnetismo y las fases topológicas.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
5.6.1 Estructuras Cristalinas y Redes Recíprocas (Pág. 206)			No
5.6.2 Difracción de Rayos X y Dinámica de Red (Pág. 206)			No
5.6.3 Modelo de Electrones Libres y Teoría de Bandas de Sólidos (Pág. 207)			No
5.6.4 Semiconductores, Aislantes y Metales (Pág. 207)			No
5.6.5 Propiedades Magnéticas: Para-, Dia- y Ferromagnetismo (Pág. 207)			No
5.6.6 Superconductividad: Teoría BCS y Alta Tc (Pág. 208)			No
5.6.7 Física de la Materia Blanda y Polímeros (Pág. 208)			No
5.6.8 Nanoestructuras y Sistemas de Baja Dimensionalidad (Pág. 208)			No
5.6.9 Espintrónica y Aislantes Topológicos (Pág. 209)			No
5.6.10 Sistemas de Electrones Fuertemente Correlacionados (Pág. 209)			No

5.6.1. CMP/Estructuras Cristalinas y Redes Recíprocas

La descripción matemática de los arreglos atómicos periódicos en el espacio real y de momentos.

Temas:

Core Tier1

- Las 14 redes de Bravais y simetrías de grupos puntuales
- Celdas unitarias primitivas, celdas de Wigner-Seitz y la base de átomos
- Índices de Miller para planos y direcciones cristalinas
- Construcción de la red recíproca: $\vec{b}_i = 2\pi \frac{\vec{a}_j \times \vec{a}_k}{\vec{a}_i \cdot (\vec{a}_j \times \vec{a}_k)}$
- La primera zona de Brillouin y su significado físico en el espacio k

Core Tier2

- Fracciones de empaquetamiento y números de coordinación para SC, BCC, FCC y HCP

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Determinar los vectores de la red recíproca para un cristal cúbico centrado en las caras (FCC) [Usar]
2. Calcular el factor de empaquetamiento atómico para una estructura de diamante [Evaluar]

Core-Tier2:

3. Identificar las operaciones de simetría rotacional y traslacional en un mosaico cristalino 2D [Usar]

5.6.2. CMP/Difracción de Rayos X y Dinámica de Red

Sondeo de estructuras cristalinas y cuantización de vibraciones colectivas (fonones).

Temas:

Core Tier1

- Ley de Bragg de difracción: $2d\sin\theta = n\lambda$
- Condición de Laue para interferencia constructiva: $\Delta\vec{k} = \vec{G}$
- El factor de estructura geométrico S_G y ausencias sistemáticas en la difracción
- Relaciones de dispersión de fonones para cadenas monoatómicas y diatómicas

Core Tier2

- Distinción física entre ramas de fonones acústicas y ópticas
- Dispersión inelástica de neutrones como herramienta para medir espectros de fonones

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular el ángulo de difracción para el plano (111) del silicio usando rayos X de longitud de onda 1.54 Å [Usar]

Core-Tier2:

2. Analizar la aparición de una brecha de frecuencia en una curva de dispersión de red diatómica [Evaluar]

5.6.3. CMP/Modelo de Electrones Libres y Teoría de Bandas de Sólidos

El comportamiento de los electrones en potenciales periódicos y el origen de las bandas de energía.

Temas:

Core Tier1

- El modelo clásico de Drude: conductividad, velocidad de deriva y efecto Hall
- El modelo de Sommerfeld: energía de Fermi, superficie de Fermi y densidad de estados
- Teorema de Bloch: funciones de onda en un potencial periódico $\psi(\vec{r}) = u(\vec{r})e^{i\vec{k}\cdot\vec{r}}$
- El modelo de Kronig-Penney y la emergencia de brechas de energía

Core Tier2

- La aproximación del electrón casi libre y perturbación en los bordes de zona
- La aproximación de enlace fuerte para orbitales atómicos localizados

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la energía de Fermi y la temperatura de Fermi para el cobre [Usar]

Core-Tier2:

2. Evaluar la masa efectiva de un electrón a partir de una relación de dispersión $E(k)$ dada [Evaluar]

5.6.4. CMP/Semiconductores, Aislantes y Metales

Clasificación de sólidos basada en el transporte electrónico y el dopaje.

Temas:

Core Tier1

- Semiconductores intrínsecos: concentración de portadores y ley de acción de masas
- Semiconductores extrínsecos: dopaje tipo n y tipo p, niveles de donantes/aceptores
- Física de la unión p-n: región de agotamiento y potencial de contacto

Core Tier2

- Dependencia de la movilidad de portadores y conductividad con la temperatura
- Propiedades ópticas: semiconductores de brecha directa vs. indirecta

Alternativo

- Principios de operación de MOSFETs y celdas solares

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Determinar la concentración de portadores del silicio a 300K dada su brecha de energía [Usar]
2. Analizar cómo se desplaza el nivel de Fermi al aumentar la concentración de donantes [Evaluar]

5.6.5. CMP/Propiedades Magnéticas: Para-, Dia- y Ferromagnetismo

Orígenes de los momentos magnéticos y el ordenamiento colectivo en sólidos.

Temas:

Core Tier1

- Diamagnetismo de Langevin y ley de Curie para el paramagnetismo
- Campo molecular de Weiss e interacción de intercambio de Heisenberg
- La ley de Curie-Weiss y la transición de fase paramagnética-ferromagnética

Core Tier2

- Antiferromagnetismo (temperatura de Néel) y ferrimagnetismo
- Energía de pared de dominio y el ciclo de histéresis magnética
- Ondas de espín cuantizadas (magnones) y su excitación térmica

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la susceptibilidad magnética de una sal paramagnética a una temperatura específica [Usar]

Core-Tier2:

2. Comparar las curvas de susceptibilidad de materiales ferromagnéticos y antiferromagnéticos [Evaluar]

5.6.6. CMP/Superconductividad: Teoría BCS y Alta Tc

Resistencia cero y diamagnetismo perfecto en fluidos cuánticos.

Temas:

Core Tier1

- El efecto Meissner-Ochsenfeld y la expulsión del flujo magnético
- Superconductores Tipo I y Tipo II: campos críticos y vórtices

Core Tier2

- Ecuaciones de London y longitud de penetración magnética λ_L
- Teoría BCS: pares de Cooper, atracción mediada por fonones y brecha de energía
- Efectos Josephson DC y AC en uniones superconductoras

Alternativo

- Perspectiva de los cupratos de alta Tc y superconductividad no convencional

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Describir la dependencia del campo magnético crítico con la temperatura [Familiarizarse]

Core-Tier2:

2. Calcular la profundidad de penetración de London para un superconductor de plomo [Usar]

5.6.7. CMP/Física de la Materia Blanda y Polímeros

Complejidad en sistemas donde la energía térmica es comparable a la energía de interacción.

Temas:

Core Tier2

- Cristales líquidos: fases nemática, esméctica y colestérica
- Modelos de cadena ideal: caminos aleatorios y teoría de Flory
- Viscoelasticidad y transición vítrea en sólidos amorfos

Alternativo

- Estabilidad coloidal, teoría DLVO y autoensamblaje

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Analizar el origen entrópico de la elasticidad en polímeros similares al caucho [Evaluar]

5.6.8. CMP/Nanoestructuras y Sistemas de Baja Dimensionalidad

Efectos de confinamiento cuántico en materiales 0D, 1D y 2D.

Temas:

Core Tier1

- Densidad de estados en pozos cuánticos (2D), hilos (1D) y puntos (0D)

Core Tier2

- Estructura electrónica del grafeno y propiedades de los nanotubos de carbono

- Tunelamiento de un solo electrón y bloqueo de Coulomb
- Efecto Hall cuántico entero y niveles de Landau

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Calcular el desplazamiento del primer nivel de energía en un pozo cuántico de GaAs de 10nm [Usar]

5.6.9. CMP/Espintrónica y Aislantes Topológicos

Explotación del espín del electrón y la topología para la próxima generación de electrónica.
Temas:
Core Tier2

- Magnetorresistencia gigante (GMR) y sensores de válvula de espín
- Acoplamiento espín-órbita y el efecto Spin Hall

Alternativo

- Aislantes topológicos: correspondencia bulto-borde y estados protegidos
- Estados ligados de Majorana en superconductores topológicos

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier2:

1. Analizar el papel de la dispersión dependiente del espín en dispositivos GMR [Evaluar]

5.6.10. CMP/Sistemas de Electrones Fuertemente Correlacionados

Física más allá de la aproximación de una sola partícula: efectos de muchos cuerpos.
Temas:
Core Tier2

- Aislantes de Mott y el colapso de la teoría de bandas
- El efecto Kondo y la dispersión por impurezas magnéticas

Alternativo

- El modelo de Hubbard para electrones correlacionados en una red
- Sistemas de fermiones pesados y renormalización de masa

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier2:

1. Evaluar cuándo la repulsión electrón-electrón domina sobre la energía cinética en el modelo de Hubbard [Evaluar]

5.7. Física Nuclear y de Partículas (NPP)

La Física Nuclear y de Partículas explora los constituyentes fundamentales de la materia y las fuerzas que gobiernan sus interacciones. Este campo abarca desde la estructura y estabilidad del núcleo atómico hasta las interacciones de alta energía de quarks y leptones descritas por el Modelo Estándar.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
5.7.1 Estructura Nuclear, Energía de Enlace y Modelo de Capas (Pág. 210)			No
5.7.2 Radiactividad y Desintegraciones Nucleares (Alfa, Beta, Gamma) (Pág. 210)			No
5.7.3 Reacciones Nucleares, Fisión y Fusión (Pág. 211)			No
5.7.4 Aceleradores y Detectores de Partículas (Pág. 211)			No
5.7.5 El Modelo Estándar de la Física de Partículas (Pág. 211)			No
5.7.6 Quarks, Leptones y Bosones de Norma (Pág. 212)			No
5.7.7 Simetrías y Leyes de Conservación (CPT) (Pág. 212)			No
5.7.8 Interacciones Fundamentales y Diagramas de Feynman (Pág. 212)			No
5.7.9 Mecanismo de Higgs y más allá del Modelo Estándar (Pág. 213)			No
5.7.10 Física de Neutrinos y Oscilación (Pág. 213)			No

5.7.1. NPP/Estructura Nuclear, Energía de Enlace y Modelo de Capas

Propiedades estáticas de los núcleos y los modelos que describen la organización de los nucleones.

Temas:

Core Tier1

- Radios nucleares, distribución de carga y la ley de escala $R = R_0 A^{1/3}$
- El Modelo de la Gota Líquida y la Fórmula Semiempírica de Masa (SEMF)
- Curva de energía de enlace por nucleón y tendencias de estabilidad nuclear
- El Modelo de Capas Nuclear: Evidencia de los números mágicos
- Acoplamiento espín-órbita fuerte y reordenamiento de los niveles de energía nucleares

Core Tier2

- Momentos dipolares magnéticos y cuadripolares eléctricos del núcleo
- Modelo colectivo: Estados excitados rotacionales y vibracionales en núcleos
- Preparación remota de estados e intercambio de entrelazamiento (entanglement swapping)

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la energía de enlace total de un núcleo de Carbono-12 usando la SEMF [Usar]
2. Predecir el espín y paridad del estado fundamental de núcleos con A impar usando el Modelo de Capas [Evaluar]

Core-Tier2:

3. Determinar la energía de separación de neutrones para un isótopo específico a partir de datos de masa [Usar]

5.7.2. NPP/Radiactividad y Desintegraciones Nucleares (Alfa, Beta, Gamma)

La física de los núcleos inestables y los tres modos principales de desintegración radiactiva.

Temas:

Core Tier1

- La ley de desintegración exponencial, vida media y conceptos de vida promedio
- Desintegración Alfa: Teoría de Gamow del tunelamiento cuántico a través de la barrera de Coulomb
- Desintegración Beta (β^-, β^+) y Captura Electrónica: El papel del neutrino
- Teoría de Fermi de la desintegración beta y el gráfico de Kurie
- Desintegración Gamma: Transiciones multipolares y conversión interna

Core Tier2

- Reglas de selección para transiciones nucleares basadas en el momento angular y la paridad
- Cadenas de desintegración radiactiva y equilibrio secular/transitorio

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la actividad de una muestra radiactiva después de cinco vidas medias [Usar]

2. Analizar por qué el espectro de partículas beta es continuo mientras que el de alfa es discreto [Evaluar]

Core-Tier2:

3. Evaluar si una transición E1 o M1 está permitida entre dos estados nucleares [Usar]

5.7.3. NPP/Reacciones Nucleares, Fisión y Fusión

Energética y mecanismos de transmutaciones nucleares y liberación de energía.

Temas:

Core Tier1

- Valor Q de la reacción y energía umbral para reacciones endotérmicas
- Secciones eficaces nucleares y fórmula de resonancia de Breit-Wigner
- Mecanismos de fisión inducida y el principio de reacción en cadena
- Fusión nuclear: La cadena p-p y el ciclo CNO en la nucleosíntesis estelar

Core Tier2

- Moderación de neutrones, fórmula de los cuatro factores y criticidad del reactor
- Clasificación de reacciones: Reacciones directas frente a formación de núcleo compuesto

Alternativo

- Fusión por confinamiento inercial y magnético: Criterio de Lawson

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la energía cinética mínima requerida para una reacción (α, n) [Usar]

Core-Tier2:

2. Analizar la distribución de masa de los fragmentos de fisión en el U-235 [Evaluar]
3. Calcular la energía liberada en un solo evento de fusión Deuterio-Tritio [Usar]

5.7.4. NPP/Aceleradores y Detectores de Partículas

Herramientas experimentales para la física de alta energía y medición de radiación.

Temas:

Core Tier1

- Principios de Aceleradores Lineales (Linacs) y cavidades de RF
- Ciclotrones, Sincrotrones y el principio de estabilidad de fase
- Cámaras de ionización, contadores proporcionales y tubos Geiger-Müller
- Contadores de centelleo y tubos fotomultiplicadores (PMTs)

Core Tier2

- Detectores de tiras de silicio y de germanio de alta pureza (HPGe)
- Calorímetros electromagnéticos/hadrónicos y detectores de radiación Cherenkov
- Física de colisionadores: Energía en el centro de masas y luminosidad integrada

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la frecuencia de ciclotrón para un protón en un campo de 1.5 Tesla [Usar]

Core-Tier2:

2. Seleccionar el tipo de detector apropiado para espectroscopía de rayos gamma de alta resolución [Evaluar]

5.7.5. NPP/El Modelo Estándar de la Física de Partículas

El marco unificado que describe el electromagnetismo y las interacciones débil y fuerte.

Temas:

Core Tier1

- Las cuatro fuerzas fundamentales: rango, intensidad y partículas mediadoras
- Clasificación de partículas: Fermiones (materia) y Bosones (portadores de fuerza)
- Estructura conceptual del Lagrangiano del Modelo Estándar
- Invarianza de norma local como generadora de interacciones
- Las tres generaciones de quarks y leptones

Core Tier2
- Unificación electrodébil y la escala de la Teoría de la Gran Unificación (GUT)
- Libertad asintótica y la evolución de las constantes de acoplamiento

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Enumerar todas las partículas del Modelo Estándar y sus propiedades básicas (espín, carga) [Familiarizarse]

Core-Tier2:
2. Contrastar las propiedades del Gluón y el Fotón [Evaluar]

5.7.6. NPP/Quarks, Leptones y Bosones de Norma

Propiedades específicas de los bloques fundamentales del universo.

Temas:
Core Tier1
- Sabores de quarks (u, d, s, c, b, t) y carga de color (R, G, B)
- Clasificación de hadrones: Bariones (qqq) y Mesones (q-antiquark)
- Familias de leptones y la conservación del número leptónico (L_e, L_μ, L_τ)
- Los bosones W^+, W^- y Z^0: Mediadores de la fuerza débil
- Gluones y el principio de confinamiento de color

Core Tier2
- Extrañeza y el descubrimiento de kaones e hiperones
- Charmonium y Bottomonium: Espectroscopía de quarks pesados

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Determinar el contenido de quarks para un protón, neutrón y pion [Usar]
2. Calcular la carga eléctrica total y extrañeza de un barión Ω^- [Evaluar]

5.7.7. NPP/Simetrías y Leyes de Conservación (CPT)

La conexión profunda entre las simetrías y los invariantes físicos.

Temas:
Core Tier1
- Teorema de Noether: Simetrías espacio-temporales y conservación de E, p, L
- Conservación del número bariónico (B) y el número leptónico (L)
- Simetría de Isospín en la interacción fuerte (simetría p/n)
- Violación de la Paridad (P) en interacciones débiles (experimento de Wu)
- Conjugación de Carga (C) y la simetría C

Core Tier2
- Violación CP en el sistema de kaones neutros
- El Teorema CPT y sus implicaciones para la paridad de masa partícula-antipartícula

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Comprobar si una reacción de partículas propuesta viola la conservación de B, L o Q [Usar]

Core-Tier2:
2. Evaluar la paridad de un sistema de dos piones en un estado orbital específico [Evaluar]

5.7.8. NPP/Interacciones Fundamentales y Diagramas de Feynman

Visualización y cálculo de interacciones de partículas mediante diagramas de intercambio.

Temas:
Core Tier1
- Reglas básicas para dibujar diagramas de Feynman: Vértices y propagadores
- Vértices de Electrodinámica Cuántica (QED) y acoplamiento electrón-fotón
- Vértices débiles: Corriente cargada ($W^\pm$) y corriente neutra (Z)
- Vértices de Cromodinámica Cuántica (QCD) y auto-interacción de gluones

Core Tier2

- Relación entre diagramas de Feynman y elementos de la matriz de transición
- Partículas fuera de la capa de masa y significado físico de los propagadores
- Introducción a los diagramas de bucle y conceptos de renormalización

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Dibujar un diagrama de Feynman para la desintegración de un muón en un electrón y neutrinos [Usar]
2. Identificar si un diagrama representa una interacción EM, débil o fuerte basada en los vértices [Evaluar]

5.7.9. NPP/Mecanismo de Higgs y más allá del Modelo Estándar

Origen de la masa y limitaciones de la teoría actual.

Temas:

Core Tier2

- Ruptura espontánea de la simetría y bosones de Goldstone
- El potencial del campo de Higgs (sombrero mexicano) y valor esperado del vacío
- Generación de masa a través del acoplamiento de Yukawa con el campo de Higgs
- El problema de la jerarquía y el ajuste fino en el Modelo Estándar

Alternativo

- Introducción a la Supersimetría (SUSY) y supercompañeros
- Teorías de Gran Unificación (GUTs) y candidatos WIMP para la materia oscura

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Explicar cómo el mecanismo de Higgs proporciona masa a los bosones W y Z [Familiarizarse]

5.7.10. NPP/Física de Neutrinos y Oscilación

La naturaleza elusiva de los neutrinos y la evidencia de una masa no nula.

Temas:

Core Tier1

- Fuentes de neutrinos: Neutrinos solares, atmosféricos y de reactor
- El experimento de Reines-Cowan y escalas de la sección eficaz de los neutrinos

Core Tier2

- Mezcla de sabores de neutrinos y la matriz PMNS
- Fórmula de oscilación de neutrinos: Aproximación de dos sabores
- El problema de los neutrinos solares y el efecto MSW

Alternativo

- Neutrinos de Dirac frente a Majorana y desintegración beta doble sin neutrinos

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Calcular la longitud de oscilación para un neutrino de energía 1 GeV [Usar]
2. Evaluar las implicaciones de las oscilaciones de neutrinos para la suposición de masa nula del Modelo Estándar [Evaluar]

5.8. Información y Computación Cuántica (QIC)

La Información y Computación Cuántica aprovecha los principios de la mecánica cuántica (superposición, entrelazamiento e interferencia) para procesar y transmitir información de formas que superan los límites clásicos. Esta área cubre los fundamentos teóricos, el desarrollo de algoritmos y las arquitecturas físicas del hardware cuántico.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
5.8.1 Qubits, Superposición y Entrelazamiento (Pág. 214)			No
5.8.2 Comuertas y Circuitos Lógicos Cuánticos (Pág. 214)			No
5.8.3 Algoritmos Cuánticos (Shor, Grover) (Pág. 215)			No
5.8.4 Teletransportación y Criptografía Cuántica (Pág. 215)			No
5.8.5 Decoherencia y Corrección de Errores Cuánticos (Pág. 215)			No
5.8.6 Realizaciones Físicas de Computadoras Cuánticas (Pág. 216)			No
5.8.7 Sensores Cuánticos y Metrología (Pág. 216)			No
5.8.8 Redes Cuánticas y Comunicación (Pág. 216)			No

5.8.1. QIC/Qubits, Superposición y Entrelazamiento

Las unidades fundamentales de la información cuántica y sus correlaciones no clásicas.

Temas:

Core Tier1

- La Esfera de Bloch: Visualización de qubits en estados puros
- Principio de superposición lineal y estados de la base computacional $|0\rangle, |1\rangle$
- Los cuatro estados de Bell y pares máximamente entrelazados
- Formalismo de la matriz de densidad para estados puros y mixtos

Core Tier2

- Descomposición de Schmidt y el concepto de purificación de estados
- Traza parcial de subsistemas y operadores de densidad reducida

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Representar un estado cuántico dado como coordenadas en la Esfera de Bloch [Usar]
2. Utilizar la traza parcial para determinar si un estado de dos qubits está entrelazado o es separable [Evaluar]

Core-Tier2:

3. Calcular la entropía de Von Neumann para cuantificar el grado de entrelazamiento en un sistema [Usar]

5.8.2. QIC/Comuertas y Circuitos Lógicos Cuánticos

Transformaciones unitarias y la construcción de circuitos cuánticos universales.

Temas:

Core Tier1

- Comuertas de un solo qubit: Pauli X, Y, Z, Hadamard (H) y fases (S, T)
- Comuertas de dos qubits: Operaciones CNOT, CZ y SWAP
- Definición de conjuntos de comuertas universales para la computación cuántica

Core Tier2

- Comuertas de tres qubits: Comuertas Toffoli (CCNOT) y Fredkin (CSWAP)
- El principio de computación reversible en circuitos cuánticos
- Descomposición de matrices unitarias arbitrarias en comuertas elementales

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Construir un circuito cuántico usando comuertas H y CNOT para generar un estado de Bell [Usar]
2. Evaluar el vector de estado resultante tras aplicar una secuencia de comuertas de Pauli [Evaluar]

Core-Tier2:

3. Sintetizar una comuerta Toffoli usando solo comuertas CNOT de 2 qubits y comuertas de 1 qubit [Usar]

5.8.3. QIC/Algoritmos Cuánticos (Shor, Grover)

Aprovechamiento de la aceleración cuántica para factorización, búsqueda y simulación.

Temas:

Core Tier1

- La Transformada de Fourier Cuántica (QFT) y su implementación en circuitos
- El algoritmo de Deutsch-Jozsa y el concepto de aceleración basada en oráculos

Core Tier2

- Algoritmo de Shor: Búsqueda de periodos y la amenaza al criptosistema RSA
- Algoritmo de Grover: Amplificación de amplitud y el operador de difusión
- El protocolo de Estimación de Fase Cuántica (QPE)

Alternativo

- Variational Quantum Eigensolver (VQE) y QAOA para dispositivos NISQ

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Aplicar la QFT a un registro de 3 qubits y determinar el estado de salida [Usar]

Core-Tier2:

2. Analizar la aceleración $O(\sqrt{N})$ del algoritmo de Grover en comparación con la búsqueda lineal clásica [Evaluar]
3. Seguir los pasos del algoritmo de Estimación de Fase para hallar el autovalor de una comuerta T [Usar]

5.8.4. QIC/Teletransportación y Criptografía Cuántica

Protocolos para la transferencia segura de información y reconstrucción de estados.

Temas:

Core Tier1

- El protocolo de Teletransportación Cuántica: Medición y corrección clásica
- El Teorema de No-Clonación y sus implicaciones para la seguridad cuántica
- El protocolo BB84 de Distribución de Claves Cuánticas (QKD)

Core Tier2

- Protocolo Ekert91: QKD basado en entrelazamiento y la desigualdad de Bell
- Codificación superdensa: Envío de dos bits clásicos mediante un solo qubit

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Diagramar la secuencia de comuertas y mediciones requeridas para teletransportar un qubit [Usar]
2. Identificar la presencia de espionaje en un protocolo BB84 analizando las tasas de error en la clave filtrada [Evaluar]

Core-Tier2:

3. Ejecutar el mapeo de bits a comuertas para una transmisión de codificación superdensa [Usar]

5.8.5. QIC/Decoherencia y Corrección de Errores Cuánticos

Protección de la información cuántica frente al ruido y las interacciones ambientales.

Temas:

Core Tier1

- Tipos de ruido cuántico: Canales de bit-flip, phase-flip y de despolarización
- Escalas de tiempo de relajación (T_1) y desfasamiento (T_2)

Core Tier2

- El código de bit-flip de 3 qubits y votación por mayoría
- El código de 9 qubits de Shor: Corrección de errores arbitrarios en un solo qubit
- El formalismo estabilizador para códigos de corrección de errores cuánticos

Alternativo

- Introducción a los códigos de superficie y protección topológica

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Evaluar el impacto del desfasamiento en los elementos fuera de la diagonal de una matriz de densidad [Evaluar]

Core-Tier2:

2. Calcular el síndrome de error para un código de 3 qubits usando mediciones de estabilizadores [Usar]
3. Analizar el teorema del umbral para la computación cuántica tolerante a fallos [Evaluar]

5.8.6. QIC/Realizaciones Físicas de Computadoras Cuánticas

La ingeniería de sistemas físicos para satisfacer los criterios de DiVincenzo.

Temas:

Core Tier1

- Criterios de DiVincenzo para una computadora cuántica escalable
- Qubits superconductores: Cajas de pares de Cooper y qubits Transmon

Core Tier2

- Computación cuántica de iones atrapados: Trampas de Paul y comuertas de Cirac-Zoller
- Computación cuántica óptica lineal (LOQC) y estados comprimidos
- Arreglos de átomos neutros e interacciones de bloqueo de Rydberg

Alternativo

- Puntos cuánticos semiconductores y qubits basados en espín

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Comparar los tiempos de coherencia y velocidades de comuerta de qubits superconductores frente a trampas de iones [Familiarizarse]

Core-Tier2:

2. Analizar el Hamiltoniano de un qubit Transmon y su anarmonicidad [Evaluar]
3. Diseñar una configuración de cadena de iones 1D para una operación de comuerta específica [Usar]

5.8.7. QIC/Sensores Cuánticos y Metrología

Uso de estados cuánticos para lograr mediciones de precisión más allá del Límite Cuántico Estándar.

Temas:

Core Tier2

- El Límite Cuántico Estándar y el Límite de Heisenberg
- Luz comprimida (squeezed light) y su aplicación en la detección de ondas gravitacionales (LIGO)
- Centros de vacante de nitrógeno (NV) en diamante para la detección de campos magnéticos
- Interferometría de Ramsey y la física de los relojes atómicos
- Información de Fisher cuántica y precisión en la estimación de fase

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Calcular la escala de precisión de medición para N partículas entrelazadas frente a N partículas independientes [Usar]
2. Evaluar la sensibilidad de un sensor de centro NV ante un campo magnético de escala nano-Tesla [Evaluar]
3. Modelar una secuencia de pulsos de Ramsey para medir un desplazamiento de frecuencia [Usar]

5.8.8. QIC/Redes Cuánticas y Comunicación

Escalado de sistemas cuánticos mediante distribución de entrelazamiento y repetidores cuánticos.

Temas:

Core Tier2

- Repetidores cuánticos: Purificación de entrelazamiento y mediciones del estado de Bell
- Intercambio de entrelazamiento para comunicación cuántica de larga distancia
- Memoria cuántica: Almacenamiento de estados cuánticos en interfaces luz-materia

- Comunicación cuántica en espacio libre y satelital (misión Micius)

Alternativo

- Arquitecturas para una internet cuántica global

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Diseñar una cadena de repetidores cuánticos para conectar dos nodos separados por 1000 km [Usar]
2. Evaluar la fidelidad de un estado cuántico tras su recuperación de un medio de almacenamiento [Evaluar]
3. Calcular la probabilidad de éxito en la distribución de entrelazamiento sobre un enlace de fibra con pérdidas [Usar]

5.9. Óptica y Fotónica (OPH)

La Óptica y la Fotónica estudian el comportamiento y las propiedades de la luz y su interacción con la materia. Este campo abarca las leyes clásicas de reflexión y refracción, la naturaleza ondulatoria de la luz manifestada en interferencias y difracción, y los avances modernos en láseres, fibra óptica y nanofotónica.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
5.9.1 Óptica Geométrica e Instrumentos Ópticos (Pág. 217)			No
5.9.2 Óptica Ondulatoria: Interferencia y Difracción (Pág. 218)			No
5.9.3 Polarización y Óptica de Cristales (Pág. 218)			No
5.9.4 Física de Láseres: Principios y Aplicaciones (Pág. 218)			No
5.9.5 Óptica No Lineal y Conversión de Frecuencia (Pág. 219)			No
5.9.6 Fibra Óptica y Cristales Fotónicos (Pág. 219)			No
5.9.7 Óptica Ultrarrápida y Espectroscopía (Pág. 219)			No
5.9.8 Nano-óptica y Plasmónica (Pág. 220)			No

5.9.1. OPH/Óptica Geométrica e Instrumentos Ópticos

La aproximación de rayos de la luz y el diseño de sistemas de formación de imágenes.

Temas:

Core Tier1

- Principio de Fermat y la derivación de las leyes de reflexión/refracción
- La fórmula del fabricante de lentes y ecuaciones de espejos esféricos
- Lentes gruesas, puntos cardinales y matrices de sistema
- Teoría de Abbe de formación de imágenes y límites de resolución
- Microscopios compuestos, telescopios keplerianos/galileanos y cámaras

Core Tier2

- Trazado de rayos paraxiales usando el formalismo de matrices ABCD
- Las cinco aberraciones de Seidel: Esférica, Coma, Astigmatismo, Curvatura, Distorsión

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Diseñar un sistema de telescopio con un aumento angular específico usando dos lentes delgadas [Usar]
2. Evaluar la posición y naturaleza de una imagen formada por una combinación de espejo y lente [Evaluar]

Core-Tier2:

3. Calcular la matriz ABCD equivalente para un tren óptico complejo de tres componentes [Usar]

5.9.2. OPH/Óptica Ondulatoria: Interferencia y Difracción

Fenómenos de naturaleza ondulatoria de la luz y la resolución de sistemas ópticos.

Temas:

Core Tier1

- Principio de superposición y representación fasorial de las ondas
- Experimento de la doble rendija de Young e intensidad de las franjas de interferencia
- Interferómetro de Michelson: Medición de desplazamientos pequeños
- Difracción de Fraunhofer por rendijas simples, dobles y múltiples (N)
- Difracción por aberturas circulares y el disco de Airy

Core Tier2

- El criterio de Rayleigh para el límite de resolución
- Difracción de Fresnel y placas de zona de Fresnel
- Interferencia de haces múltiples e interferómetro de Fabry-Perot

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular el espaciado de franjas en una pantalla para un montaje de doble rendija con luz de 532 nm [Usar]
2. Determinar la separación mínima resoluble de dos estrellas usando un telescopio de 10 pulgadas [Evaluar]

Core-Tier2:

3. Calcular la fineza y el rango espectral libre de una cavidad Fabry-Perot [Usar]

5.9.3. OPH/Polarización y Óptica de Cristales

La naturaleza vectorial de la luz y su interacción con medios anisotrópicos.

Temas:

Core Tier1

- Estados de polarización lineal, circular y elíptica
- Ley de Malus para la intensidad de la luz polarizada
- Polarización por reflexión y ángulo de Brewster

Core Tier2

- Birrefringencia en cristales uniaxiales: Rayos ordinarios y extraordinarios
- Placas de cuarto de onda y media onda: Retardo de fase
- Vectores y matrices de Jones para el análisis de estados de polarización
- Actividad óptica y rotación de Faraday

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Determinar la intensidad final de luz no polarizada que pasa por tres polarizadores con desfases de 45 grados [Usar]

Core-Tier2:

2. Analizar el estado de polarización de salida de luz linealmente polarizada tras pasar por una placa de cuarto de onda [Evaluar]
3. Usar el cálculo de Jones para hallar el estado resultante tras un polarizador y un retardador [Usar]

5.9.4. OPH/Física de Láseres: Principios y Aplicaciones

Generación de luz coherente mediante emisión estimulada.

Temas:

Core Tier1

- Coeficientes A y B de Einstein para absorción y emisión
- Condiciones para la acción láser: Inversión de población y bombeo óptico
- Resonadores ópticos: Modos longitudinales y transversales (TEM)
- Esquemas de niveles de energía: Láseres de 3 niveles frente a 4 niveles

Core Tier2

- Propagación de haces gaussianos: Cintura del haz, rango de Rayleigh y divergencia

- Operación pulsada: Técnicas de Q-switching y Mode-locking
- Características de los láseres de He-Ne, Nd:YAG y de diodo

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la ganancia umbral requerida para la acción láser en una cavidad con pérdidas de espejo conocidas [Usar]
2. Explicar por qué un sistema de 2 niveles no puede lograr una inversión de población en estado estacionario [Evaluar]

Core-Tier2:

3. Determinar el tamaño del punto enfocado de un haz láser gaussiano usando una lente dada [Usar]

5.9.5. OPH/Óptica No Lineal y Conversión de Frecuencia

Interacciones luz-materia donde la polarización depende de forma no lineal del campo eléctrico.

Temas:

Core Tier2

- Tensores de susceptibilidad no lineal ($\chi^{(2)}, \chi^{(3)}$)
- Generación de segundo armónico (SHG) y duplicación de frecuencia
- Condición de ajuste de fase (phase matching): Conservación del momento en procesos no lineales
- Efectos electro-ópticos: Efecto Pockels y efecto Kerr
- Oscilación paramétrica óptica (OPO) y conversión descendente (down-conversion)

Alternativo

- Autoenfoque, automodulación de fase y solitones ópticos

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Calcular la eficiencia de conversión de SHG para una longitud específica de un cristal no lineal [Usar]
2. Evaluar la orientación requerida del cristal para lograr el ajuste de fase tipo I [Evaluar]
3. Modelar el desplazamiento de fase inducido por el efecto Kerr en una fibra óptica [Usar]

5.9.6. OPH/Fibra Óptica y Cristales Fotónicos

Guía de luz en guías de onda y estructuras dieléctricas periódicas.

Temas:

Core Tier1

- Reflexión interna total y estructuras de fibra de índice escalonado
- Apertura Numérica (NA) y el cono de aceptación de una fibra
- Fibras monomodo frente a multimodo y el número V

Core Tier2

- Degradación de señal: Atenuación y dispersión cromática/modal
- Cristales fotónicos y la emergencia de brechas de banda fotónicas (bandgaps)
- Fibras microestructuradas y de cristal fotónico (PCFs)

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la apertura numérica de una fibra con índice de núcleo 1.48 y revestimiento 1.46 [Usar]
2. Determinar la longitud de onda máxima para operación monomodo en una fibra dada [Evaluar]

Core-Tier2:

3. Evaluar el ensanchamiento del pulso en un enlace de 50 km debido a la dispersión cromática [Usar]

5.9.7. OPH/Óptica Ultrarrápida y Espectroscopía

Generación y medición de pulsos de luz de femtosegundos y attosegundos.

Temas:

Core Tier2

- Generación de pulsos de femtosegundos mediante láseres de Ti:Zafiro
- El producto tiempo-ancho de banda y pulsos limitados por transformada
- Dispersión de velocidad de grupo (GVD) y chirrido (chirping) de pulsos
- Caracterización de pulsos usando autocorrelación de intensidad
- Técnicas de espectroscopía de bomba-sonda (pump-probe) resueltas en el tiempo

Alternativo

- Generación de armónicos de alto orden y ciencia de attosegundos

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier2:

1. Calcular el ancho de pulso ensanchado tras pasar por un bloque de vidrio con una GVD conocida [Usar]
2. Interpretar una traza de autocorrelación experimental para estimar la duración del pulso [Evaluar]
3. Modelar el decaimiento de una población en estado excitado usando datos de bomba-sonda [Usar]

5.9.8. OPH/Nano-óptica y Plasmónica

Interacción de la luz con estructuras de sub-longitud de onda y nanoestructuras metálicas.
Temas:
Core Tier1

- Ruptura del límite de difracción: Óptica de campo cercano frente a campo lejano

Core Tier2

- Plasmones de superficie polaritones (SPPs) en interfaces metal-dieléctrico
- Resonancia de plasmón de superficie localizado (LSPR) en nanopartículas metálicas
- Microscopía óptica de barrido de campo cercano (SNOM)
- Metamateriales y estructuras de índice de refracción negativo

Alternativo

- Pinzas ópticas: Fuerza y torque sobre nanopartículas

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:

1. Describir el decaimiento de las ondas evanescentes y su papel en la óptica de campo cercano [Familiarizarse]

Core-Tier2:

2. Analizar el desplazamiento en la frecuencia LSPR debido a cambios en el índice de refracción local [Evaluar]
3. Calcular la longitud de onda de un SPP en una interfaz oro-aire para una luz de excitación dada [Usar]

5.10. Física Computacional y Basada en Datos (CDP)

Este área de conocimiento se centra en la intersección del análisis numérico, la computación de alto rendimiento y la ciencia de datos moderna. Proporciona a los físicos las herramientas para simular sistemas que carecen de soluciones analíticas y para extraer leyes fundamentales de conjuntos masivos de datos mediante el aprendizaje automático y la inferencia estadística.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
5.10.1 Métodos Numéricos para Ecuaciones Diferenciales en Física (Pág. 221)			No
5.10.2 Métodos y Simulaciones de Monte Carlo (Pág. 221)			No
5.10.3 Computación de Alto Rendimiento (HPC) para Simulaciones a Gran Escala (Pág. 222)			No
5.10.4 Aprendizaje Automático para el Descubrimiento Físico (Pág. 222)			No
5.10.5 Análisis de Datos y Cuantificación de la Incertidumbre (Pág. 222)			No
5.10.6 Modelado de Sistemas Adaptativos Complejos (Pág. 223)			No
5.10.7 Redes Neuronales para Sistemas Cuánticos (Pág. 223)			No
5.10.8 Redes Neuronales Informadas por la Física (PINNs) (Pág. 224)			No

5.10.1. CDP/Métodos Numéricos para Ecuaciones Diferenciales en Física

Aproximaciones discretas para resolver las ecuaciones fundamentales de movimiento y teorías de campos.

Temas:

Core Tier1

- Métodos de Runge-Kutta (RK2, RK4) para problemas de valor inicial
- Diferencias Finitas en el Dominio del Tiempo (FDTD) para las ecuaciones de Maxwell
- Problemas de valor en la frontera: Métodos de disparo y técnicas de relajación
- Integradores simplécticos: Algoritmos de Verlet y leapfrog para sistemas Hamiltonianos

Core Tier2

- Análisis de estabilidad y la condición de Courant-Friedrichs-Lewy (CFL)
- Método de Elementos Finitos (FEM) para geometrías complejas en elasticidad y flujo de fluidos

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Implementar un algoritmo de Verlet de velocidad para simular un oscilador armónico 1D [Usar]
2. Evaluar la escala de los errores de discretización para diferentes órdenes de integración numérica [Evaluar]

Core-Tier2:

3. Resolver la ecuación de Laplace en 2D para un conjunto dado de potenciales de frontera mediante la iteración de Jacobi [Usar]

5.10.2. CDP/Métodos y Simulaciones de Monte Carlo

Técnicas de muestreo estocástico para la integración multidimensional y la física estadística.

Temas:

Core Tier1

- Algoritmos de generación de números pseudoaleatorios y cuasialeatorios
- El algoritmo de Metropolis-Hastings para Monte Carlo por Cadenas de Markov (MCMC)
- Simulación de Monte Carlo del modelo de Ising y transiciones de fase

Core Tier2

- Muestreo por importancia y técnicas de reducción de varianza
- Monte Carlo Cinético (KMC) para procesos estocásticos dependientes del tiempo
- Estimación de error estadístico y tiempo de autocorrelación en cadenas MCMC

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Estimar el valor de π utilizando un método de integración de Monte Carlo de acierto o error [Usar]

2. Analizar el efecto del ralentizamiento crítico cerca de la temperatura de Curie en una simulación de Ising [Evaluar]

Core-Tier2:

3. Desarrollar un esquema de muestreo por importancia para integrar una función multidimensional con picos pronunciados [Usar]

5.10.3. CDP/Computación de Alto Rendimiento (HPC) para Simulaciones a Gran Escala

Arquitecturas y modelos de programación para la paralelización masiva de código de física.

Temas:

Core Tier1

- Programación de memoria compartida mediante directivas OpenMP
- Sistemas de memoria distribuida y la Interfaz de Paso de Mensajes (MPI)
- Aceleración por GPU y modelos de programación GPGPU (CUDA, OpenCL)

Core Tier2

- Análisis de escalabilidad: Ley de Amdahl y Ley de Gustafson
- Métodos de descomposición de dominio para la paralelización de mallas espaciales
- Optimización para la localidad de caché y límites de ancho de banda de memoria

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Paralelizar una adición de vectores a gran escala utilizando sincronización de hilos OpenMP [Usar]

Core-Tier2:

2. Medir la aceleración (speedup) y la eficiencia paralela de un código en múltiples nodos de cómputo [Evaluar]
3. Implementar comunicación MPI no bloqueante para un intercambio de celdas fantasma en una simulación de fluidos [Usar]

5.10.4. CDP/Aprendizaje Automático para el Descubrimiento Físico

Aplicación del aprendizaje estadístico para identificar patrones y modelos subrogados en los datos.

Temas:

Core Tier1

- Aprendizaje supervisado: Regresión lineal, Lasso y Ridge para modelos físicos
- Aprendizaje no supervisado: Clustering y PCA para la detección de transiciones de fase
- Ingeniería de características físicas y reducción dimensional

Core Tier2

- Regresión por Procesos Gaussianos para modelado subrogado consciente de la incertidumbre
- Regresión simbólica para el descubrimiento de leyes físicas analíticas a partir de datos
- Métodos de kernel para la aproximación de estados cuánticos de muchos cuerpos

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Entrenar un modelo de regresión para predecir la superficie de energía potencial de una molécula [Usar]
2. Evaluar el sobreajuste (overfitting) del modelo mediante validación cruzada en un conjunto de datos espectroscópicos [Evaluar]

Core-Tier2:

3. Aplicar Análisis de Componentes Principales (PCA) para visualizar configuraciones de espín de alta dimensión [Usar]

5.10.5. CDP/Análisis de Datos y Cuantificación de la Incertidumbre

Marcos estadísticos para la validación de modelos y la cuantificación de errores experimentales.

Temas:

Core Tier1

- Inferencia bayesiana: Priores, verosimilitudes y distribuciones a posteriori
- Estimación de Máxima Verosimilitud (MLE) para el ajuste de parámetros
- Propagación lineal y no lineal de incertidumbres experimentales

Core Tier2

- Técnicas de remuestreo: Métodos de Bootstrapping y Jackknife
- Criterios de selección de modelos: AIC, BIC y razones de evidencia
- Análisis de sensibilidad global e índices de Sobol

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Realizar un ajuste de parámetros bayesiano para un conjunto de datos de desintegración radiactiva [Usar]
2. Calcular intervalos de confianza para una constante física derivada de mediciones ruidosas [Evaluar]

Core-Tier2:

3. Calcular la matriz de Información de Fisher para determinar los límites de precisión de una estimación de parámetros [Usar]

5.10.6. CDP/Modelado de Sistemas Adaptativos Complejos

Dinámica de sistemas con comportamiento emergente e interacciones no lineales.

Temas:

Core Tier2

- Modelado basado en agentes (ABM) para sistemas sociales y biológicos
- Ciencia de redes: Dinámica de redes de mundo pequeño y libres de escala
- Criticidad autoorganizada (SOC) y distribuciones de ley de potencia
- Autómatas celulares: Juego de la Vida y modelos de flujo de fluidos (LBM)
- Sincronización colectiva y el modelo de Kuramoto

Alternativo

- Teoría de juegos evolutiva y estabilidad ecológica

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Simular el flujo de un fluido en 2D alrededor de un obstáculo mediante el método de Lattice Boltzmann [Usar]
2. Analizar la robustez de una red libre de escala frente a ataques dirigidos [Evaluar]
3. Modelar una pila de arena para demostrar los principios de la criticidad autoorganizada [Usar]

5.10.7. CDP/Redes Neuronales para Sistemas Cuánticos

Representación y resolución de problemas cuánticos de muchos cuerpos mediante redes neuronales artificiales.

Temas:

Core Tier2

- Máquinas de Boltzmann Restringidas (RBM) como Estados Cuánticos Neuronales (NQS)
- Monte Carlo Variacional utilizando funciones de onda de prueba de redes neuronales
- Tomografía de estado cuántico basada en redes neuronales
- CNNs para identificar fases topológicas en redes de espín
- Autoencoders para comprimir datos de funciones de onda de muchos cuerpos

Alternativo

- Modelos generativos (GANs, VAEs) para el muestreo de configuraciones cuánticas

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Entrenar una RBM para aproximar el estado fundamental de una cadena de Ising de campo transversal [Usar]
2. Comparar la precisión de los Estados Cuánticos Neuronales frente a los métodos tradicionales de Hartree-Fock [Evaluar]

3. Reconstruir una matriz de densidad a partir de datos de medición experimentales utilizando una red neuronal [Usar]

5.10.8. CDP/Redes Neuronales Informadas por la Física (PINNs)

Integración de leyes físicas directamente en las funciones de pérdida de arquitecturas de aprendizaje profundo.

Temas:

Core Tier1

- Incorporación de PDEs como residuales en las funciones de pérdida de redes neuronales
- Aprovechamiento de la diferenciación automática para restricciones de derivadas exactas

Core Tier2

- Aprendizaje eficiente en datos: Resolución de problemas inversos con datos escasos
- Aplicación de PINNs a las ecuaciones de Navier-Stokes
- PINNs bayesianas para la cuantificación de la incertidumbre en soluciones de PDE

Alternativo

- Introducción a los Operadores Neuronales (DeepONet) para PDEs paramétricas

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Resolver la ecuación de Burgers 1D utilizando una arquitectura PINN [Usar]

Core-Tier2:

2. Comparar el coste computacional y la precisión de una PINN frente a un resolvedor tradicional de Elementos Finitos [Evaluar]
3. Usar una PINN para estimar el coeficiente de viscosidad de un fluido a partir de datos de visualización de flujo [Usar]

5.11. Biofísica y Física Médica (BMP)

La Biofísica y la Física Médica aplican las teorías y métodos de la física para comprender los sistemas biológicos y mejorar la salud humana. Desde las fuerzas mecánicas dentro de una sola célula hasta la compleja física de las radiaciones utilizada en la terapia contra el cáncer.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
5.11.1 Física de las Macromoléculas Biológicas (Pág. 224)			No
5.11.2 Biomecánica y Fuerzas Celulares (Pág. 225)			No
5.11.3 Física Neuronal y Procesamiento de Información en el Cerebro (Pág. 225)			No
5.11.4 Física de Imágenes (MRI, CT, PET, Ultrasonido) (Pág. 225)			No
5.11.5 Radioterapia y Dosimetría (Pág. 226)			No
5.11.6 Mecánica Estadística de Redes Biológicas (Pág. 226)			No
5.11.7 Microfluídica y Lab-on-a-Chip (Pág. 227)			No

5.11.1. BMP/Física de las Macromoléculas Biológicas

Propiedades estructurales y termodinámicas del ADN, proteínas y lípidos.

Temas:

Core Tier1

- Plegamiento de proteínas: Energética, entropía y la paradoja de Levinthal
- Elasticidad del ADN: El modelo de cadena tipo gusano (WLC)
- Mecánica de las bicapas lipídicas y transiciones de fase en membranas

Core Tier2

- Electrostática en células: Teoría de Poisson-Boltzmann y Debye-Hückel
- Motores moleculares: Kinesina y Miosina como trinquetes brownianos
- Técnicas de molécula individual: Pinzas ópticas y magnéticas

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Calcular la longitud de persistencia de una molécula de ADN a partir de una curva de fuerza-extensión [Usar]
2. Evaluar la estabilidad térmica de una proteína utilizando su paisaje de energía libre de Gibbs [Evaluar]

Core-Tier2:
3. Modelar la eficiencia de un motor molecular utilizando un enfoque estocástico de Langevin [Usar]

5.11.2. BMP/Biomecánica y Fuerzas Celulares

Fuerzas físicas que rigen la forma, motilidad y adhesión celular.
Temas:
Core Tier1
- Elasticidad del citoesqueleto: Filamentos de actina y microtúbulos
- Modelos viscoelásticos de células: Representaciones de Maxwell y Kelvin-Voigt
- Principios de la Microscopía de Fuerza de Tracción (TFM)

Core Tier2
- Mecanismos de motilidad celular: Protrusión y retracción de lamelipodios
- Mecanotransducción: Cómo las células convierten estímulos mecánicos en señales químicas
- Presión osmótica y la regulación del volumen celular

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Calcular el módulo de Young de una célula utilizando datos de Microscopía de Fuerza Atómica (AFM) [Usar]

Core-Tier2:
2. Analizar la deformación por fluencia (creep) de una membrana celular bajo estrés constante [Evaluar]
3. Modelar el hinchamiento dependiente del tiempo de una célula en una solución hipotónica [Usar]

5.11.3. BMP/Física Neuronal y Procesamiento de Información en el Cerebro

Propiedades eléctricas y estadísticas de las neuronas y redes neuronales.
Temas:
Core Tier1
- El modelo de Hodgkin-Huxley de la propagación del potencial de acción
- Potenciales de equilibrio: Las ecuaciones de Nernst y de Goldman-Hodgkin-Katz
- Transmisión sináptica y modelos de "Integrate-and-Fire" con fugas

Core Tier2
- Teoría de cables para la atenuación de señales en dendritas y axones
- Codificación neuronal: Estrategias de codificación por tasa frente a codificación temporal
- Redes de Hopfield y la física de la memoria asociativa

Aprendizaje esperado (*Learning Outcomes*):
Core-Tier1:
1. Calcular el potencial de Nernst para iones de potasio a través de una membrana neuronal [Usar]
2. Analizar el papel de los canales dependientes de voltaje en la iniciación de un potencial de acción [Evaluar]

Core-Tier2:
3. Aplicar la ecuación de cable para determinar la constante de longitud de una dendrita pasiva [Usar]

5.11.4. BMP/Física de Imágenes (MRI, CT, PET, Ultrasonido)

Principios físicos de la visualización no invasiva del cuerpo humano.
Temas:
Core Tier1

- Atenuación de rayos X, ley de Beer-Lambert y Tomografía Computarizada (CT)
- Resonancia Magnética Nuclear (NMR) y las ecuaciones de Bloch para MRI
- Mecanismos de contraste: Tiempos de relajación T1 y T2 en MRI
- Impedancia acústica y reflexión en ultrasonido médico

Core Tier2

- Física de la Tomografía por Emisión de Positrones (PET) y detección por coincidencia
- Reconstrucción de imágenes: Retroproyección filtrada y métodos de Fourier

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular el número CT (unidades Hounsfield) para hueso y tejido blando [Usar]
2. Contrastar la apariencia de un tumor en imágenes de MRI ponderadas en T1 frente a T2 [Evaluar]

Core-Tier2:

3. Determinar la resolución axial de una sonda de ultrasonido basada en la duración del pulso [Usar]

5.11.5. BMP/Radioterapia y Dosimetría

Interacción de la radiación ionizante con el tejido para fines terapéuticos.

Temas:

Core Tier1

- Efecto fotoeléctrico, dispersión de Compton y producción de pares en el tejido
- Definiciones de Dosis Absorbida (Gray), Dosis Equivalente (Sievert) y KERMA
- Poder de frenado y el pico de Bragg en protonterapia

Core Tier2

- Radiobiología: El modelo lineal-cuadrático para la supervivencia celular
- Principios de la Radioterapia de Intensidad Modulada (IMRT)
- Protocolos de dosimetría y calibración de cámaras de ionización

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la dosis absorbida a una profundidad específica utilizando una curva de Dosis en Profundidad Porcentual (PDD) [Usar]
2. Comparar la distribución de dosis de un haz de rayos X de 6MV frente a un haz de protones de 200 MeV [Evaluar]

Core-Tier2:

3. Aplicar el modelo lineal-cuadrático para estimar el efecto del fraccionamiento de la dosis [Usar]

5.11.6. BMP/Mecánica Estadística de Redes Biológicas

Comportamiento colectivo en la regulación genética, el metabolismo e interacciones proteicas.

Temas:

Core Tier2

- Dinámica de redes de regulación genética: La ecuación de Hill y cooperatividad
- Modelado basado en restricciones y Análisis de Flujo Metabólico (MFA)
- Algoritmo de Gillespie para la expresión genética estocástica
- Identificación de motivos de red: Bucles de retroalimentación y alimentación directa
- Teoría de la información en la señalización biológica: Información mutua y capacidad
- Adaptación perfecta y robustez en la señalización por quimiotaxis

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Modelar un interruptor genético (toggle switch) utilizando un par de ecuaciones diferenciales acopladas [Usar]
2. Analizar cómo los bucles de retroalimentación negativa suprimen el ruido estocástico en la expresión genética [Evaluar]
3. Calcular la capacidad de información de un sensor bioquímico simple [Usar]

5.11.7. BMP/Microfluídica y Lab-on-a-Chip

Física de fluidos a escala micrométrica para el análisis biológico.

Temas:

Core Tier1

- Flujo a bajo número de Reynolds y la reversibilidad del flujo de Stokes
- Tensión superficial, ecuación de Young-Laplace y acción capilar
- Flujo electroosmótico y potencial Zeta en microcanales

Core Tier2

- Microfluídica basada en gotas y cribado de alto rendimiento
- Mezclado a microescala: Número de Peclet y límites difusivos
- Principios de Organ-on-a-Chip: Imitación de microambientes fisiológicos

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la altura de ascenso capilar de la sangre en un tubo de vidrio de 50 micrómetros [Usar]

Core-Tier2:

2. Evaluar el tiempo requerido para que una proteína se difunda a través de un canal microfluídico [Evaluar]
3. Diseñar una geometría de micro-mezclador para optimizar la eficiencia de mezclado a $Re < 1$ [Usar]

5.12. Astrofísica y Ciencias del Espacio (ASS)

La Astrofísica y las Ciencias del Espacio aplican las leyes de la física al estudio del universo. Este área abarca el nacimiento, vida y muerte de las estrellas, la formación de sistemas planetarios, la estructura de las galaxias y los fenómenos de alta energía.

Unidad de Conocimiento (KU)	Core Tier1	Core Tier2	Alternativo
5.12.1 Estructura y Evolución Estelar (Pág. 227)			No
5.12.2 Ciencia Planetaria y Exoplanetas (Pág. 228)			No
5.12.3 Dinámica y Estructura Galáctica (Pág. 228)			No
5.12.4 Astrofísica de Altas Energías (Rayos X, Rayos Gamma) (Pág. 228)			No
5.12.5 Técnicas Observacionales y Telescopios (Pág. 229)			No
5.12.6 Física Solar y Clima Espacial (Pág. 229)			No
5.12.7 Astronomía de Multimensajeros (Ondas Gravitacionales + Neutrinos) (Pág. 229)			No

5.12.1. ASS/Estructura y Evolución Estelar

Los procesos físicos que rigen los ciclos de vida de las estrellas.

Temas:

Core Tier1

- Equilibrio hidrostático y el Teorema del Virial para estrellas
- Nucleosíntesis estelar: cadena pp, ciclo CNO y proceso triple alfa
- El diagrama de Hertzsprung-Russell (H-R) y clasificación estelar
- Transporte de energía: Transferencia radiativa y zonas de convección

Core Tier2

- Presión de degeneración de electrones/neutrones y los límites de Chandrasekhar y TOV
- Evolución post-secuencia principal: Gigantes rojas, enanas blancas y supernovas

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Estimar el tiempo de vida en la secuencia principal de una estrella basado en su masa y luminosidad [Usar]

2. Analizar por qué las estrellas de alta masa tienen núcleos convectivos mientras que las de baja masa tienen núcleos radiativos [Evaluar]

Core-Tier2:

3. Determinar la masa máxima de una enana blanca no rotatoria [Usar]

5.12.2. ASS/Ciencia Planetaria y Exoplanetas

Formación y características de los planetas dentro y fuera de nuestro Sistema Solar.

Temas:

Core Tier1

- La Hipótesis Nebular: Acreción y formación de planetesimales
- Detección de exoplanetas: Fotometría de tránsito y análisis de curvas de luz
- Método de velocidad radial: Desplazamientos Doppler y el movimiento reflejo de las estrellas

Core Tier2

- Composición atmosférica, efecto invernadero y la Zona Habitable
- Interacciones de marea: Migración orbital y rotación síncrona
- Imágenes directas de alto contraste e interferometría de anulación

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular el radio de un exoplanet dada la profundidad del tránsito de su estrella anfitriona [Usar]
2. Evaluar si un planeta a 0.5 UA de una estrella tipo K se encuentra en la zona habitable [Evaluar]

Core-Tier2:

3. Estimar la masa mínima ($M \sin i$) de un planeta a partir de la amplitud de su velocidad radial [Usar]

5.12.3. ASS/Dinámica y Estructura Galáctica

La organización, cinemática y evolución de las galaxias.

Temas:

Core Tier1

- La secuencia de Hubble: Galaxias elípticas, espirales e irregulares
- Curvas de rotación galáctica y evidencia de la Materia Oscura
- Núcleos Galácticos Activos (AGN) y agujeros negros supermasivos

Core Tier2

- Cinemática estelar y las ecuaciones de Jeans para el equilibrio galáctico
- Teoría de la estructura espiral: Ondas de densidad frente a formación estocástica
- Interacciones de galaxias, fusiones y formación jerárquica de estructuras

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la masa de materia oscura requerida para explicar una curva de rotación plana a 20 kpc [Usar]
2. Comparar las poblaciones estelares y el contenido de gas de las galaxias elípticas frente a las espirales [Evaluar]

Core-Tier2:

3. Aplicar el Teorema del Virial para estimar la masa total de un cúmulo de galaxias [Usar]

5.12.4. ASS/Astrofísica de Altas Energías (Rayos X, Rayos Gamma)

Procesos físicos extremos y aceleradores cósmicos.

Temas:

Core Tier1

- Física de discos de acreción: Luminosidad de Eddington
- Emisión no térmica: Radiación de sincrotrón y de curvatura
- Dispersión Compton inversa y el efecto Sunyaev-Zel'dovich

Core Tier2

- Brotes de rayos gamma (GRB): Hipernovas y fusiones de estrellas de neutrones

- Física de púlsares: Radiación de dipolo magnético y frenado por rotación
- Mecanismo de aceleración de Fermi en remanentes de supernova

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la luminosidad de Eddington para un agujero negro de 10 masas solares [Usar]

Core-Tier2:

2. Analizar la evolución del periodo de un púlsar debido a la pérdida de energía por radiación de dipolo [Evaluar]
3. Determinar la energía máxima de un fotón dispersado por electrones ultra-relativistas [Usar]

5.12.5. ASS/Técnicas Observacionales y Telescopios

Métodos para detectar y medir la radiación electromagnética del espacio.

Temas:

Core Tier1

- Telescopios refractores y reflectores: Apertura y poder de resolución
- Principios de los detectores CCD, eficiencia cuántica y fuentes de ruido
- Técnicas espectroscópicas: Dispersión, resolución y perfiles de línea

Core Tier2

- Radioastronomía: Síntesis de apertura y técnicas de VLBI
- Óptica adaptativa: Sensor de frente de onda y espejos deformables
- Instrumentación para observatorios espaciales de rayos X y rayos gamma

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la relación señal-ruido para una estrella dado un tiempo de exposición y fondo de cielo [Usar]
2. Evaluar el límite de difracción teórico de un telescopio terrestre de 30 metros [Evaluar]

Core-Tier2:

3. Diseñar una configuración de arreglo interferométrico para lograr una resolución angular específica a 1.4 GHz [Usar]

5.12.6. ASS/Física Solar y Clima Espacial

La actividad del Sol y su impacto en la heliosfera y la Tierra.

Temas:

Core Tier1

- Estructura solar: Núcleo, zona radiativa, zona convectiva y fotosfera
- El ciclo solar de 11 años y el modelo de dinamo de Babcock-Leighton
- El viento solar: Teoría de Parker de la expansión isotérmica

Core Tier2

- Llamaradas solares y eyecciones de masa coronal (CME): Reconexión magnética
- Clima espacial: Tormentas geomagnéticas y perturbaciones ionosféricas
- Helioseismología: Uso de ondas acústicas para sondear el interior solar

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier1:

1. Calcular la constante solar (irradiancia) a la distancia de Marte [Usar]

Core-Tier2:

2. Analizar cómo la reconexión magnética desencadena la liberación de energía en las llamaradas solares [Evaluar]
3. Modelar la velocidad del viento solar como función de la distancia radial al Sol [Usar]

5.12.7. ASS/Astronomía de Multimensajeros (Ondas Gravitacionales + Neutrinos)

Integración de señales electromagnéticas, gravitacionales y de partículas de eventos cósmicos.

Temas:

Core Tier2

- Fuentes de ondas gravitacionales: Fusiones de sistemas binarios de agujeros negros y estrellas de neutrones
- Interferometría LIGO/Virgo: Configuración Michelson con cavidades Fabry-Perot
- Neutrinos cósmicos de alta energía y detección en IceCube
- Sirenas estándar: Uso de señales GW para medir la constante de Hubble
- La conexión física entre rayos cósmicos de ultra-alta energía y neutrinos

Alternativo

- Detección de GW en el espacio: LISA y arreglos de cronometría de púlsares

Aprendizaje esperado (*Learning Outcomes*):

Core-Tier2:

1. Calcular la masa de chirp de un sistema binario a partir de la evolución de su frecuencia de onda gravitacional [Usar]
2. Evaluar el flujo de neutrinos esperado de una supernova por colapso de núcleo cercana [Evaluar]
3. Estimar la distancia de luminosidad a una fusión de estrellas de neutrones binarias usando datos de deformación de GW [Usar]

Capítulo 6

Plan de estudios

6.1. Codificación de los cursos

Los cursos son clasificados en 4 niveles:

- **Introductorios** Código 1.. ejemplo: CS100
- **Intermedios** Código 2.. ejemplo: MA200
- **Avanzados** Código 3.. y ejemplo: IS370
- **Trabajo de final de carrera** Código 4.. ejemplo: CE403

Los cursos se encuentran codificados bajo el esquema que se muestra en la Figura 6.1.

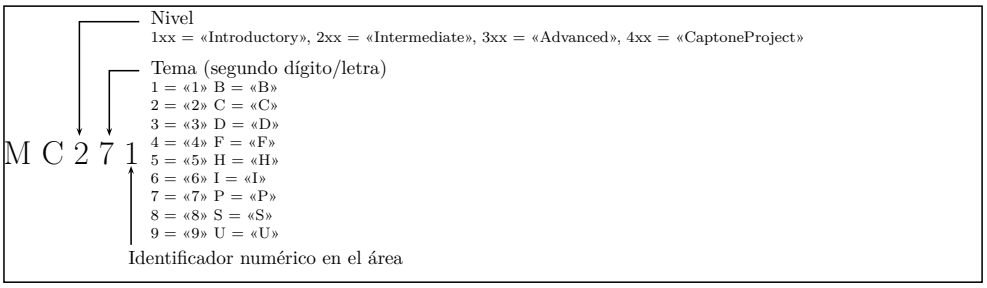


Figura 6.1: Esquema de codificación para los cursos.

El tipo de curso esta determinado por las letras iniciales. Los posibles códigos para estos tipos son:

1. **CS** Área de Ciencia de la Computación (*Computer Science* – CS);
2. **AI** Área de Inteligencia Artificial (*Artificial Intelligence* – AI);
3. **MA** Matemáticas (MA);
4. **STA** Estadística (STA);
5. **FI** Área de Física;
6. **QC** Computación Cuántica
7. **FG** Área de Formación General;
8. **ID** Área de Idiomas.

Al mismo tiempo, los cursos están clasificados en:

- Estudios generales
- Específicos
- Especialización

6.2. Estructura Curricular

La relación de cursos se muestra a continuación:

Primer Semestre							
Código	**Curso**	**Área**	**HT**	**HP**	**CR**	**TIPO**	**Requisitos**
CS111	Introducción a la Programación (Pág 253)	Específicos	3	4	5	O	
MA100	Matemática Básica (Pág 253)	Estudios generales	3	4	5	O	
FG101	Lenguaje (Pág 253)	Estudios generales	1	2	2	O	
FG105	Género y Sociedad (Pág 253)	Estudios generales	1	2	2	O	
FG109	Ambiente y Desarrollo Sostenible (Pág 253)	Estudios generales	2	2	3	O	
FG112	Métodos de gestión personal universitaria (Pág 254)	Estudios generales	1	2	2	O	
FG140	Fundamentos de Investigación Científica (Pág 254)	Estudios generales	2	2	3	O	
					22		

Segundo Semestre							
Código	**Curso**	**Área**	**HT**	**HP**	**CR**	**TIPO**	**Requisitos**
CS100	Introducción a la Ciencia de la Computación (Pág 254)	Específicos	2	2	3	O	
CS112	Programación Orientada a Objetos I (Pág 254)	Específicos	2	4	4	O	CS111 (1er Sem)
AI161	IA Aplicada (Pág 254)	Específicos	2	4	4	O	CS111 (1er Sem)
MA111	Cálculo I (Pág 255)	Estudios generales	3	4	5	O	MA100 (1er Sem)
FI101	Física General (Pág 255)	Estudios generales	3	4	5	O	MA111 (2do Sem)
					21		

Tercer Semestre							
Código	**Curso**	**Área**	**HT**	**HP**	**CR**	**TIPO**	**Requisitos**
CS113	Programación Orientada a Objetos II (Pág 255)	Específicos	1	4	3	O	CS112 (2do Sem)
CS1D1	Estructuras Discretas I (Pág 255)	Específicos	2	4	4	O	MA100 (1er Sem)
CS2B1	Desarrollo Basado en Plataformas (Pág 255)	Específicos	2	4	4	O	CS112 (2do Sem)
MA112	Cálculo II (Pág 255)	Estudios generales	3	2	4	O	MA111 (2do Sem)
MA121	Algebra Lineal I (Pág 255)	Específicos	2	4	4	O	MA100 (1er Sem)
ID101	Inglés I (Pág 256)	Estudios generales	1	4	3	O	
					22		

Cuarto Semestre							
Código	**Curso**	**Área**	**HT**	**HP**	**CR**	**TIPO**	**Requisitos**
CS1D2	Estructuras Discretas II (Pág 256)	Específicos	2	2	3	O	CS1D1 (3er Sem)
CS210	Algoritmos y Estructuras de Datos (Pág 256)	Específicos	2	4	4	O	CS113 (3er Sem)
CS271	Bases de Datos I (Pág 256)	Específicos	1	4	3	O	CS112 (2do Sem), CS1D1 (3er Sem)
MA122	Algebra Lineal II (Pág 256)	Específicos	2	4	4	O	MA121 (3er Sem)
MA210	Cálculo III (Pág 256)	Estudios generales	2	4	4	O	MA112 (3er Sem)
ID102	Inglés II (Pág 256)	Estudios generales	1	6	4	O	ID101 (3er Sem)
					22		

Quinto Semestre							
Código	**Curso**	**Área**	**HT**	**HP**	**CR**	**TIPO**	**Requisitos**
CS251	Computación Gráfica (Pág 257)	Especialización	1	4	3	O	CS210 (4to Sem)
MA211	Cálculo IV (Pág 257)	Específicos	1	4	3	O	MA210 (4to Sem), MA121 (3er Sem)
MA221	Análisis Numérico I[1] (Pág 257)	Específicos	1	4	3	O	MA112 (3er Sem), MA122 (4to Sem), AI161 (2do Sem)
MA222	Análisis Real I (Pág 257)	Específicos	3	4	5	O	MA112 (3er Sem), ID101 (3er Sem)
MA251	Introducción a la Estadística (Pág 257)	Estudios generales	1	4	3	O	AI161 (2do Sem)
MA281	Teoría de números (Pág 257)	Estudios generales	1	2	2	O	MA100 (1er Sem)
ID201	Inglés III (Pág 257)	Estudios generales	1	4	3	O	ID102 (4to Sem)
					22		

Sexto Semestre							
Código	**Curso**	**Área**	**HT**	**HP**	**CR**	**TIPO**	**Requisitos**
MA212	Ecuaciones Diferenciales Ordinarias (Pág 258)	Específicos	4	4	6	O	CS112 (2do Sem), MA121 (3er Sem), MA222 (5to Sem)
MA325	Análisis Numérico II (Pág 258)	Específicos	2	4	4	O	MA221 (5to Sem)
MA371	Análisis funcional I (Pág 258)	Específicos	3	4	5	O	MA222 (5to Sem), ID201 (5to Sem), MA122 (4to Sem)
MA351	Estadística y Probabilidades (Pág 258)	Específicos	2	4	4	O	MA251 (5to Sem), CS112 (2do Sem)
ID202	Inglés IV (Pág 258)	Estudios generales	1	4	3	O	ID201 (5to Sem)
					22		

[1]Se recomienda que el alumno haya llevado: line:189 CS210-CS2023 Algoritmos y Estructuras de Datos(4to Sem).

Séptimo Semestre							
Código	**Curso**	**Área**	**HT**	**HP**	**CR**	**TIPO**	**Requisitos**
CS261	Inteligencia Artificial (Pág 258)	Especialización	1	4	3	O	MA251 (5to Sem)
CS3P1	Computación Paralela y Distribuída (Pág 258)	Especialización	1	4	3	O	CS210 (4to Sem)
MA213	Ecuaciones Diferenciales Parciales (Pág 259)	Específicos	2	4	4	O	MA212 (6to Sem), MA211 (5to Sem)
MA326	Análisis Numérico III (Pág 259)	Específicos	2	4	4	O	MA325 (6to Sem)
MA361	Modelado Matemático (Pág 259)	Estudios generales	2	4	4	O	MA325 (6to Sem)
ID203	Inglés V (Pág 259)	Estudios generales	1	6	4	O	ID202 (6to Sem)
					22		

	Octavo Semestre							
	Código	**Curso**	**Área**	**HT**	**HP**	**CR**	**TIPO**	**Requisitos**
	AI263	Introducción al Aprendizaje de Máquina (Pág 259)	Específicos	2	4	4	O	CS261 (7mo Sem)
	MA401	Proyecto de final de carrera I (Pág 259)	Especialización	1	4	3	O	MA361 (7mo Sem), MA371 (6to Sem), 140cr
Alternativos	CS2S1	Sistemas Operativos (Pág 260)	Específicos	3	4	5	E	CS210 (4to Sem)
Alternativos	CS3I1	Seguridad en Computación (Pág 260)	Específicos	3	4	5	E	CS210 (4to Sem)
	QC251	Mecánica Cuántica (Pág 260)	Especialización	4	2	5	E	FI101 (2do Sem), MA371 (6to Sem)
Alternativos	CS253	Análisis exploratorio de datos (Pág 260)	Especialización	3	4	5	E	MA351 (6to Sem)
Alternativos	CS370	Big Data y Analítica de Datos (Pág 260)	Especialización	3	4	5	E	CS271 (4to Sem)
Alternativos	STA252	Estadística Inferencial (Pág 260)	Especialización	3	4	5	E	MA351 (6to Sem)
Alternativos	MA327	Análisis Numérico IV (Pág 260)	Específicos	4	2	5	E	MA326 (7mo Sem), MA213 (7mo Sem)
Alternativos	MA352	Procesos estocásticos (Pág 261)	Específicos	3	4	5	E	MA351 (6to Sem), MA210 (4to Sem), CS261 (7mo Sem)
Alternativos	MA372	Análisis funcional II (Pág 261)	Específicos	3	4	5	E	MA371 (6to Sem)
						22		

	Noveno Semestre							
	Código	**Curso**	**Área**	**HT**	**HP**	**CR**	**TIPO**	**Requisitos**
	AI264	Aprendizaje Profundo (Pág 261)	Específicos	3	4	5	O	AI263 (8vo Sem)
	MA402	Proyecto de final de carrera II (Pág 261)	Especialización	1	8	5	O	MA401 (8vo Sem), ID203 (7mo Sem)
Alternativos	CS231	Redes y Comunicación (Pág 261)	Específicos	2	4	4	E	CS2S1 (8vo Sem)
	MA381	Estructuras Algebraicas (Pág 261)	Estudios generales	2	4	4	E	MA281 (5to Sem)
Alternativos	AI262	Procesamiento de Lenguaje Natural (Pág 261)	Específicos	2	4	4	E	AI263 (8vo Sem)
	CS273	Análisis de series temporales (Pág 262)	Especialización	2	4	4	E	MA351 (6to Sem)
Alternativos	MA341	Optimización I (Pág 262)	Especialización	2	4	4	E	MA221 (5to Sem)
	MA355	Teoría de Aprendizaje Estadístico (Pág 262)	Estudios generales	2	4	4	E	MA352 (8vo Sem), MA372 (8vo Sem)
						22		

	Décimo Semestre							
	Código	**Curso**	**Área**	**HT**	**HP**	**CR**	**TIPO**	**Requisitos**
	CS400	Prácticas Pre-profesionales (Pág 262)	Especialización	1	4	3	O	110Cr
	AI268	Visión Computacional (Pág 262)	Especialización	3	4	5	O	AI264 (9no Sem)
	AI367	Tópicos en Inteligencia Artificial (Pág 262)	Especialización	3	4	5	O	AI264 (9no Sem)
	MA403	Proyecto de final de carrera III (Pág 263)	Especialización	1	8	5	O	MA402 (9no Sem)
Alternativos	QC381	Introduction a la Computación Cuántica (Pág 263)	Especialización	2	4	4	E	QC251 (8vo Sem), MA381 (9no Sem)
Alternativos	CS254	Visualización de Datos Avanzada (Pág 263)	Especialización	2	4	4	E	CS253 (8vo Sem)
						26		

Total de créditos de la carrera: 223 .

6.3. Distribución de cursos en la carrera

Esta propuesta puede ser analizada por el número de créditos dedicados a cada área, por niveles de cursos (Introductorios, Intermedios, Avanzados y Proyectos).

	Estudios generales	Específicos	Especialización	
Primer Sem	17	5		22
Segundo Sem	10	11		21
Tercer Sem	7	15		22
Cuarto Sem	8	14		22
Quinto Sem	8	11	3	22
Sexto Sem	3	19		22
Séptimo Sem	8	8	6	22
Octavo Sem		9	13	22
Noveno Sem	8	5	9	22
Décimo Sem			26	26
Total	69	97	57	223
	30.9 %	43.4 %	25.5 %	

Cuadro 6.1: Distribución de cursos por áreas

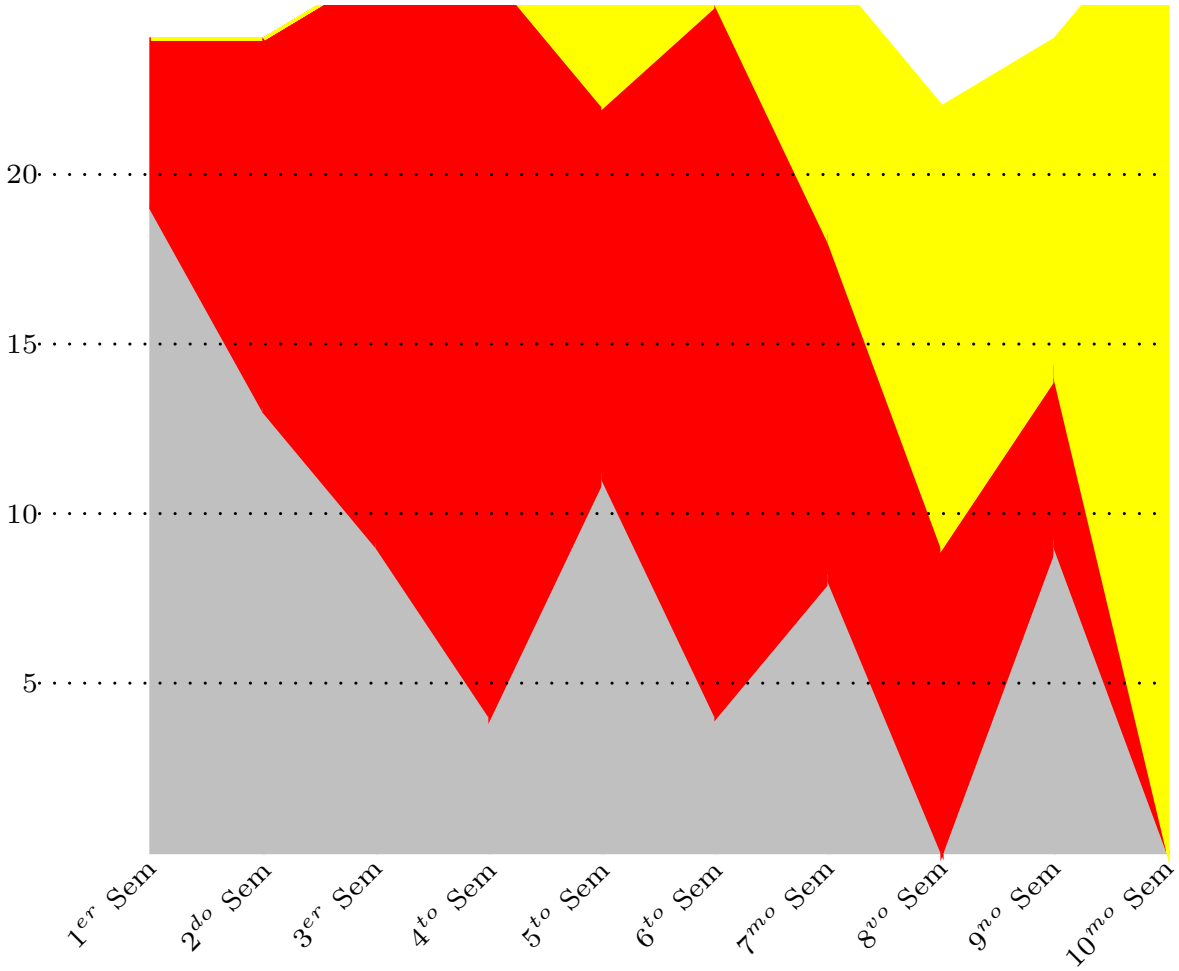


Figura 6.2: Créditos por área por semestre

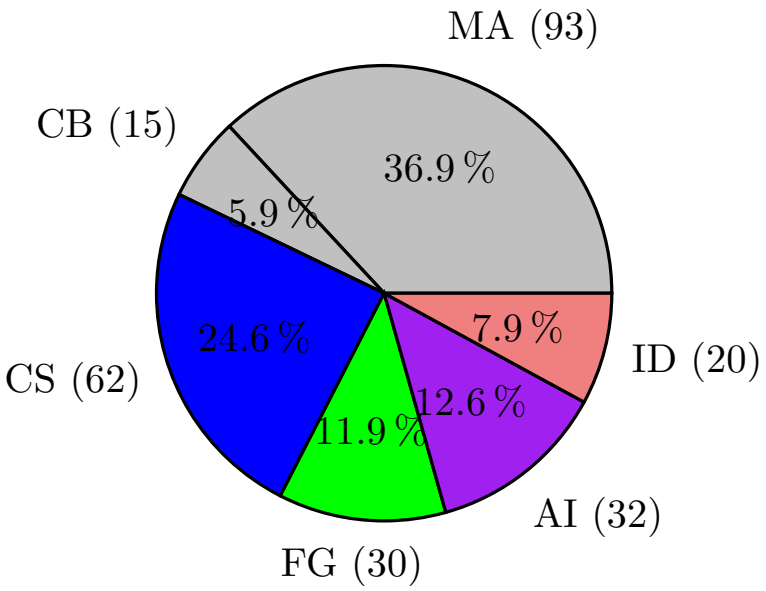


Figura 6.3: Distribución de cursos por áreas considerando creditaje.

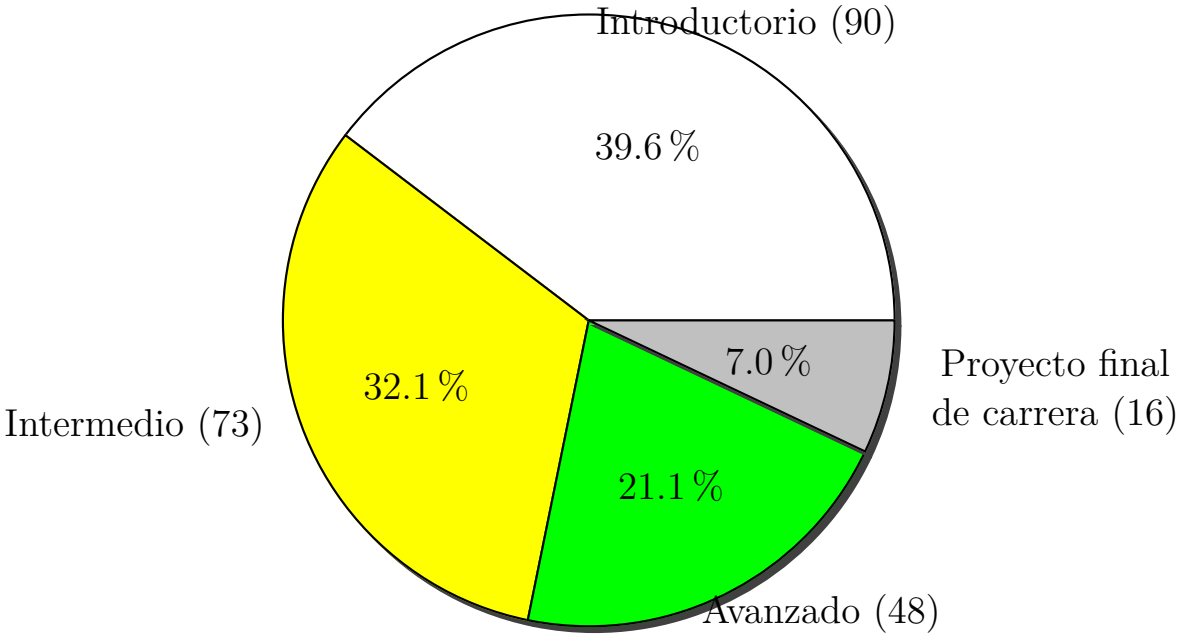


Figura 6.4: Distribución de créditos por niveles de cursos.

Referencias Bibliográficas

Capítulo 7

Equivalencias con otros planes curriculares

A continuación se pueden observar las equivalencias de la presente malla con el(los) Plan(es) curricular(es) anterior(ES). Para ver mayores detalles de la malla propuesta observar la sección 6.2 Pág. 232.

7.1. Equivalencia del Plan 2018 al Plan2026

Primer Semestre – Plan 2018			**Plan2026**			
Código	**Curso**	**Cr**	**Código**	**Curso**	**Sem**	**Cr**
CBO106	Biología					
CBO103	Gestión Personal					
CBO102	Métodos de Estudio Universitario	3	FG112	Métodos de gestión personal universitaria	1er Sem	2
CBO101	Lenguaje	4	FG101	Lenguaje	1er Sem	2
CBE004	Género y Sociedad					
CBO105	Matemática Básica	4	MA100	Matemática Básica	1er Sem	5
CBO104	Cálculo I	4	MA111	Cálculo I	2do Sem	5

Segundo Semestre – Plan 2018			Plan2026			
Código	**Curso**	**Cr**	**Código**	**Curso**	**Sem**	**Cr**
CBO206	Física General	4	FI101	Física General	2do Sem	5
CBO205	Química Inorgánica y Orgánica					
CBE003	Economía General					
CBO201	Fundamentos de Investigación Científica	3	FG140	Fundamentos de Investigación Científica	1er Sem	3
			AI161	IA Aplicada	2do Sem	4
CBO203	Realidad Nacional y Mundial					
CBO204	Cálculo II	4	MA112	Cálculo II	3er Sem	4
CBO202	Medio Ambiente y Desarrollo Sostenible	4	FG109	Ambiente y Desarrollo Sostenible	1er Sem	3

Tercer Semestre – Plan 2018			Plan2026			
Código	**Curso**	**Cr**	**Código**	**Curso**	**Sem**	**Cr**
183033	Algorítmica y Fundamentos de Programación	6	CS111	Introducción a la Programación	1er Sem	5
			CS2B1	Desarrollo Basado en Plataformas	3er Sem	4
183032	Elementos de Álgebra y Geometría	6	MA121	Algebra Lineal I	3er Sem	4
183034	Elementos de Matemática Discreta y Computacional	4	CS1D1	Estructuras Discretas I	3er Sem	4
183031	Cálculo III	6	MA210	Cálculo III	4to Sem	4
183035	Inglés I	3	ID101	Inglés I	3er Sem	3

Cuarto Semestre – Plan 2018			Plan2026			
Código	**Curso**	**Cr**	**Código**	**Curso**	**Sem**	**Cr**
			CS1D2	Estructuras Discretas II	4to Sem	3
183045	Inglés II	3	ID102	Inglés II	4to Sem	4
183044	Estructura de Datos	6	CS210	Algoritmos y Estructuras de Datos	4to Sem	4
183042	Álgebra	6	MA122	Algebra Lineal II	4to Sem	4
183041	Cálculo IV	6	MA211	Cálculo IV	5to Sem	3
183043	Modelos Matemáticos de la Física					

Quinto Semestre – Plan 2018			Plan2026			
Código	**Curso**	**Cr**	**Código**	**Curso**	**Sem**	**Cr**
183053	Base de Datos e Ingeniería de Software	6	CS271	Bases de Datos I	4to Sem	3
183055	Inglés III	3	ID201	Inglés III	5to Sem	3
			MA251	Introducción a la Estadística	5to Sem	3
183054	Métodos Numéricos y Programación I	5	MA221	Análisis Numérico I	5to Sem	3
183051	Análisis Real	6	MA222	Análisis Real I	5to Sem	5
183052	Ecuaciones Diferenciales Ordinarias	6	MA212	Ecuaciones Diferenciales Ordinarias	6to Sem	6

Sexto Semestre – Plan 2018			Plan2026			
Código	**Curso**	**Cr**	**Código**	**Curso**	**Sem**	**Cr**
183062	Probabilidad e Inferencia Estadística	5	STA252	Estadística Inferencial	8vo Sem	5
183064	Sistemas Operativos	5	CS2S1	Sistemas Operativos	8vo Sem	5
183063	Métodos Númericos y Programación II	6	MA325	Análisis Numérico II	6to Sem	4
183061	Análisis Funcional I	5	MA371	Análisis funcional I	6to Sem	5
183065	Técnicas de Modelamiento	5	MA361	Modelado Matemático	7mo Sem	4
			ID202	Inglés IV	6to Sem	3

Séptimo Semestre – Plan 2018			Plan2026			
Código	**Curso**	**Cr**	**Código**	**Curso**	**Sem**	**Cr**
			ID203	Inglés V	7mo Sem	4
183073	Modelos Matemáticos de la Ciencia I					
183072	Computación Gráfica	6	CS251	Computación Gráfica	5to Sem	3
183071	Análisis Funcional II	5	MA372	Análisis funcional II	8vo Sem	5
183074	Inteligencia Artificial	5	CS261	Inteligencia Artificial	7mo Sem	3

Octavo Semestre – Plan 2018			Plan2026			
Código	**Curso**	**Cr**	**Código**	**Curso**	**Sem**	**Cr**
183084	Proyecto de Investigación	4	MA401	Proyecto de final de carrera I	8vo Sem	3
			MA352	Procesos estocásticos	8vo Sem	5
183081	Ecuaciones Diferenciales Parciales I	6	MA213	Ecuaciones Diferenciales Parciales	7mo Sem	4
183082	Modelos Matemáticos de la Ciencia II					
183123	Mecánica Cuántica	5	QC251	Mecánica Cuántica	8vo Sem	5
183083	Redes, Arquitecturas y Comunicaciones	6	CS231	Redes y Comunicación	9no Sem	4

Noveno Semestre – Plan 2018			**Plan2026**			
Código	**Curso**	**Cr**	**Código**	**Curso**	**Sem**	**Cr**
			MA355	Teoría de Aprendizaje Estadístico	9no Sem	4
183093	Métodos Númericos y Programación III	6	MA326	Análisis Numérico III	7mo Sem	4
183094	Desarrollo del Proyecto de Investigación	5	MA402	Proyecto de final de carrera II	9no Sem	5
183092	Modelos Matemáticos de la Ciencia III					
183091	Optimización	5	MA341	Optimización I	9no Sem	4

Décimo Semestre – Plan 2018			**Plan2026**			
Código	**Curso**	**Cr**	**Código**	**Curso**	**Sem**	**Cr**
			CS254	Visualización de Datos Avanzada	10mo Sem	4
183104	Presentación y Sustentación del Proyecto de Investigación	6	MA403	Proyecto de final de carrera III	10mo Sem	5
183102	Formulación, evaluación y gestión de proyectos					
183101	Modelos Matemáticos de la Ciencia IV					
183103	Práctica Preprofesional	4	CS400	Prácticas Pre-profesionales	10mo Sem	3

7.2. Equivalencia del Plan Plan2026 al 2018

Primer Semestre – Plan2026			Plan 2018			
Código	**Curso**	**Cr**	**Código**	**Curso**	**Sem**	**Cr**
CS111	Introducción a la Programación	5	183033	Algorítmica y Fundamentos de Programación	3er Sem	6
MA100	Matemática Básica	5	CBO105	Matemática Básica	1er Sem	4
FG101	Lenguaje	2	CBO101	Lenguaje	1er Sem	4
FG109	Ambiente y Desarrollo Sostenible	3	CBO202	Medio Ambiente y Desarrollo Sostenible	2do Sem	4
FG112	Métodos de gestión personal universitaria	2	CBO102	Métodos de Estudio Universitario	1er Sem	3
FG140	Fundamentos de Investigación Científica	3	CBO201	Fundamentos de Investigación Científica	2do Sem	3

Segundo Semestre – Plan2026			Plan 2018			
Código	**Curso**	**Cr**	**Código**	**Curso**	**Sem**	**Cr**
CS100	Introducción a la Ciencia de la Computación	3			2do Sem	
CS112	Programación Orientada a Objetos I	4			2do Sem	
AI161	IA Aplicada	4			2do Sem	
MA111	Cálculo I	5	CBO104	Cálculo I	1er Sem	4
FI101	Física General	5	CBO206	Física General	2do Sem	4

Tercer Semestre – Plan2026			Plan 2018			
Código	**Curso**	**Cr**	**Código**	**Curso**	**Sem**	**Cr**
CS113	Programación Orientada a Objetos II	3			3er Sem	
CS1D1	Estructuras Discretas I	4	183034	Elementos de Matemática Discreta y Computacional	3er Sem	4
CS2B1	Desarrollo Basado en Plataformas	4			3er Sem	
MA112	Cálculo II	4	CBO204	Cálculo II	2do Sem	4
MA121	Algebra Lineal I	4	183032	Elementos de Álgebra y Geometría	3er Sem	6
ID101	Inglés I	3	183035	Inglés I	3er Sem	3

Cuarto Semestre – Plan2026			Plan 2018			
Código	**Curso**	**Cr**	**Código**	**Curso**	**Sem**	**Cr**
CS1D2	Estructuras Discretas II	3			4to Sem	
CS210	Algoritmos y Estructuras de Datos	4	183044	Estructura de Datos	4to Sem	6
CS271	Bases de Datos I	3	183053	Base de Datos e Ingeniería de Software	5to Sem	6
MA122	Algebra Lineal II	4	183042	Álgebra	4to Sem	6
MA210	Cálculo III	4	183031	Cálculo III	3er Sem	6
ID102	Inglés II	4	183045	Inglés II	4to Sem	3

Quinto Semestre – Plan2026			Plan 2018			
Código	**Curso**	**Cr**	**Código**	**Curso**	**Sem**	**Cr**
CS251	Computación Gráfica	3	183072	Computación Gráfica	7mo Sem	6
MA211	Cálculo IV	3	183041	Cálculo IV	4to Sem	6
MA221	Análisis Numérico I	3	183054	Métodos Numéricos y Programación I	5to Sem	5
MA222	Análisis Real I	5	183051	Análisis Real	5to Sem	6
MA251	Introducción a la Estadística	3			5to Sem	
MA281	Teoría de números	2			5to Sem	
ID201	Inglés III	3	183055	Inglés III	5to Sem	3

Sexto Semestre – Plan2026			Plan 2018			
Código	**Curso**	**Cr**	**Código**	**Curso**	**Sem**	**Cr**
MA212	Ecuaciones Diferenciales Ordinarias	6	183052	Ecuaciones Diferenciales Ordinarias	5to Sem	6
MA325	Análisis Numérico II	4	183063	Métodos Númericos y Programación II	6to Sem	6
MA371	Análisis funcional I	5	183061	Análisis Funcional I	6to Sem	5
ID202	Inglés IV	3			6to Sem	

Séptimo Semestre – Plan2026			Plan 2018			
Código	**Curso**	**Cr**	**Código**	**Curso**	**Sem**	**Cr**
CS261	Inteligencia Artificial	3	183074	Inteligencia Artificial	7mo Sem	5
CS3P1	Computación Paralela y Distribuída	3			7mo Sem	
MA213	Ecuaciones Diferenciales Parciales	4	183081	Ecuaciones Diferenciales Parciales I	8vo Sem	6
MA326	Análisis Numérico III	4	183093	Métodos Númericos y Programación III	9no Sem	6
MA361	Modelado Matemático	4	183065	Técnicas de Modelamiento	6to Sem	5
ID203	Inglés V	4			7mo Sem	

Octavo Semestre – Plan2026			Plan 2018			
Código	**Curso**	**Cr**	**Código**	**Curso**	**Sem**	**Cr**
AI263	Introducción al Aprendizaje de Máquina	4			8vo Sem	
MA401	Proyecto de final de carrera I	3	183084	Proyecto de Investigación	8vo Sem	4
CS251	Sistemas Operativos	5	183064	Sistemas Operativos	6to Sem	5
CS311	Seguridad en Computación	5			8vo Sem	
QC251	Mecánica Cuántica	5	183123	Mecánica Cuántica	8vo Sem	5
CS253	Análisis exploratorio de datos	5			8vo Sem	
CS370	Big Data y Analítica de Datos	5			8vo Sem	
STA252	Estadística Inferencial	5	183062	Probabilidad e Inferencia Estadística	6to Sem	5
MA327	Análisis Numérico IV	5			8vo Sem	
MA352	Procesos estocásticos	5			8vo Sem	
MA372	Análisis funcional II	5	183071	Análisis Funcional II	7mo Sem	5

Noveno Semestre – Plan2026			Plan 2018			
Código	**Curso**	**Cr**	**Código**	**Curso**	**Sem**	**Cr**
AI264	Aprendizaje Profundo	5			9no Sem	
MA402	Proyecto de final de carrera II	5	183094	Desarrollo del Proyecto de Investigación	9no Sem	5
CS231	Redes y Comunicación	4	183083	Redes, Arquitecturas y Comunicaciones	8vo Sem	6
MA381	Estructuras Algebraicas	4			9no Sem	
AI262	Procesamiento de Lenguaje Natural	4			9no Sem	
CS273	Análisis de series temporales	4			9no Sem	
MA341	Optimización I	4	183091	Optimización	9no Sem	5
MA355	Teoría de Aprendizaje Estadístico	4			9no Sem	

Décimo Semestre – Plan2026			Plan 2018			
Código	**Curso**	**Cr**	**Código**	**Curso**	**Sem**	**Cr**
CS400	Prácticas Pre-profesionales	3	183103	Práctica Preprofesional	10mo Sem	4
AI268	Visión Computacional	5			10mo Sem	
AI367	Tópicos en Inteligencia Artificial	5			10mo Sem	
MA403	Proyecto de final de carrera III	5	183104	Presentación y Sustentación del Proyecto de Investigación	10mo Sem	6
QC381	Introduction a la Computación Cuántica	4			10mo Sem	
CS254	Visualización de Datos Avanzada	4			10mo Sem	

Capítulo 8

Laboratorios

CS111. Introducción a la Programación (Obligatorio) 1er Sem, Lab: 4 hr(s)

Laboratorio Básico de Computación:

- **Hardware**: 30 equipos (Intel Core i5, 8GB RAM, SSD 256GB, gráficos/sonido integrados, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Ubuntu LTS) / Windows 11 Pro (arranque dual).
- **Lenguajes**: Python, Java, C++ (C++11/14/17/20), C#, Prolog, Lex/Yacc.
- **Herramientas**: Git, VS Code, Eclipse, Jupyter Notebook.

CS112. Programación Orientada a Objetos I (Obligatorio) 2do Sem, Lab: 4 hr(s)

Laboratorio Básico de Computación:

- **Hardware**: 30 equipos (Intel Core i5, 8GB RAM, SSD 256GB, gráficos/sonido integrados, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Ubuntu LTS) / Windows 11 Pro (arranque dual).
- **Lenguajes**: Python, Java, C++ (C++11/14/17/20), C#, Prolog, Lex/Yacc.
- **Herramientas**: Git, VS Code, Eclipse, Jupyter Notebook.

AI161. IA Aplicada (Obligatorio) 2do Sem, Lab: 4 hr(s)

Laboratorio Básico de Computación:

- **Hardware**: 30 equipos (Intel Core i5, 8GB RAM, SSD 256GB, gráficos/sonido integrados, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Ubuntu LTS) / Windows 11 Pro (arranque dual).
- **Lenguajes**: Python, Java, C++ (C++11/14/17/20), C#, Prolog, Lex/Yacc.
- **Herramientas**: Git, VS Code, Eclipse, Jupyter Notebook.

CS113. Programación Orientada a Objetos II (Obligatorio) 3er Sem, Lab: 2 hr(s)

Laboratorio Básico de Computación:

- **Hardware**: 30 equipos (Intel Core i5, 8GB RAM, SSD 256GB, gráficos/sonido integrados, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Ubuntu LTS) / Windows 11 Pro (arranque dual).
- **Lenguajes**: Python, Java, C++ (C++11/14/17/20), C#, Prolog, Lex/Yacc.
- **Herramientas**: Git, VS Code, Eclipse, Jupyter Notebook.

CS1D1. Estructuras Discretas I (Obligatorio) 3er Sem, Lab: 2 hr(s)

Laboratorio Básico de Computación:

- **Hardware**: 30 equipos (Intel Core i5, 8GB RAM, SSD 256GB, gráficos/sonido integrados, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Ubuntu LTS) / Windows 11 Pro (arranque dual).
- **Lenguajes**: Python, Java, C++ (C++11/14/17/20), C#, Prolog, Lex/Yacc.
- **Herramientas**: Git, VS Code, Eclipse, Jupyter Notebook.

CS2B1. Desarrollo Basado en Plataformas (Obligatorio) 3er Sem, Lab: 2 hr(s)

Laboratorio Avanzado de Computación:

- **Hardware**: 30 PCs de alto rendimiento (Intel Core i7, 16GB RAM, SSD NVMe 512GB, NVIDIA GTX 1660, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Fedora) / Windows 11 Pro.
- **Lenguajes**: C++17/20, Python (con librerías científicas).
- **Software**:
  - *Desarrollo*: J2EE, .NET 6, MATLAB, GCC/Clang.
  - *Bases de datos/Cloud*: PostgreSQL, MongoDB, Docker.
  - *Especializado*: Rational Rose, VTK, LEDA, emuladores 8086.

CS1D2. Estructuras Discretas II (Obligatorio) 4to Sem, Lab: 2 hr(s)

Laboratorio Básico de Computación:

- **Hardware**: 30 equipos (Intel Core i5, 8GB RAM, SSD 256GB, gráficos/sonido integrados, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Ubuntu LTS) / Windows 11 Pro (arranque dual).
- **Lenguajes**: Python, Java, C++ (C++11/14/17/20), C#, Prolog, Lex/Yacc.
- **Herramientas**: Git, VS Code, Eclipse, Jupyter Notebook.

CS210. Algoritmos y Estructuras de Datos (Obligatorio) 4to Sem, Lab: 2 hr(s)

Laboratorio Avanzado de Computación:

- **Hardware**: 30 PCs de alto rendimiento (Intel Core i7, 16GB RAM, SSD NVMe 512GB, NVIDIA GTX 1660, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Fedora) / Windows 11 Pro.
- **Lenguajes**: C++17/20, Python (con librerías científicas).
- **Software**:
  - *Desarrollo*: J2EE, .NET 6, MATLAB, GCC/Clang.
  - *Bases de datos/Cloud*: PostgreSQL, MongoDB, Docker.
  - *Especializado*: Rational Rose, VTK, LEDA, emuladores 8086.

CS271. Bases de Datos I (Obligatorio) 4to Sem, Lab: 4 hr(s)

Laboratorio Avanzado de Computación:

- **Hardware**: 30 PCs de alto rendimiento (Intel Core i7, 16GB RAM, SSD NVMe 512GB, NVIDIA GTX 1660, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Fedora) / Windows 11 Pro.
- **Lenguajes**: C++17/20, Python (con librerías científicas).
- **Software**:
  - *Desarrollo*: J2EE, .NET 6, MATLAB, GCC/Clang.

- *Bases de datos/Cloud*: PostgreSQL, MongoDB, Docker.
- *Especializado*: Rational Rose, VTK, LEDA, emuladores 8086.

CS251. Computación Gráfica (Obligatorio) 5to Sem, Lab: 4 hr(s)

Laboratorio Avanzado de Computación:

- **Hardware**: 30 PCs de alto rendimiento (Intel Core i7, 16GB RAM, SSD NVMe 512GB, NVIDIA GTX 1660, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Fedora) / Windows 11 Pro.
- **Lenguajes**: C++17/20, Python (con librerías científicas).
- **Software**:
  - *Desarrollo*: J2EE, .NET 6, MATLAB, GCC/Clang.
  - *Bases de datos/Cloud*: PostgreSQL, MongoDB, Docker.
  - *Especializado*: Rational Rose, VTK, LEDA, emuladores 8086.

CS261. Inteligencia Artificial (Obligatorio) 7mo Sem, Lab: 2 hr(s)

Laboratorio Básico de Computación:

- **Hardware**: 30 equipos (Intel Core i5, 8GB RAM, SSD 256GB, gráficos/sonido integrados, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Ubuntu LTS) / Windows 11 Pro (arranque dual).
- **Lenguajes**: Python, Java, C++ (C++11/14/17/20), C#, Prolog, Lex/Yacc.
- **Herramientas**: Git, VS Code, Eclipse, Jupyter Notebook.

CS3P1. Computación Paralela y Distribuída (Obligatorio) 7mo Sem, Lab: 2 hr(s)

Laboratorio Avanzado de Computación:

- **Hardware**: 30 PCs de alto rendimiento (Intel Core i7, 16GB RAM, SSD NVMe 512GB, NVIDIA GTX 1660, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Fedora) / Windows 11 Pro.
- **Lenguajes**: C++17/20, Python (con librerías científicas).
- **Software**:
  - *Desarrollo*: J2EE, .NET 6, MATLAB, GCC/Clang.
  - *Bases de datos/Cloud*: PostgreSQL, MongoDB, Docker.
  - *Especializado*: Rational Rose, VTK, LEDA, emuladores 8086.

MA361. Modelado Matemático (Obligatorio) 7mo Sem, Lab: 2 hr(s)

Laboratorio Avanzado de Computación:

- **Hardware**: 30 PCs de alto rendimiento (Intel Core i7, 16GB RAM, SSD NVMe 512GB, NVIDIA GTX 1660, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Fedora) / Windows 11 Pro.
- **Lenguajes**: C++17/20, Python (con librerías científicas).
- **Software**:
  - *Desarrollo*: J2EE, .NET 6, MATLAB, GCC/Clang.
  - *Bases de datos/Cloud*: PostgreSQL, MongoDB, Docker.
  - *Especializado*: Rational Rose, VTK, LEDA, emuladores 8086.

AI263. Introducción al Aprendizaje de Máquina (Obligatorio) 8vo Sem, Lab: 4 hr(s)

Laboratorio Básico de Computación:

- **Hardware**: 30 equipos (Intel Core i5, 8GB RAM, SSD 256GB, gráficos/sonido integrados, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Ubuntu LTS) / Windows 11 Pro (arranque dual).
- **Lenguajes**: Python, Java, C++ (C++11/14/17/20), C#, Prolog, Lex/Yacc.
- **Herramientas**: Git, VS Code, Eclipse, Jupyter Notebook.

CS2S1. Sistemas Operativos (Alternativo) 8vo Sem, Lab: 4 hr(s)

Laboratorio Avanzado de Computación:

- **Hardware**: 30 PCs de alto rendimiento (Intel Core i7, 16GB RAM, SSD NVMe 512GB, NVIDIA GTX 1660, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Fedora) / Windows 11 Pro.
- **Lenguajes**: C++17/20, Python (con librerías científicas).
- **Software**:
  - *Desarrollo*: J2EE, .NET 6, MATLAB, GCC/Clang.
  - *Bases de datos/Cloud*: PostgreSQL, MongoDB, Docker.
  - *Especializado*: Rational Rose, VTK, LEDA, emuladores 8086.

CS3I1. Seguridad en Computación (Alternativo) 8vo Sem, Lab: 4 hr(s)

Laboratorio Avanzado de Computación:

- **Hardware**: 30 PCs de alto rendimiento (Intel Core i7, 16GB RAM, SSD NVMe 512GB, NVIDIA GTX 1660, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Fedora) / Windows 11 Pro.
- **Lenguajes**: C++17/20, Python (con librerías científicas).
- **Software**:
  - *Desarrollo*: J2EE, .NET 6, MATLAB, GCC/Clang.
  - *Bases de datos/Cloud*: PostgreSQL, MongoDB, Docker.
  - *Especializado*: Rational Rose, VTK, LEDA, emuladores 8086.

CS370. Big Data y Analítica de Datos (Alternativo) 8vo Sem, Lab: 4 hr(s)

Laboratorio Avanzado de Computación:

- **Hardware**: 30 PCs de alto rendimiento (Intel Core i7, 16GB RAM, SSD NVMe 512GB, NVIDIA GTX 1660, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Fedora) / Windows 11 Pro.
- **Lenguajes**: C++17/20, Python (con librerías científicas).
- **Software**:
  - *Desarrollo*: J2EE, .NET 6, MATLAB, GCC/Clang.
  - *Bases de datos/Cloud*: PostgreSQL, MongoDB, Docker.
  - *Especializado*: Rational Rose, VTK, LEDA, emuladores 8086.

AI264. Aprendizaje Profundo (Obligatorio) 9no Sem, Lab: 4 hr(s)

Laboratorio Básico de Computación:

- **Hardware**: 30 equipos (Intel Core i5, 8GB RAM, SSD 256GB, gráficos/sonido integrados, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Ubuntu LTS) / Windows 11 Pro (arranque dual).

- **Lenguajes**: Python, Java, C++ (C++11/14/17/20), C#, Prolog, Lex/Yacc.
- **Herramientas**: Git, VS Code, Eclipse, Jupyter Notebook.

CS231. Redes y Comunicación (Alternativo) 9no Sem, Lab: 4 hr(s)

Laboratorio Avanzado de Computación:

- **Hardware**: 30 PCs de alto rendimiento (Intel Core i7, 16GB RAM, SSD NVMe 512GB, NVIDIA GTX 1660, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Fedora) / Windows 11 Pro.
- **Lenguajes**: C++17/20, Python (con librerías científicas).
- **Software**:
  - *Desarrollo*: J2EE, .NET 6, MATLAB, GCC/Clang.
  - *Bases de datos/Cloud*: PostgreSQL, MongoDB, Docker.
  - *Especializado*: Rational Rose, VTK, LEDA, emuladores 8086.

AI262. Procesamiento de Lenguaje Natural (Alternativo) 9no Sem, Lab: 4 hr(s)

Laboratorio Avanzado de Computación:

- **Hardware**: 30 PCs de alto rendimiento (Intel Core i7, 16GB RAM, SSD NVMe 512GB, NVIDIA GTX 1660, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Fedora) / Windows 11 Pro.
- **Lenguajes**: C++17/20, Python (con librerías científicas).
- **Software**:
  - *Desarrollo*: J2EE, .NET 6, MATLAB, GCC/Clang.
  - *Bases de datos/Cloud*: PostgreSQL, MongoDB, Docker.
  - *Especializado*: Rational Rose, VTK, LEDA, emuladores 8086.

MA341. Optimización I (Alternativo) 9no Sem, Lab: 2 hr(s)

Laboratorio Avanzado de Computación:

- **Hardware**: 30 PCs de alto rendimiento (Intel Core i7, 16GB RAM, SSD NVMe 512GB, NVIDIA GTX 1660, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Fedora) / Windows 11 Pro.
- **Lenguajes**: C++17/20, Python (con librerías científicas).
- **Software**:
  - *Desarrollo*: J2EE, .NET 6, MATLAB, GCC/Clang.
  - *Bases de datos/Cloud*: PostgreSQL, MongoDB, Docker.
  - *Especializado*: Rational Rose, VTK, LEDA, emuladores 8086.

AI268. Visión Computacional (Obligatorio) 10mo Sem, Lab: 2 hr(s)

Laboratorio Avanzado de Computación:

- **Hardware**: 30 PCs de alto rendimiento (Intel Core i7, 16GB RAM, SSD NVMe 512GB, NVIDIA GTX 1660, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Fedora) / Windows 11 Pro.
- **Lenguajes**: C++17/20, Python (con librerías científicas).
- **Software**:
  - *Desarrollo*: J2EE, .NET 6, MATLAB, GCC/Clang.
  - *Bases de datos/Cloud*: PostgreSQL, MongoDB, Docker.
  - *Especializado*: Rational Rose, VTK, LEDA, emuladores 8086.

AI367. Tópicos en Inteligencia Artificial (Obligatorio) 10mo Sem, Lab: 2 hr(s)

Laboratorio Avanzado de Computación:

- **Hardware**: 30 PCs de alto rendimiento (Intel Core i7, 16GB RAM, SSD NVMe 512GB, NVIDIA GTX 1660, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Fedora) / Windows 11 Pro.
- **Lenguajes**: C++17/20, Python (con librerías científicas).
- **Software**:
  - *Desarrollo*: J2EE, .NET 6, MATLAB, GCC/Clang.
  - *Bases de datos/Cloud*: PostgreSQL, MongoDB, Docker.
  - *Especializado*: Rational Rose, VTK, LEDA, emuladores 8086.

QC381. Introduction a la Computación Cuántica (Alternativo) 10mo Sem, Lab: 4 hr(s)

Laboratorio Avanzado de Computación:

- **Hardware**: 30 PCs de alto rendimiento (Intel Core i7, 16GB RAM, SSD NVMe 512GB, NVIDIA GTX 1660, Ethernet Gigabit).
- **Sistemas Operativos**: Linux (Fedora) / Windows 11 Pro.
- **Lenguajes**: C++17/20, Python (con librerías científicas).
- **Software**:
  - *Desarrollo*: J2EE, .NET 6, MATLAB, GCC/Clang.
  - *Bases de datos/Cloud*: PostgreSQL, MongoDB, Docker.
  - *Especializado*: Rational Rose, VTK, LEDA, emuladores 8086.

Capítulo 9

Sumillas

9.1. Primer Semestre

9.1.1. CS111. Introducción a la Programación

Este es el primer curso de la secuencia de cursos introductorios a la Ciencia de la Computación. Este curso introduce a los participantes a los conceptos fundamentales de programación usando Python. Los temas incluyen tipos de datos, estructuras de control, funciones, listas, recursión, y la mecánica de ejecución, prueba y depuración. La programación es uno de los pilares de la Ciencia de la Computación; cualquier profesional en el área necesitará programar para materializar sus modelos y propuestas.

9.1.2. MA100. Matemática Básica

El curso incluye conocimientos sobre Nociones de Lógica Proposicional y teoría de conjuntos, sistemas de los números Reales, números Enteros, Principio de Inducción Matemática, polinomios de una variable. Matrices, determinantes y sus aplicaciones en problemas que involucren a los seres vivos. Introducción a la estadística.

9.1.3. FG101. Lenguaje

El curso de Lenguaje es fundamental para el desarrollo de las habilidades comunicativas del estudiante universitario. Se centra en el dominio de las reglas y principios que regulan el uso del español estándar, abarcando los niveles fonético, sintáctico, léxico y pragmático. Asimismo, busca fortalecer la capacidad de comprensión lectora, síntesis y redacción académica bajo normas internacionales, asegurando que el egresado pueda comunicar sus ideas con coherencia, ética y rigor académico.

9.1.4. FG105. Género y Sociedad

La asignatura de Género y Sociedad busca familiarizar al estudiante con la categoría de género como herramienta de análisis para comprender la organización social. El curso aborda de manera crítica las implicancias de la desigualdad y la discriminación en la producción de conocimiento y en la estructura sociohistórica. A través de este análisis, se promueve una visión reflexiva sobre el entorno, fomentando el respeto por la diversidad y la búsqueda de la equidad en el ejercicio de la ciudadanía y la práctica profesional.

9.1.5. FG109. Ambiente y Desarrollo Sostenible

La asignatura de Medio Ambiente y Desarrollo Sostenible es de naturaleza teórico-práctica y busca sensibilizar al estudiante sobre la importancia de la conservación de los ecosistemas. El curso aborda los principios de la ecología, el impacto de las actividades humanas en la biosfera y las estrategias globales para alcanzar un desarrollo que equilibre el crecimiento económico, la equidad social y la protección ambiental. Estos conocimientos son cruciales para que el profesional de computación diseñe soluciones tecnológicas con un enfoque de responsabilidad ambiental y sostenibilidad.

9.1.6. FG112. Métodos de gestión personal universitaria

La asignatura de Métodos de Gestión Personal Universitaria es de naturaleza teórico-práctica y tiene como propósito empoderar al estudiante mediante el desarrollo de competencias intrapersonales y estrategias de trabajo intelectual. El curso integra procesos de autoconocimiento y planificación estratégica con técnicas avanzadas de aprendizaje y herramientas digitales. A través de este enfoque integral, el estudiante podrá gestionar su comportamiento ante el cambio, optimizar su rendimiento académico y realizar investigaciones con rigor ético, fomentando una visión estratégica para enfrentar los retos del entorno universitario y profesional contemporáneo.

9.1.7. FG140. Fundamentos de Investigación Científica

La asignatura de Fundamentos de Investigación Científica brinda al estudiante los conocimientos y herramientas básicas para el desarrollo de una investigación científica de calidad. El curso aborda la problematización de la realidad, la fundamentación teórica, la metodología y la ética en la investigación. Busca desarrollar en el estudiante la capacidad de observación crítica, el rigor académico y la honestidad intelectual necesarios para producir nuevos conocimientos que contribuyan al avance de la ciencia y la tecnología.

9.2. Segundo Semestre

9.2.1. CS100. Introducción a la Ciencia de la Computación

Este curso sirve como base para comprender los conceptos fundamentales del pensamiento computacional aplicables en diversas profesiones.

El curso proporciona, partiendo desde cero, una visión panorámica de: pensamiento computacional introductorio, almacenamiento de datos, arquitectura de computadoras, sistemas operativos, redes e Internet, algoritmos, métodos de ordenamiento, ingeniería de software, bases de datos, estructuras de datos, ingeniería de software, gráficos por computadora. Además, todos los conceptos son integrados con los nuevos conceptos de la inteligencia artificial.

Diseñado como un curso introductorio a la Ciencia de la Computación, los conceptos se presentan de manera lúdica y utilizando una metodología de Aprendizaje Activo. A lo largo del curso, se fomenta la participación activa de la audiencia, similar a una representación teatral.

Las áreas de conocimiento relacionadas cubiertas están directamente alineadas con los Planes de Estudio de Computación ACM/IEEE-CS.

El curso **no requiere** conocimientos previos en temas de manejo de computadoras y puede ser tomado por estudiantes de cualquier campo.

9.2.2. CS112. Programación Orientada a Objetos I

Este es el segundo curso en la secuencia introductoria de programación en Ciencia de la Computación. Incorpora una transición de lenguaje de programación como estrategia pedagógica y se enfoca en los conceptos centrales de Programación Orientada a Objetos (POO) en C++17/C++20, expandiendo las bases del primer curso mientras introduce conceptos de programación de bajo nivel. El curso proporciona una base sólida para los temas avanzados cubiertos en el curso subsiguiente del plan de estudios.

9.2.3. AI161. IA Aplicada

Este curso proporciona una introducción práctica a la Inteligencia Artificial (IA) para estudiantes de todas las disciplinas científicas y de ingeniería. Centrado en desarrollar alfabetización en IA y habilidades prácticas, cubre conceptos fundamentales, herramientas modernas de IA (incluyendo plataformas occidentales y chinas), y uso responsable. Los estudiantes aprenderán a interactuar efectivamente con diversos sistemas de IA, escribir prompts de calidad y aplicar soluciones de IA a problemas en diversos dominios, comprendiendo las implicaciones éticas y contextos culturales del despliegue de IA.

9.2.4. MA111. Cálculo I

El curso incluye conocimientos sobre relaciones binarias, funciones reales de variable real. Límites, Continuidad y Derivadas de Funciones reales, aplicaciones (máximos y mínimos) y Diferenciales aplicados a la solución de problemas que involucren a los seres vivos.

9.2.5. FI101. Física General

El curso incluye conocimientos sobre los principios y leyes fundamentales de la Física, Mecánica Clásica de Newton, Cinemática, Estática, Dinámica, Energía y Trabajo y Fuerzas conservativas y no conservativas, Energía e Impulso. Pone énfasis en los fenómenos acústicos, ondas electromagnéticas y óptica con aplicaciones en las ciencias biológicas.

9.3. Tercer Semestre

9.3.1. CS113. Programación Orientada a Objetos II

Este es el tercer curso en la secuencia de cursos introductorios en ciencia de la computación. Este curso explora características avanzadas de C++17/C++20 centrándose en el desarrollo de sistemas de alto rendimiento. Los temas clave incluyen metaprogramación con plantillas, semántica de movimiento, reenvío perfecto, optimización RAII, patrones de herencia múltiple/virtual, concurrencia con std::thread y async/await, programación sin bloqueo, paradigmas modernos de C++ como CRTP y plantillas de expresión, e interfaz de C++ con otros lenguajes. El curso prepara a los estudiantes para motores de juego, computación de alto rendimiento y desarrollo embebido donde C++ es dominante.

9.3.2. CS1D1. Estructuras Discretas I

Las estructuras discretas son la columna vertebral de la ciencia de la computación. Este primer curso se centra en la lógica fundamental y el razonamiento formal necesarios para evaluar la corrección de los procesos computacionales. Los estudiantes dominarán el lenguaje formal de las matemáticas para evitar ambigüedad y aprenderán diversas técnicas de demostración esenciales para el análisis riguroso de algoritmos y especificaciones de software.

9.3.3. CS2B1. Desarrollo Basado en Plataformas

El desarrollo basado en plataformas se enfoca en el diseño y la implementación de aplicaciones dentro de entornos informáticos específicos. Este curso utiliza el marco de Proceso y Diseño de Software (SPD) para explorar las restricciones y capacidades únicas de las plataformas Web, Móviles e Interactivas. Los estudiantes aprenderán a aplicar preocupaciones arquitectónicas compartidas y APIs específicas de plataforma para construir soluciones de software robustas, escalables y centradas en el usuario.

9.3.4. MA112. Cálculo II

El curso incluye conocimientos sobre Cálculo Integral: la Integral Indefinida (antiderivada). Técnicas de Integración. Integral Definida. Integración de funciones hiperbólicas e inversas. Aplicaciones de la Integral Definida (Áreas de regiones planas, Volúmenes, Longitud, Área de Superficie). Integral Impropia. Aplicaciones en problemas que involucren a los seres vivos.

9.3.5. MA121. Algebra Lineal I

Este curso introduce los conceptos fundamentales del álgebra lineal, proporcionando las bases matemáticas para el estudio de espacios vectoriales, transformaciones lineales y sistemas de ecuaciones lineales. Desarrolla habilidades de pensamiento abstracto y resolución de problemas mediante matrices, determinantes y vectores. El curso es fundamental para aplicaciones en ciencias, ingeniería y computación.

9.3.6. ID101. Inglés I

Una parte fundamental de la formación integral de un profesional es la capacidad de comunicarse en un idioma extranjero además del propio idioma nativo. No solo amplía su horizonte cultural sino que permite una visión más humana y completa de la vida de las personas. En el caso de los idiomas extranjeros, el inglés es sin duda el más práctico porque se habla en todo el mundo.

9.4. Cuarto Semestre

9.4.1. CS1D2. Estructuras Discretas II

Continuando desde Estructuras Discretas I, este curso explora estructuras matemáticas más complejas como conjuntos, relaciones y funciones, las cuales constituyen la base para el modelado de datos en bases de datos y lenguajes de programación. Adicionalmente, los estudiantes estudiarán técnicas de conteo y relaciones de recurrencia, proporcionando las herramientas analíticas necesarias para estimar la complejidad computacional de los algoritmos.

9.4.2. CS210. Algoritmos y Estructuras de Datos

Este curso proporciona un estudio exhaustivo de las estructuras de datos y algoritmos fundamentales con énfasis en su implementación utilizando características avanzadas de C++. Basándose en conocimientos previos de C++, los estudiantes implementarán estructuras de datos genéricas usando plantillas y rasgos, asegurando flexibilidad de tipo y rendimiento. El curso cubre estructuras lineales (vectores, varias listas enlazadas), estructuras de árbol (binario, AVL, B-trees), montículos, tablas hash, tries y estructuras de datos concurrentes. Se pone especial atención en técnicas modernas de C++ incluyendo plantillas variádicas para operaciones genéricas y programación concurrente para estructuras de datos seguras en hilos.

9.4.3. CS271. Bases de Datos I

Este curso introduce conceptos y técnicas fundamentales en gestión de datos, centrándose en sistemas de bases de datos relacionales. Los estudiantes aprenderán modelado de datos usando diagramas entidad-relación, principios de diseño de bases de datos relacionales, teoría de normalización y Lenguaje de Consulta Estructurado (SQL). El curso cubre componentes centrales de sistemas de bases de datos, gestión de transacciones y procesamiento básico de consultas. Se pone énfasis en habilidades prácticas para diseñar, implementar y consultar bases de datos, así como en comprender el rol de los datos en las organizaciones y las consideraciones éticas asociadas en la gestión de datos.

9.4.4. MA122. Algebra Lineal II

Este curso profundiza en los conceptos avanzados del álgebra lineal, enfocándose en espacios con producto interno, transformaciones lineales avanzadas, descomposiciones matriciales y aplicaciones computacionales. Desarrolla habilidades para analizar estructuras algebraicas complejas y aplicar métodos matriciales a problemas de ciencia de datos, procesamiento de señales y optimización.

9.4.5. MA210. Cálculo III

La naturaleza del curso es teórica-práctica, su propósito es desarrollar en los estudiantes un marco conceptual y metodológico orientado a proporcionar una sólida formación matemática para la comprensión de todo fenómeno natural. Comprende las siguientes unidades temáticas: Cálculo diferencial en varias variables. Cálculo integral en varias variables. Cálculo vectorial.

9.4.6. ID102. Inglés II

Una parte fundamental de la formación integral de un profesional es la capacidad de comunicarse en un idioma extranjero además del propio idioma nativo. No solo amplía su horizonte cultural sino

que permite una visión más humana y completa de la vida de las personas. En el caso de los idiomas extranjeros, el inglés es sin duda el más práctico porque se habla en todo el mundo.

9.5. Quinto Semestre

9.5.1. CS251. Computación Gráfica

Los Gráficos por Computadora son un área fundamental de la Ciencia de la Computación que se enfoca en las técnicas matemáticas y computacionales para generar y manipular imágenes. Este curso proporciona a los estudiantes los fundamentos del modelado 2D y 3D, las canalizaciones de renderizado y los principios físicos de la luz y el color, permitiéndoles desarrollar aplicaciones visuales, simulaciones y sistemas interactivos de alto rendimiento.

9.5.2. MA211. Cálculo IV

La naturaleza del curso es teórica-práctica, su propósito es desarrollar un marco conceptual y metodológico orientado a proporcionar una sólida formación matemática para la comprensión de todo fenómeno natural. Comprende las siguientes unidades temáticas: Sucesiones en R^n, C, (R^m, R). Derivadas direccionales y Derivación implícita. Isometrías de R^n, cónicas y superficies cuadráticas. Campos vectoriales.

9.5.3. MA221. Análisis Numérico I

Este curso introduce los métodos numéricos fundamentales para resolver problemas matemáticos mediante algoritmos computacionales. Desarrolla habilidades para implementar y analizar métodos numéricos en álgebra lineal, ecuaciones no lineales e interpolación, con énfasis en el análisis de error y la estabilidad numérica.

9.5.4. MA222. Análisis Real I

La asignatura corresponde al área de la Estudios Específicos, la naturaleza del curso es teórica-práctica, su propósito está orientado a proporcionar una sólida formación matemática para la comprensión de todo fenómeno natural. Comprende las siguientes unidades temáticas: Numerabilidad. Sucesiones de números reales. Topología de R. Límite y continuidad de funciones reales de una variable real. Derivación. Integral de Riemann-Stieltjes.

9.5.5. MA251. Introducción a la Estadística

Escribir la justificación del curso aquí ...

9.5.6. MA281. Teoría de números

La Teoría de Números es una rama fundamental de las matemáticas puras que estudia las propiedades de los números enteros, siendo la base de múltiples aplicaciones en matemática computacional, criptografía y teoría de la información. Este curso proporciona al estudiante las herramientas teóricas y algorítmicas para analizar problemas de divisibilidad, primalidad, congruencias y ecuaciones diofánticas, desarrollando un pensamiento lógico-abstracto riguroso. El enfoque computacional permite implementar y validar los conceptos teóricos, sentando las bases para áreas como la criptografía de clave pública y los algoritmos de factorización.

9.5.7. ID201. Inglés III

Una parte fundamental de la formación integral de un profesional es la capacidad de comunicarse en un idioma extranjero además del propio idioma nativo. No solo amplía su horizonte cultural sino que permite una visión más humana y completa de la vida de las personas. En el caso de los idiomas extranjeros, el inglés es sin duda el más práctico porque se habla en todo el mundo.

9.6. Sexto Semestre

9.6.1. MA212. Ecuaciones Diferenciales Ordinarias

El curso de Ecuaciones Diferenciales Ordinarias proporciona las herramientas matemáticas para analizar sistemas que evolucionan respecto a una sola variable independiente. Se enfoca en la modelación de sistemas dinámicos mediante ecuaciones de primer orden, orden superior y sistemas de ecuaciones, utilizando métodos analíticos y numéricos. Es fundamental para la comprensión de la mecánica, circuitos eléctricos y modelos poblacionales.

9.6.2. MA325. Análisis Numérico II

Este curso avanza en el estudio de métodos numéricos para problemas matemáticos avanzados, enfocándose en ecuaciones diferenciales, integración numérica y problemas de valores propios. Desarrolla habilidades para implementar y analizar algoritmos numéricos sofisticados con aplicaciones en ingeniería y ciencias computacionales, con énfasis en estabilidad y eficiencia computacional.

9.6.3. MA371. Análisis funcional I

El análisis funcional es un pilar central de las matemáticas modernas, proporcionando el marco para resolver problemas en ecuaciones diferenciales, mecánica cuántica y análisis numérico. Este curso cierra la brecha entre el álgebra lineal y los espacios de dimensión infinita. Al introducir la Teoría de la Medida como una base rigurosa para la integración, el curso permite el estudio de los espacios de Banach y Hilbert, los operadores lineales y los teoremas fundamentales que rigen los sistemas funcionales.

9.6.4. MA351. Estadística y Probabilidades

El cálculo de probabilidades y la estadística son fundamentales en la ciencia de la computación para el análisis de algoritmos, el modelado de sistemas, la toma de decisiones en situaciones de incertidumbre y el análisis de datos. Este curso integra teoría de probabilidades con métodos estadísticos, enfocándose en aplicaciones computacionales como análisis de algoritmos, machine learning, modelado de sistemas y ciencia de datos.

9.6.5. ID202. Inglés IV

Una parte fundamental de la formación integral de un profesional es la capacidad de comunicarse en un idioma extranjero además del propio idioma nativo. No solo amplía su horizonte cultural sino que permite una visión más humana y completa de la vida. En el caso de los idiomas extranjeros, el inglés es sin duda el más práctico porque se habla en todo el mundo. Además de ser vital para su carrera profesional, permite el acceso a documentación técnica actualizada.

9.7. Séptimo Semestre

9.7.1. CS261. Inteligencia Artificial

La Inteligencia Artificial (IA) es un campo fundamental de la ciencia de la computación que busca crear sistemas capaces de realizar tareas que normalmente requieren inteligencia humana. Este curso proporciona una introducción a las técnicas utilizadas para construir dichos sistemas, incluyendo algoritmos de búsqueda, razonamiento probabilístico y aprendizaje automático. Los estudiantes aprenderán a modelar problemas complejos e implementar agentes inteligentes, mientras consideran las implicaciones sociales y éticas de estas tecnologías.

9.7.2. CS3P1. Computación Paralela y Distribuída

Con el fin del escalado de frecuencia en procesadores de un solo núcleo, la computación paralela y distribuida se ha vuelto esencial para el desarrollo de software de alto rendimiento. Este curso

introduce a los estudiantes a los modelos de programación, patrones de comunicación y mecanismos de coordinación necesarios para explotar sistemas de múltiples núcleos y clústeres distribuidos. Los estudiantes aprenderán a diseñar algoritmos escalables y evaluar su rendimiento bajo diferentes restricciones computacionales.

9.7.3. MA213. Ecuaciones Diferenciales Parciales

La asignatura de Ecuaciones Diferenciales Parciales (MA213) estudia fenómenos que dependen de múltiples variables, como la propagación del calor, ondas y potenciales eléctricos. El curso introduce las series de Fourier y los métodos de separación de variables para resolver las ecuaciones clásicas de la física matemática (Calor, Onda y Laplace). Es una pieza clave para la simulación numérica avanzada y el análisis de medios continuos.

9.7.4. MA326. Análisis Numérico III

El curso de Análisis Numérico 3 se centra en la solución computacional de Ecuaciones Diferenciales Ordinarias (EDO). En la matemática computacional, muchos sistemas no pueden resolverse analíticamente; por lo tanto, comprender la convergencia, estabilidad y eficiencia de los integradores numéricos es vital. Este curso abarca desde los métodos básicos de un paso hasta los esquemas avanzados de Runge-Kutta y el tratamiento especializado de ecuaciones rígidas (stiff), los cuales son críticos para simular reacciones químicas, circuitos eléctricos y mecánica estructural.

9.7.5. MA361. Modelado Matemático

Este curso introduce los principios fundamentales del modelado matemático para representar, analizar y predecir el comportamiento de sistemas en diversas áreas como ciencias, ingeniería y negocios. Se enfatiza la formulación de modelos a partir de problemas reales, su resolución mediante herramientas matemáticas y computacionales, y la interpretación crítica de los resultados. El curso desarrolla la capacidad de abstracción y la aplicación de matemáticas avanzadas para resolver problemas complejos del mundo real.

9.7.6. ID203. Inglés V

Una parte fundamental de la formación integral de un profesional es la capacidad de comunicarse en un idioma extranjero además del propio idioma nativo. No solo amplía su horizonte cultural sino que permite una visión más humana y completa de la vida. En el caso de los idiomas extranjeros, el inglés es sin duda el más práctico porque se habla en todo el mundo. En la carrera de ingeniería, el inglés técnico es la herramienta más importante para acceder a la vanguardia del conocimiento y la investigación.

9.8. Octavo Semestre

9.8.1. AI263. Introducción al Aprendizaje de Máquina

Este curso introduce los fundamentos del aprendizaje automático, cubriendo algoritmos clásicos y modernos para problemas de clasificación, regresión y agrupamiento. Se enfoca en la implementación práctica usando scikit-learn y TensorFlow, con aplicaciones en visión por computadora y procesamiento de lenguaje natural.

9.8.2. MA401. Proyecto de final de carrera I

Este curso representa el primer semestre del proyecto de final de carrera en matemáticas computacionales. Los estudiantes desarrollan un proyecto de investigación aplicada o desarrollo tecnológico que integra y aplica los conocimientos adquiridos durante la carrera. El curso enfatiza la formulación de problemas, revisión de literatura, diseño metodológico, y desarrollo inicial de soluciones computacionales bajo supervisión de un profesor tutor. Los proyectos deben demostrar capacidad de investigación, innovación y aplicación de matemáticas computacionales a problemas del mundo real.

9.8.3. CS2S1. Sistemas Operativos

El sistema operativo es una colección de servicios necesarios para interconectar de manera segura el hardware con las aplicaciones. Este curso explora los mecanismos y políticas necesarios para virtualizar el cómputo, la memoria y las E/S. Los estudiantes comprenderán conceptos centrales como concurrencia, persistencia y límites de confianza, los cuales son fundamentales para desarrollar sistemas de software robustos y eficientes.

9.8.4. CS3I1. Seguridad en Computación

En un mundo cada vez más digital, la seguridad ya no es una característica opcional sino un requisito fundamental. Este curso proporciona una introducción completa a los principios y prácticas de la ciberseguridad, cubriendo desde los fundamentos teóricos hasta la codificación segura y la informática forense digital. Los estudiantes adquirirán las habilidades necesarias para diseñar, implementar y gobiernar sistemas seguros, comprendiendo las responsabilidades éticas y profesionales involucradas en la protección de la información y la infraestructura.

9.8.5. QC251. Mecánica Cuántica

La mecánica cuántica es fundamental para comprender los principios físicos que gobiernan los sistemas a escala atómica y subatómica, los cuales son la base de tecnologías computacionales modernas como la computación cuántica, criptografía cuántica, semiconductores y dispositivos de estado sólido. Este curso proporciona las bases matemáticas y conceptuales de la mecánica cuántica, con especial énfasis en sus aplicaciones en ciencias de la computación, incluyendo algoritmos cuánticos, simulación cuántica y el desarrollo de hardware cuántico.

9.8.6. CS253. Análisis exploratorio de datos

El Análisis Exploratorio de Datos (EDA) es una etapa crítica en cualquier proyecto de ciencia de datos. Este curso proporciona las herramientas para entender la estructura, detectar anomalías y extraer patrones iniciales de los datos antes del modelado formal. Se enfoca en el uso de estadísticas descriptivas, limpieza de datos y visualización preliminar para validar supuestos y guiar la selección de modelos complejos, asegurando la integridad y calidad de los hallazgos científicos.

9.8.7. CS370. Big Data y Analítica de Datos

En la era actual, el volumen, velocidad y variedad de los datos superan las capacidades de los sistemas tradicionales de almacenamiento y procesamiento. Este curso introduce los conceptos y tecnologías fundamentales del Big Data. Cubre sistemas de archivos distribuidos, frameworks de procesamiento paralelo, bases de datos NoSQL y flujo de datos en tiempo real. Los estudiantes aprenderán a diseñar arquitecturas escalables para extraer valor de conjuntos de datos masivos.

9.8.8. STA252. Estadística Inferencial

La estadística inferencial es crucial en ciencias de la computación para analizar datos, extraer conclusiones y tomar decisiones basadas en evidencia. Este curso cubre los principios de la inferencia estadística, incluyendo estimación puntual, intervalos de confianza y pruebas de hipótesis.

9.8.9. MA327. Análisis Numérico IV

Análisis Numérico 4 proporciona las herramientas computacionales necesarias para resolver Ecuaciones Diferenciales Parciales (EDP), las cuales describen una amplia gama de fenómenos en física, ingeniería y finanzas. El curso se centra en los tres tipos fundamentales de EDP de segundo orden: elípticas, parabólicas e hiperbólicas. Los estudiantes aprenderán a discretizar dominios continuos, analizar la estabilidad y convergencia de esquemas numéricos e implementar algoritmos para problemas complejos con condiciones de contorno.

9.8.10. MA352. Procesos estocásticos

Este curso introduce la teoría de procesos estocásticos, proporcionando las bases matemáticas para el estudio de sistemas que evolucionan aleatoriamente en el tiempo. Desarrolla habilidades para modelar fenómenos aleatorios en finanzas, ingeniería y ciencias mediante cadenas de Markov, procesos de Poisson y movimiento browniano, con aplicaciones en simulación y análisis de sistemas estocásticos.

9.8.11. MA372. Análisis funcional II

Este curso profundiza en los fundamentos teóricos de la medida y la integración, proporcionando las herramientas esenciales para el análisis avanzado en espacios de dimensión infinita. Se centra en la construcción rigurosa de medidas abstractas y la operatividad de la integral de Lebesgue. Finalmente, se estudia la estructura geométrica y de operadores en espacios de Hilbert, fundamentales para el desarrollo de métodos computacionales zy modelos en física matemática.

9.9. Noveno Semestre

9.9.1. AI264. Aprendizaje Profundo

Este curso cubre los fundamentos del aprendizaje profundo moderno, incluyendo redes neuronales convolucionales, arquitecturas de transformers, y técnicas de entrenamiento para sistemas de inteligencia artificial avanzada.

9.9.2. MA402. Proyecto de final de carrera II

Este curso constituye la fase final del proyecto de titulación en matemáticas computacionales, donde los estudiantes completan, validan y presentan un proyecto de investigación aplicada o desarrollo tecnológico. Se enfatiza la implementación completa de soluciones computacionales, el análisis riguroso de resultados, la validación metodológica y la comunicación profesional de hallazgos. El proyecto debe demostrar dominio de las matemáticas computacionales, capacidad de innovación y contribución al conocimiento o solución de problemas del mundo real, culminando en un trabajo de calidad profesional y una defensa pública ante un jurado especializado.

9.9.3. CS231. Redes y Comunicación

Las redes y la comunicación son la columna vertebral de los sistemas distribuidos modernos y de la Internet global. Este curso proporciona una introducción completa a los modelos OSI y TCP/IP, cubriendo desde la transmisión física hasta los protocolos de capa de aplicación. Comprender estas capas es esencial para diseñar aplicaciones e infraestructuras de red seguras, escalables y eficientes.

9.9.4. MA381. Estructuras Algebraicas

El curso de Estructuras Algebraicas proporciona la base formal y abstracta esencial para las matemáticas avanzadas y sus aplicaciones computacionales. Este curso introduce las estructuras fundamentales del álgebra moderna —grupos, anillos y campos— desarrollando la capacidad de pensar en términos de axiomas, homomorfismos e invariantes. El dominio de estos conceptos es crucial para comprender los fundamentos teóricos de áreas como la teoría de códigos, la criptografía, el diseño de algoritmos y la geometría algebraica. El curso enfatiza la interacción entre la teoría abstracta y los ejemplos concretos, particularmente aquellos que surgen en matemáticas discretas y computacionales.

9.9.5. AI262. Procesamiento de Lenguaje Natural

Este curso introduce los fundamentos teóricos y prácticos del Procesamiento de Lenguaje Natural (NLP), cubriendo desde técnicas clásicas hasta modelos modernos basados en deep learning. Los estudiantes aprenderán a implementar pipelines completos de NLP para aplicaciones como análisis de sentimientos, traducción automática y chatbots.

9.9.6. CS273. Análisis de series temporales

En diversas áreas como las finanzas, la meteorología y la ingeniería, los datos se recolectan secuencialmente a través del tiempo, presentando dependencias temporales únicas. Este curso introduce los fundamentos teóricos y prácticos para analizar, modelar y pronosticar series de tiempo. Se abordan desde métodos clásicos de descomposición hasta modelos estocásticos avanzados (ARIMA, GARCH), permitiendo al estudiante extraer información relevante de procesos dinámicos y tomar decisiones basadas en predicciones rigurosas.

9.9.7. MA341. Optimización I

Este curso introduce los fundamentos matemáticos de la optimización, cubriendo tanto problemas sin restricciones como con restricciones. Se estudian métodos clásicos de optimización continua, condiciones de optimalidad, algoritmos numéricos y aplicaciones en diversos campos. El curso proporciona las bases teóricas y computacionales para formular y resolver problemas de optimización en ciencia, ingeniería y economía, desarrollando habilidades analíticas para caracterizar y encontrar soluciones óptimas.

9.9.8. MA355. Teoría de Aprendizaje Estadístico

Este curso proporciona los fundamentos teóricos del aprendizaje estadístico, cubriendo los principios matemáticos que sustentan los algoritmos de aprendizaje automático. Se enfoca en el análisis teórico de la capacidad de generalización, complejidad de modelos, y garantías de rendimiento para diversos métodos de aprendizaje. El curso conecta la teoría con la práctica a través del estudio de cotas de generalización, compensación sesgo-varianza, y fundamentos de la teoría de aprendizaje computacional.

9.10. Décimo Semestre

9.10.1. CS400. Prácticas Pre-profesionales

Este curso permite a los estudiantes aplicar los conocimientos adquiridos durante su formación académica en un entorno laboral real, bajo la supervisión de una empresa y la universidad. Las prácticas pre-profesionales son esenciales para desarrollar competencias profesionales, éticas y técnicas, asegurando que el estudiante adquiera experiencia en proyectos relacionados con la carrera.

9.10.2. AI268. Visión Computacional

La Visión por Computadora es un área clave de la Inteligencia Artificial que permite a las máquinas interpretar y comprender el mundo visual. Este curso cubre técnicas fundamentales para procesamiento de imágenes, extracción de características y reconocimiento de objetos. Con el advenimiento del aprendizaje profundo, la visión por computadora se ha vuelto esencial para vehículos autónomos, imágenes médicas y realidad aumentada. Los estudiantes aprenderán los fundamentos matemáticos y enfoques algorítmicos para resolver problemas de percepción visual del mundo real.

9.10.3. AI367. Tópicos en Inteligencia Artificial

La Inteligencia Artificial es un campo en rápida evolución. Este curso explora temas avanzados y contemporáneos que están dando forma actualmente a la industria y la investigación. Se pone énfasis en la IA Generativa, Modelos de Lenguaje Grandes (LLMs) y Aprendizaje por Refuerzo Avanzado. Los estudiantes analizarán las arquitecturas detrás de los sistemas modernos de IA, como los Transformers, y discutirán los desafíos técnicos y sociales de desplegar agentes inteligentes autónomos a gran escala.

9.10.4. MA403. Proyecto de final de carrera III

La asignatura de Tesis de Grado en Matemática Computacional (MA403) es la culminación del proceso de formación académica e investigación del estudiante. Este curso se enfoca en la consolidación del informe final de tesis, asegurando el cumplimiento de los estándares científicos y de probidad académica. El estudiante deberá demostrar una integración total de conocimientos matemáticos y habilidades computacionales para resolver un problema de investigación, finalizando con la presentación formal del manuscrito y la preparación técnica para la defensa ante la comunidad académica y profesional.

9.10.5. QC381. Introduction a la Computación Cuántica

Este curso introduce los principios fundamentales de la computación cuántica, incluyendo cúbits, superposición, entrelazamiento, algoritmos cuánticos y hardware. Los estudiantes contrastarán modelos clásicos y cuánticos, explorando aplicaciones en criptografía, optimización, simulación física y aprendizaje automático cuántico, usando frameworks como Qiskit o Cirq. Incluye un módulo práctico sobre arquitecturas de hardware (superconductores, trampas de iones) y su impacto en el diseño de algoritmos.

9.10.6. CS254. Visualización de Datos Avanzada

En la era del Big Data, la capacidad de comunicar información compleja de manera visual y efectiva es fundamental. Este curso profundiza en los principios de la percepción visual, el diseño de tableros (*dashboards*) interactivos y el uso de herramientas avanzadas para la narrativa de datos (*storytelling*). Se centra en transformar análisis abstractos en representaciones visuales que faciliten la toma de decisiones estratégicas, integrando estética, funcionalidad y ética en el manejo de la información.